IMPERIAL

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Mitigating Distribution Shift and Label Noise in Deep Neural Network Based Network Intrusion Detection Systems

Fahad Mastor T Alotaibi

Supervised by Associate Professor Dr Sergio Maffeis

Submitted in part fulfilment of the requirements for the degree of
Doctor of Philosophy in Computing of Imperial College London
September 2025

Declaration of Originality

I, Fahad Alotaibi, hereby declare that the work presented in this thesis is my own, except where explicit acknowledgment is given. All external sources and contributions have been properly cited.

Abstract

Deep Neural Network (DNN)-based Network Intrusion Detection Systems (NIDS) have shown strong performance on offline benchmarks under controlled conditions. However, real-world deployment remains challenging. This thesis focuses on two key challenges: (i) distribution shift, in which benign traffic evolves and attackers continuously devise new strategies; and (ii) label noise, arising from imperfect automated labelling of large volumes of network traffic.

To address these challenges, the thesis first provides a critical literature review and develops a structured taxonomy of techniques for handling distribution shift in NIDS and label noise in both NIDS and malware datasets. Building on these insights, it introduces three frameworks. `Mateen` adapts one-class anomaly detection to evolving benign traffic by combining selective labelling with an ensemble-based mechanism that identifies and responds to shifts with minimal manual effort. `Rasd` extends multi-class NIDS to detect and integrate newly emerging attack classes, substantially reducing labelling costs through strategic selection of a small, informative, and diverse subset. `SLB` mitigates label noise by partitioning the dataset into clean and noisy sets and iteratively refining both the model and the labels.

Each framework is evaluated extensively across multiple datasets and compared with state-of-the-art baselines. `Mateen` improves the anomaly-detection F1 score by 4.13% under a light-shift scenario (CICIDS2017) and by 72.6% under an extreme-shift scenario (Kitsune). `Rasd` increases the novel class detection F1 score by 6.83% on CICIDS2017 and by 19.21% on CSE-CIC-IDS2018. `SLB` reduces the noise rate in CICIDS2017 (with 30% injected random noise) to below 1.2% and outperforms the vanilla baseline by 11.83% in macro F1.

This thesis serves as a reference for researchers and practitioners in cyber security and artificial intelligence. Beyond its literature review and taxonomy, it contributes three frameworks that collectively enhance the robustness of DNN-based NIDS, achieving state-of-the-art results on the evaluated benchmarks.

Acknowledgements

During this journey, I have had the privilege of receiving support from numerous individuals and organisations. First and foremost, I express my deepest gratitude to my supervisor, Dr. Sergio Maffeis. His constant support, kindness, and insightful guidance have made this journey significantly easier. I am deeply grateful for his understanding and hope that our correspondence continues well into the future. I am equally thankful to Najran University for their generous full scholarship and continuous support, which provided me with the essential resources and funding necessary to achieve this work.

I also wish to thank my colleagues in the Security & Machine Learning (SML) Lab: Almuthanna Alageel, Mohamad Hazim, Kate Highnam, Myles Foley, Abdullah Aldaihan, Adam Jones, Yunan Peng, and Xin Fan Guo, for their kindness, friendship, and professionalism. Beyond the lab, I am grateful to Abdullah Alotaibi (KCL); Abdulmajed Alasmari (KCL); Ahmed Aljameel (ICL); Awadh Alotaibi (Leeds); Faisal Algoblan (KCL); Hani Alotaibi (Newcastle); Imran Alturki (ICL); Meshal Alotaibi (Reading); Muhammed Alotaibi (ICL); Muhammed Alotaibi (QMUL); Saad Asiri (UCL); and Shuayl Alotaibi (York) for their friendship and steady encouragement.

Dedication

I dedicate this thesis to my beloved family. To my father, Mastor, whose lifelong commitment to learning has been a constant source of inspiration. To my mother, Fawziah, whose unconditional love and strength have been the foundation upon which I stand. To my wife, Aliyah, whose patience, encouragement, and unwavering support have made this journey possible. To my sons, Abdullah and Fawaz, whose smiles bring me daily motivation and joy. To my brothers: Abdullah, Majed, Salman, and Hussain, and to my sisters, Sarah and Salma, whose encouragement has continually lifted my spirit. Finally, to my in-laws in Leeds: Maram, who is pursuing her PhD, and her family, Basel and Faraj, for their encouragement along the way.

Contents

List of Figures

List of Tables

List of Abbreviations

AE AutoEncoder

AMMD Adversarial Maximum Mean Discrepancy

AUC Area Under the Curve

AV Anti-Virus

CB Class-Balanced

CIC Canadian Institute for Cybersecurity

CL Confident Learning

CPU Central Processing Unit

CSE Communications Security Establishment

DDoS Distributed Denial of Service

DNN Deep Neural Network

DNNs Deep Neural Networks

DrLIM Dimensionality Reduction by Learning an Invariant Mapping

DTree Decision Tree

ECDFs Empirical Cumulative Distribution Functions

EN Effective Number

ERM Empirical Risk Minimisation

EVT Extreme Value Theory

FFT Farthest-First Traversal

FN False Negatives

FP False Positives

FPR False Positive Rate

GA Genetic Algorithm

GAN Generative Adversarial Network

GMM Gaussian Mixture Model

HIDS Host Intrusion Detection Systems

HTTP Hypertext Transfer Protocol

HTTPS Hypertext Transfer Protocol Secure

IDS Intrusion Detection Systems

IoT Internet of Things

IP Internet Protocol

KL Kullback–Leibler

KNN K-Nearest Neighbors

KS Kolmogorov–Smirnov

LAN Local Area Network

LLM Large Language Model

LR Logistic Regression

LSL Lifted Structured Loss

ML Machine Learning

MLP MultiLayer Perceptron

MSE Mean Squared Error

NCN Nearest Centroid Neighbour

NIDS Network Intrusion Detection Systems

PCA Principal Component Analysis

PCs Personal Computers

ReLU Rectified Linear Unit

RF Random Forest

ROC Receiver Operating Characteristic

SGD Stochastic Gradient Descent

SLB Selective Label Bootstrapping

SoTA State of The Art

SQL Structured Query Language

SVM Support Vector Machine

TCP Transmission Control Protocol

TN True Negatives

TP True Positives

TPR True Positive Rate

VAE Variational AutoEncoder

VT VirusTotal

XSS Cross-Site Scripting

Chapter 1

Introduction

Enterprise networks form the core of modern business operations, connecting data centres, cloud services, on-premises servers, and endpoints. This expansive connectivity empowers real-time collaboration, boosts productivity, and accelerates decision-making across geographical and organisational boundaries. However, the ubiquity and critical role of these networks make them prime targets for malicious actors, including organised crime groups, state-sponsored entities, and opportunistic cybercriminals. These adversaries, motivated by financial gain, espionage, or sabotage, employ sophisticated and evolving tactics including ransomware, exploitation of unpatched software, compromise of weak authentication mechanisms, and social engineering.

The consequences of successful attacks are severe, including substantial financial losses, significant operational disruptions, reputational damage, and exposure of sensitive information. For instance, in 2024, UnitedHealth Group's subsidiary Change Healthcare experienced a ransomware attack that led to an immediate quarterly loss of $872 million [214]. Equifax agreed to a $425 million settlement for a data breach affecting 147 million individuals, caused by exploitation of an unpatched Apache server [80]. Taiwan Semiconductor Manufacturing Company (TSMC) incurred approximately $170 million in losses after a three-day operational halt following a ransomware attack [33, 263]. In more extreme scenarios, cyberattacks have led to bankruptcy and extensive job losses, as evidenced by the cases of Travelex and Colorado Timberline [257, 222].

A practical defence against such threats is the deployment of Network Intrusion Detection Systems (NIDS). The underlying principle is simple: leverage existing knowledge to differentiate malicious traffic from legitimate activity. NIDS operate primarily via two approaches: signature-based detection and anomaly-based detection. Signature-based systems compare network traffic against a database of known attack patterns (signatures) and raise alerts when a match is found. Anomaly-based systems, on the other hand, learn a statistical model of normal behaviour and flag deviations as potentially malicious.

1.1 Motivation

However, both detection approaches have well-known limitations. Signature-based detection recognises only attacks that have already been recorded in the database. It depends on a never-ending cycle of threat surveillance, detailed analysis, rule generation, and careful database updates. As traffic volume and adversary creativity grow, that cycle becomes increasingly cumbersome and slower to complete. Even a subtle modification to a packet or flow can cause a signature to fail to match, allowing an attacker to go unnoticed. Completely novel attacks, meanwhile, are absent from the database and therefore invisible to the system. Anomaly-based detection can spot both modified and entirely new attacks because it flags any traffic that deviates from the learned baseline of normal network behaviour. Yet this strength comes at a cost: by definition, anything unusual, including harmless deviations, can trigger alarms, yielding high false-alarm rates. Over time, the model of normal behaviour changes, so analysts must periodically gather fresh baseline data and update the system to keep error rates in check—a process that is likewise slow and labour-intensive.

To mitigate these limitations, industry and academia alike have turned to Deep Neural Networks (DNNs) for intrusion detection (*e.g.* [177, 224, 74, 187, 255, 152, 108, 49, 215, 111, 78, 12, 193, 6, 120, 233, 127]). DNN-based NIDS learn non-linear patterns in raw packet and flow features that reveal malicious intent, reducing reliance on hand-crafted signatures and large, manually curated rule sets. This has led to strong adoption: commercial systems such as Palo Alto Networks' PAN-OS 10.2 Nebula [193] claim improved accuracy over signature-based and traditional anomaly detectors, while academic studies report performance exceeding 99% for both anomaly detection [111, 215] and multi-class intrusion classification [6, 128].

However, the generalisability of DNN-based NIDS remains limited. These models perform well only on traffic close to their training distribution, implicitly assuming relatively stationary network behaviour. In practice, the distribution of traffic observed at deployment (test time) changes over time: new services are launched, configurations and policies evolve, user behaviour shifts with workflows and applications, and adversaries devise new attack techniques. This mismatch between the training distribution and the deployment distribution is known as distribution shift [274, 181, 88, 159]. When distribution shift occurs, performance often degrades in both anomaly detection [107, 194, 276] and explicit attack classification [295, 18, 142]. DNN-based NIDS therefore need frequent updates; if left unchanged, even the most accurate model will quickly fall behind the threat landscape. Moreover, the performance of DNN-based NIDS heavily depends on large-scale, accurately labelled training datasets. Given the immense volume of network traffic, manually labelling all samples is prohibitively expensive and time-consuming. Consequently, automated labelling methods are often used, which inevitably introduce label noise by mislabelling a portion of the data [77, 156]. When trained on such noisy datasets, DNN-based intrusion detection models overfit to this noise and consequently perform poorly on classification and detection tasks [304, 305, 202].

A straightforward way to mitigate distribution shift is to update models frequently [17, 238, 43] or to monitor performance in real time and react accordingly [18, 276]. Both strategies often require

ongoing manual labelling of incoming data, which is costly and difficult to sustain at scale. Label noise, on the other hand, can be addressed by auditing the labelling pipeline, validating each step, and comparing the results with trusted evidence [77, 156]. Lessons from this audit are then applied to correct the data labels. Nevertheless, this remediation requires a thorough understanding of the operational environment, the labelling procedures, and the threat landscape. To this end, this thesis aims to empirically study the effects of distribution shift and label noise on DNN-based NIDS, and to devise efficient adaptation and denoising strategies that improve performance with minimal human supervision.

1.2 Objectives

To achieve this aim, the thesis pursues five objectives:

1. Survey and critically analyse the literature on distribution shift and label noise in DNN-based NIDS, identifying strengths, limitations, and gaps.
2. Develop adaptive methods for one-class anomaly detection that reduce performance degradation from benign shifts under tight labelling budgets.
3. Develop novel class detection and adaptation techniques for multi-class NIDS that correctly identify emerging attack classes and incorporate them with low labelling budgets.
4. Design strategies that mitigate the impact of label noise through automatic identification, correction, and noise-tolerant learning.
5. Implement and evaluate the proposed methods across multiple datasets and benchmarks against State of The Art (SoTA) baselines.

1.3 Contributions

To address the research aim and objectives set out earlier, this thesis makes the following contributions:

Literature Review and Taxonomy. In Section 2.4, we provide a survey of distribution shift in DNN-based NIDS. We structure the literature along three dimensions: the training paradigm (one-class, binary, and multi-class), the type of shift (benign shift, in-class evolution, and novel-class emergence), and the mitigation stage (shift detection, sample selection, sample labelling, model adaptation, and robust learning). For each mitigation stage, we thoroughly taxonomise the specific techniques employed (*e.g.*, pseudo-labelling and manual labelling for the sample labelling stage). To the best of our knowledge, this taxonomy is the first to cover all three dimensions, setting our review apart from previous surveys [274, 237]. In Section 2.5, we extend our survey to learning under label noise for both NIDS and malware datasets. Here, we build a taxonomy that considers the training paradigm (binary or multi-class), the application domain (NIDS, malware, or domain-agnostic), and the mitigation strategy (sample selection, label correction, robust training, and semi-supervised learning).

Benign Shift Mitigation in One-Class Anomaly Detection. In Chapter 3, we study how benign distribution shift affects one-class AutoEncoder (AE)–based intrusion detectors. Across the datasets we study, such shifts often cause a noticeable degradation in detection performance. To address this, we propose `Mateen`, a four-stage adaptive framework. `Mateen` (i) detects benign shifts, (ii) selects a concise, representative coreset of new samples for manual labelling, (iii) adapts an ensemble of AE to the updated distribution, and (iv) prunes the ensemble to manage computational costs effectively. We evaluate `Mateen` on three public NIDS datasets and two extensions, showing that it consistently improves detection performance under diverse benign shift types and surpasses recent SoTA methods. Moreover, its coreset selection and ensemble-adaptation stages enable low-supervision operation: only a small number of samples require manual labelling, and the ensemble adapts effectively within this limited budget.

Novel Class Detection and Adaptation in Multi-Class DNN-based NIDS. In Chapter 4, we argue that contrastive distance losses yield a more reliable signal for detecting previously unseen attack classes than softmax probabilities, yet their susceptibility to class imbalance and training overhead limit practical use. To address this, we refine a centroid-based distance loss to be robust to imbalance while preserving its lightweight computation, and integrate it into `Rasd`, a two-stage framework comprising (i) shift detection and (ii) sample selection and labelling. In the first stage, `Rasd` runs a distance-based novel-class detector in parallel with the classifier. In the second stage, a computational geometry method selects informative, diverse samples for manual labelling and subsequent model updates. We evaluate `Rasd` on two public, time-aware datasets and observe consistent gains over two SoTA baselines in both novel-class detection and adaptation. Finally, we show that the selection method improves classifier performance even with a very low budget (*e.g.*, labelling only 0.5% of detected novel-class samples), enabling efficient adaptation with low human supervision.

Label Noise Mitigation in Security Datasets. In Chapter 5, we examine how label noise affects DNN classifiers on NIDS and malware datasets, finding that label noise consistently degrades performance across feature spaces and model complexities. To address this challenge, we introduce Selective Label Bootstrapping (`SLB`), a framework that denoises datasets while simultaneously training a reliable classifier. `SLB` leverages prediction consistency to identify samples that are likely to be correctly labelled and to propose plausible corrections for mislabelled samples, thereby producing a refined dataset. Subsequently, it employs a continuous revision strategy to train safely on this refined dataset. We evaluate `SLB` on five diverse public datasets and four extensions, demonstrating that it substantially reduces label noise and yields robust classifiers without human supervision. Comparative experiments against two recent SoTA methods show that `SLB` surpasses them in both robustness and training efficiency.

1.4 Publications

Over the course of my PhD, I contributed to five peer-reviewed publications. Three form the scientific backbone of this thesis; the remaining two fall outside its scope.

Core papers incorporated in this thesis:

- **Fahad Alotaibi** and Sergio Maffeis, "Rasd: Semantic Shift Detection and Adaptation for Network Intrusion Detection," *Proceedings of the IFIP International Conference on ICT Systems Security and Privacy Protection (IFIP SEC 2024)*, 2024. DOI: 10.1007/978-3-031-65175-5_2.
  (Discussed in Chapter 4.)

- **Fahad Alotaibi** and Sergio Maffeis, "Mateen: Adaptive Ensemble Learning for Network Anomaly Detection," *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defences (RAID 2024)*, 2024. DOI: 10.1145/3678890.3678901.
  (Discussed in Chapter 3.)

- **Fahad Alotaibi**, Euan Goodbrand, and Sergio Maffeis, "Deep Learning from Imperfectly Labelled Malware Data," *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS 2025)*, 2025. DOI: 10.1145/3719027.3765197.
  (Discussed in Chapter 5.)

Manuscript in preparation:

- **Fahad Alotaibi** and Sergio Maffeis, "Distribution Shift in DNN-based NIDS: A Survey and Taxonomy," *Manuscript in preparation*.
  (Detailed in Section 2.4.)

Authorship contribution. Sergio Maffeis served as supervisor for all projects, providing guidance, feedback, and substantial improvements to the manuscript. Euan Goodbrand provided editorial input to the final CCS 2025 manuscript and carried out the initial exploratory research on feature noise, label noise, and class imbalance. As his research represents a distinct methodological and conceptual contribution, it will be reported in a separate publication.

Additional publications (outside thesis scope):

- Abdullah Aldaihan, **Fahad Alotaibi** and Sergio Maffeis, "Clouseau: Intelligent Agents for Autonomous Attack Investigation," *2025 Annual Computer Security Applications Conference (ACSAC)*, 2025.

- Lucy Steele, **Fahad Alotaibi**, and Sergio Maffeis, "(Poster) Randomness Unmasked: Reproducible Benchmarks for Shift-Aware Deep-NIDS Models," *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS 2025)*, 2025. DOI: 10.1145/3719027.3760743.

1.5 Thesis Outline

The rest of this thesis is structured as follows. Chapter 2 provides foundational concepts related to network intrusions, intrusion detection systems, and DNNs. It also includes comprehensive surveys on distribution shift in DNN-based NIDS and label noise in NIDS and malware

datasets. Chapter 3 focuses specifically on mitigating benign distribution shift that affects one-class anomaly detection systems. It introduces the `Mateen` framework and evaluates its performance against existing methods. Chapter 4 examines the detection and adaptation to novel attack classes within multi-class scenarios, presenting the `Rasd` framework, which integrates shift detection with a sample selection technique. Chapter 5 tackles the issue of label noise prevalent in security datasets by proposing `SLB`, a framework that both corrects noisy labels and trains classifiers resilient to such noise. Finally, Chapter 6 concludes the thesis by summarising key contributions, reflecting on limitations encountered, and outlining potential directions for future research.

Research trajectory. The project progressed through four milestones: (i) a survey of related work, iteratively refined throughout the PhD; (ii) development of `Rasd` to handle emerging attack classes; (iii) creation of `Mateen` for anomaly detection under evolving benign traffic; and (iv) design of `SLB` for label noise mitigation. Chronologically, `Rasd` preceded `Mateen`. However, the chapters are organised conceptually: we present anomaly detection first (`Mateen`), as it naturally precedes multi-class classification (`Rasd`), and address label noise mitigation last (`SLB`), after introducing both the classifier and its training data.

Chapter 2

Background and Literature Review

This chapter provides the foundational background and relevant literature supporting the contributions of this thesis. Section 2.1 introduces intrusions and their detection systems; Section 2.2 surveys DNN fundamentals, focusing on fully connected, feed-forward architectures, which form the core model used in this thesis; Section 2.3 provides details on the NIDS datasets used; Section 2.4 presents background, a taxonomy, and an in-depth review of distribution shift mitigation for DNN-based NIDS; Section 2.5 offers a parallel taxonomy and literature review on handling label noise in DNN-based NIDS and malware detection; Section 2.6 specifies the threat model assumed throughout the thesis; and Section 2.7 concludes the chapter.

2.1 Network Intrusion Detection

Network intrusions are a critical threat to modern organisations: they can damage reputations, expose secret information [268, 266], and disrupt day-to-day operations [263, 35]. One of the most effective first lines of defence is timely detection, typically provided by an intrusion detection system. In this section, we first define network intrusions and present recent real-world examples (§2.1.1). Then, we introduce intrusion detection systems and outline the main design approaches (§2.1.2).

2.1.1 Intrusions

Intrusions can be defined as unauthorised or malicious actions, whether attempted or successful, that compromise at least one element of the confidentiality, integrity, and availability triad of an information asset [101]. In other words, an intrusion is any attempt by an internal or external actor to gain unauthorised access to data or system resources, to modify them, or to disrupt their availability.

Confidentiality. Confidentiality ensures that information remains accessible only to those who have explicit authorisation to view it. Intrusions threatening confidentiality include credential-guessing (brute force) attacks, where automated tools systematically try username–password pairs

until one works, and data exfiltration, where information is secretly transferred beyond an organisation's secure perimeter.

For example, on 22 April 2024 Change Healthcare disclosed that ransomware attackers had exfiltrated customer data; by 24 January 2025 the company confirmed that roughly 190 million people had been affected [268, 281]. Another instance is the UK Ministry of Justice (Legal Aid Agency) data breach, disclosed in May 2025, which stemmed from inadequate security maintenance [266, 146]. Finally, in April 2025, Australian superannuation funds fell victim to a credential-guessing campaign that used previously exposed credentials (a technique called credential stuffing) to gain unauthorised access [147, 29].

Integrity. Integrity ensures that information remains accurate and unaltered from its intended state; only approved entities may create, modify, or delete data, and any legitimate changes must be traceable. One prominent intrusion is the software supply-chain attack, whereby an attacker covertly adds or alters code during the software development or update cycle, causing the compromised program to execute malicious instructions once deployed.

A notable example is the CCleaner supply-chain compromise. In March 2017, attackers used stolen TeamViewer credentials to access a developer workstation, moved laterally to the build server, and inserted a backdoor into an otherwise legitimate release [30, 31, 53]. The tampered, digitally signed build was then distributed via Piriform's official update channel from August 15 to September 12, 2017, leading more than two million Personal Computers (PCs) to install what appeared to be a routine update but was, in fact, vendor-signed spyware [53, 31].

Availability. Availability guarantees that authorised users and processes can reach the network, servers, and other resources whenever they need them. In other words, every component must stay online and responsive, and no malicious activity should be able to block or degrade that access. Intrusions that undermine this objective often flood bandwidth, overwhelm Central Processing Unit (CPU), or crash critical services, thereby preventing legitimate users from gaining access.

One notable example is the Mirai botnet incident of 2016, in which over 600,000 Internet of Things (IoT) devices (including certain IP cameras) were hijacked [20]. Attackers took advantage of 62 default username–password pairs (an elementary dictionary-based attack) to seize control of these devices, forming a large-scale botnet capable of launching massive Distributed Denial of Service (DDoS) attacks. One of its targets was Dyn DNS, whose subsequent downtime disrupted well-known websites such as Spotify, GitHub, and Reddit [35].

2.1.2 Detection Systems

Intrusion Detection Systems (IDS) help mitigate intrusions by continuously monitoring activity and comparing it against an established baseline or known signatures to identify malicious or unwanted behaviour. Broadly, IDS can be classified by deployment scope and detection methodology.

Deployment scope. IDS can be placed directly on a system (host-based IDS, or HIDS), at key aggregation points in the network (network-based IDS, or NIDS), or in a layered (hybrid) arrangement that combines the two [223, 174, 261, 40, 223].

HIDS run on each protected system (*e.g.*, server, workstation, or virtual machine) and can observe detailed local data, such as audit logs, kernel or system-call events, file-system changes, and process activity [40]. This level of granularity allows the system to identify which user account launched a suspicious executable or which process edited a sensitive configuration file. However, because HIDS must be installed and maintained on every host, they introduce per-host overhead and lack visibility into broader network-level events.

In contrast, NIDS are typically deployed out-of-band, connected to a switch's SPAN (mirror) port or a dedicated network tap, enabling them to inspect network traffic flowing among multiple devices [223]. By examining packet headers, flow records (*e.g.*, NetFlow), and any unencrypted payload data, they can detect patterns such as reconnaissance scans, command-and-control beacons, or DDoS attacks. On the downside, NIDS cannot observe purely local events (*e.g.*, file modifications) and their ability to inspect packet contents may be restricted by encryption.

Detection methodology. IDS, whether host-based or network-based, generally employ one of two common detection strategies: signature-based or anomaly-based [223, 174, 261, 40, 223].

Signature-based detection works by comparing incoming flows or packets against a curated dataset of known attack patterns. Each pattern, or signature, captures a distinctive trait of a particular exploit, malware, or technique. If the system observes traffic that matches one of these patterns, it flags it as malicious. This method is highly effective against repeat attacks that look identical (or very similar) to past incidents, but it may miss zero-day or polymorphic attacks for which no signature yet exists.

```
alert tcp $EXTERNAL_NET any -> $HTTP_SERVERS $HTTP_PORTS \
(msg:"SQL Injection Attempt"; \
  flow:to_server,established; \
  content:"' OR 1=1-"; nocase; http_uri; \
  sid:1000001; rev:1;)
```

Table 2.1: Example Snort signature that targets a classic SQL-injection probe.

Table 2.1 illustrates a simple signature-based rule. It triggers whenever an HTTP request headed towards a protected web server contains the canonical SQL-injection test string `' OR 1=1-` in its URI. The `flow:to_server,established` option restricts detection to traffic travelling towards the server on an already established TCP session, while `nocase` makes the match case-insensitive. Because the rule looks for one exact byte sequence, attackers can evade it with trivial modifications, such as using a different tautology such as `2=2`.

Anomaly-based detection establishes a model of normal behaviour using statistical modelling (*e.g.*, univariate or multivariate analysis [299]) or machine learning techniques (*e.g.*, AE [177]), then identifies suspicious activity when observed events deviate from the established baseline [134]. This method is particularly useful for detecting unknown or zero-day exploits because

it does not rely on prior knowledge of malicious actions. However, anomaly-based systems often produce more false alarms, since legitimate changes in network traffic or system workloads may appear unusual.

To remain effective, both detection strategies require ongoing maintenance. Security analysts must continuously collect up-to-date traffic and event data, examine them for new attack patterns or shifts in legitimate behaviour, and feed these findings back into the IDS. In practice, this involves adding or adjusting signatures in signature-based systems and periodically updating anomaly-detection models (*e.g.*, retraining DNN models or redefining statistical thresholds) so that the baseline for normal activity keeps pace with changes in benign behaviour.

2.2 Deep Neural Networks

DNNs are among the most influential models in deep learning—a core area of modern machine learning. Over the past decade, they have set the SoTA across a wide range of tasks, particularly in computer vision [114, 254, 119, 71] and natural language processing [65, 41, 262, 2, 154]. Motivated by these breakthroughs, security researchers and practitioners have begun applying DNNs to security problems, with network intrusion detection emerging as a particularly active topic [78, 253, 12, 96, 89, 32].

This section provides the groundwork on DNNs needed for the remainder of the thesis. First, we introduce the fundamental building blocks of neural networks and show how they compose to form computation graphs (§2.2.1). Next, we examine how networks are trained under the supervised, semi-supervised, and unsupervised learning regimes (§2.2.2). We then review the optimisation algorithms used during training (§2.2.3) and discuss the key hyperparameters that govern their behaviour (§2.2.4). Finally, we summarise standard data split practices and the evaluation metrics commonly used to assess model effectiveness (§2.2.5).

2.2.1 Architecture

A neural network is fundamentally a function f_θ, parameterised by weights and biases, designed to map input vectors to corresponding outputs [99]. At its simplest, a neural model can be a single computational unit called a neuron [171, 217, 99].

A neuron performs computations on an input vector $\mathbf{x} \in \mathbb{R}^d$ and produces a scalar output via a weighted sum followed by a bias term and a nonlinear activation function:

$$\hat{o} = \sigma\big(\mathbf{w}^\top \mathbf{x} + b\big) = \sigma\Big(\sum_{i=1}^{d} w_i x_i + b\Big), \tag{2.1}$$

where $\mathbf{w} \in \mathbb{R}^d$ denotes the weights, $b \in \mathbb{R}$ represents the bias term, and $\sigma(\cdot)$ is a nonlinear activation function. Figure 2.1 illustrates this computation for $d = 3$: the neuron receives a three-dimensional feature vector (*e.g.*, the numeric encoding of a network flow), multiplies each component by its corresponding weight, adds the bias, and then applies the activation to produce an output.

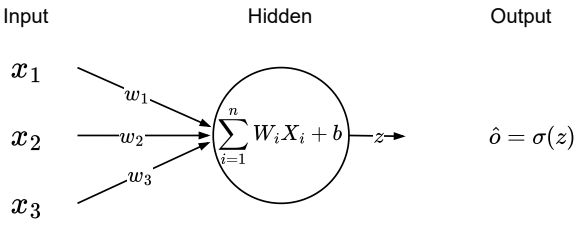


Figure 2.1: Computational diagram of a single fully connected neuron.

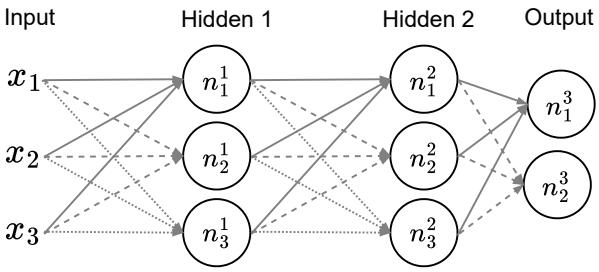


Figure 2.2: Example of a DNN (Deep Neural Network) with two hidden layers. Neuron n_i^l denotes the i^{th} neuron in layer l.

Stacking multiple neurons forms a fully connected layer, where each neuron computes in parallel without lateral connections, resulting in a vector of outputs. Mathematically, the operation of a fully connected layer comprising m neurons can be succinctly expressed in matrix form as:

$$\hat{\mathbf{o}} = \sigma\big(W\,\mathbf{x} + \mathbf{b}\big), \tag{2.2}$$

with weight matrix $W \in \mathbb{R}^{m \times d}$ and bias vector $\mathbf{b} \in \mathbb{R}^m$.

Networks comprising only a single hidden layer are often referred to as shallow neural networks. By contrast, modern neural networks typically consist of several hidden layers, resulting in deep neural networks. Numerous studies have demonstrated empirically that increasing depth often enables hierarchical, increasingly abstract representations and can boost performance on complex recognition tasks [141, 116, 252]. For instance, deeper architectures have been key in advancing object recognition in computer vision [141], speech recognition [116], and machine translation [252]. Figure 2.2 exemplifies such a deep network structure.

The computational flow through a DNN is performed layer by layer in a process known as forward propagation. Given an initial input vector $\mathbf{a}^0 := \mathbf{x}$, each subsequent layer l computes an intermediate linear transformation followed by a nonlinear activation:

$$\mathbf{z}^l = W^l\,\mathbf{a}^{l-1} + \mathbf{b}^l, \tag{2.3}$$

$$\mathbf{a}^l = \sigma^l\big(\mathbf{z}^l\big). \tag{2.4}$$

Here, $W^l \in \mathbb{R}^{m_l \times m_{l-1}}$, $\mathbf{b}^l \in \mathbb{R}^{m_l}$, and $\mathbf{a}^l \in \mathbb{R}^{m_l}$, with $m_0 = d$ and $m_L = M$ denoting the number of

output units. For clarity, the calculation for a single neuron n_i^l within layer l is

$$a_i^l = \sigma\Big(\sum_{j=1}^{m_{l-1}} w_{ij}^l a_j^{l-1} + b_i^l\Big), \tag{2.5}$$

which directly generalises the single-neuron computation described by Eq. (2.1).

The activation function σ is important because it allows the neural network to learn nonlinear relationships from the data. Common choices for hidden layers include the hyperbolic tangent (tanh), Rectified Linear Unit (ReLU) [183], and the sigmoid function, each with distinct advantages depending on the application:

$$\text{tanh:} \quad \sigma_{\tanh}(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \tag{2.6}$$

$$\text{ReLU:} \quad \sigma_{\text{ReLU}}(z) = \max(0,z), \tag{2.7}$$

$$\text{sigmoid:} \quad \sigma_{\text{sigmoid}}(z) = \frac{1}{1+e^{-z}}. \tag{2.8}$$

Here, z denotes the pre-activation value introduced in Eq. (2.3), and $e \approx 2.71828$ is the base of the natural logarithm. The final activation is task-dependent and can differ from the hidden-layer choice; this will be explained in the next section.

2.2.2 Supervision Level

The fully connected feedforward network, also known as a MultiLayer Perceptron (MLP), introduced above must be tailored to the amount of supervision available in the training dataset. Let

$$D_{\text{train}} = \{\mathbf{x}_i\}_{i=1}^N, \quad \mathbf{x}_i \in \mathbb{R}^d, \tag{2.9}$$

denote the training samples, and let $f_\theta : \mathbb{R}^d \to \mathbb{R}^M$ be the network with parameters θ. If labels are present, define a labelled subset $\{(\mathbf{x}_i, y_i)\}_{i \in I_{\text{lab}}}$; otherwise only $\mathbf{x}_i$ is observed. Training uses Empirical Risk Minimisation (ERM) with a regime-specific target $y_i^{(\text{proxy})}$:

$$\theta^\star = \arg\min_\theta \frac{1}{N} \sum_{i=1}^N \ell\big(f_\theta(\mathbf{x}_i), y_i^{(\text{proxy})}\big), \tag{2.10}$$

where $y_i^{(\text{proxy})}$ is the true label in supervised learning; a pseudo-label or consistency target in semi-supervised learning; or a signal derived from $\mathbf{x}_i$ in unsupervised learning (*e.g.*, reconstruction). The remainder of this subsection specifies the output activation, the loss ℓ, and any architectural adjustments for each regime.

Supervised learning. In the fully labelled regime for classification, let C denote the number of classes and M the number of output logits. For multi-class problems we typically set $M = C$, whereas for binary classification one may use either $M = 1$ (single-logit sigmoid) or $M = 2$ (two-logit softmax).

For binary classification with a single logit ($M = 1$, $C = 2$), let the final layer produce $z_i^L \in \mathbb{R}$ for sample i. The predicted probability of the positive class is $\hat{y}_i = \sigma(z_i^L)$, where $\sigma(\cdot)$ is the sigmoid

of Eq. (2.8), and $y_i \in \{0,1\}$. The per-example binary cross-entropy is

$$\ell_{\mathrm{BCE}}(y_i, z_i^L) \;=\; -\Big[y_i \log \sigma(z_i^L) + \big(1-y_i\big) \log\big(1-\sigma(z_i^L)\big)\Big]. \tag{2.11}$$

The empirical objective (empirical risk over all N points) is

$$L_{\mathrm{BCE}}(\theta; D_{\mathrm{train}}) = \frac{1}{N} \sum_{i=1}^{N} \ell_{\mathrm{BCE}}(y_i, z_i^L). \tag{2.12}$$

For multi-class classification ($C \geq 2$) with $M = C$, let the final layer output a logit vector $\mathbf{z}_i^L = (z_{i,1}^L, \ldots, z_{i,C}^L)^\top \in \mathbb{R}^C$. Class probabilities are given by the softmax

$$\sigma_{\mathrm{softmax}}(\mathbf{z})_c = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}}, \qquad c = 1, \ldots, C. \tag{2.13}$$

Writing $p_{i,c} = \sigma_{\mathrm{softmax}}(\mathbf{z}_i^L)_c$ and letting $y_i \in \{1, \ldots, C\}$ be the ground-truth index, the per-example categorical cross-entropy is

$$\ell_{\mathrm{CE}}(i) \;=\; -\log p_{i,y_i}. \tag{2.14}$$

The empirical objective (empirical risk over all N points) is

$$L_{\mathrm{CE}}(\theta; D_{\mathrm{train}}) \;=\; \frac{1}{N} \sum_{i=1}^{N} \ell_{\mathrm{CE}}(i). \tag{2.15}$$

Note that both softmax and categorical cross-entropy can also be used for binary classification ($C = 2$). Instead of a single logit and sigmoid with binary cross-entropy, the network can produce two logits and apply softmax, then train with the same categorical cross-entropy typically used for $C > 2$. This can simplify code reuse or facilitate an easier transition to scenarios with more than two classes. In this thesis, we use a three-hidden-layer, ReLU-activated MLP with a softmax output, trained with categorical cross-entropy, as our main classifier in `Rasd` (§4) and `SLB` (§5).

Semi-supervised learning. Assume the training set is partitioned into a labelled subset D_{lab} and an unlabelled subset D_{unlab} with $D_{\mathrm{lab}} \uplus D_{\mathrm{unlab}} = D_{\mathrm{train}}$ and both parts non-empty. Let I_{lab} and I_{unlab} be their index sets with sizes N_{lab} and N_{unlab} ($N = N_{\mathrm{lab}} + N_{\mathrm{unlab}}$). We write the objective as a weighted sum of per-subset empirical risks, each normalised by its own cardinality:

$$L_{\mathrm{SSL}}(\theta; D_{\mathrm{train}}) \;=\; \frac{1}{N_{\mathrm{lab}}} \sum_{i \in I_{\mathrm{lab}}} \ell_{\mathrm{CE}}(i) \;+\; \lambda \, \frac{1}{N_{\mathrm{unlab}}} \sum_{i \in I_{\mathrm{unlab}}} \ell_{\mathrm{cons}}(\mathbf{x}_i; \theta), \tag{2.16}$$

where ℓ_{cons} is a consistency-style loss on unlabelled examples and $\lambda > 0$ controls the relative weight of the unlabelled term.

Two influential instantiations illustrate ℓ_{cons}. `Mean-Teacher` [256] maintains a teacher[1] whose parameters are an exponential moving average of the student's; for each unlabelled $\mathbf{x}$, the loss penalises the discrepancy between student and teacher predictions on two stochastically augmented

[1] Here, the student is the network updated by gradient descent on the training objective, while the teacher is a second network used to provide stable targets for consistency.

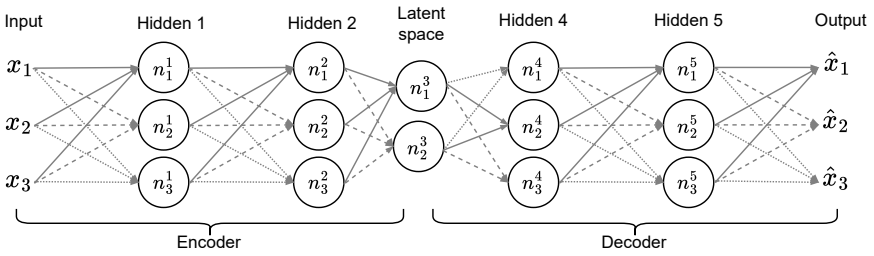


Figure 2.3: An AE (AutoEncoder): the encoder maps samples to a capacity-limited latent space (bottleneck), from which the decoder reconstructs them.

views. `FixMatch` [240] omits a separate teacher: a pseudo-label from a weakly augmented view is accepted only if its confidence exceeds a threshold, and the model is trained to match it on a strongly augmented view. Both plug into the objective above, differing only in the specific form of ℓ_{cons}, and leave the architecture unchanged.

However, both approaches rely on a carefully tuned augmentation pipeline. A weak augmentation introduces minor, label-preserving changes (*e.g.*, horizontal flip in images [240]; brief time masking in speech [196]), providing a stable reference prediction. A strong augmentation applies broader but still label-preserving transformations (*e.g.*, RandAugment [59], CutOut [66] in images; back-translation in text [227]), testing prediction stability. Across domains, an augmentation is useful only if it preserves the true label and semantics.

Unsupervised learning. When no labels are available, the training set reduces to $D_{\text{train}} = \{\mathbf{x}_i\}_{i=1}^{N}$. The ERM framework still applies, but the target is generated directly from $\mathbf{x}_i$. A well-known example is the AE (Figure 2.3), which factorises the network into an encoder $g_{\theta_{\text{enc}}}$ and a decoder $h_{\theta_{\text{dec}}}$. The composite mapping $f_\theta = h_{\theta_{\text{dec}}} \circ g_{\theta_{\text{enc}}}$ is trained to reconstruct its input using, for example, the Mean Squared Error (MSE) reconstruction loss:

$$\ell_{\text{MSE}}(\mathbf{x}_i;\theta) \;=\; \frac{1}{d}\left\|f_\theta(\mathbf{x}_i)-\mathbf{x}_i\right\|_2^2. \tag{2.17}$$

The empirical objective (empirical risk over all N points) is

$$L_{\text{MSE}}(\theta;D_{\text{train}}) \;=\; \frac{1}{N}\sum_{i=1}^{N}\ell_{\text{MSE}}(\mathbf{x}_i;\theta). \tag{2.18}$$

The encoder $g_{\theta_{\text{enc}}}$ maps each input $\mathbf{x}_i$ to a latent vector $\mathbf{z}_i$ (often via a bottleneck), while the decoder $h_{\theta_{\text{dec}}}$ reconstructs the input. Such models are frequently used for anomaly detection [177, 276, 215, 95]: if trained mainly on normal data (*e.g.*, zero-positive learning in NIDS), the per-sample reconstruction error

$$r(\mathbf{x}) \;=\; \ell_{\text{MSE}}(\mathbf{x};\theta)$$

tends to be low for in-distribution (normal) examples and higher for out-of-distribution (anomalous) ones. A threshold t can be chosen so that samples with $r(\mathbf{x}) > t$ are flagged as outliers.

In this thesis, we use an AE with a four-layer encoder (three ReLU-activated hidden layers and a

linear bottleneck) and a symmetric decoder. The model is trained with MSE and serves as a one-class anomaly detector in `Mateen` (§3). We also reuse the encoder architecture as the backbone of the shift detector in `Rasd` (§4).

2.2.3 Optimisation

Once the architecture is specified and a regime-appropriate loss function $L(\theta)$ is defined, the network learns by iteratively adjusting its parameters $\{W^l, \mathbf{b}^l\}_{l=1}^{L}$ so as to minimise the empirical risk. Conceptually, the goal is to move the parameters in the direction that reduces the loss—this direction is provided by the gradient of the loss with respect to the parameters.

In principle, one may perform full-dataset gradient descent by computing $\nabla_\theta L(\theta_t)$ over all N examples at each step. However, for computational efficiency, most deep learning methods use a mini-batch of size $B \ll N$ at each iteration. The resulting mini-batch gradient $\mathbf{g}_t = \nabla_\theta L_B(\theta_t)$ is an estimate of $\nabla_\theta L(\theta_t)$, and the Stochastic Gradient Descent (SGD) optimiser update is

$$\theta_{t+1} \;=\; \theta_t \;-\; \alpha\, \mathbf{g}_t,$$

where α is the learning rate. To update θ, we need partial derivatives like $\frac{\partial L_B}{\partial W_{ij}^l}$ or $\frac{\partial L_B}{\partial b_i^l}$. These derivatives indicate how small changes in the parameters affect $L_B(\theta)$. They are computed via the chain rule of calculus in reverse—an algorithm historically called backpropagation [219]. Modern deep learning frameworks such as PyTorch [197] and TensorFlow [1] implement backpropagation using reverse-mode automatic differentiation: they record the operations from the forward pass and then automatically derive the corresponding gradients in a backward pass.

Another optimiser is Adam [138], which is frequently used in security research [288, 85, 13, 164, 18, 295, 239, 286, 149]. At each iteration it forms exponential moving averages of the mini-batch gradient (first moment) and its elementwise square (second moment), applies a standard bias correction, and then takes an adaptive per-parameter step:

$$\theta_{t+1} \;=\; \theta_t \;-\; \alpha\, \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon},$$

where $\hat{\mathbf{m}}_t$ and $\hat{\mathbf{v}}_t$ are the bias-corrected first- and second-moment estimates computed from $\mathbf{g}_t$, and $\varepsilon > 0$ stabilises the denominator.

We adopt Adam as the default optimiser across all frameworks (§3, §4, §5).

2.2.4 Hyperparameter

Designing and training a deep neural network involves two broad categories of hyperparameters: those specifying the network's structure (architecture hyperparameters) and those controlling the learning process (training hyperparameters). Unlike the weights $\{W^l, \mathbf{b}^l\}_{l=1}^{L}$, which the model learns via gradient descent, hyperparameters must be chosen or tuned before and sometimes during experimentation.

Architecture Hyperparameters

Four key hyperparameters define a feed forward DNN's overall structure:

a) **Depth L.**
The number of hidden layers (Fig. 2.2). Increasing L typically increases the model's capacity to learn hierarchical representations but also demands more data and careful tuning.

b) **Width m_l.**
The number of neurons in each hidden layer l. This directly sets the shape of $W^l \in \mathbb{R}^{m_l \times m_{l-1}}$. Each layer can have a different width. Wider layers capture more complex functions but again it needs careful tuning and more data.

c) **Hidden Activations $\sigma_l(\cdot)$.**
The nonlinearities applied after each hidden layer (*e.g.*, ReLU, tanh, sigmoid; see Eqs. 2.6–2.8).

d) **Output Layer Design.**
The number of neurons m_L in the final layer and its activation should suit the task: for binary classification, either $m_L = 1$ with a sigmoid (single-logit) or $m_L = 2$ with a softmax (two-logit); for multi-class with $C > 2$, set $m_L = C$ with a softmax.

Training Hyperparameters

Once the network's shape is fixed, we must specify how to minimise the loss in Eq. (2.10). The following hyperparameters control the learning process:

1. Batch Size B. At each iteration, only a subset (mini-batch) of the full dataset is processed to estimate the gradient. Typical batch sizes range from $B = 8$ to $B = 128$, influenced by hardware constraints and dataset scale [286, 85, 164, 149, 60]. Selecting an appropriate batch size involves balancing computational efficiency and model generalisation. Smaller batch sizes result in more updates per epoch, which can help regularise the training, but they also increase the overall training cost [170]. In contrast, larger batch sizes may lower the cost but could lead to poorer generalisation [132].

2. Learning Rate α. This is the initial step size in gradient-based updates. Too large a learning rate can cause divergence, while too small slows convergence. Values around 0.01 to 0.00001 are common [85, 149, 18, 48, 295, 60].

3. Epochs. One epoch is a single pass over the entire training set. Training for more epochs provides additional gradient updates, which can boost accuracy but also heighten the risk of overfitting—where the model memorises training examples rather than learning patterns that generalise. In such cases, the model may achieve very high accuracy on the training set yet perform poorly on the test set.

4. Optional Regularisation. Regularisation hyperparameters help reduce overfitting. Common strategies include:

Table 2.2: Example confusion matrix for binary intrusion detection (attack = positive, benign = negative). Rows are ground truth; columns are predictions.

	Predicted label	
True label	Attack (+)	Benign (-)
Attack (+)	TP = 80	FN = 20
Benign (-)	FP = 10	TN = 90

- **Weight Decay (L2 regularisation):** Adds a penalty term $\lambda_{\text{wd}}\|\theta\|_2^2$ to the loss, shrinking weights towards zero. The scalar $\lambda_{\text{wd}} > 0$ is tuned empirically.
- **Dropout** [246]: Randomly zeroes out a fraction $p \in (0,1)$ of the activations within a layer at training time. Larger p implies stronger regularisation.
- **Early Stopping:** Monitors validation loss during training and stops when improvement stalls, preventing over-training on the training data. The patience (number of epochs to wait before stopping) is a hyperparameter here.

5. Task-Specific Hyperparameters. Certain training objectives introduce additional hyperparameters. For example:

- **FixMatch:** Requires a confidence threshold for accepting pseudo-labels, a coefficient λ for the consistency loss, and definitions of the weak and strong augmentations.

2.2.5 Evaluation

Having trained a model on the dataset D_{train} defined in Eq. (2.9), we must assess how well it generalises to unseen data. Typically, the full dataset D is split into at least two non-overlapping parts: D_{train} (for learning model parameters) and D_{test} (for final evaluation). Often, an additional D_{val} is set aside to tune hyperparameters without polluting the test set.

Data Splits. There are three standard protocols to create these splits [208]:

Hold-out validation. The simplest approach is to randomly partition D into D_{train}, D_{val}, and D_{test}. For example, 60% of the data for training, 20% for validation (to tune hyperparameters or decide early stopping), and 20% for final testing. This method is fast but can yield performance estimates that vary depending on the split. If hyperparameter tuning and early stopping are not required, one can omit D_{val} and just split into train/test.

k-fold cross-validation. Partition D into k roughly equal folds. For each fold j (where $j = 1,\ldots,k$), train on the other $k-1$ folds and test on fold j. The final performance is the average across all k folds. This approach provides a more stable estimate than a single hold-out but is more computationally expensive.

Leave-one-out cross-validation. This is the extreme case of $k = N$, in which each sample is held out once. While this yields the most exhaustive estimate, it is too expensive for large datasets and deep neural networks.

Evaluation metrics. Once the model is trained on D_{train}, it is evaluated on the held-out test set D_{test}. The choice of metrics depends on whether the task is binary ($C = 2$) or multi-class ($C > 2$).

In a binary setting, one class is labelled positive and the other negative. A confusion matrix summarises the outcomes in a 2×2 table by counting predictions against the ground truth (true labels versus predicted labels). Table 2.2 shows an example. The confusion matrix consists of four counts:

$$\text{TP} = \text{true positives}, \quad \text{TN} = \text{true negatives}, \quad \text{FP} = \text{false positives}, \quad \text{FN} = \text{false negatives}.$$

For instance, attack can serve as the positive class, while benign is negative:

- TP: Number of attacks that are correctly detected (predicted attack).
- TN: Number of benign samples correctly classified as benign.
- FP: Number of benign samples incorrectly classified as attack (false alarms).
- FN: Number of attacks missed by the system, predicted as benign.

From these four values, we derive common metrics:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{2.19}$$

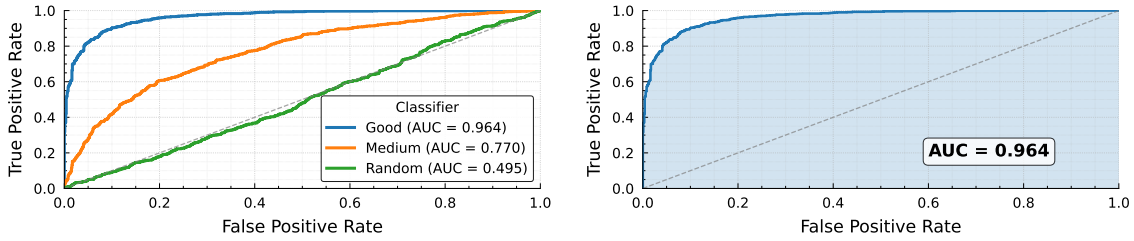
$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{2.20}$$

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \tag{2.21}$$

$$\text{F1} = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2.22}$$

- **Precision:** Of all samples predicted as attacks, what fraction are truly attacks? A high value means few false alarms.
- **Recall:** Of all actual attacks, what fraction are detected? A high recall means fewer missed attacks.
- **F1 Score:** The harmonic mean of precision and recall, commonly used to balance these two aspects.
- **Accuracy:** The fraction of correctly classified samples overall.

Unlike the metrics above, which evaluate a classifier at a single decision threshold, the Receiver Operating Characteristic (ROC) curve shows performance at all thresholds. For every threshold t applied to the model score $s(\mathbf{x})$ (probability, reconstruction error, ...), we calculate the True



(a) ROC curve

(b) AUC-ROC

Figure 2.4: ROC (Receiver Operating Characteristic) and its area.

Positive Rate (TPR) and False Positive Rate (FPR) as follows:

$$\mathrm{TPR}(t) = \frac{\mathrm{TP}(t)}{\mathrm{TP}(t) + \mathrm{FN}(t)}, \qquad \mathrm{FPR}(t) = \frac{\mathrm{FP}(t)}{\mathrm{FP}(t) + \mathrm{TN}(t)}.$$

Varying t continuously from $+\infty$ (no positives predicted) to $-\infty$ (all samples predicted positive) traces a parametric curve in the unit square (Figure 2.4a). The Area Under this Curve (AUC-ROC) condenses the full trajectory into a single scalar,

$$\mathrm{AUC} = \int_0^1 \mathrm{TPR}(x)\,dx, \qquad x \equiv \mathrm{FPR},$$

which equals the probability that a randomly chosen positive sample receives a higher score than a randomly chosen negative one [64, 97]. Thus, AUC$= 0.5$ corresponds to chance-level discrimination, while AUC$= 1.0$ indicates perfect separation (Figure 2.4b).

When $C > 2$, the confusion matrix extends to a $C \times C$ table. One can still define $\mathrm{TP}, \mathrm{FP}, \mathrm{TN}, \mathrm{FN}$ for each class by treating that class as positive and all others as negative, then averaging the metrics:

- **Macro averaging:** Compute precision, recall, and F1 for each class, then average. Each class is treated equally, which can highlight poor performance on minority classes.
- **Micro averaging:** Aggregate $\mathrm{TP}, \mathrm{FP}, \mathrm{FN}$ across all classes first, then compute precision, recall, or F1. This treats each sample equally, which can overshadow smaller classes if they are outnumbered by larger ones.

A multi-class extension of ROC analysis is more complex, typically done through one-vs.-all or one-vs.-one schemes. In this thesis, we report macro-averaged F1-scores, along with accuracy, as our primary metrics. We use the terms macro F1-score, mF1-score, and mF1 interchangeably, all referring to macro-averaged F1-scores, and we use Acc interchangeably with accuracy. Additional metrics are included when appropriate—for example, we report the AUC-ROC when evaluating `Mateen`, our adaptive anomaly detection system (§3).

2.3 NIDS Datasets

Researchers evaluating learning-based NIDS depend on public datasets that mix benign and malicious traffic, allowing them to measure detection and classification performance across diverse

attacks. For shift-aware DNN-based studies, however, a dataset is suitable only if it is (i) well established and documented, (ii) recorded as a continuous timeline so that temporal correlations and distribution shift remain intact, (iii) built on genuine user or device traffic rather than traffic generators, (iv) commonly adopted in the shift-aware NIDS literature as a benchmark dataset, and (v) rich enough to cover several attack families so multi-class evaluation is possible.

Among the eight prominent datasets reviewed (DARPA 1998, NSL-KDD, CTU-13, UNSW-NB15, CICIDS2017, CSE-CIC-IDS2018, Kitsune, and CIC-DDoS2019), only three satisfy at least four of the above criteria: CICIDS2017, CSE-CIC-IDS2018, and Kitsune.

The two CIC datasets provide one- to three-week laboratory recordings in which benign traffic generated by profiling tools is interleaved with a broad suite of scripted attacks, providing controlled, multiclass testbeds used throughout this thesis. Both datasets are also commonly used in shift-aware NIDS evaluation (*e.g.,* CICIDS2017 in `INSOMNIA` [18] and CSE-CIC-IDS2018 in `CADE` [295]). Kitsune was captured on a live network, pairing authentic background traffic with nine injected attacks; because it targets one-class anomaly detection, it is used only in the `Mateen` chapter (§3). Thus, while these three datasets collectively supply time-aware, multi-attack benchmarks that underlie the thesis, each chapter draws on the subset that best matches its methodological focus.

2.3.1 CICIDS2017

The CICIDS2017 (or CIC-IDS-2017) dataset [230], developed by the Canadian Institute for Cybersecurity (CIC), was collected in a controlled lab environment designed to simulate small-enterprise network activity. The testbed consists of two isolated networks: a victim network with 10 workstations, two servers, and supporting infrastructure, and an attacker network with four workstations. Traffic was captured over five consecutive days: the first day contains only benign activity, while each subsequent day introduces a distinct set of attack scenarios. In total, the dataset includes 14 different attack types.

Benign background traffic was generated using a profiling tool [231] that emulates human behavior across standard protocols such as HTTP, HTTPS, FTP, SSH, and email. The tool leverages 25 user profiles to create diverse traffic patterns.

The dataset is publicly available and includes both raw packet captures and preprocessed network flow records. In this dataset, a data record refers to one bidirectional flow summary produced by CICFlowMeter [145], which aggregates packets sharing the same 5-tuple (source IP, destination IP, source port, destination port, protocol). CICFlowMeter emits a record when it closes the flow; in the original CICIDS2017 processing pipeline, closure is governed by an inactivity timeout (120 *seconds*) and, for TCP, is triggered upon observing the first FIN-carrying packet [77]. For each flow, 83 statistical features are computed.

Labels are assigned at the flow level by correlating each flow's timestamp and 5-tuple (IPs, ports, and protocol) with the published attack execution schedule and the known attacker/victim hosts [230]. As a result, CICIDS2017 is engineered around pre-coordinated attack executions conducted in dedicated time windows [230], with the intent of ensuring no scenario-level overlap between

Table 2.3: Distribution of traffic classes in the enhanced CICIDS2017 dataset over the five-day capture period (Day 1 = Monday . . . Day 5 = Friday).

Class	Day	Count	Percentage (%)
Benign	1–5	1,666,837	79.3424
FTP-Patator	2	3,973	0.1891
SSH-Patator	2	2,980	0.1418
DoS Hulk	3	158,469	7.5432
DoS GoldenEye	3	7,567	0.3602
DoS slowloris	3	4,001	0.1904
DoS SlowHTTPTest	3	1,742	0.0829
Heartbleed	3	11	0.0005
Web Brute Force	4	151	0.0072
Infiltration	4	32	0.0015
XSS	4	27	0.0013
SQL Injection	4	12	0.0006
PortScan	5	159,151	7.5757
DDoS	5	95,123	4.5279
Bot	5	738	0.0351
Total		2,100,814	100%

attack categories (while benign traffic continues throughout). Put differently, attacks are executed in separate time periods rather than concurrently.

In this thesis, we use an enhanced version of the dataset [77], which we refer to as IDS17, that addresses known issues in CICFlowMeter's flow construction and labelling. For example, some flows labelled as payload-based attacks (*e.g.*, Cross-Site Scripting (XSS)) contained no packet data. Additionally, the original tool prematurely terminated flows upon encountering the first FIN flag, omitting subsequent packets and resulting in fragmented flows. These fragments, containing only trailing ACK and FIN packets, were assigned the same label as the original flow. The enhanced dataset contains 2,100,814 flows, with the distribution detailed in Table 2.3.

2.3.2 CICIDS2018

The CICIDS2018 (or CSE-CIC-IDS2018) dataset is an extended version of CICIDS2017, jointly developed by the CIC and the CSE of Canada. As with its predecessor, it was generated in a controlled lab environment, but features a significantly larger and more complex network architecture, as well as a longer capture period. The testbed includes approximately 420 workstations and 30 servers distributed across five simulated organisational departments, in addition to 50 machines dedicated to conducting attacks. Traffic was captured over 10 non-consecutive days spanning a three-week period. Each day contains both benign traffic and attack activity. In contrast to CICIDS2017, several attacks in CICIDS2018 span multiple days.

As with CICIDS2017, CICIDS2018 flow records are bidirectional 5-tuple flow summaries produced by CICFlowMeter and exported when the tool closes a flow. Flow labels are assigned by matching each flow's timestamp and endpoints to the published attack schedule and known attacker/victim hosts, with attacks executed in scheduled windows to ensure no scenario-level overlap. In this thesis, we use the original release for `Rasd`, and for `Mateen` we adopt the corrected

Table 2.4: Distribution of traffic classes in the enhanced CICIDS2018 dataset over the three-week capture period.

Class	Week	Count	Percentage (%)
Benign	1–3	59,659,723	94.4055
SSH Patator	1	94,197	0.1491
DoS GoldenEye	1	22,560	0.0357
DoS Slowloris	1	8,490	0.0134
DoS Hulk	1	1,803,160	2.8533
DDoS LOIC HTTP	2	289,328	0.4578
DDoS LOIC UDP	2	2,527	0.0040
DDoS HOIC	2	1,082,293	1.7126
Web Brute Force	2	131	0.0002
XSS	2	113	0.0002
SQL Injection	2	39	0.0001
Infiltration	3	289	0.0005
Portscan	3	89,374	0.1414
Bot	3	142,921	0.2262
Total		63,195,145	100%

variant by Liu *et al.* [156], which we denote IDS18. IDS18 addresses these issues and contains 63,195,145 flows; their distribution is shown in Table 2.4.

2.3.3 Kitsune

The Kitsune dataset [177] was collected in a semi-realistic environment rather than a tightly controlled laboratory. Benign traffic was captured on two live networks: (i) an IP video surveillance network with eight HD PoE cameras and a DVR, and (ii) a heterogeneous IoT network hosting about 12 machines such as thermostats, baby monitors, webcams, and desktop PCs.

Against this authentic background, the authors generated nine attack scenarios, including ARP man in the middle, denial of service floods, and a Mirai botnet infection. Each scenario was logged into a separate file that interleaves the malicious activity with its benign background traffic. Every record in these files corresponds to a single packet and consists of packet-level statistics computed on the fly with exponentially-damped sliding windows, yielding a 115-dimensional feature vector per packet. Labels are provided at the packet level using the known attack injection interval for each scenario, so packets observed during the injected attack activity are marked as malicious while the surrounding background packets remain benign. Since each file contains only one injected attack scenario, different attack types do not overlap across files. Table 2.5 summarises, for each file, the distribution of benign and attack classes.

2.3.4 Dataset Limitations

The datasets used in this work are widely adopted, but they have limitations that affect generalisability, especially when drawing conclusions about distribution shift.

First, they are collected in controlled or semi-controlled settings. CICIDS2017 and CSE-CIC-IDS2018 are generated in a testbed with scripted user profiles and injected attacks, while Kitsune

Table 2.5: Distribution of benign and malicious records per scenario file in the Kitsune dataset.

File	Class	Count	Percentage (%)
ARP MitM	Benign	1,358,995	54.2672
	Malicious	1,145,272	45.7328
Active Wiretap	Benign	1,355,473	59.4848
	Malicious	923,216	40.5152
Fuzzing	Benign	1,811,356	80.7149
	Malicious	432,783	19.2850
OS Scan	Benign	1,632,151	96.1304
	Malicious	65,700	3.8696
Mirai Bot	Benign	121,620	15.9160
	Malicious	642,516	84.0840
Video Injection	Benign	2,369,902	95.8543
	Malicious	102,499	4.1457
SYN DoS	Benign	2,764,238	99.7460
	Malicious	7,038	0.2540
SSDP Flood	Benign	2,637,662	64.6919
	Malicious	1,439,604	35.3081
SSL renegotiation	Benign	2,114,919	95.8029
	Malicious	92,652	4.1970
	Benign Total	16,166,316	
	Malicious Total	4,851,280	
	Total	21,017,596	

includes benign traffic captured from a real IoT/surveillance deployment but still relies on injected attack executions. These settings can under-represent the heterogeneity of operational networks, which may reduce how directly results transfer to deployment.

Second, intra-class diversity can be limited for both attacks and benign activity. Attacks are often generated using a small number of tools and fixed parameterisations, so models may partially rely on tool- or configuration-specific artefacts rather than the underlying attack semantics [18, 103]. Similarly, benign traffic in the testbed-style datasets is driven by a limited set of scripted behaviours, which can narrow the range of benign variability represented during training and evaluation.

Third, the capture durations are relatively short (five working days for CICIDS2017, roughly three weeks for CSE-CIC-IDS2018, and short scenario-based traces for Kitsune). These horizons restrict long-term shift and seasonal effects, and can create sharp change points (benign to injected attack) that are less representative of gradual evolution in practice.

Fourth, protocol and service coverage is limited. The CIC datasets emphasise a small set of common services, while Kitsune is domain-specific (IoT/surveillance) and dominated by device and streaming traffic. As a result, shifts in protocol mix that occur in practice, such as changes in deployed services or increased use of a specific application, are only weakly represented, which constrains generalisation to networks with different service profiles.

Overall, these limitations indicate that the results of this thesis provide strong evidence of effectiveness on widely used public datasets, which offer a standard and reproducible basis for evaluating shift-aware NIDS methods. However, operational networks vary in traffic composition, protocol

mix, host populations, and policies, so performance may differ across deployments and should be validated in the intended environment. Developing broader and more operationally representative public datasets remains an active research area [22]. Recent efforts have moved toward captures from operational networks, such as university-campus settings combined with malware activity [21], but comprehensive and longer-term benchmarks are still needed.

2.4 Distribution Shift

Distribution shift occurs when the data distribution observed during model deployment differs from the distribution experienced during training. Over time, such discrepancies can significantly degrade model performance, a phenomenon often termed time decay or model aging [124]. Although the problem of model aging has been extensively studied across various domains, including image recognition [249] and natural language processing [91], it has only recently gained attention within NIDS.

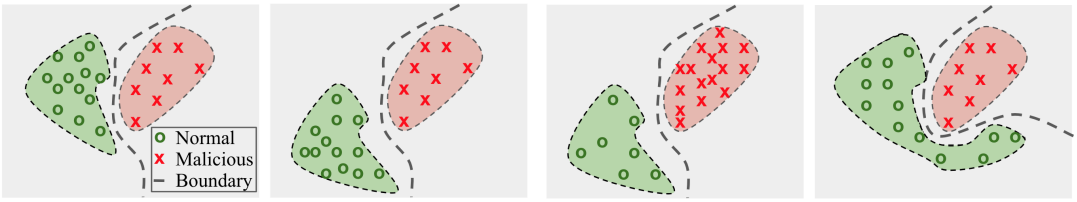
This section provides an overview of distribution shift as it pertains to NIDS. Initially, we present a general definition of distribution shift in learning-based systems (§2.4.1). Subsequently, we adapt this general definition specifically to the context of security applications that employ machine learning (§2.4.2). We then outline two prevalent strategies used to mitigate the impact of distribution shift (§2.4.3). Following that, we offer a detailed survey and taxonomy of existing techniques aimed at mitigating distribution shift in DNN-based NIDS (§2.4.4—2.4.8). Finally, we conclude the section by identifying the research gaps that this thesis aims to address (§ 2.4.9).

2.4.1 Definition and Types of Shift

Definition. Distribution shift, also known as dataset shift, occurs when the joint distribution $P(x,y)$ encountered at test time differs from the one observed during training [181, 88, 159]. Concretely, suppose we train a parametric model f_θ on a dataset D_{train}, whose samples are drawn i.i.d. (independent and identically distributed) from some underlying training distribution $P_{\text{train}}(x,y)$. We then deploy the learned parameters θ to classify new samples drawn from a test distribution $P_{\text{test}}(x,y)$. In classical learning theory, it is standard to assume stationarity, meaning that $P_{\text{train}}(x,y) = P_{\text{test}}(x,y)$. In other words, it presumes that both the training and testing data arise from the same underlying distribution. Distribution shift specifically refers to the violation of this assumption, formally expressed as $P_{\text{train}}(x,y) \neq P_{\text{test}}(x,y)$.

The stationary assumption is common in NIDS research, either explicitly or implicitly. For example, several works [78, 253, 12] apply k-fold cross-validation to the CICIDS2017 dataset, effectively ensuring that both the training set D_{train} and the test set D_{test} are drawn from the same distribution. Under this assumption, reported performance metrics frequently exceed 99%.

However, real-world networks naturally exhibit distribution shift as both malicious and benign behaviors evolve. To illustrate its impact, Andresini *et al.* [18] employed a time-aware split of the CICIDS2017 dataset by training on the first two days of data and testing on subsequent days. This



(a) Train (b) Covariate (c) Ratio (d) Concept

Figure 2.5: Distribution shift examples: (a) Training Distribution - data on which the model was initially trained. (b) Covariate Shift - benign data distribution changes, but the decision boundary remains the same. (c) Ratio Shift - reduction in benign data and increase in malicious data. (d) Concept Shift - a new concept that changes the decision boundary.

simple alteration led to a severe drop in performance, with the F1-score approaching zero.[2]

Types of shift. Distribution shift can be characterised by which component of the joint distribution $P(x, y)$ changes between training and deployment [181] (see Figure 2.5).

Covariate Shift, sometimes referred to as feature shift, feature-space concept drift, or virtual concept drift, arises when $P(x)$ changes over time but $P(y \mid x)$ remains essentially the same. For instance, in a NIDS trained on packet payloads, attackers might alter those payloads (*e.g.*, by using different syntax or encoding in SQL injection attacks) to achieve the same malicious goal. If these modifications do not change the underlying decision boundary, they create a distribution shift without affecting $P(y \mid x)$, as illustrated in Figure 2.5b. Formally:

$$P_{\text{train}}(x) \neq P_{\text{test}}(x) \quad \text{while} \quad P_{\text{train}}(y \mid x) = P_{\text{test}}(y \mid x).$$

Because $P(y \mid x)$ remains unchanged, covariate shift typically has limited direct impact on a model's performance [50]. Nonetheless, covariate shift can indirectly affect system performance, often by reducing the model's confidence in its otherwise correct predictions. This loss of confidence has various practical implications, such as:

- *Higher Uncertainty or Missed Detections*: When decisions rely on a probability threshold (*e.g.*, flagging a network packet as malicious only if confidence exceeds 0.9), reduced confidence may lead to more false negatives, allowing attacks or unwanted behavior to remain undetected.
- *Increased Escalations or Workload*: Uncertain cases may be escalated for manual review or secondary checks. Lower confidence on otherwise accurate predictions can unnecessarily increase the workload for security analysts.
- *Potential Loss of Trust*: When operators rely on confidence scores to interpret outputs, persistently low confidence can diminish their trust in the system, even if the predictions remain accurate.

[2] When a model produces no true positives, recall is zero, causing the F1-score to collapse.

- *Reduced Automation*: If a high-confidence prediction is required to trigger automated actions (*e.g.*, blocking suspicious traffic), lower confidence could cause the system to revert to manual intervention, hindering operational efficiency.

Ratio Shift, also called prior probability or label shift, represents the converse scenario in which the distribution of labels $P(y)$ changes but the conditional distribution $P(x \mid y)$ remains stable. In a network context, this occurs if the proportion of malicious traffic increases or decreases substantially at deployment (see Figure 2.5c). Formally:

$$P_{\text{train}}(y) \neq P_{\text{test}}(y) \quad \text{while} \quad P_{\text{train}}(x \mid y) = P_{\text{test}}(x \mid y).$$

A sudden surge in DoS attacks exemplifies this scenario, as the ratio of malicious to benign flows shifts significantly.

Concept Shift, sometimes referred to as *semantic shift* or *concept drift*, arises when the underlying relationship between features and labels $P(y \mid x)$ changes over time, effectively altering the decision boundary (see Figure 2.5d). Formally, this shift can be expressed as

$$P_{\text{train}}(x, y) \neq P_{\text{test}}(x, y)$$

in a manner that does not preserve $P(y \mid x)$. Under these conditions, the model may encounter instances whose feature-label relationship it no longer recognises, resulting in incorrect predictions. We will elaborate on this type of shift in the following section.

Temporal Patterns. These distribution shift types can be categorised according to their temporal characteristics as gradual, sudden, incremental, or recurrent [88]:

- *Sudden Shift:* This occurs when the distribution changes abruptly, often over a very short period. For instance, a NIDS accustomed to normal traffic may suddenly observe a surge of malicious packets due to a newly discovered exploit. The old distribution is immediately replaced by a new, distinct one.

- *Gradual Shift:* In this scenario, old and new distributions overlap for a period before the new distribution eventually becomes dominant. An example might be an organisation where employees begin using a new service intermittently, causing the traffic profile to blend old and new patterns until the new service takes hold.

- *Incremental Shift:* Here, the distribution evolves slowly and subtly over a longer time frame. Small, continuous changes in user behavior, business operations, or attack techniques accumulate, causing the old distribution to transition almost imperceptibly into a new one.

- *Recurrent Shift:* A previously observed distribution reappears after some interval, creating a cyclical or periodic pattern. Weekends and holidays, for example, may produce distinctive network traffic volumes and types that recur regularly, returning the system to a distribution it has seen before.

2.4.2 Concept Shift in Security

Among the various forms of distribution shift, *concept shift* is particularly challenging for security systems. It fundamentally changes the decision boundary, invalidating assumptions established during model training and thereby leading to model aging. As a result, concept shift has recently attracted significant attention in security domains such as network intrusion detection [295, 18], malware classification [124, 34], and anomaly detection [276, 107].

Depending on the classification setting (*e.g.*, one-class, binary, or multi-class), concept shift can be categorised into three subtypes: *normality shift* [107], *in-class evolution* [18], and *introduction of a new class* [295].

Normality shift, also referred to as *benign shift*, arises in one-class anomaly detection systems (commonly used in zero-positive learning), which are trained solely on benign traffic. These models are inherently robust to variations in malicious behavior, as they are designed to detect deviations from the benign class. However, when the distribution of benign traffic itself evolves over time (*e.g.*, due to changes in user behavior, software updates, or protocol changes), these models may increasingly misclassify benign samples as malicious, leading to elevated false positive rates.

In-class evolution occurs in both binary and multi-class settings, where the semantics of an existing class gradually change. For instance, an attacker might modify their techniques to evade detection while still pursuing the same objective. This causes the observed distribution of that class to shift away from what the model originally learned, potentially shifting decision boundaries and degrading classifier performance. Critically, this phenomenon does not require introducing a new class label; rather, it reflects a refinement or evolution of the same underlying concept.

Introduction of a new class, also referred to as *novel class*, *unknown class*, or the $C+1$ problem, is specific to multi-class settings. In this scenario, the model encounters traffic from a class it has never seen during training (*i.e.*, one that falls outside the original set of training classes). For example, a model trained to recognise Benign, DDoS, Port Scan, and Exfiltration traffic will inevitably misclassify a novel SQL Injection sample as one of the known classes, potentially causing critical false negatives or misattributions.

While it may appear that such new class scenarios are limited to multi-class classifiers, binary classifiers can also face the same challenges. For example, consider a binary model trained on benign versus malicious traffic, where the malicious class comprises only DDoS, Port Scan, and Exfiltration samples. If SQL Injection traffic appears at inference, some may argue that this is the introduction of a new class under a binary framework. However, our view is that this can be treated as an *in-class evolution* scenario because, within a binary benign/malicious setting, different attack types are subtypes of the single malicious class rather than separate classes. Moreover, a well-trained binary model may successfully generalise to other malicious behaviors, even previously unseen ones, if they share enough features with known malicious traffic. Thus, the model might still correctly classify SQL Injection as malicious, indicating it does not necessarily constitute a wholly new class from the model's perspective.

2.4.3 Shift Mitigation Strategies in DNN-based NIDS

To address concept shift and its subtypes, the literature broadly follows two complementary directions: (i) robust training, and (ii) detection-and-adaptation pipelines.

The first direction, sometimes referred to as robust learning, focuses on constructing architectures, training regimes, and feature representations that are inherently resistant to distributional changes. This includes efforts to build a feature space that remains representative under class evolution, as well as curating training data that captures the expected variability within a class. In particular, robustness to in-class evolution can be enhanced by selecting semantically grounded features that generalise across behavioral variants [238]. Furthermore, some approaches proactively expand the range of possible class variations, often through techniques such as data augmentation [104], to capture a broader spectrum of potential shifts and strengthen resilience against emerging threats.

The second line of work centers on explicit detection and adaptation pipelines. These systems typically follow a staged process: first, the model monitors for potential distributional shifts. Once a shift is detected, the system identifies or selects a subset of samples from the new (test) distribution. These samples may then be labelled manually or automatically, and are used to update the model. While some frameworks integrate the full pipeline from detection to adaptation, others focus on individual stages, such as shift detection [293], or adaptation [15].

Relation to Detection Frameworks. Within this broader context of handling concept shift, several lines of recent research describe shift detection mechanisms under three well-known frameworks, each defined by distinct goals and assumptions [294]:

Anomaly Detection (AD). Here, all observed training data are treated as one normal class. Any significant deviation from this normal profile is tagged as an anomaly. As a result, anomaly detection naturally indicates distributional changes, whether they arise from gradual shifts in normal behavior or from new (previously unseen) attack classes ($C+1$ classes). Because AD generally operates in a binary (normal vs. anomalous) fashion, it often serves as a standalone shift detector, separate from a multi-class classifier.

Novelty Detection (ND). This framework separately models each class from the training set (*e.g.*, multiple known attack types and benign traffic). At inference, ND checks whether an incoming sample fits any existing class model or constitutes a novel category. In NIDS, ND is particularly relevant for identifying new attack types (*i.e.*, $C+1$ classes) that were absent in the original training data. ND can function as a standalone shift detector or be integrated into a classifier. For instance, `CADE` [295] is a specialised novel class detector designed to operate alongside a classification model.

Open Set Recognition (OSR). Aims to simultaneously classify known classes accurately and identify unknown (new) classes. This dual objective ensures both high accuracy on known classes and the ability to detect samples that lie outside the known set. For instance, `CVAE-EVT` [293] shows how OSR can be adapted to intrusion detection by jointly optimising for correct classification and new class identification.

Table 2.6: A summary of recent techniques for mitigating concept shift in DNN-based NIDS.

Paper Info			Problem Setup		Method Modules				
ID	**Paper**	**Year**	**Setting**	**Shift Type**	**Detect**	**Select**	**Label**	**Adapt**	**Robust**
1	CADE [295]	2021	Multi-Class	Novel Class	✓	✓	✓	✗	✗
2	INSOMNIA [18]	2021	Binary	In-Class Evolution	✗	✓	✓	✓	✗
3	Str-MINDFUL [17]	2021	Binary	In-Class Evolution	✓	✗	✓	✓	✗
4	CVAE-EVT [293]	2021	Multi-Class	Novel Class	✓	✗	✗	✗	✗
5	Kuppa and Le-Khac [142]	2022	Multi-Class	Novel Class	✓	✗	✓	✓	✗
6	ENIDrift [276]	2022	Zero-Positive	Normality Shift	✗	✗	✓	✓	✗
7	Pawlicki *et al.* [198]	2022	Binary	In-Class Evolution	✗	✗	✗	✓	✗
8	OWAD [107]	2023	Zero-Positive	Normality Shift	✓	✓	✓	✓	✗
9	Amalapuram *et al.* [14]	2023	Binary	In-Class Evolution	✗	✗	✗	✓	✗
10	DOC++ [241]	2023	Multi-Class	Novel Class	✓	✗	✓	✓	✗
11	OpenCADP [201]	2023	Multi-Class	Novel Class	✓	✗	✗	✗	✗
12	OIDMD [158]	2023	Multi-Class	Novel Class	✓	✗	✗	✗	✗
13	Corsini and Yang [55]	2023	Multi-Class	Novel Class	✓	✗	✗	✗	✗
14	Rasd [11]	2024	Multi-Class	Novel Class	✓	✓	✓	✓	✗
15	Mateen [10]	2024	Zero-Positive	Normality	✓	✓	✓	✓	✗
16	ReCDA [297]	2024	Binary	In-Class Evolution	✗	✗	✗	✗	✓
17	SPIDER [15]	2024	Binary	In-Class Evolution	✗	✓	✓	✓	✗
18	NSFDA [194]	2024	Zero-Positive	Normality Shift	✓	✓	✓	✓	✗
19	ICE-CP [162]	2024	Multi-Class	Novel Class	✓	✗	✗	✗	✗
20	Trident [313]	2024	Multi-Class	Novel Class	✓	✗	✓	✓	✗
21	RFG-HELAD [315]	2024	Multi-Class	Novel Class	✓	✗	✓	✓	✗
22	Agate *et al.* [3]	2024	Anomaly Detection	Novel Class	✓	✗	✗	✓	✗
23	AIS-NIDS [79]	2024	Multi-Class	Novel Class	✓	✗	✓	✓	✗
24	SwapCon [275]	2024	Binary	In-Class Evolution	✗	✗	✗	✗	✓
25	BFS-NID [73]	2024	Multi-Class	Novel Class	✗	✗	✓	✓	✗
26	ECNet [110]	2024	Binary	In-Class Evolution	✓	✗	✗	✗	✗
27	VAEMax [203]	2024	Multi-Class	Novel Class	✓	✗	✗	✗	✗
28	Gaudreault and Branco [93]	2024	Multi-Class	Novel Class	✓	✗	✗	✓	✗
29	Deep ResNIDS [118]	2024	Binary	In-Class Evolution	✓	✗	✓	✓	✗
30	CAEAID [300]	2025	Binary	In-Class Evolution	✓	✗	✓	✓	✗
31	NetGuard [104]	2025	Multi-Class	In-Class & Novel	✓	✓	✓	✓	✓
32	IFDA-GPC [238]	2025	Binary	In-Class Evolution	✓	✗	✓	✓	✓
33	SSF [312]	2025	Binary	In-Class Evolution	✓	✓	✓	✓	✗
34	CDDA-MD [43]	2025	Binary	In-Class Evolution	✓	✗	✓	✓	✗
35	OC-MAL [173]	2025	One-Class	Novel Class	✓	✗	✗	✗	✗
36	CND-IDS [86]	2025	Binary	In-Class Evolution	✓	✗	✓	✓	✗
37	HCRP-OSD [42]	2025	Multi-Class	Novel Class	✓	✗	✗	✗	✗
38	Uddin *et al.* [265]	2025	Multi-Class	Novel Class	✓	✓	✓	✓	✗
39	nNFST [188]	2025	Multi-Class	Novel Class	✓	✗	✗	✗	✗

2.4.4 Review of Current Research

Several recent studies have focused on mitigating concept shift in DNN-based NIDS. Table 2.6 provides an overview of 39 representative works, categorising each according to five core methodological components: shift detection (Detect), sample selection (Select), sample labelling (Label), shift adaptation (Adapt), and robust learning (Robust). A check mark (✓) denotes the presence of a given component, while a cross (✗) signifies its absence. Notably, the majority of these works have been published after 2023, reflecting a growing research focus on enhancing the adaptability of DNN-based NIDS to evolving network environments. The gray-highlighted rows correspond to `Rasd` and `Mateen`, which constitute two key contributions of this thesis.

This review does not provide a quantitative comparison of prior work because reported performance is highly sensitive to experimental choices that differ across studies. These choices include dataset versions and preprocessing, the train-test splitting protocol, the temporal ordering of attacks, and the evaluation metrics. As a result, headline numbers are often not directly comparable and may reflect the evaluation setup as much as the underlying method. We therefore emphasise methodological structure and stated assumptions.

A further motivation for this framing is that some splitting protocols can yield overly optimistic conclusions (*e.g.*, [295, 293, 142]). In particular, splits that do not reflect deployment constraints, such as those that do not enforce strict temporal ordering between training and test traffic, can inflate reported performance [24]. For example, `CADE` [295] uses benign traffic and only three attack classes from CICIDS2018, applies a random 80:20 split, and simulates shift by withholding one entire attack class from training and introducing it only at test time. Under this protocol, most of the test set remains in-distribution due to the random split, while the shift is isolated to a single unseen class. This concentration can make shift detection easier and can also simplify downstream stages, because sample selection can focus on a clearly single novel class and adaptation can be driven primarily by that class, rather than by continuous temporal changes affecting benign traffic and multiple attacks.

After analysing these studies, we derived a taxonomy of the techniques employed across the five dimensions. Figure 2.6 offers a high-level overview of this taxonomy, and the following sections discuss each dimension in detail: Detection (§2.4.5); Selection & Labelling (§2.4.6); Adaptation (§2.4.7); and Robust Learning (§2.4.8).

2.4.5 Shift Detection

Existing shift detection techniques can be roughly grouped into two operational regimes. Some methods examine each incoming sample in isolation, deciding on the fly whether it belongs to a shifted distribution (sample-by-sample detection); others accumulate a window of recent traffic and look for aggregate changes in its statistical profile (distribution-based detection).

Regardless of the regime, most approaches draw evidence from five recurrent signal families: (i) online performance, (ii) probabilistic confidence, (iii) reconstruction error, (iv) distance-based measures, and (v) hybrid techniques that combines two or more of these signals.

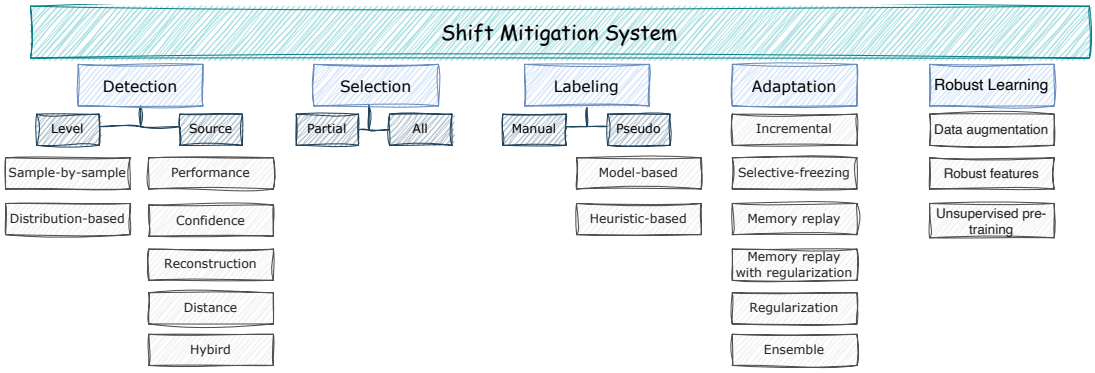


Figure 2.6: Taxonomy of currently employed techniques for mitigating concept shift in DNN-based NIDS.

Online Performance. When real-time ground-truth labels are available, monitoring predictive performance is the most direct way to identify shift samples. `IFDA-GPC` [238] and `CDDA-MD` [43] track the rate of misclassification in consecutive windows of test data and signal a change if the rate remains elevated.

Confidence. Because real-time ground-truth labels are often unavailable, many methods use model confidence as a proxy. In particular, they track each sample's predicted probability distribution across the known (training) classes, using these probabilities to gauge the model's familiarity with the input. A common approach measures the maximum softmax probability over the C known classes, $s_{\text{conf}}(\mathbf{x}) := \max_c p_c(\mathbf{x})$. If the model confidence $s_{\text{conf}}(\mathbf{x})$ is lower than a threshold t, the input $\mathbf{x}$ is flagged as a shift sample.

`ECNet` [110] assigns each input both a predicted class and a confidence score, treating all low-confidence samples as shifts. `DOC++` [241] employs a one-vs-rest formulation with per-class thresholds: the model produces one sigmoid output per known class and applies an independent threshold to each. If all class-specific outputs fall below their thresholds, the sample is classified as a shift sample. Gaudreault and Branco [93] train one binary classifier per known class and flag shift samples if all classifiers' probabilities fall below their respective thresholds. `SSF` [312], on the other hand, applies a Kolmogorov–Smirnov (KS) test to the distribution of predicted probabilities, and concludes that a shift has occurred if the distribution of newly arrived traffic diverges from historical data.

Reconstruction error. One well-known limitation of confidence-based methods is that DNNs often produce high-confidence predictions even for unrelated inputs [185]. An alternative is to use reconstruction error as a signal of abnormality. Reconstruction-based methods typically employ an autoencoder or a Variational AutoEncoder (VAE) to learn a representation of either only benign traffic or the entire training dataset (benign and malicious). At inference time, the reconstruction loss

$$r(\mathbf{x}) = \frac{1}{d} \left\| \mathbf{x}_i - \hat{\mathbf{x}}_i \right\|_2^2,$$

where $\hat{\mathbf{x}}$ is the model's reconstruction of the input $\mathbf{x}$, serves as an anomaly score. Large reconstruction errors usually indicate that $\mathbf{x}$ lies outside the training distribution.

Most reconstruction-based methods operate on a sample-by-sample basis, using a threshold t derived from training data or fitted via extreme-value modeling. `CVAE-EVT` [293] fits an Extreme Value Theory (EVT) model to the tail of the reconstruction-error distribution for each known class; any sample exceeding the EVT threshold is flagged as a shift sample. `Trident` [313] trains a separate per class model (*e.g.*, AE) and marks an input as shift sample if its reconstruction error exceeds the threshold for all classes. `Deep ResNIDS` [118] first uses a supervised DNN to detect known malicious inputs; only those predicted as benign are passed to an autoencoder trained exclusively on benign data. If the reconstruction error on such an input exceeds t, `Deep ResNIDS` flags it as a shift sample. Uddin *et al.* [265] first filter out benign traffic using a one-class anomaly detector trained on normal data; the remaining suspicious samples are tested for distribution shift with a second one-class detector trained on known malicious traffic. `CND-IDS` [86] maintains a continually updated feature extractor and performs a Principal Component Analysis (PCA)-based reconstruction check on the encoded representations. If the reconstruction error exceeds a threshold t, the sample is flagged as a shifted sample.

`OWAD` [107], on the other hand, proposes a distribution-based detection approach. First, it calibrates the raw anomaly scores (*e.g.*, reconstruction errors). Then, whenever new data arrive, `OWAD` stores them in a window (also referred to as a batch or chunk) and compares the distribution of their calibrated scores against that of the previously learned distribution. Concretely, the method bins both new and old calibrated scores into histograms, computes the Kullback–Leibler (KL) divergence, and applies a permutation-based hypothesis test to decide whether the two distributions differ significantly. If they do, the system raises an alert that a shift has likely occurred.

Distance. Using reconstruction error has a key limitation: autoencoders can sometimes generalise so effectively that they also reconstruct out-of-distribution inputs with low error [98]. This can happen if shift data share local patterns with the training samples or if the decoder is powerful enough to reproduce features it has not explicitly encountered. As a result, shift samples may remain undetected when reconstruction error is the only signal. To address this shortcoming, distance-based methods have been proposed. In such methods, each sample is mapped into a latent space where known classes form distinct clusters. The distance between the new sample and each class centroid is then computed, with a common detection rule given by

$$\min_k \left\| f_\theta(\mathbf{x}) - c_k \right\| > t \implies \text{shifted sample.} \tag{2.23}$$

Here, c_k denotes the centroid of class k, and t is a threshold typically estimated from within-class scatter.

Several methods employ contrastive or related objectives to shape the latent space. For instance, `CADE` [295] and Kuppa and Le-Khac [142] use contrastive autoencoders that both bring together embeddings of the same class (intra-class clustering) and push apart embeddings of different classes (inter-class separation). `ICE-CP` [162], by contrast, adopts a VAE with KL-divergence

and cross-entropy losses to align sample distributions with class-specific priors while maintaining discriminative boundaries. OIDMD [158] extends these ideas to the penultimate layer of a classifier (the layer preceding the output layer), adding M-Fisher loss for cluster compactness and inter-class separation, and AMMD loss for pushing embeddings away from adversarially synthesised out-of-distribution points.

Beyond centroid-based rules, RFG-HELAD [315] fuses the penultimate-layer embedding of a contrastive classifier with features from a Generative Adversarial Network (GAN) discriminator, and considers a sample novel if its k-nearest-neighbor distance exceeds a threshold. In contrast, HCRP-OSD [42] defines per-class rejection regions in the embedding space; if a test sample falls within all of them, it is flagged as a shifted sample.

Hybrid. Although distance-based methods are effective for detecting distribution shift, they can be computationally expensive and sensitive to class imbalance. For instance, contrastive learning often requires large batch sizes and lengthy training to yield robust representations [47], and minority classes may struggle to form coherent clusters [48]. In response, various hybrid methods have emerged that combine multiple signals, either sequentially or in parallel, to improve shift detection.

For instance, OpenCADP [201] first applies softmax thresholding to reject low-confidence inputs, directly tagging them as shifted samples. For high-confidence predictions, it then invokes a class-specific one-class classifier to verify the sample's membership in the predicted class; if this verification fails, the sample is also flagged as shifted. VAEMax [203] first applies OpenMax [36]; if the sample prediction is the unknown class, the sample is considered as shift sample. Otherwise, the sample is routed to the assigned known class's VAE, which marks it as a shift if its reconstruction loss exceeds that class's threshold. Meanwhile, AIS-NIDS [79] trains an initial multi-class model on benign and known attack classes. Samples predicted as benign are subsequently evaluated by a stacking ensemble over benign sub-clusters; a meta-learner aggregates the ensemble outputs to decide whether a sample is a shift.

OC-MAL [173] combines reconstruction error with the distance of a sample from the training centroid in latent space. If the resulting score exceeds a threshold, the sample is flagged as a shifted instance. Str-MINDFUL [17] applies three concurrent Page-Hinkley tests: one on classification errors from a supervised classifier and two on reconstruction errors separately computed for malicious and benign classes. A shift alarm is triggered when any of these thresholds is exceeded. NSFDA [194], on the other hand, operates at the distribution level by comparing representative chunks of historical and newly arrived data. Its normality-shift detector evaluates three metrics between the two chunks: (i) the difference in Shannon entropy, (ii) the cosine similarity of latent embeddings produced by an autoencoder, and (iii) a KS test on reconstruction-error distributions. The final decision is made through an ensemble voting strategy: if any of these metrics exceed their predefined thresholds, NSFDA raises an alert indicating that a distribution shift has occurred.

2.4.6 Sample Selection and Labelling

Once a shift is detected, the affected samples must be examined and labelled. In principle, all shifted samples could be manually inspected (*e.g.*, as in [17, 315, 276, 118, 43, 241]), but in practice this is often prohibitively expensive and time-consuming. As a result, existing approaches either *automatically* assign labels to the shifted data or *selectively* request manual labels for only a small subset.

The literature typically employs two strategies for automatic labelling: (i) using model predictions as pseudo-labels, which is common in binary or one-class settings, and (ii) applying heuristic-based labelling, which relies on assumptions about the data and is often used in multi-class scenarios.

In model-based approaches, the system generally relies on one or more classifiers trained on previously labelled data. For example, `SPIDER` [15] randomly selects a subset of shifted samples and uses the existing classifier to pseudo-label them as benign or malicious. `INSOMNIA` [18] identifies the least confident samples from the main classifier and assigns pseudo-labels via a separate oracle (a Nearest Centroid Neighbour (NCN) classifier trained in parallel). `CAEAID` [300] employs two parallel models and accepts a pseudo-label only if both models agree. Finally, `CND-IDS` [86] applies K-Means clustering [166] to shifted samples and uses a small clean reference set to pseudo-label clusters as benign (if they contain at least one reference sample) or anomalous (if they do not).

In contrast, heuristic-based pseudo-labelling is often used to discover or manage new classes without relying on direct model predictions. Such methods typically involve clustering or distance-based rules to group unlabelled samples, assigning each discovered cluster a new category (*e.g.*, New Class 1, New Class 2, ...). For example, Kuppa and Le-Khac [142] accumulate shifted samples and assign a novel label if they exceed a distance threshold from known classes. `Trident` [313] clusters the shifted samples and assigns each emergent cluster a new class label, while `AIS-NIDS` [79] creates a new label whenever five or more consecutive samples deviate from all known categories.

However, ensuring high-quality pseudo-labels remains challenging, and incrementally retraining a model on imperfect labels often leads to performance degradation over time [129]. As a result, selectively requesting manual labels for a limited number of samples is generally more reliable, particularly under a labelling budget—a practical limit on the number of samples that can be manually labelled due to resource or time constraints.

Several methods exemplify this selective labelling approach. Uddin *et al.* [265] cluster the shifted samples and request manual labels only for the largest cluster. `CADE` [295] selects the most distant samples from the latent space of training classes for manual labelling under a budget constraint, and `NSFDA` [194] follows a similar strategy. `OWAD` [107] proposes an optimisation-based method to select the smallest possible subset of shifted samples. Specifically, it models the error (*e.g.*, reconstruction error) of both old and new samples as histograms, then merges just enough new samples so that the combined histogram remains closely aligned with that of the new distribution. `NetGuard` [104] fits two Gaussian Mixture Model (GMM) instances: one on the old labelled data

and another on the shifted unlabelled data. For each unlabelled sample, it computes the likelihood under both GMMs and takes their difference. Samples that are typical under the unlabelled GMM but atypical under the labelled GMM receive higher scores, indicating that they are likely to reflect new or evolving behaviours. These high-scoring samples are then prioritised for labelling within the labelling budget.

2.4.7 Shift Adaptation

Whenever newly labelled shifted samples are collected, the model must be updated to accommodate the emerging distribution. Existing approaches to this *shift adaptation* typically employ incremental learning, selective freezing, memory replay, memory replay with regularisation, pure regularisation, or ensemble-based strategies. Each of these methods seeks to integrate new information while mitigating catastrophic forgetting and controlling computational cost.

Incremental. Incremental learning provides a straightforward strategy for adapting a model to distributional shifts. At time step t, the model parameters θ_t are retrained (either from scratch or by fine-tuning) on an expanding training set:

$$D_t^{\text{train}} = D_{t-1}^{\text{train}} \cup D_t^{\text{shift}},$$

which contains all previously collected and labelled data together with the newly labelled shifted samples D_t^{shift}. This process begins at $t = 0$, where D_0^{train} corresponds to the original training set (D_{train}, as defined in Eq. 2.9).

`INSOMNIA` [18] adopts this incremental retraining strategy for both its DNN and NCN classifiers. Similarly, Kuppa and Le-Khac [142], `DOC++` [241], `RFG-HELAD` [315], `CAEAID` [300], and Uddin *et al.* [265] employ the same approach. `NetGuard` [104] also follows this expanding training set paradigm but further augments the data by synthesising additional samples for minority classes using a deep generative model. This augmentation helps to mitigate class imbalance and improve the classifier's robustness against evolving attack behaviours.

However, incremental learning can be cost-inefficient, as it requires substantial storage space and computational resources to update the model. A straightforward workaround is to reuse the model weights trained on previous distributions and fine-tune them on the new distribution. `CDDA-MD` [43] adopts this strategy; however, it may result in catastrophic forgetting—a phenomenon in which retraining on new data causes the model to lose knowledge acquired from earlier distributions.

Selective freezing. To address catastrophic forgetting, several strategies have been proposed. One approach is to freeze certain layers (*i.e.*, keep them unaltered) while optimising only the remaining layers on the new distribution. For example, Pawlicki *et al.* [198] freeze the backbone of the previously trained model and, upon detecting a distribution shift, remove its final softmax layer. This layer is then replaced with a newly initialised one, which is retrained on the shifted data while the earlier layers remain unchanged. Each time a distribution shift is detected, however, the newly initialised softmax layer discards the knowledge gained from earlier updates. As a result,

the model at time t retains its original learned features together with the most recent shift information, but loses the adaptations acquired during previous updates (*e.g.*, at steps $1, 2, \ldots, t-1$). Meanwhile, `NSFDA` [194] preserves the original training layers and introduces new layers trained on each newly detected distribution. Although this design retains past knowledge and accommodates future distributions, it increases both model size and training cost. `Deep ResNIDS` [118] follows a similar strategy by adding new layers and unfreezing a random subset of neurons in adjacent layers. However, over time, this approach leads to increased training cost and still risks catastrophic forgetting.

Memory replay. Another strategy for reducing training cost while mitigating catastrophic forgetting is memory replay. Instead of retraining on all previously seen data, memory replay methods maintain a bounded memory buffer M_t that stores a curated subset of past and newly acquired samples. Formally, when newly labelled shifted data D_t^{shift} arrive at time t, the buffer is updated according to

$$M_t = \text{UpdateBuffer}\big(M_{t-1}, D_t^{\text{shift}}\big), \quad \text{with} \quad M_t \subseteq M_{t-1} \cup D_t^{\text{shift}}, \tag{2.24}$$

where $\text{UpdateBuffer}(\cdot)$ denotes a selection or replacement policy (*e.g.*, reservoir sampling, class-balanced sampling). The model parameters θ_t are then updated by training on the buffer M_t, which serves as a compact but representative approximation of the data observed up to time t. By replaying stored samples alongside newly arrived ones, memory replay helps preserve knowledge of past distributions while requiring significantly less storage and computation than retraining on the entire historical dataset.

`Str-MINDFUL` [17] employs a straightforward memory replay strategy based on a fixed-size buffer. During initial training, this buffer is populated with a balanced (stratified) sample of both malicious and benign flows. Whenever a distribution shift is detected, `Str-MINDFUL` inserts the newly arrived samples into the buffer and randomly discards an equal number of older samples from the same class to maintain its fixed capacity. `AIS-NIDS` [79], in contrast, constructs a growing memory by randomly sampling $s\%$ of each known class. When a new class is detected, `AIS-NIDS` first retrains the model using only that class's data and then retrains again on the expanded memory set (which now includes the new class) to preserve previously learned knowledge. Amalapuram *et al.* [14] adopt a more advanced buffer-maintenance strategy. They extended the class-balanced reservoir sampling by tracking global class frequencies to guide sample replacement. Furthermore, they identify the most critical (*i.e.*, maximally interfering) stored samples and ensure these are replayed alongside the newly shifted data.

Memory replay with regularisation. The effectiveness of memory replay depends critically on the quality of the stored samples, which is difficult to guarantee over time. A complementary strategy is to combine memory replay with regularisation, so that the model not only replays a curated subset of past data but also applies a penalty term to preserve important parameters. Formally, let $L_{\text{task}}(\theta; M_t)$ denote the empirical risk on the current buffer M_t (cf. Eq. (2.10)), and let θ_{old} represent the previous model parameters. The hybrid objective then augments this task

loss with a regularisation term $R(\theta, \theta_{\text{old}})$:

$$\theta_t \;=\; \arg\min_{\theta} L_{\text{task}}\big(\theta; M_t\big) \;+\; \lambda\, R(\theta, \theta_{\text{old}}),$$

where $\lambda > 0$ controls the trade-off between fitting the current buffer and retaining past knowledge.

OWAD [107] first calibrates raw anomaly scores into reliable one-dimensional outputs and then constructs histograms for both old and new distributions. It formulates an optimisation problem over two sets of mask vectors (for old and new samples) that approximate these histograms while adding as few new samples as possible. Once the mask vectors are binarised, OWAD obtains a concise memory buffer containing old (still-relevant) samples together with a minimal set of newly shifted ones. It then estimates parameter importance by aggregating gradient magnitudes on the old samples (weighted by the mask scores) and retrains the model on this curated buffer with a regularisation penalty applied to the highly weighted parameters.

Similarly, SSF [312] discretises old and new output distributions into histograms and solves a KL-divergence-based optimisation problem for mask vectors that identify representative old samples and essential new ones. The buffer is updated accordingly: non-representative samples are discarded and replaced with newly shifted ones. SSF then retrains on the updated buffer, assigning higher loss weights to newly added samples and applying a regularisation term to preserve existing parameter values if no shift.

SPIDER [15] combines memory replay with a constraint-based mechanism akin to a specialised form of regularisation. It constructs a replay buffer by randomly selecting n samples per class from both the new distribution (*i.e.*, the shifted data) and the old one. To protect previously learned representations, it employs a gradient projection memory: before each update, SPIDER uses singular value decomposition to identify the subspace of parameters crucial to older distributions. The new gradient is then projected onto the orthogonal complement of this subspace, eliminating any component that overlaps with critical parameters from past distributions.

Finally, CND-IDS [86] implements a latent-space regularisation scheme. It fine-tunes the feature extractor on newly arrived shifted data while combining three objectives: promoting cluster separation between normal and anomalous embeddings, preserving fidelity of latent representations through reconstruction, and penalising deviations from the embeddings produced by previous model snapshots.

Ensemble. The approaches discussed so far (*e.g.*, memory replay, selective freezing, and replay with regularisation) all rely on a single model that is continually updated to accommodate distribution shift. In contrast, ensemble-based strategies maintain multiple sub-models. Formally, let $\{D_1, \ldots, D_t\}$ denote the historical sequence of distributions. When a novel distribution D_{t+1} is detected, the ensemble adds a new sub-model $f(\cdot; \theta^{(t+1)})$ specialised for D_{t+1}, rather than overwriting existing parameters. At inference, the predictions of the relevant sub-models are fused via an aggregation operator,

$$\hat{\mathbf{y}}(\mathbf{x}) \;=\; \text{Agg}\big(f(\mathbf{x}; \theta^{(1)}), \ldots, f(\mathbf{x}; \theta^{(t+1)})\big),$$

typically instantiated as a majority vote, a weighted average, or another fusion rule.

`ENIDrift` [276] trains a new sub-classifier whenever a data chunk meets a criterion and appends it to the existing ensemble. The entire pool of sub-classifiers is then evaluated on that chunk, and each model's weight is updated according to its performance. Models that adapt well to the shift are assigned higher weights, while poorly performing ones are gradually muted. The ensemble's final prediction is obtained by aggregating these weighted outputs, ensuring that the most effective sub-classifiers exert the greatest influence. A natural drawback of this approach is that the ensemble size tends to grow over time.

Agate *et al.* [3] address this issue by capping the ensemble size and reusing previously trained models when possible. When a new data chunk arrives, the chunk's confidence distribution is compared with each model's reference distribution using a KS-based procedure. If a match is found, the corresponding model is reactivated rather than training a new one; otherwise, a new sub-model is created. When the ensemble has reached full capacity, the oldest model is discarded. Importantly, only a single model is used for inference at any given time: the reactivated or newly trained model remains active until the next shift occurs.

`IFDA-GPC` [238] trains a new base classifier for each shifted data chunk and incorporates it into a fixed-size ensemble. First, a dynamic feature-selection mechanism identifies the most relevant subset of features in the shifted data. The reduced dataset is then used to train a new classifier, which is added to the ensemble. If the ensemble is already full, the least effective classifier is discarded. Finally, a genetic programming module evolves an aggregation function to fuse the outputs of all sub-classifiers, including the newly added one.

`Trident` [313] employs unsupervised clustering to group newly arrived shifted samples and then trains a one-class model for each discovered cluster. At inference time, any sample rejected by all existing models is flagged as anomalous. This design can cause the ensemble to grow rapidly, since every new cluster introduces an additional learner. Gaudreault and Branco [93] adopt a related strategy by training a new binary classifier for each novel class (*i.e.*, one-versus-all). At inference, each classifier determines whether a sample belongs to its corresponding class; if all reject it, the sample is deemed novel.

`BFS-NID` [73] maintains a frozen feature-extraction backbone. For each newly detected shifted batch of data, a dedicated projection layer is fine-tuned, initialised from the original projection layer trained on the initial dataset. These batch-specific projection layers are stored in a repository. During inference, feature embeddings are processed through all stored projection layers to compute similarity scores, and the final class prediction is made by selecting the label with the highest fused similarity score across layers.

2.4.8 Robust Learning

A central goal of robust learning is to mitigate performance degradation under distribution shift (particularly in-class evolution), where the defining characteristics of a class change over time. Within NIDS research, several strategies have been proposed, including data augmentation via generative models (*e.g.*, `NetGuard` [104]), robust feature selection (*e.g.*, `IFDA-GPC` [238]), and

specialised training mechanisms (*e.g.*, `SwapCon` [275] and `ReCDA` [297]). These methods aim to ensure that the learned representations remain effective despite continuous changes in benign or malicious network traffic patterns.

Both `SwapCon` and `ReCDA` build on unsupervised pre-training using a SimCLR-style framework [47], which has proven effective for constructing models resistant to distribution shift in non-security domains [235, 92]. SimCLR operates by creating positive pairs from two augmented views of the same sample, compelling the model to learn semantic invariances in its feature space. As a result, even when only 10% of the data are labelled, fine-tuning a classifier on top of these representations can substantially outperform purely supervised training (*e.g.*, 92.6% vs. 80.4% accuracy on ImageNet [47]). Leveraging this principle, `SwapCon` and `ReCDA` seek to counteract class evolution by preserving high-quality, robust features under evolving network conditions.

However, the augmentation schemes in `SwapCon` and `ReCDA` raise domain-specific concerns. In image-based SimCLR, simple transformations such as random crops or flips preserve semantic meaning—a flipped cat is still recognisably a cat. By contrast, `SwapCon` shuffles feature positions, and `ReCDA` swaps selected features between samples, which may generate traffic patterns that do not faithfully reflect the behaviour of the underlying class. Nonetheless, both methods demonstrate measurable improvements in robustness to in-class evolution. A key research priority, therefore, is the design of augmentation strategies that preserve the semantics of real network behaviour while retaining these gains in resilience.

2.4.9 Discussion

Our survey identifies the limitation of every technique, highlighting gaps that invite further research. Here, we narrow the discussion to the specific limitations this thesis sets out to resolve.

Novel Class Detection. Ideally, we would like to reuse the classifier's own outputs to detect novel classes [241, 93]. However, confidence scores often fail to capture a significant portion of novel samples, as DNNs are known to produce overconfident predictions [185]. An alternative is contrastive learning, which has recently gained traction for this task [295, 142, 48]. While promising, contrastive learning is often computationally expensive and sensitive to class imbalance. In this thesis, we aim to harness the strengths of contrastive learning without inheriting its drawbacks. To achieve this, we introduce a distance-based method that approximates the contrastive learning mechanism but offers lower computational cost and improved detection performance (`Rasd`, §4.3.1).

Benign Shift Detection. Detecting shifts in benign behavior is especially challenging in the one-class setting of anomaly detection. Unlike supervised classification, where samples can be compared across multiple known classes, this setting relies on a single reference class, namely benign traffic. This lack of contrast makes it difficult to determine whether deviations reflect actual anomalies or simply changes in normal behavior. Recent work [107] has addressed this issue by using distribution-based methods applied to one-class anomaly detection outputs, such as reconstruction error. In this method, the threshold for the statistical test can be tuned to reduce false alarms when detecting shifts. In this thesis, we follow a similar approach (`Mateen`, §3.3.2).

Sample Selection. The most reliable and least noisy approach to handling distribution shift is to manually label a small number of selected samples. SoTA methods typically rely on one-shot[3] uncertainty sampling, which selects the most uncertain samples for manual labelling. For example, `CADE` [295] selects samples that are farthest from the known training classes. However, this approach has a key limitation: the most uncertain samples often share similar characteristics. As a result, the labelling budget is spent on nearly identical samples, while other important shift samples, such as those from different classes or with distinct semantics, are likely to be missed. To overcome this issue, we propose two selection strategies. For the multi-class setting, we propose an extended farthest-first traversal that reduces redundancy and spreads selections across both minority and majority classes (`Rasd`, §4.3.2). For one-class setting, we introduce a new selection method that aims to capture both informative and diverse samples from different regions of the distribution (`Mateen`, §3.3.4). We compare both approaches against various uncertainty sampling methods and show that they offer improved performance, both in terms of overall accuracy and per-class effectiveness (§3.5.4 and §4.5.3).

Model Adaptation. A range of methods have been proposed for model adaptation, with incremental learning being the most common and effective. However, this approach faces three major challenges: the need for additional storage to accommodate an ever-growing dataset, the high computational cost of retraining on accumulated data, and slow adaptation. The latter occurs because newly introduced shift samples are typically underrepresented relative to existing training data. As a result, unless these rare samples are repeatedly observed across multiple updates, the model struggles to learn them effectively. In this thesis, we focus on addressing the challenge of slow adaptation in the one-class setting. Specifically, we augment incremental learning with adaptive ensemble learning (`Mateen`, §3.3.3). We demonstrate that this combination enables anomaly detection models to adapt both efficiently and rapidly to different types of distribution shift, including concept shift and covariate shift (`Mateen`, §3.5.4).

2.5 Label Noise

Label noise can severely degrade the performance and trustworthiness of machine-learning models. Both NIDS and malware-analysis pipelines are highly susceptible to it: data is often collected in the wild, ground truth is reconstructed retrospectively, and adversaries deliberately obfuscate their behavior, making it difficult to distinguish malicious from benign activity. In what follows we (i) characterise the main noise sources in NIDS and malware datasets, (ii) review generic defences developed in the broader machine-learning literature, (iii) highlight domain-specific advances, and (iv) outline promising directions for future work.

2.5.1 Labelling Strategies

Access to large-scale security data is strongly domain-specific. In the malware realm, tens of millions of malicious and benign binaries are readily downloadable from public aggregation ser-

[3]Uncertainty sampling is usually performed iteratively: select a few samples, label them, retrain the model, and repeat until the budget is exhausted [213]. However, in security applications, this process is often performed only once [18, 199, 48].

vices such as AndroZoo[4], VirusTotal (VT)[5], and VirusShare[6]. For instance, as of April 14, 2025, AndroZoo alone hosts more than 25 million Android APK files.

By contrast, realistic network traces for NIDS are rarely public; they are usually collected and shared only within organisations that are contractually or legally allowed to capture production traffic—*e.g.*, security vendors, managed-security service providers, or in-house security teams. Regardless of how abundant the raw data may be, the bottleneck quickly becomes labelling. The literature identifies two main approaches to labelling: manual and automated labelling [102, 264].

Manual labelling. Manual annotation provides the most trustworthy ground truth but is also the most expensive option for both malware and NIDS data. For malware, analysts disassemble or decompile binaries, execute them in sandboxes, manually inspect their behavior, and cross-reference the results with threat-intelligence feeds [125, 284]. Mohaisen and Alrawi report a median of 10 hours of expert effort to label a single previously unseen sample [179]. Labelling network traffic is even more labor-intensive than malware. Empirical studies of enterprise networks report that a single 1 Gbps link can generate nearly a terabyte of packet-level data per day [117]. To annotate such data, analysts must replay packet captures, reconstruct flows using tools such as Wireshark or Zeek, manually inspect traffic, and correlate their observations with host-level telemetry and security logs (*e.g.*, firewalls or IDS) before deciding whether each flow is benign or malicious. Given these volumes and the dependence on contextual information, full manual annotation is feasible only for small validation subsets or narrowly scoped research traces; larger datasets inevitably rely on automated labelling, which in turn introduces label noise.

Automatic labelling in Malware. VT-based labelling is the de-facto standard for large-scale malware labelling because it is fast, reproducible, and scales to millions of samples. Security analysts can query VT using a file hash to retrieve the latest available report or submit the file (by hash or upload) to trigger a re-scan. VT returns verdicts from multiple anti-virus engines, and dataset labels are typically derived by applying simple voting rules [258, 25, 199, 175, 271, 140, 44], often informed by earlier dynamic-analysis research [316, 272, 200, 52]. For example, Pendlebury *et al.* consider a sample malicious if at least four engines flag it and benign if none do [199]. After the binary class is fixed, family names are commonly derived using AVClass2 [226], which normalizes heterogeneous AV vendor labels into canonical names (*e.g.*, WannaCry).

Despite its scalability, VT-based labelling introduces label noise, leading to imperfectly labelled datasets. For example, Joyce *et al.* [125] analysed discrepancies between labels obtained from previously published security reports and VT-based labelling, finding that only 62.10% of binary labels and 46.78% of multi-class labels matched. Researchers therefore adopt robust labelling practices. One line of work re-queries VT at multiple time points (for example at day 0, after one month, and after one year) and then aggregates the votes [44, 207]; this temporal smoothing improves labels accuracy but postpones dataset release by months or years.

[4] https://androzoo.uni.lu
[5] https://www.virustotal.com
[6] https://www.virusshare.com

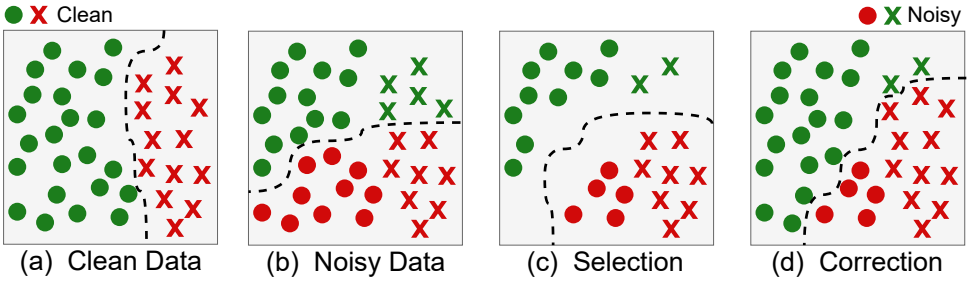


Figure 2.7: Learning from label noise.

Automatic labelling in NIDS. Two main strategies are used to label NIDS samples automatically [102]. (i) Instrumentation-based labelling. In production networks the captured traffic is re-played through security tools—*e.g.*, signature-based IDS. Any flow that fires a rule is tagged malicious, while the remainder are treated as benign [269, 45]. This approach inevitably mislabels attacks that lack signatures. A related tactic deploys honeypots: every flow that reaches a honeypot, or that later originates from a host confirmed as compromised, is likewise labelled malicious [245, 244]. (ii) Test-bed labelling. Security researchers often prefer a fully controlled environment in which benign background traffic is generated and attacks are injected synthetically. Because the attacker and victim IP addresses, ports and injection times are known a priori, labels can be derived deterministically—for example in the CICIDS2017 dataset [230]. If designed carefully, a test-bed can still produce an accurately labelled dataset, but the resulting benign traffic may be less representative of real-world behaviour.

2.5.2 Noise Learning Techniques

DNN-based models require large and diverse datasets to achieve strong generalisation. However, manual labelling at scale is costly and time-consuming, making it impractical for many fields beyond security, including computer vision and natural language processing. To address this challenge, researchers have explored alternative approaches to label large datasets. For example, a common strategy is using crowdsourcing platforms such as Amazon Mechanical Turk (MTurk), where multiple human annotators contribute to dataset labelling [278, 209, 63, 5, 282]. Although this method provides scalability, it introduces variability in label quality due to differences in annotator expertise, leading to potential human errors [278, 190, 283, 115]. Wei *et al.* studied this issue by using MTurk to label CIFAR-10, assigning each sample to three annotators [278]. Their analysis revealed that even with majority voting, the dataset contained 9.03% label noise, highlighting the challenges of ensuring annotation accuracy.

Due to these challenges, extensive research efforts have focused on developing methods to mitigate label noise during model training across various domains. Figure 2.7 illustrates the most common strategies, which are sample selection and label correction.

Sample selection identifies a subset of training data, known as the clean set, that is likely free of label noise. The model is then trained using only this subset, computing the supervised loss exclusively on the selected samples. A common approach to constructing the clean set is to use the small-loss trick, which assumes that samples with low-loss values are more likely to be correctly

Table 2.7: A summary of recent methods for mitigating label noise in NIDS and Malware.

Paper Info			Setting		Domain			Method Modules			
ID	Paper	Year	Binary	Multi-Class	NIDS	Malware	Generalisable	Selection	Correction	Robust	Semi-Supervised
1	DT [289]	2021	✓	✗	✗	✓	✓	✓	✓	✗	✗
2	MalWhiteout [273]	2022	✓	✗	✗	✓	✗	✓	✓	✗	✗
3	Morse [286]	2023	✓	✓	✗	✓	✓	✓	✗	✗	✓
4	BoAu [304]	2023	✓	✓	✓	✗	✓	✗	✗	✓	✗
5	MCRe [305]	2024	✓	✓	✓	✗	✓	✓	✗	✓	✗
6	RAPIER [202]	2024	✓	✗	✓	✓	✓	✓	✓	✗	✗
7	SLB [9]	2025	✓	✓	✓	✓	✓	✓	✓	✗	✗

labelled [106, 303]. Based on this principle, the model selects low-loss samples and trains only on them. Figure 2.7c illustrates this process, comparing it to a training set that is noise-free (Figure 2.7a) and a noisy training dataset (Figure 2.7b). As observed, the selection process favors easier samples—those in densely populated regions and far from the decision boundary. Although this approach mitigates the effect of label noise, it can result in an oversimplified decision boundary by overlooking hard-to-classify, correctly labelled samples that are potentially highly informative. To address this limitation, subsequent research has incorporated non-selected samples into training by down-weighting their influence [236, 212] or applying semi-supervised learning techniques, such as Mean Teacher [256] and FixMatch [240], to retain useful information [186, 82, 310, 279].

Label correction leverages the entire training dataset by revising incorrectly labelled samples rather than discarding them (Figure 2.7d). The goal is to dynamically adjust incorrect labels during training, improving data quality and model reliability. Unlike sample selection, which filters out suspected mislabelled data (Figure 2.7c), label correction retains all data points, allowing more samples to contribute to supervised learning and potentially enhancing model performance. Reed *et al.* [210] introduced a bootstrapping method that, in each training epoch, blends the original labels with the model's current predictions to gradually correct mislabelled data. Lu and He [161] refined this approach using self-ensembling, which blends the original labels with predictions aggregated over multiple epochs, resulting in more robust and reliable label corrections. However, because these methods revise every sample, they risk changing labels that were originally correct. To address this, Zheng *et al.* [314] proposed error-bounded correction, modifying labels only when the model is sufficiently confident, reducing the risk of incorrect revising.

Applications to security data. Despite their success in non-security domains, these methods often struggle when applied directly to noisy security datasets, particularly malware datasets. Wang *et al.* [273] emphasised that these methods require further revisions to achieve meaningful results on malware datasets. Wu *et al.* [286] expanded on this by systematically evaluating eight label noise handling methods on malware datasets and found that they underperformed. They attributed this to three key reasons. First, real-world malware datasets contain a significantly higher proportion of incorrect labels compared to other datasets. Second, noise is distributed unevenly across classes, with some being disproportionately affected. Third, malware datasets often suffer from severe class imbalances, a challenge that these methods are not inherently designed to handle effectively.

2.5.3 Literature Review

Recently, mitigating label noise in security datasets has received increased attention, particularly in malware [286, 289, 273] and NIDS [304, 305, 202]. We summarise these works in Table 2.7, focusing on setting, domain, and modules. In the setting dimension, we examine whether a given approach can adapt to both binary and multi-class classification; systems tagged solely for binary classification would require a fundamental redesign to scale to multi-class problems. Under the domain category, we note the specific security area in which the approach was developed, and assess whether it is generalisable to other security areas without major modifications. Finally, we classify each method based on the modules it includes: sample selection (Selection), label correction (Correction), robust training mechanisms (Robust), and semi-supervised learning.

Techniques on NIDS Literature. `BoAu` [304] begins with a brief warm-up during which its autoencoder is trained solely with a reconstruction objective. That pre-training step gives the encoder a label-agnostic starting point, reducing the chance that it will memorise early label errors. Once warm-up finishes, three objectives are optimised simultaneously. First, the boundary-augmentation loss clusters each mini-batch with K-Means [166], then forces every sample to lie close to its cluster-mates and far from the nearest competing cluster centre, effectively tightening class boundaries. Second, a conventional cross-entropy term supplies limited supervision from the noisy labels. Third, the reconstruction loss is preserved to keep the latent space faithful to the raw traffic. After training, the encoder's embeddings are re-clustered with K-Means [166]; the Hungarian algorithm then pairs each cluster with the most compatible class label, producing the final classifier.

`MCRe` [305] extends `BoAu` by providing sample selection mechanism, discarding the warm-up stage, and providing new loss. It still uses reconstruction and the same cluster-separability loss, and it retains cross-entropy to inject weak supervision. Its distinctive feature is the core-proximity loss. During training, any instance whose predicted label matches its noisy label is treated as clean and projected into a secondary embedding space. For each class, a momentum-averaged prototype is maintained from those clean representations. The core-proximity term penalises large distances between any sample and its class prototype. At inference time the model can be used exactly like `BoAu`, but the prototype distances also provide a simple criterion for discarding suspected noisy labels. By measuring how far each sample lies from the assigned class prototype, one can simply discard any sample beyond a certain distance threshold.

Both `BoAu` and `MCRe` are considerably more expensive than standard supervised training. `BoAu` incurs heavy cost by running K-Means [166] inside every mini-batch and again globally at the end of training, with losses that require repeated distance computations to multiple cluster centers. `MCRe` removes the warm-up but adds its own burden: maintaining momentum-updated prototypes in a secondary embedding space and enforcing per-sample proximity to them, on top of clustering. These extra clustering, prototype, and dual-space operations substantially increase training time and memory use, limiting scalability to very large datasets.

`RAPIER` [202] is a framework that explicitly reduces the influence of label noise on malware-

detection models for encrypted traffic.[7] A bidirectional gated recurrent unit autoencoder is first trained and then used to produce embeddings. RAPIER next trains the autoregressive density estimator MADE [94] solely on the encoder embeddings of samples originally labelled as benign, thereby learning a density landscape for normal traffic. Each sample receives a benign-likelihood score, and the top $s\%$ of samples are provisionally accepted as benign. Within this subset, each sample's embedding average Euclidean distance to all other members is computed. The half with the smallest averages (*i.e.*, the densest portion) is then relabelled as definitively benign. For the remaining training samples, the distance to that definitive benign subset is computed. The most distant subset, equal in size to the definitive benign group, is redesignated as malicious. Finally, a diverse ensemble of seven machine learning classifiers (*e.g.*, Random Forest [39]) is trained on this cleaned split and used to pseudo-label the residual unlabelled traffic, resulting in a fully annotated set. However, this approach is designed only for binary classification and cannot easily scale to multi-class settings.

Techniques on Malware Literature. MalWhiteout [273] extends Confident Learning (CL) [191] to specifically address noisy labels in binary Android malware datasets by combining ensemble-based noise detection with developer-based label correction. First, it trains multiple base Android malware classifiers (*i.e.*, Drebin [25], CSBD [8], and MalScan [287]) and uses 5-fold cross-validation to obtain out-of-sample predicted probabilities for each application. These outputs are aggregated into a meta-learner (via stacking), which CL then uses to identify incorrectly labelled samples. Next, MalWhiteout refines these results by clustering apps according to developer metadata (*e.g.*, signature, developer ID) and assigning a majority label to each cluster—any app whose original label contradicts the cluster's majority is deemed noisy and flipped. This final calibration step leverages the assumption that malicious developers often release multiple malicious apps, so apps sharing a developer metadata typically share labels. As a result, MalWhiteout produces a cleaner training subset with fewer false positives and false negatives than naive CL alone. However, this approach is suitable only for binary Android problems, and cannot scale to multi-class settings or other domains.

DT [289] is a label-noise-correction framework designed for binary Android-malware datasets. It trains two deep neural networks: one on the entire dataset and another on a down-sampled subset. The per-sample loss vectors emitted by these models are passed to an ensemble of 13 unsupervised outlier detectors (*e.g.*, one-class support vector machine [225]). Detection thresholds are calibrated using the known global noise rate. Samples flagged as outliers have their labels flipped to the opposite class with a random acceptance probability. Although effective, DT requires substantial computation because each iteration retrains two deep models and executes all 13 detectors, and its binary label-flipping mechanism does not generalise naturally to multi-class settings.

MORSE [286] is a semi-supervised noise mitigation framework tailored to malware classification tasks under high noise rates and extreme class imbalance. First, it warms up a deep neural network by briefly training it on the entire dataset for a few epochs. Then, in each training epoch, it partitions the dataset by designating the lowest-loss examples in each class as labelled and treating

[7] Although RAPIER also incorporates a generative module to address limited data, that component lies outside the present review, which focuses solely on label-noise mitigation.

the rest as unlabelled. Next, MORSE applies FixMatch-style consistency regularisation: for every unlabelled sample, it creates two augmented views via random feature replacement (*i.e.*, replacing different subsets of the sample's features with those from two randomly selected samples). The weak augmentation replaces fewer features, thus producing a perturbed version that remains closer to the original, whereas the strong augmentation replaces more features, leading to a substantially altered variant. The first (weak) augmentation is used to predict a pseudo-label; if the prediction's maximum probability exceeds a confidence threshold t, the second (strong) augmentation is forced to share that same pseudo-label via a consistency loss. To address severe class imbalance, MORSE assigns per-class weights inversely proportional to the effective sample size, thereby boosting minority classes. Although this setup reduces dependence on large, perfectly labelled datasets, the comparatively limited labelled subset may constrain a deep model's capacity to learn robust representations—especially given that larger labelled datasets generally enhance performance [250].

2.5.4 Discussion

Our survey reveals several pressing research gaps that deserve focused attention.

Realistic yet controllable NIDS datasets. Capturing perfectly labelled traffic from production systems is nearly impossible: packet traces are massive, user behavior is highly dynamic, and attack activity is both rare and difficult to observe in real time [102, 168, 267]. A promising alternative is to build a closed, fully instrumented testbed in which benign traffic emerges naturally from realistic user workflows and attack traffic reflects the latest tactics, techniques, and procedures. We argue that such realism is now feasible through an agentic framework in which cooperating Large Language Model (LLM)–driven agents perform everyday business activities such as email, web browsing, and file sharing, while adversarial agents simultaneously probe the same network, identify vulnerabilities, and generate exploits in real time. Because every component of the environment is instrumented, each flow receives a deterministic label, yielding datasets that combine authentic background traffic with up-to-date attacks—something rarely achieved with live captures or scripted replays.

Label noise in VT-based annotations. Most research that investigates VT dynamic focuses on how verdicts changes when samples are rescanned over time [316, 272, 200, 52], yet there is still no thorough empirical study of the noise already present in a single VT snapshot. We do not know the baseline error rate, how mistakes are distributed across malware families or vendors, or which classes suffer most from noise. Nor has anyone quantified the real impact of the common robust labelling practices, such as issuing repeated scans at fixed intervals [207], restricting the vote to a curated set of high-trust engines [25], or requiring stricter consensus thresholds [48]. A systematic investigation that measures the noise, characterises its vendor and class bias, and tests these practices side by side would establish how much label quality actually improves and where further effort is needed. Such evidence could guide the creation of a principled pipeline for curating large VT-annotated malware datasets and provide a solid baseline for future work on learning with noisy labels.

Noise-learning techniques. Recent security works that tackle label noise often claim that algo-

rithms from vision or language tasks perform poorly on malware data, yet they still import many of the same assumptions and design choices from those non-security domains [286, 273]. A pressing research need is a systematic study that tests a broad spectrum of noise-learning methods on representative security datasets. Such an investigation should evaluate dozens of algorithms across multiple malware and network datasets, measure where each method succeeds or fails, and identify the factors that drive those outcomes. The resulting evidence would show which techniques generalise across security tasks and datasets, and provide concrete design rules for noise-learning pipelines tailored to security data.

Broadly applicable frameworks for label correction. Label correction methods are attractive because they keep every example in play and are typically architecture-agnostic, plugging in at the label/loss level. Compared with sample selection or semi-supervised schemes, they can deliver high data efficiency by upgrading noisy annotations rather than discarding or down-weighting them. The drawback is that existing correction systems (`DT`, `RAPIER`, and `MalWhiteOut`) are tailored to specific security domains and operate only in binary settings, which limits their reach. A key research direction is, therefore, to design label-correction frameworks that are broadly applicable across security datasets and scalable to multiple classes, while recognising that reliable deployment may benefit from domain- or protocol-specific cues.

Because two earlier chapters of this thesis already explore concept shift, the remaining time does not allow a thorough treatment of every research avenue identified above. We therefore devote the final technical chapter to a broadly applicable label-correction framework (`SLB`, §5).

2.6 Threat Model

Before concluding this chapter, we define the threat model assumed throughout this thesis. Our goal is to make explicit (i) what the defender observes and optimises, (ii) which classes of attackers are represented by our benchmarks, and (iii) which failure modes we attribute to natural distribution shift and unintentional label noise rather than to adaptive adversarial machine learning.

System Model and Defender Objectives. We consider a defender operating a passive NIDS deployed at a strategic observation point. The NIDS monitors traffic and converts it into feature vectors $\mathbf{x}$ derived from either (i) bidirectional flows (5-tuple aggregation with flow statistics) or (ii) packet-level temporal statistics (*e.g.,* exponentially decayed, windowed features). A deep model f_θ then maps each $\mathbf{x}$ to a label in either a binary setting (benign vs. malicious) or a multi-class setting (benign plus known attack types), depending on the regime in each chapter.

The defender's primary objective is to maintain high detection performance at low false-alarm rates under two non-adversarial stressors: (i) distribution shift ($P_{\text{train}}(x,y) \neq P_{\text{test}}(x,y)$), including benign shift and novel classes as defined in §2.4; and (ii) label noise in the training data, arising from imperfect automatic labelling as discussed in §2.5. We assume the defender can periodically retrain or update the model to respond to distribution shift, but lacks continuous, real-time ground truth labels for all observed traffic. Any manual inspection is budget-limited and therefore applies only to a small subset of samples.

Attacker Goals and Capabilities. Attackers aim to violate the confidentiality, integrity, or availability of networked assets (§2.1.1). They can generate and shape network traffic by interacting with exposed services, compromising endpoints, or operating from within the network. The NIDS is passive: the attacker cannot directly tamper with the sensor, its feature-extraction module, or the model parameters, except indirectly by changing the observed traffic distribution.

Attacker Profiles. We distinguish two broad operational profiles, aligned with the datasets used in this thesis:

- **External and perimeter-focused attackers.** These actors operate outside the network perimeter and target exposed services. Typical behaviours include scanning, credential attacks (*e.g.,* brute force), web exploitation (*e.g.,* SQL injection and XSS), and availability attacks (DoS/DDoS). These behaviours are represented in the CICIDS2017 and CSE-CIC-IDS2018 datasets.

- **Proximity-based and internal attackers.** These actors have local-network proximity (*e.g.,* the same broadcast domain) or can occupy an on-path position. Their behaviours include LAN reconnaissance, ARP-based man-in-the-middle interception or manipulation, and LAN-local disruption, including malware/botnet activity. This profile is characterised most directly by the Kitsune dataset, which is captured on IoT and surveillance networks and includes explicit on-path (man in the middle) scenarios and device-centric attacks. We do not rely on Kitsune to model full enterprise post-compromise playbooks such as extensive lateral movement or structured data exfiltration; rather, we use it to study benign shift and anomaly detection in realistic local-network environment.

Shift and Noise as the Primary Stressors. We attribute performance degradation primarily to naturally arising data issues rather than to an adaptive, ML-aware adversary:

1. **Distribution shift (deployment-time).** We assume that both benign activity and malicious behaviours evolve over time. This includes: (i) benign shift driven by changes in users, applications, or configurations; and (ii) novel classes where previously unseen attack families appear (the $C+1$ setting in §2.4.2). These changes may be gradual or abrupt, and the evaluation protocols in later chapters reflect the time-aware perspective discussed in §2.4.

2. **Unintentional label noise (training-time).** We assume training labels may be imperfect due to the practicalities of labelling security data (*e.g.,* heuristic/testbed labelling rules, or human error under limited manual inspection). This thesis treats such noise as unintentional and focuses on learning procedures that remain robust when labels are unreliable (§2.5 and Chapter SLB).

Out-of-Scope Adversarial ML. To keep the scope aligned with the thesis objectives, we explicitly exclude: (i) query-adaptive evasion and gradient-based adversarial examples crafted against DNN-based NIDS; (ii) intentional poisoning where an adversary injects or modifies training samples specifically to subvert learning; and (iii) attacks that assume compromise of the NIDS infrastructure itself (sensor tampering, feature-extractor compromise, or model-parameter manipulation). We nevertheless include naturally stealthy attacks present in the datasets (*i.e.,* the scarce Heart-

bleed samples in CICIDS2017), and we treat their difficulty as part of the real-world distributional variability the defender must handle. Accordingly, any degradation observed in our experiments is interpreted as a consequence of distribution shift and/or unintentional label noise, not an adaptive adversary optimising against DNN-based NIDS internals.

2.7 Conclusion

In this chapter, we established the conceptual foundation for the remainder of the thesis. We began by defining network intrusions and discussing the two main categories of intrusion detection systems: signature-based and anomaly-based. The discussion then turned to deep neural networks, covering feed-forward architectures, supervision levels, optimisation algorithms, and the key hyperparameters and evaluation metrics that shape model performance. Since the reliability of experimental results depends critically on data quality, we next justified the selection of three modern, time-aware NIDS datasets (IDS17, IDS18, and Kitsune), detailing their collection processes, class distributions, and known limitations.

Two long-standing challenges to building robust and deployable DNN-based NIDS were then examined in depth. First, we formalised the concept of distribution shift in the context of network security, distinguishing between in-class evolution, benign shift, and novel classes. We also presented a taxonomy of recent mitigation techniques, including reconstruction-based and distance-based detectors, as well as memory replay and ensemble learners for model adaptation. Second, we analysed the widespread impact of label noise, identifying its origins in automated labelling pipelines such as VirusTotal-based and testbed heuristics. We reviewed recent techniques from both NIDS and malware research, highlighting their strengths and limitations.

We also discussed the key challenges that motivate our design choices. For example, in novel class detection, we argued that contrastive learning is more effective than relying on the classifier's own outputs. However, it comes with significant drawbacks, including high training costs and sensitivity to class imbalance. We further highlighted that incremental learning often suffers from slow adaptation, and that the commonly used uncertainty sampling method tends to select highly similar samples, leading to poor diversity and coverage.

Finally, we defined the threat model assumed throughout the thesis. In particular, we consider a passive NIDS whose performance is primarily stressed by naturally arising distribution shift at deployment time and unintentional label noise at training time, rather than by an adaptive adversary optimising directly against the detector. This framing clarifies what is measured in the subsequent chapters, what the experiments are intended to represent, and how we interpret failures: as evidence of shift and noisy supervision rather than deliberate evasion or poisoning. With this foundation in place, the next chapters build and evaluate methods that detect, mitigate, and adapt to these stressors under realistic operational constraints.

Chapter 3

Mateen: Adaptive One-Class Anomaly Detection

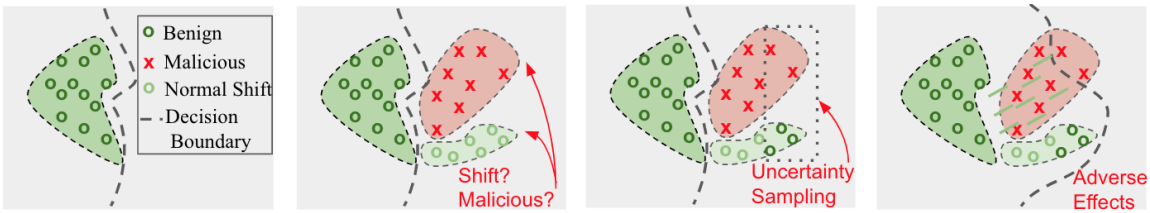
In the previous chapter, we discussed the challenge of distribution shift in DNN-based NIDS, emphasising that this issue can arise at any detection level: one-class, binary, or multi-class (§2.4). As the first technical contribution of this thesis, we now introduce Mateen, a framework designed to mitigate the impact of benign shift on one-class anomaly detectors, with a particular emphasis on autoencoder-based architectures. This work has been published as:

Fahad Alotaibi and Sergio Maffeis, "**Mateen: Adaptive Ensemble Learning for Network Anomaly Detection".** In *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID) 2024*, September 30–October 02, 2024, Padua, Italy. ACM, New York, NY, USA.
DOI: 10.1145/3678890.3678901

3.1 Introduction

AutoEncoders (AEs) form the foundation of many zero-positive anomaly detectors in network security [177, 224, 74, 187, 255, 152, 108, 49, 215, 111]. For instance, KitNET [177] trains an ensemble of AEs to model normal traffic in IoT environments and flag deviations as potential attacks. These systems often report strong performance, but typically under offline evaluation settings where the model is fixed after training and the test data is assumed to follow the same distribution as the training data.

In real-world deployments, however, benign behaviour evolves and attackers continuously adapt, leading to distribution shifts that violate this assumption [24, 48, 107]. Since zero-positive AEs are trained exclusively on benign traffic, they are inherently insensitive to novel attack patterns and particularly vulnerable to changes in normal behaviour—so-called benign shifts. As the benign data distribution shifts, reconstruction errors increase, causing the model to misclassify normal traffic as malicious. As a result, a model trained offline can quickly become outdated [242, 199,



(a) Train (b) Test (c) Selection (d) Adaptation

Figure 3.1: Key challenges in adapting to benign shifts.

24, 26]. To remain effective, the model must be continuously updated to keep pace with the evolving environment.

Challenges. Updating an outdated AE presents three main challenges. The first challenge is detecting benign shifts in a zero-positive setting. In this context, the AE, trained exclusively on benign instances (Figure 3.1a), interprets deviations as potential malicious behaviours. This introduces complexity, as illustrated in Figure 3.1b, where deviations may indicate malicious activities, benign shifts, or a combination of both. The second challenge lies in selecting a small, representative subset of samples from the shifted distribution for manual labelling and subsequent model updating. As illustrated in Figure 3.1d, updating the AE with non-representative samples can lead to poor generalisation and degraded performance. In particular, uncertainty-based methods often select samples that are far from the decision boundary, which may fail to capture the full diversity of the benign shift, as shown in Figure 3.1c. A model updated with such samples might not effectively capture the full distribution, risking the misclassification of malicious samples as benign (Figure 3.1d), or incorrectly identifying benign shift samples, especially those significantly different from the ones chosen for manual labelling, as malicious. The third challenge is achieving effective adaptation to the shifted distribution while preserving knowledge of the original benign behaviour. Because only a limited number of samples are selected and labelled, the model may struggle to generalise beyond them, leading to slow adaptation, insufficient learning of the new behaviour, or forgetting of previously learned patterns, all of which degrade performance and increase the need for human supervision (*e.g.*, increasing the subset size).

Our Method. In response to the abovementioned challenges, this chapter introduces `Mateen`, a framework designed for AEs-based NIDS to detect and adapt to intricate and varied changes in benign traffic patterns. `Mateen` addresses the first challenge by employing a statistical analysis method to detect shift that demonstrates lower sensitivity to the distribution's tail. This approach allows `Mateen` to focus on deviations near the center of the distribution, where benign shifts are likely to occur. To address the second challenge, `Mateen` leverages a new sample selection technique designed to construct a compact coreset from the new data distribution in an unsupervised fashion. This coreset is carefully selected to encapsulate the entirety of the new distribution, thereby significantly enhancing the model's capacity to generalise across it. To address the third challenge, `Mateen` implements an ensemble of AEs updated collaboratively and selectively to adapt to shifts. In this process, a long-lived AE is updated with newly selected benign samples and historical data. Concurrently, a temporary AE, derived from the long-lived AE, is trained

exclusively on the new benign samples. By doing so, Mateen ensures that the ensemble models remain diverse and relevant to both historical and recently captured distributions. Finally, Mateen keeps the ensemble as compact as possible by merging high-performing AEs and discarding underperforming ones. In this way, Mateen streamlines ensemble complexity without sacrificing performance.

Evaluation. We carried out a comprehensive evaluation of Mateen using three NIDS datasets: IDS17 [77, 230], IDS18 [156, 230], the IoT-focused Kitsune dataset [177], and two modified versions of Kitsune, each designed to represent a distinct type of distribution shift. We compared Mateen with recent SoTA methods for managing distribution shift in NIDS. These are INSOMNIA [18] for supervised adaptation, CADE [295] for out-of-distribution detection and sample selection, and OWAD [107] for unsupervised detection and adaptation to benign shifts. The results show that Mateen consistently outperforms these baselines across multiple detection metrics and shift scenarios. Notably, Mateen achieves AUC-ROC scores of 96.79% on IDS17, 98.17% on IDS18, and 94.76% on modified Kitsune (mKitsune), exceeding the strongest baseline by 2.23%, 4.36%, and 14.78% respectively. We also conducted an ablation study to assess the contribution of each subcomponent, as well as a hyperparameter sensitivity analysis to examine the effect of configuration choices. The findings show that every component contributes to Mateen's overall effectiveness. Moreover, Mateen maintains strong performance even with sub-optimal hyperparameters, demonstrating its robustness in practical settings.

Contributions. This chapter has four main contributions:

- We designed a new adaptive ensemble learning approach, explicitly crafted to address the challenge of various forms of benign shift effectively.
- We enhanced the adaptive ensemble learning approach with three techniques focused on benign shift detection, sample selection, and ensemble complexity reduction, each contributing to a more robust and effective system.
- We incorporated these components into Mateen, a unified framework designed to allow an offline AE to detect and adapt to benign shifts at the lowest possible cost. We have made the code publicly available[1], facilitating future research in this field.
- To the best of our knowledge, we provide the first evaluation of shift-aware NIDS on different scenarios of benign shift. We demonstrate the ability of Mateen to adapt to varying distributions, and explore the impact of its key components.

3.2 Preliminaries

In this section, we present an overview and literature review of techniques relevant to Mateen. We begin by discussing ensemble learning, with a particular focus on adaptive ensemble methods and

[1] https://github.com/ICL-ml4csec/Mateen/

the challenges they face. We then review existing approaches to sample selection and highlight the key differences between them.

3.2.1 Ensemble Learning

Ensemble learning is a machine learning technique that combines multiple base models, or learners, to construct a single, more robust predictive model. The one predictive model is derived by aggregating the outputs of individual learners through methods like weighted voting, majority voting, or selecting the best performer. Typically, these learners are developed in an offline setting, trained once on a substantial dataset, and then deployed without subsequent updates. However, in environments characterised by frequent distribution shifts, such static learners quickly become outdated.

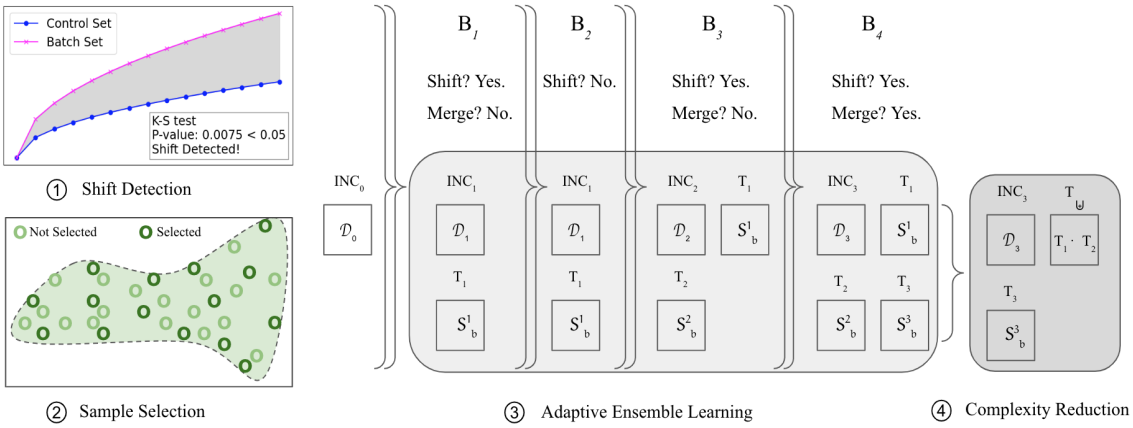
Adaptive ensemble learning addresses the challenge of distribution shift by continuously updating the ensemble to reflect changes in incoming data. This is typically achieved in two ways. One approach is incremental learning, where the training set is enriched with new samples from the shifted distribution before retraining the existing model [17, 130]. Another, more flexible approach is batch-based ensemble learning, which avoids reliance on a fixed set of models trained on the same distribution. Instead, it incrementally trains a new sub-model for each incoming data batch that meets certain criteria—such as evidence of distribution shift [301, 292] or a drop in detection performance [70, 76]. Each sub-model is then added to a growing model pool, and ensemble predictions are made through strategies such as weighted voting [276, 160, 301, 292].

Batch-based learning strategies vary widely depending on how batches are formulated. Common methods include fixed-batching [76, 70, 301, 292] and dynamic-batching [37, 38, 160, 276], with training typically carried out in either an unsupervised manner [301, 292] or using labelled benign samples (*i.e.* zero positive) [276]. However, both strategies pose challenges in security applications such as NIDS. Unsupervised training risks self-poisoning, as malicious samples unknowingly included during updates are treated as benign. This can lead the model to learn malicious behaviour, increasing the likelihood of misclassifying similar attacks as normal in the future. In contrast, benign-only training assumes access to accurate labels for all samples in each batch, which is often unrealistic in practice due to the cost and difficulty of obtaining large labelled data in security.

3.2.2 Sample Selection

Sample selection consists of selecting particular examples from a dataset to boost a model's performance, and is applicable to both labelled and unlabelled datasets. For labelled data, this reduces to constructing a coreset: the smallest subset that still mirrors the full distribution's geometry. Training on such a distilled set can achieve performance comparable to using the entire dataset while substantially reducing computation and storage requirements [19, 153, 135, 54, 298, 178, 136].

With unlabelled datasets, the focus is on selecting data points that, once labelled, significantly enhance model performance, balancing informativeness (points about which the model is most uncertain), representativeness (closeness to dense regions of the data manifold), and diversity



Dataset	Description	Model	Description
B_i	The incoming batch of data at step i.	INC_i	The *incremental model* trained on D_i.
S_b^i	Samples selected from B_i and manually labelled as *benign*.	T_i	A *temporary model* exclusively trained on S_b^i.
D_i	Incremental dataset at step i: $D_i = D_{i-1} \cup S_b^i$.	$T_{\uplus}$	The *merged model* averaging T_1 and T_2.

Figure 3.2: Overview of `Mateen`.

(distinctness from previously selected points) [213]. In some cases, a sample can be both representative and informative—for example, when it originates from a shifted distribution and captures a large portion of the new data. Such selection strategies may rely on assessing the intrinsic properties of the data or on feedback from the model's predictions [285, 18, 48, 213]

Active learning, also often associated with selection from unlabelled datasets, involves querying a pool of unlabelled samples to identify those that require labelling [228, 213]. This selection process can occur once or iteratively on an n-by-n basis, where n represents either one sample or a batch of samples. This iterative approach integrates model feedback at each step, allowing for continuous refinement of the selection process. The most common active learning technique in security applications is uncertainty sampling [18, 295, 48]. It focuses on choosing samples that present the highest degree of uncertainty to the model. Such uncertainty is typically derived from various metrics, including low probability scores [18], low fitness scores (e.g., high distance to established training classes) [295, 48], and high reconstruction errors [107].

3.3 The `Mateen` Framework

This section presents `Mateen`, a framework that detects and adapts to benign shifts in AE-based NIDS. `Mateen` comprises four sub-modules: the shift detection module, responsible for detecting benign shifts; the sample selection module, which selects relevant samples for labelling and updates; the adaptive ensemble learning module, responsible for shift adaptation; and a complexity reduction module, designed to decrease `Mateen`'s overall complexity.

3.3.1 Overview and Simplified Example

Figure 3.2 illustrates the high-level workflow of `Mateen`. `Mateen` operates in an online manner, predicting incoming samples as they arrive and adapting to benign shifts. `Mateen` classifies incoming samples by using the model that showed the best performance during the previous update

round. Simultaneously, these samples are aggregated into a count-based batch B_i. When B_i reaches full capacity, `Mateen` performs hypothesis testing to determine whether a shift has occurred statistically, checking if the distribution of B_i aligns with that of the previous training data D_{i-1} (Figure 3.2, ①). If the distributions are aligned, no update is needed. If instead a distribution shift is detected, `Mateen` selects a stratified subset from B_i that accurately represents the overall distribution, as illustrated in Figure 3.2 ②, for annotation by human experts. This labelled subset is then employed to update the models.

`Mateen` performs two types of updates on its ensemble of models: a major update and a minor update (Figure 3.2, ③). In the major update, a new temporary model, T_i, is created and trained on the benign samples from the labelled subset, denoted as S_b^i. In the minor update, `Mateen` augments the training data to include S_b^i, resulting in D_i, and retrains the previous incremental model, INC_{i-1}, on D_i, to produce INC_i. Finally, `Mateen` optimizes the complexity of the ensemble by enforcing a maximum size limit (Figure 3.2, ④). Upon reaching this limit, high-performing models are combined in a merged model $T_{\uplus}$, and low-performing ones are discarded.

3.3.2 Shift Detection

The `Mateen` shift detection module is designed to identify occurrences of benign shift. In contrast to existing methods that operate in a supervised setting, where an instance is compared against both malicious and benign baselines [295, 142, 48], `Mateen` functions within a one-class AD framework. Hence, we need to identify shift occurrences without relying on a direct comparison to a malicious baseline. Our strategy is to compare the distribution of two sets of equal size: the incoming batch of data B_i and a control set D_c. The control set comprises the most recent samples from D_{i-1}, ensuring it has the same number of samples as B_i. However, it is hard to derive a strong distribution comparison using the sample features, because of the high dimensionality. Instead, we leverage the current AE (the highest-performing model from the most recent update cycle) to map each sample to a single dimension. We calculate the MSE score for each sample x_i as follows:

$$\text{MSE} = \frac{1}{d} \left\| \text{AE}(\mathbf{x}_i) - \mathbf{x}_i \right\|_2^2. \tag{3.1}$$

With the one-dimensional scores for each set in hand, our objective is to statistically compare their distributions. The null hypothesis is that D_c and B_i originate from the same distribution, indicating no shift. Note that, D_c is presumed to be benign, as it comprises instances that have been collected, labelled, and filtered to ensure they represent only benign behaviour. In contrast, we cannot assert that B_i contains only benign instances. Indeed, if B_i comprises a mix of malicious and non-shifted benign instances (i.e., benign samples drawn from the same distribution as D_c), the comparative analysis of the distributions for D_c and B_i might reveal a significant divergence, indicating that shift has occurred.

To compare distributions, we use the two-sample KS test. Although it has been criticized for being less sensitive to outlier values at the tails of a distribution, such as extreme values [169, 167, 81], this characteristic is actually advantageous in our specific context. Typically, malicious instances exhibit high MSE scores, but occur less frequently than benign instances. Thus, the KS test's lower

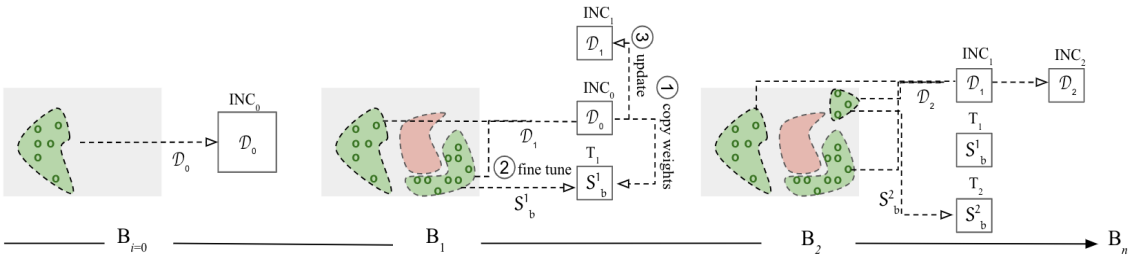


Figure 3.3: Mateen's adaptive ensemble learning module.

sensitivity to such extreme values allows for an equitable comparison across the full spectrum of both distributions, ensuring that rare, extreme malicious samples do not bias the results.

Let $F_{D_c}(r)$ and $F_{B_i}(r)$ represent the Empirical Cumulative Distribution Functions (ECDFs) corresponding to the MSE scores derived from the control set D_c and the incoming batch B_i, respectively, where r denotes the MSE score values for which the ECDFs are calculated. We calculate the KS test as follows:

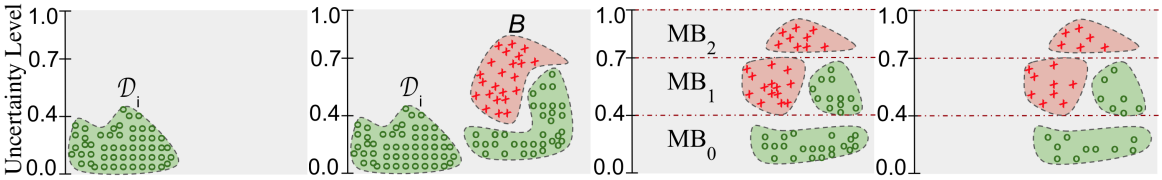
$$KS_{D_c,B_i} = \sup_{r \in \mathbb{R}} |F_{D_c}(r) - F_{B_i}(r)| \tag{3.2}$$

The term $\sup_{r \in \mathbb{R}}$ indicates the supremum, or the maximum deviation, between the two ECDFs across all possible values of MSE scores observed in both D_c and B_i. The resulting p-value from the KS test is then compared against the conventional significance threshold of 0.05. In statistical terms, a p-value below this threshold typically suggests a significant difference [195, 126, 302, 155, 69], hence supporting the alternative hypothesis that a distributional shift has occurred. Conversely, a p-value above this threshold would support the null hypothesis, indicating no significant shift between the distributions. Analysts may adjust the threshold value to better reflect the malicious-to-benign ratio in the specific network being secured, thus tailoring the sensitivity of the test to the context of their network. For instance, by setting a higher significance threshold (*e.g.*, 0.1), the test becomes more permissive, thereby increasing its sensitivity to smaller deviations between distributions.

3.3.3 Adaptive Ensemble Learning

To adapt to distribution shifts, `Mateen` employs adaptive ensemble learning, a method that generates new models to encompass emerging distributions. While various approaches have been devised to apply this concept across different scenarios, they often fall short in the context of AE-based NIDS for a crucial reason.

Commonly, these approaches generate a new model using the entirety of the incoming data batch [70, 76, 276, 160, 301, 292]. This batch may be processed in two ways: either by being labelled and cleansed of malicious instances before training [276, 160], or by allowing the model to update itself in an unsupervised manner without any prior filtering [301, 292]. The challenge with the former is the impracticality of labelling every piece of incoming data, while the latter poses a risk of self-poisoning, potentially causing the model to incorrectly classify malicious activity as benign. Furthermore, limiting the model's training to a small set of labelled benign samples can



(b) 1. $\boldsymbol{D_i}$ (c) 2. $\boldsymbol{B}$ (d) 3. Partition (e) 4. Selection

Figure 3.4: The proposed sample selection method.

cause overfitting, where the model fails to generalise well beyond the specific instances it was trained on. Our approach, detailed below and illustrated in Figure 3.3, mitigates these issues.

If data batch B_i exhibits a distribution shift, we employ the selection method outlined in §3.3.4 to select a subset S_i of representative samples for manual labelling. S_i is then used to assess the need for updating the ensemble. If the model's performance on S_i, measured using the F1 score, surpasses a predefined threshold t, no update is necessary[2]. Otherwise, the samples from S_i are filtered to eliminate instances of attacks, obtaining the benign-only subset S_b^i, which is utilised for two types of updates: minor and major.

For the minor update, we augment the benign data accumulated so far with S_b^i, resulting in an updated dataset $D_i = D_{i-1} \cup S_b^i$. We then update INC_{i-1} using D_i to obtain INC_i (Figure 3.3, ③). This incremental model is capable of adapting to minor distribution shifts, such as covariate shifts, because it continuously learns from a growing dataset and only needs to adjust to changes within the feature space. For instance, if a user's browsing behaviour changes, INC_i can reassess the importance of the features to establish a more effective decision boundary.

For the major update, we clone INC_{i-1} and fine-tune it on S_b^i to obtain the new temporary model T_i, which we add to the ensemble (see ① and ② in Figure 3.3). Leveraging the weights from the incremental model, instead of initializing T_i with random weights, allows INC_{i-1} to transfer its deep understanding of benign traffic patterns to T_i, avoiding the pitfall of learning from scratch, which can lead to overfitting. This strategy presents multiple advantages. It prepares the temporary models to efficiently manage new data distributions, or concept shifts, despite having limited data for training. This capability starkly contrasts with the incremental model, which, due to empirical risk minimization principles, may display a bias towards the dominant patterns within the training data. Moreover, not every statistical representation benefits from fine-tuning, leaving some statistics unchanged and thus still reflective of prior distributions. As a result, the temporary models, even after adaptation, are expected to make fewer mistakes on previous distributions compared to models that begin with random weights.

After the updates, we select the model that achieves the highest F1 score on the S_i labelled samples for use as the reference model in the next prediction and shift detection cycle.

[2]We set t at 99%, as this level of performance is typically observed when evaluating the model on network data from the same distribution (e.g., [111, 4, 215]).

3.3.4 Selection Method

The adaptive ensemble learning module necessitates a labelled dataset for its update step. Due to the impracticality of labelling an entire batch B, we need a method for selecting a representative subset S from B. For the update to be effective, this S needs to satisfy three key criteria: it must encompass samples from every segment of B's distribution, ensuring a comprehensive representation; the selected samples need to be highly informative to enhance the learning process; and the sample set must display adequate diversity to reduce any potential bias towards overly specific patterns in the data.

To construct S, `Mateen` introduces a new selection method tailored to fulfill these requirements. The intuition is illustrated in Figure 3.4 and the procedure is sketched in Algorithm 1. As an initial step, we partition B into mini-batches MB_n of size ρ (step ③ in Figure 3.4). Specifically, we first calculate the MSE score for every individual sample in B. Following this, we arrange the samples in ascending order according to their MSE scores. Then, we distribute these ordered samples into the mini-batches MB_n in a sequential manner. By selecting samples from each mini-batch, we ensure comprehensive representation across the entire distribution of B (Figure 3.4, ④).

To perform selection from each mini-batch, we compute two key metrics: informativeness ($\hat{I}$) and uniqueness ($\hat{U}$), as outlined in Algorithm 1. Informativeness is designed to measure a sample's value to the overall learning process, indicating that a highly informative sample is expected to significantly decrease the model's error. Uniqueness assesses how distinct a sample is relative to others, with a unique sample potentially introducing novel and less common patterns from B. In the rest of this Section, we detail our approach to calculate $\hat{I}$ and $\hat{U}$, and the criteria for selecting samples based on these scores.

INFORMATIVENESS. This metric is designed to meet the first criterion: the identification of informative samples that contribute to a reduction in model error. Since we lack access to the actual labels of the entire batch, we cannot directly measure model error. Instead, we assess the informativeness of samples through a distance-based measure. The underlying assumption here is that samples that are more similar to a larger group of other samples are more informative because they represent a greater portion of the data. However, relying solely on similarity can be problematic as it is sensitive to noise and may lead to over-selecting samples from similar patterns, thus missing out on diverse, yet informative samples from other patterns.

To address this, we use a selective method to ensure a broad representation of informative samples. We iteratively select the first sample from each distinct pattern, based on its temporal order, while removing samples that are too similar to it beyond a specific threshold. Consequently, a sample is deemed informative if its selection results in the removal of many other similar samples.

Setting the threshold can be challenging due to the variability in similarity levels across mini-batches. Therefore, we employ a binary search algorithm to determine the threshold that allows us to select a specific percentage, σ, of samples from each mini-batch MB_j, thereby creating a condensed mini-batch MB_j^-. The choice of σ is flexible, tailored to the user's needs, and can be adjusted based on the desired balance between redundancy and the degree of similarity among the

Algorithm 1 Pseudocode for `Mateen`'s selection method.

Input: a batch of data B, a model M, the size of minibatch ρ, the retention rate σ, and the selection rate δ.
Output: The subset S of B to be labelled.

function MATEENSELECTOR(B, M, ρ, σ, δ)
  ▷ Initialise S
  $S \leftarrow$ empty list
  ▷ Distribute B into bins of size ρ
  $IDX_{batch} \leftarrow$ DATATOBINS(M, B, ρ)
  ▷ Iterate over each bin index j in IDX_{batch}
  for $j \in IDX_{batch}$ **do**
    ▷ Get the minibatch MB_j from B using the index j
    $MB_j \leftarrow B[j]$
    ▷ Calculate the number of samples to select from MB_j
    $N \leftarrow \lfloor \delta \times \rho \rfloor$
    ▷ Get MB_j^- of MB_j based on a retention rate σ
    ▷ Calculate informativeness scores I for samples in MB_j^-
    $MB_j^-, I \leftarrow$ INFORMATIVENESS(M, MB_j, σ)
    ▷ Calculate uniqueness scores U for samples in MB_j^-
    $U \leftarrow$ UNIQUENESS(M, MB_j^-)
    ▷ Normalize I and U using min-max scaling
    $\hat{I} \leftarrow \frac{I - \min(I)}{\max(I) - \min(I)}$
    $\hat{U} \leftarrow \frac{U - \min(U)}{\max(U) - \min(U)}$
    ▷ Calculate the combined weight W using $\hat{U}$ and $\hat{I}$
    $W \leftarrow \lambda_0 \cdot \hat{U} + \lambda_1 \cdot \hat{I}$
    ▷ Selects the samples with the highest values in W
    $IDX_{top} \leftarrow \{i \mid W[i] \in$ top N highest values in $W\}$
    $S \leftarrow S \cup \{MB_j^-[i] \mid i \in IDX_{top}\}$
  end for
  return S
end function

data in the given network.

For a sample s, let $\phi(s)$ be its reconstruction via $AE(s)$, and let $\varepsilon(s)$ be its feature error vector, where each component $\varepsilon_j(s)$ is computed as $(s_j - \text{AE}(s)_j)^2$ for each feature index j. Given two samples s_1 and s_2, our similarity measure is the Euclidean distance

$$||\phi(s_1)::\varepsilon(s_1) - \phi(s_2)::\varepsilon(s_2)||^2 \tag{3.3}$$

where :: denotes vector concatenation.

We include the feature error vector ε term because of the dynamics of our ensemble update process, where we repeatedly train our incremental model (*INC*) on a growing, large dataset, and train the temporary models (T) on small sets of samples. This could lead to the latent representations, or the reconstructions ϕ, of different inputs to be too similar to each other [206, 58]. In such cases, ε serves as a critical measure, capturing the genuine discrepancy between the feature space of a sample, s, and its reconstructed form via AE(s).

UNIQUENESS. This metric is crafted to fulfill the second criterion: identifying a diverse set of samples to ensure that the model trained on this set is not biased towards specific data patterns. Given the samples within MB_j^-, we compute a uniqueness value, u, for each sample, s, as follows:

$$u(s, MB_j^-) = \sum_{s' \in MB_j^- \setminus \{s\}} ||\varepsilon(s) - \varepsilon(s')|| \tag{3.4}$$

Here, u represents the cumulative distance of sample s to all other samples in MB_j^-. A higher u score indicates that the sample is distinct compared to others in the batch.

Selection. We derive a vector U, where each element corresponds to the u value for each s within MB_j^-. Similarly, we compute a vector I of informativeness values. Each value in I indicates the number of samples excluded by INFORMATIVENESS due to their similarity or redundancy with the corresponding sample in MB_j^-.

Both vectors U and I are normalized using min-max scaling to obtain $\hat{U}$ and $\hat{I}$. The final weights for each sample in a condensed mini-batch are calculated using the formula $\lambda_0\hat{U} + \lambda_1\hat{I}$, where λ_0 and λ_1 are hyperparameters that adjust the balance between uniqueness and informativeness. Samples with the highest weights from each minibatch are chosen based on a user-specified selection rate (δ), resulting in a subset S that represents the entire distribution of B.

3.3.5 Complexity Reduction

The proposed adaptive learning module in `Mateen` faces a key challenge: as time progresses, the number of models in the ensemble increases. On the one hand, discarding models risks losing valuable insights; on the other hand, retaining all models leads to increased complexity. Hence, a balanced approach is necessary. We, therefore, propose a dynamic scheme that balances the retention of valuable knowledge against the escalating complexity through selective model merging and elimination.

Suppose the number of models reaches a critical threshold. In that case, we reduce the models' number to 3, keeping the incremental model INC_i, the temporary model T_i (trained on the most recent data S_b^i), and adding a new merged model $T_{\uplus}$.

To obtain $T_{\uplus}$, we start by evaluating the F1 scores of the remaining temporary models on the latest subset S_i, arranging these models in descending order based on their scores $(T^0, \ldots, T^k)$. We then proceed with a recursive merging process for those $n \leq k$ models whose performance is within 10% of the highest-scoring model, denoted T^0. This emphasis on F1 scores is due to the likelihood that models with similar performance metrics share akin parameter spaces and exhibit similar detection errors, rendering them nearly equivalent [184]. Thus, merging these models into a unified global model is expected to maintain overall performance on distributions learned from its constituent models and decrease redundancy within the pool [301, 151, 172].

The merging process employs the parameter averaging technique [172, 301], but adds a weighting mechanism. The merger is formalized by the recursive equations:

$$\begin{aligned} T_{\uplus}^0 &= T^0, \\ T_{\uplus}^{j+1} &= \alpha \cdot T_{\uplus}^j + (1-\alpha) \cdot T^{j+1}, \quad \text{for } j = 0, 1, \ldots, n-1 \end{aligned} \tag{3.5}$$

Here, α is the weighting coefficient, optimized to refine $T_{\uplus}$'s performance on the recent subset S_i, ensuring the global model effectively encapsulates the strengths of its components while being specifically attuned to the latest distribution. To find the best α, we employ a direct empirical method: testing values from 0 to 1 in 0.01 increments. For each value, we calculate the F1 score against the subset S_i. The α that delivers the highest F1 score is selected.

The models which fall outside the 10% performance margin are eliminated. While this might be perceived as a loss of valuable knowledge, the incremental model INC_i still retains insights gained from data corresponding to the removed models.

3.4 Experimental Setup

We evaluated `Mateen` alongside four baselines on three public network datasets, each chosen to induce distinct distribution-shift scenarios. The rest of this section explains our experimental setup in detail.

3.4.1 Datasets

We conduct our experiments on the three public datasets already described in §2.3: Kitsune [177], IDS17 [77], and IDS18 [156]. The Kitsune dataset consists of nine independent packet-capture files, each containing a specific attack scenario along with background benign traffic collected in its own network setup [232]. Since the captures were obtained separately, the benign distributions differ across files, and each file reflects a distinct combination of network environment and attack vector. As a result, models trained on one capture often generalise poorly to others, reflecting distributional shifts in both normal and malicious behaviours. The IDS17 dataset features a mix of both benign and malicious traffic gathered from the same network over five days. The IDS18

dataset provides a mix of traffic captured over three weeks from a larger network. IDS17 and IDS18 exhibit covariate shifts in benign traffic, marked by gradual and subtle changes in traffic properties over time, and concept shifts, defined by the sudden appearance of new attack patterns as time evolves.

Data Split. We divided each dataset into chronologically ordered training and testing sets, adopting time-aware splits to mitigate bias and emulate real-world deployment, as recommended by [199, 22]. Kitsune's feature files lack timestamps and the dataset is designed for per-file evaluation; thus, we train on the first file (ARP MitM) and test on the remaining ones. For IDS17, the first two days serve as the training set and the next three days as the test set, while for IDS18 we use the first week for training and the subsequent week for testing. Each test split is then processed sequentially in batches of 50,000 samples, following [18, 107]. Because the benign-to-malicious proportion fluctuates across these batches, each dataset naturally exhibits a ratio shift. This chronological batching can also split a single attack across consecutive batches, making B_{i-1} and B_i dependent. If an attack spans both, adapting on B_{i-1} may overstate performance on B_i because B_i continues behaviour already seen during adaptation; conversely, splitting can fragment the attack signature, so the model may need multiple batches to observe the full pattern.

Shift Scenarios. The testing sets illustrate two types of benign shift: covariate and ratio shifts, as seen in IDS17 and IDS18, and concept and ratio shifts, as seen in Kitsune. We also created additional simulated shift scenarios. From IDS18 (covariate shift), we removed the effect of ratio shift to focus solely on analyzing the impact of covariate shift. Specifically, we first calculate the benign-to-malicious sample ratio within the entire testing set. Subsequently, the testing set is partitioned into sequential batches, each comprising 50,000 samples, while maintaining the same ratio. For example, given an 80:20 benign-to-malicious ratio, each batch is formed by sequentially selecting 40,000 benign and 10,000 malicious samples. The first batch includes the initial benign and malicious samples as per the ratio, the second batch comprises the next sequence of benign and malicious samples, and this pattern continues with each batch until the testing set is fully segmented.

From Kitsune, we created a scenario maintaining a constant malicious-to-benign ratio (mKitsune), and another designed to replicate a recurring concept (rKitsune). For mKitsune, we adjusted the Kitsune testing set to keep a consistent ratio, distributing attacks across batches akin to our IDS18 strategy; this let us compare Kitsune and mKitsune results and isolate the impact of ratio shift on adaptation performance. For rKitsune, we created a test set by mixing ARP MitM and Active Wiretap files, interspersing two Active Wiretap batches after every four ARP MitM batches, aiming to test the frameworks' ability to maintain old valid distributions (ARP MitM file) while adapting to new ones (Active Wiretap file). Table 3.1 presents the statistical breakdown of the datasets post-processing, while Appendix A.1 offers further details on the datasets and their processing steps.

Table 3.1: Statistical information of the datasets.

Dataset	Train	Test	
	# Samples	# Samples	# Batches
IDS17	693K	1.4M	29
IDS18	2.5M	7.5M	151
Kitsune	751K	5.3M	107
mKitsune	751K	5.3M	107
rKitsune	751K	1.7M	35

3.4.2 Baselines

We compare `Mateen` with four baselines. The *first* baseline, `No-Update`, is an offline one-class AE trained solely on benign data from the training set; it performs no updates and thus cannot handle distribution shifts. The *second* baseline, `OWAD` [107], is the SoTA security framework that first detects benign distribution shifts and then adapts the underlying anomaly-detection models accordingly. It maintains a fixed-size memory that is periodically refreshed: when a shift is detected, outdated training samples are replaced with a subset of newly observed benign samples selected according to a predefined selection rate, and the model is then retrained on this updated memory. To prevent catastrophic forgetting, `OWAD` incorporates a customised version of `UNLEARN` [74], freezing parameters crucial to earlier valid distributions while allowing less critical ones to adapt.

The *third* baseline, `INSOMNIA` [18], is a supervised framework designed to adapt binary NIDS to distribution shifts. It does not include an explicit shift-detection step; instead, the model is updated after every incoming batch. Using uncertainty sampling, `INSOMNIA` selects the least-confident samples for labelling and then incorporates them into the training set for incremental retraining. The *fourth* baseline, `CADE`, is the SoTA in novel class detection and sample selection for multi-class security tasks. `CADE` employs supervised contrastive loss [105] to pull together samples sharing the same label and push apart those with different labels. After training, samples farthest from all label clusters are flagged as shifted, with the most distant ones chosen for labelling. We use `CADE` solely to compare its selection strategy with ours, as its primary focus is supervised shift detection and sample selection.

Training and Updates. All models are given the same training set and batches of testing data but vary in their training and sample selection strategies for updates. Each adaptive framework utilises a unique method to select an identical number of samples (δ), forming a subset that is then labelled by humans, and used for updates. `OWAD` and `Mateen` initialize with the `No-Update` model, which is exclusively trained on benign samples from the training set. Subsequently, they update their underlying AE to address benign shifts, using only the benign samples from the selected subset. `INSOMNIA` and `CADE`, as supervised frameworks, are trained on the complete training set, covering both malicious and benign samples. Their updates involve the full selected subset, integrating samples from both classes. For a detailed description of the baselines, the hyperparameter search strategy, and the hyperparameters used, please refer to Appendix A.2.

3.4.3 Evaluation Metrics.

We set the threshold for the AE-based models at the 95th percentile and computed three key metrics at this threshold: F1-Score, the harmonic mean of precision and recall; Macro F1-Score, averaging the F1-Scores for each class; and Accuracy, the proportion of correct predictions. To mitigate potential bias from the threshold setting, we also included the AUC-ROC as a metric, assessing the model's ability to differentiate between classes across various thresholds.

For the threshold-dependent metrics, we sequentially combined predictions from all batches into a single file. Next, we directly compared this consolidated predictions file with the actual labels to calculate the metrics. Concurrently, the AUC-ROC was calculated for each data batch, and the overall score was derived by averaging these individual batch scores.

It should be noted that in our process, after predicting a batch of data with a model, we stored these initial predictions for metrics calculation. If a shift was detected within this batch, we updated the model accordingly. However, we did not reapply the updated model to the same batch of data for a second round of predictions. This procedure ensures that our evaluation remains consistent with the principles of the test-then-train scheme [87].

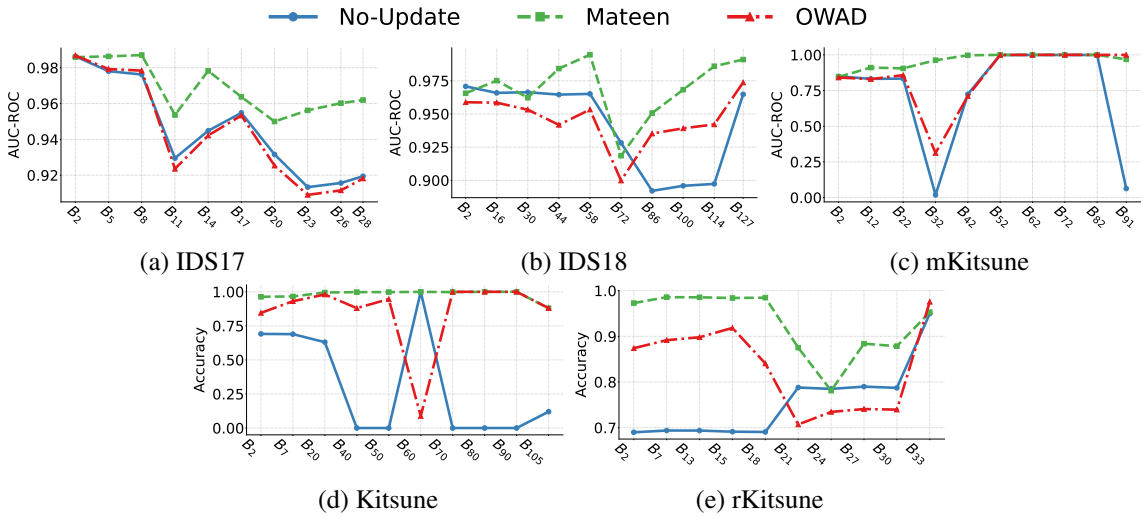
3.4.4 Computing Platform

The experiments were conducted on a Linux-based system (Ubuntu 22.04.3 LTS) with an Intel Xeon Processor (Skylake), featuring 16 cores across two NUMA nodes. The system was equipped with dual NVIDIA Tesla T4 GPUs, each offering 16 GB of memory, and was supported by 128 GB of RAM. All frameworks have been implemented using PyTorch version 2.0.1 [197], utilising CUDA version 12.2 and operating on Python 3.11.3.

3.5 Evaluation Results

Our experiments encompass five key elements, each addressing a distinct aspect of our framework's performance and characteristics:

- **End-to-End Performance** (§3.5.1): We examine the end-to-end performance of `Mateen` alongside adaptive learning baselines under diverse distribution shift conditions and varying selection rates (δ).
- **Learning Under Latency** (§3.5.2): We examine the effects of update latency and restricted sample size on adaptation performance.
- **Learning Under Label Noise** (§3.5.3) We assess the impact of mislabelling the selected samples on adaptation performance.
- **Sub-Component Analysis** (§3.5.4): We perform an ablation study to assess the impact of our proposed adaptive learning strategy, complexity reduction module, and selection method.



(a) IDS17 (b) IDS18 (c) mKitsune

(d) Kitsune (e) rKitsune

Figure 3.5: Adaptation performance over B_n.

Table 3.2: End-to-end adaptation performance under covariate shift.

Framework	F1-Score	mF1-score	Accuracy	AUC-ROC
IDS17 (Covariate & Prior)				
`No-Update`	88.09	80.53	83.46	94.56
`INSOMNIA`	78.45	49.99	64.55	49.76
`OWAD`	85.10	71.59	78.01	94.40
`Mateen`	**92.22**	**88.48**	**89.69**	**96.79**
IDS18 (Covariate & Fixed Ratio)				
`No-Update`	94.74	78.68	90.78	93.81
`INSOMNIA`	96.92	84.56	94.46	78.32
`OWAD`	85.01	56.24	85.01	91.97
`Mateen`	**97.03**	**91.32**	**95.63**	**98.17**

- **Hyperparameter Impact** (§A.2.4): We analyze the effects of varying hyperparameters, such as ρ, $\sigma\%$, maximum ensemble size, and λ_0, on `Mateen`'s performance.

3.5.1 End-to-End Performance

In this experiment, we evaluate the end-to-end anomaly-detection performance of `Mateen`. We compare `Mateen` with three baselines: (i) a static AE that is never updated after its initial pre-training (`No-update`); (ii) `OWAD`; and (iii) `INSOMNIA`. The static AE is the same model that serves as the starting point for both `OWAD` and `Mateen`.

We report two complementary results. First, we aggregate performance across all testing batches and summarise it in Tables 3.2, 3.3, and 3.4. Second, we visualize how the AE-based methods adapt over time by plotting their batch-wise scores in Figure 3.5.

During each update step, every method selects exactly 500 instances from the current batch B, corresponding to a fixed selection rate of $\delta = 1\%$. `Mateen` and `OWAD` perform an update only when a shift is detected, whereas `INSOMNIA` retrains on every incoming batch.

Table 3.3: End-to-end adaptation performance under concept shift.

Framework	F1-Score	mF1-score	Accuracy	AUC-ROC
Kitsune (Concept Shift & Prior)				
`No-Update`	24.50	27.72	27.86	63.98
`INSOMNIA`	87.23	51.38	77.81	47.31
`OWAD`	91.28	67.21	84.89	70.97
`Mateen`	**97.10**	**91.60**	**95.20**	**72.15**
mKitsune (Concept Shift & Fixed Ratio)				
`No-Update`	24.50	27.72	27.86	72.58
`INSOMNIA`	88.11	50.14	79.06	50.65
`OWAD`	93.59	81.82	89.44	79.98
`Mateen`	**95.86**	**87.41**	**93.09**	**94.76**

Covariate Shift Adaptation. The end-to-end adaptation results for the covariate-shift datasets IDS17 and IDS18 are shown in Table 3.2. Notably, `Mateen` outperforms all baseline models on both datasets across every metric. More importantly, relative to the static `No-Update` model, `Mateen` boosts the IDS17 F1-score by 4.13%, whereas `OWAD` reduces it by 2.99%. Figures 3.5a and 3.5b further illustrate that `OWAD` consistently underperforms `No-Update` across most batches, while `Mateen` achieves higher scores in the vast majority of batches.

We also note that `Mateen` markedly improves the mF1-score compared with all baselines, highlighting its ability to balance precision and recall across both benign and malicious classes. For example, on IDS17, `Mateen` attains an mF1-score of 88.48, surpassing the best baseline (`No-Update`) by 7.95%. By contrast, the other baselines exhibit a bias toward the dominant benign class: they record high overall F1-scores but mF1-scores more than 10% lower, indicating that many malicious samples are misclassified as benign.

To understand these outcomes, we examined the design choices of each framework. Both `OWAD` and `INSOMNIA` use incremental updates, merging selected samples from the current batch with historical data before retraining. However, `OWAD` employs a memory-based strategy that discards some historical samples to accommodate new ones. This can hurt performance for two reasons: (1) historical samples may still be relevant to future batches, so removing them loses valuable information; and (2) mitigating covariate shift is more effective when the model trains on the full set of historical plus new data, capturing patterns that represent both past and current distributions.

`INSOMNIA`, on the other hand, is constrained by its binary supervised framework. Unlike `OWAD` and `Mateen`, it must adapt to shifts in both malicious and benign classes, and the malicious class exhibits concept shift in both datasets. Consequently, its performance depends heavily on how attacks are represented. `INSOMNIA` excels on IDS18, where the attacks are semantically similar (*e.g.* FTP brute-force in training versus web brute-force in testing) and distributed across multiple batches. IDS17 is more challenging: the test attacks differ markedly from those in training (*e.g.* brute-force versus DDoS), and some appear in only a single batch, leading to poorer performance.

Table 3.4: End-to-end adaptation performance under recurring concept shift.

rKitsune (Recurring Concept Shift)				
Framework	F1-Score	mF1-score	Accuracy	AUC-ROC
`No-Update`	86.02	77.18	80.61	88.43
`INSOMNIA`	86.84	67.79	79.06	65.24
`OWAD`	91.09	82.89	86.82	87.14
`Mateen`	**94.52**	**89.63**	**91.93**	**93.47**

Concept Shift Adaptation. Table 3.3 presents the performance of `Mateen` and the baselines on the Kitsune and mKitsune datasets. `Mateen` demonstrates strong performance across both datasets, except the AUC-ROC score for the Kitsune dataset (only 1.18% higher than `OWAD`). The lower AUC-ROC score in Kitsune is due to its structure, which only includes two batches suitable for AUC-ROC calculation, as these are the only batches containing a mixture of benign and malicious instances.

We can also notice that `OWAD` and `Mateen` significantly enhance the evaluation metrics compared to the quickly outdated `No-Update` baseline, which suffers from concept shifts in benign traffic. Specifically, `OWAD` improves accuracy in most batches within the Kitsune dataset, as illustrated in Figure 3.5d. In this scenario, the memory-based approach proves beneficial, as it discards outdated concepts that no longer match the incoming data concepts. However, `Mateen` exhibits even stronger performance, surpassing `OWAD` in the majority of batches across both the mKitsune and Kitsune datasets, as evidenced in Figures 3.5d and 3.5c, respectively.

`INSOMNIA`, on the other hand, struggles with concept shifts, exhibiting a mF1-score around 50%. This issue originates from its incremental learning approach, which incorporates only a small number of samples (500 per batch) into the already large training set. As a result, the classifier is retrained predominantly with outdated, less relevant data and a minimal number of new distribution samples. This causes the model to focus on minimizing errors for the larger, outdated dataset, in alignment with empirical risk minimization principles.

Recurring Concept Adaptation. Table 3.4 and Figure 3.5e present the performance of `Mateen` and the baselines on the rKitsune dataset. We can notice that `Mateen` is outperforming the baselines, which suggests that `Mateen` can handle short concept shift data without sacrificing the performance on the consistent concept. For example, `Mateen`'s AUC-ROC score is 93.47, surpassing `OWAD`'s 87.14. More importantly, both `OWAD` and `Mateen` show improvements in evaluation metrics over the `No-Update`, although `OWAD`'s AUC-ROC score decreases slightly by 1.29%. However, it is crucial to note that the rKitsune dataset offers limited data (only 2 out of 35 batches) for AUC-ROC calculation. On the other hand, `INSOMNIA`, while generally less effective, demonstrates improved performance compared to its results on the Kitsune dataset. This improvement is attributed to its incremental learning method, which effectively keeps the classifier up-to-date with old concepts. However, its performance in adapting to concept shifts remains compromised.

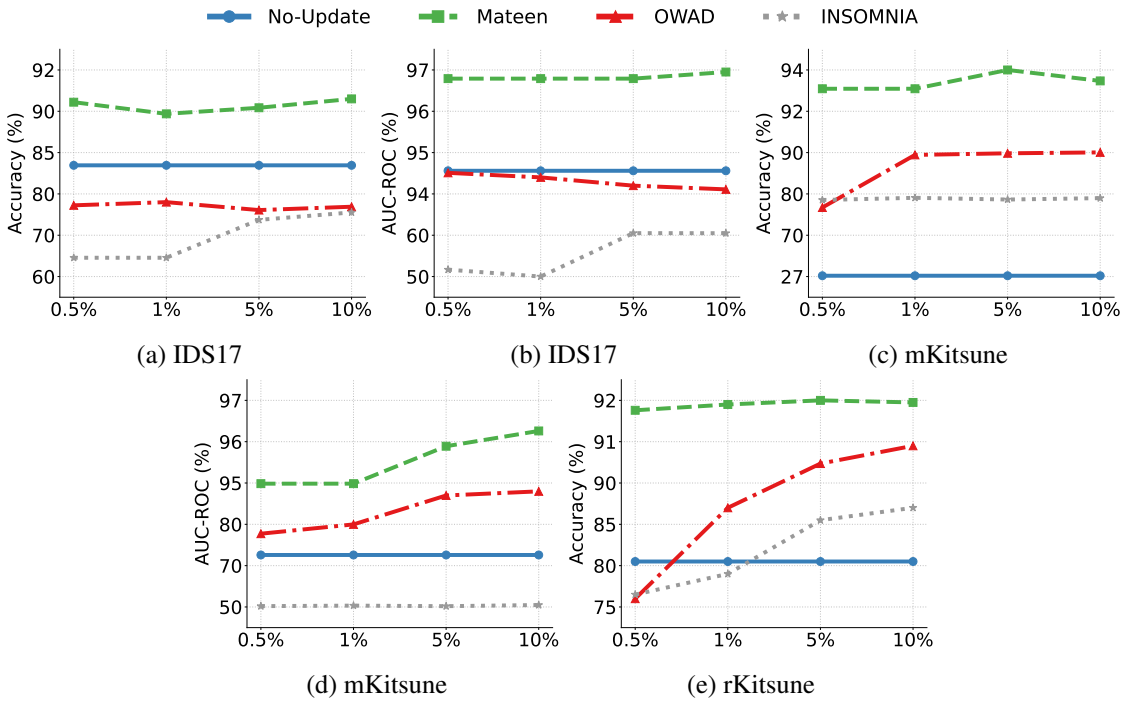


Figure 3.6: End-to-End adaptation performance with δ of 0.5%, 1%, 5%, and 10%.

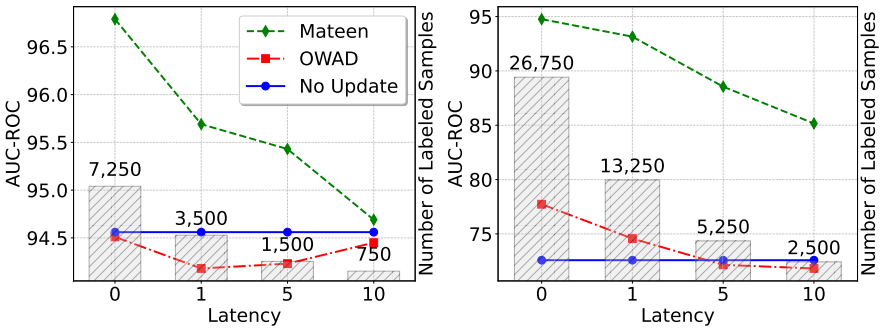
Selection Rate Impact. We here evaluate the effect of varying the δ value on the end-to-end adaptation results, utilising the IDS17, mKitsune, and rKitsune datasets with δ values of 0.5%, 1%, 5%, and 10%. These percentages correspond to the frameworks being updated with 250, 500, 2500, or 5000 samples per batch upon detecting shifts. We assessed the frameworks based on Accuracy and AUC-ROC scores, as illustrated in Figure 3.6.

We can observe three main findings. First, `Mateen` consistently outperforms all baseline frameworks across the entire range of δ values. Second, it exhibits strong performance even when only 0.5% of the samples from each shifted batch require labelling. Most notably, `Mateen`'s performance with a δ of 0.5% surpasses that of all baseline models across their full spectrum of δ values. This indicates that `Mateen` can achieve superior performance with fewer required labels. For example, in the mKitsune dataset, `Mateen` achieves an accuracy of 93.09% with only 250 labelled samples (equivalent to a δ of 0.5%), while `OWAD` and `INSOMNIA`, despite requiring 5000 labelled samples, attain accuracies of just 90.01% and 78.98%, respectively.

3.5.2 Learning Under Latency

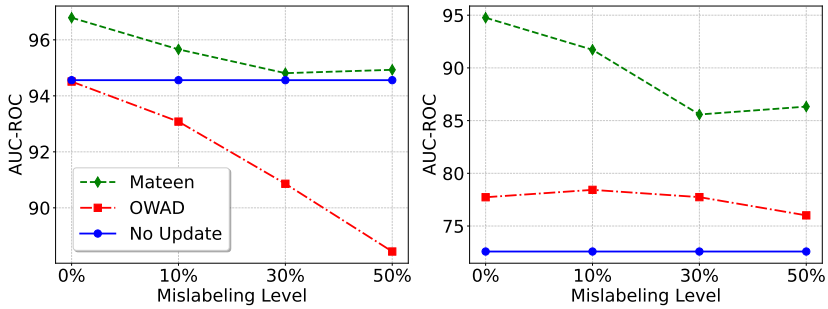
The previous experiments do not consider latency and assume that ground truth labels are immediately available and the model updates instantaneously. In reality, both manual labelling and model updates will introduce delays, and new samples continue to arrive before the update is completed. To more accurately simulate these real-world conditions, we introduce latency and a limited labelling budget into the process.

Assume updating a model takes as long as accumulating n batches of data. When a distribution shift is detected within a batch B_j, we continue using the current model M_k to classify the next



(a) IDS17 (b) mKitsune

Figure 3.7: The impact of latency and limited sample selection on performance.



(a) IDS17 (b) mKitsune

Figure 3.8: The impact of mislabelling on performance.

n batches, and in the meantime we create the updated model M_{k+1}, which will replace M_k for B_{j+n}. For updates, we use a selection rate of 0.5%, which equates to selecting only 250 samples for manual labelling and updating. Once updated, the processed batches (n batches) are excluded from the shift detection and sample selection steps. We test this setup with n set at 1, 5, and 10. For the entire testing set of IDS17, this configuration requires analysts to label 3,500 samples for n=1, 1,500 for n=5, and 750 for n=10.

Figure 3.7 shows the results of this experiment using IDS17 and mKitsune datasets. Generally, as expected, `Mateen`'s performance declines with increasing delays. However, it is noteworthy that even with significant delays (i.e., n=10) and a very low number of manually labelled samples (approximately 0.05% of the testing set), `Mateen` still outperforms the baseline models. Conversely, `OWAD` shows improved performance with delays in the IDS17 dataset, benefiting from retaining old, still-valid samples.

3.5.3 Learning Under Label Noise

Security analysts can mistakenly mislabel samples. To simulate this, we randomly changed the ground-truth labels for 10%, 30%, or 50% of the samples in each selected subset, while the rest remained correctly labelled. We then used these subsets, containing both accurate and inaccurate labels, to update the model and evaluate its performance under label noise conditions. Figure 3.8 presents the results for `Mateen` and `OWAD` on the IDS17 and mKitsune datasets. The performance of the non-updated model (`No-Update`) is also included, as it does not undergo updates and is thus

Table 3.5: Comparison of the ensemble components.

Method	F1-Score	mF1-score	Accuracy	AUC-ROC
IDS17 (Covariate & Prior)				
INC	**92.73**	**89.03**	**90.48**	96.69
T	78.32	50.28	66.1	65.29
No Transfer	91.31	87.04	88.45	96.51
No Merge	92.43	88.75	89.96	96.61
`Mateen`	92.22	88.48	89.69	**96.79**
mKitsune (Concept Shift & Fixed Ratio)				
INC	24.03	27.69	27.88	73.9
T	79.86	57.6	69.29	78.3
No Transfer	79.95	59.23	69.77	82.95
No Merge	95.5	86.72	92.53	94.01
`Mateen`	**95.86**	**87.41**	**93.09**	**94.76**

unaffected by mislabelling. This shows whether noisy updates cause the frameworks to perform worse than their non-updated counterpart. On the IDS17 dataset, `Mateen`'s performance gradually declines as noise increases, yet it still outperforms both `OWAD` and the non-updated model. In contrast, `OWAD` shows a significant decrease in performance with increasing label noise. On the mKitsune dataset, despite the noise, both `Mateen` and `OWAD` are still better than the non-updated model. However, `OWAD`'s performance is particularly affected by the highest noise level (50%). Overall, `Mateen` proves to be the most robust under label noise conditions.

3.5.4 Sub-Component Analysis

We undertook ablation studies to dissect and assess the individual contributions of `Mateen`'s components. Our focus was particularly on evaluating the effects of the adaptive learning component and the efficacy of our proposed selection method.

Adaptive Learning Strategies. We evaluate the proposed adaptive learning strategy by contrasting `Mateen` with four setups: an incrementally updated AE (*INC*), solely temporary models (T), `Mateen` without its transfer learning component (No Transfer), and `Mateen` without its merging component (No Merge). The No Merge setup specifically excludes `Mateen`'s complexity reduction module. Comparative results for the IDS17 and mKitsune datasets are detailed in Table 3.5.

We can notice three important observations. Firstly, `Mateen` demonstrates consistently high performance across both datasets, though the efficacy of its integral components, *INC* and T, varies with the type of data shift. Specifically, *INC*'s performance on the covariate shift is marginally better than `Mateen`'s overall performance; however, it is significantly worse than all baselines on the concept shift. This discrepancy arises because the *INC* model utilises its large and growing dataset to learn patterns more effectively in the covariate shift scenario. Conversely, this extensive dataset hampers its adaptability in the concept shift scenario, as acquiring representative patterns of diverse concepts proves challenging. On the other hand, the temporary models, T, underperform relative to most baselines in both datasets. This underperformance is attributed to their limited training data, which compromises their ability to learn the task adequately. Specifically,

Table 3.6: Comparison of the selection methods.

Method	F1-Score	mF1-score	Accuracy	AUC-ROC
IDS17 (Covariate & Prior)				
HRE	83.64	68.97	75.91	79.29
CADE	91.1	86.6	88.11	95.21
OWAD	91.56	87.27	88.71	96.5
Mateen	**92.22**	**88.48**	**89.69**	**96.79**
mKitsune (Concept Shift & Fixed Ratio)				
HRE	90.82	76.9	85.29	78.9
CADE	81.54	66.31	73.2	87.28
OWAD	95.29	**87.45**	92.35	93.85
Mateen	**95.86**	87.41	**93.09**	**94.76**

each model is provided with only 500 labelled samples (a δ of 1%), an amount insufficient to accurately represent the task's distribution, thereby predisposing the models to overfit.

Secondly, the transfer learning component significantly influences the effectiveness of learning. The results clearly demonstrate that models lacking transfer learning capabilities underperform markedly on the concept shift dataset compared to the complete Mateen framework, underscoring the vital role of transfer learning in improving model generalisability. However, this benefit does not apply to the covariate shift dataset. This is primarily because our best-performing model selection strategy tends to favor the *INC* model over the T model for most batches, owing to the low performance of the latter. Thirdly, Mateen's performance closely matches that of the No-Merge approach, indicating that our complexity reduction module is key to both preserving an optimal ensemble size and retaining essential knowledge. This suggests the module's effectiveness extends beyond simple size management to ensuring that critical insights are not sacrificed for efficiency.

Selection Method Comparison. In this experiment, we replace Mateen's sample-selection mechanism with three baseline strategies: CADE, High Reconstruction Error (HRE), and OWAD, to assess their effect on Mateen's adaptive ensemble. CADE and HRE both target the most uncertain samples for updates, differing in their strategies for assessing uncertainty. CADE, specifically, utilises a contrastive learning approach during training and selects samples most distant from the centroids of the training classes for labelling. HRE chooses samples with the largest reconstruction errors. Conversely, OWAD operates at a distribution level, comparing reconstruction error scores between the training data and the incoming batch data distributions. It selects samples from the batch of data that show significant deviations from the training distribution.

Table 3.6 presents the outcomes of this comparison for the IDS17 and mKitsune datasets. The HRE strategy emerges as the least effective, primarily because it selects samples with high MSE scores, which are predominantly malicious. This selection hinders the adaptive learning approach's capacity to adapt to new distributions and adversely affects the identification of the best-performing model. However, CADE demonstrates better performance compared to HRE, despite being a form of uncertainty sampling. Our analysis indicates that CADE's contrastive loss results in both benign

and malicious shifted samples being distanced from the training data, facilitating the selection of shifted samples from both classes. Nonetheless, the Mateen and OWAD selection methods exhibit significantly superior results, particularly in the mKitsune dataset, with Mateen slightly outperforming OWAD. For instance, in the mKitsune dataset, Mateen, OWAD, and CADE achieve AUC-ROC scores of 94.76, 93.85, and 87.28, respectively.

3.6 Discussion and Future Work

Hyper-parameter sensitivity. We conducted a sensitivity analysis of hyperparameters and have documented the default configuration of Mateen, along with the configurations of the considered baselines, in Appendix A.2. Through our extensive analysis, we discovered that Mateen consistently delivers robust performance across a range of hyperparameter combinations. For example, different hyperparameter combinations for IDS17 resulted in an average of 96.19 and a standard deviation of 0.64 concerning AUC-ROC scores. This observation suggests that Mateen can achieve strong results even when the optimal hyperparameters are not precisely identified. We also provide guidance on how to set these values and the factors that need to be considered in Appendix A.3.

Storage and Memory. Mateen must store both the ensemble and an ever-growing training set. To manage the dataset, a promising avenue is supervised coreset selection: choosing a compact yet representative subset that preserves performance—an idea we leave for future work. The ensemble itself is capped at a tunable maximum size, and our sensitivity analysis shows that fewer than seven members are sufficient (Appendix A.2). Despite this cap, Mateen still imposes memory overhead: each update must (i) evaluate every ensemble member, (ii) incrementally train the base model, (iii) trains a temporary model, and (iv) merges models when appropriate. We view this overhead as the price of Mateen's performance gains, but future work will explore alternative update rules to further cut these costs.

Costs of Manual labelling. Mateen adopts the most dependable updating strategy by using only a few labelled samples, a method well-regarded in security contexts [295, 107, 48, 276, 142]. However, this presents a significant challenge in highly dynamic networks where labelling even a small number of samples can become prohibitively expensive due to frequent shifts occurring in short spans of time. Recent research efforts have investigated learning without reliance on ground-truth labels, employing label estimators instead [18, 291]. Nonetheless, this method has been found to inadvertently lead to self-poisoning of models, as inaccurately labelled samples can degrade model performance [129]. This issue arises from the direct use of estimated labels for model updates without verifying their accuracy. Consequently, our future work aims to investigate learning from high-quality estimated labels, seeking methods to ensure the reliability of these labels before model integration.

Latency and Mislabelling. Manual labelling introduces two key challenges: latency and error risk. Latency occurs because analysts need time to manually label samples, and during this period, the non-updated ensemble continues to make predictions, which may not be accurate due

to shifts in data. Additionally, there is a risk of error, as analysts might mislabel some samples. These inaccuracies can lead to updates with incorrect data, resulting in degraded performance of the ensemble. Despite these challenges, our findings indicate that even with high latency and a significant level of mislabelling, `Mateen` still outperforms both the best baseline model and the non-updated version. In future work, we aim to further reduce the impact of latency and error risks in zero-positive learning settings. To achieve this, we plan to investigate three interconnected research areas: label estimation [18, 129, 291], learning from noisy data [286, 277, 82], and advanced labelling strategies [102].

Adversarial Machine Learning. `Mateen` is purpose-built to effectively handle benign shifts and improve the AE's performance. This design incorporates an updating mechanism that carefully selects samples for human labelling. However, this mechanism is a potential vector for poisoning the ensemble [139, 286, 296]. Analysts might inadvertently mislabel non-poisonous samples, as previously mentioned, or attackers could introduce poisonous samples, such as through label flipping and backdoor attacks [229, 259, 296], to deceive analysts and compromise the ensemble. Therefore, in our future research, we plan to investigate the effects of advanced poisoning attacks and explore defensive strategies, such as robust learning [212, 165, 157], to mitigate their impact. Additionally, `Mateen` may be vulnerable to evasive samples [143, 109, 113, 308], where adversaries attempt to disguise attacks as benign behaviours to evade detection. However, this susceptibility depends on the design of the underlying AE and its associated feature space. In future work, we plan to investigate the interplay between feature space, AE architecture, and `Mateen` when confronted with evasive inputs, considering the potential impact on anomaly detection accuracy and robustness.

3.7 Conclusion

In this chapter, we present `Mateen`, a framework designed for detecting and adapting to benign shifts within one-class AE-based NIDS. `Mateen` utilises a hybrid ensemble learning strategy incorporating an incremental model that is capable of adjusting to minor shifts (covariate shifts), alongside temporary models designed to tackle major shifts (concept shifts). To minimise annotation cost, the framework couples transfer learning with a coreset-inspired sampling mechanism: each incoming batch is stratified according to ensemble uncertainty, after which a subset that balances informativeness and uniqueness is selected for labelling. Redundant ensemble members are removed by a complexity reduction module, thereby preserving computational efficiency without impairing detection performance.

We evaluate `Mateen` on three public intrusion-detection datasets and two enhanced variants, and observe SoTA performance across all of them. Relative to the strongest baselines, `Mateen` raises the mF1-score by 7.95% on IDS17, 6.76 on IDS18, 24.39 on Kitsune, 5.59 on mKitsune and 6.74 on rKitsune. A component analysis shows that, on the mKitsune concept shift dataset, the full ensemble improves AUC-ROC by 20.86% over the incremental-only baseline and by 16.46% over the batch-only temporary model. The coreset-based sampler likewise outperforms conventional uncertainty sampling, boosting AUC-ROC by 15.86% when guided by reconstruction error and

by 7.48% when guided by contrastive distance. We further examine `Mateen` under realistic operational constraints—varying selection rates, update latency and up to 50% label noise. Although performance degrades under these harsher conditions, the framework consistently maintains a clear margin over the non-updated baseline.

These findings demonstrate that `Mateen` substantially strengthens the performance of one-class anomaly detection systems in dynamic environments, achieving SoTA results across multiple datasets. Nevertheless, `Mateen` has limitations that highlight promising directions for future research.

First, its adaptation strategy still relies on incremental learning, which requires storing an expanding training set and allocating memory for repeated updates. A promising alternative would be to incorporate coreset selection, which can reduce the training set size for incremental learning while maintaining near-optimal performance. Second, the framework continues to depend on manual annotation. Even with careful subset selection, this remains costly, time-consuming, and prone to errors. Future research could explore self-updating mechanisms that combine automatic label estimation with robust learning strategies to enable autonomous updates while reducing label noise. Finally, `Mateen` assumes a non-adversarial setting, leaving resilience against adversarial machine learning attacks to the underlying AE and its feature space. Future work should investigate the impact of adversarial inputs on different AE architectures and representations, and explore defences that preserve robustness under adversarial conditions.

Chapter 4

Rasd: Novel Class Detection and Adaptation

In the previous chapter, we introduced Mateen, a system that adapts to benign shifts to preserve accurate detection of malicious traffic. While such adaptation ensures reliable anomaly detection, suspicious traffic may then be forwarded to a downstream classifier to determine the specific attack type. This raises a new challenge: distribution shift in multi-class DNN-based NIDS, particularly when novel attack classes emerge that were absent during training. Existing classifiers typically misclassify these unseen attacks into known classes, leading to unreliable predictions. In this chapter, we address this problem with Rasd, a framework that detects previously unseen attacks and enables multi-class NIDS to adapt to them. This work has been published as:

Fahad Alotaibi and Sergio Maffeis, "**Rasd: Semantic Shift Detection and Adaptation for Network Intrusion Detection**." In *Proceedings of the IFIP International Conference on ICT Systems Security and Privacy Protection (IFIP SEC) 2024*.
DOI: 10.1007/978-3-031-65175-5_2

4.1 Introduction

Once an anomaly detector flags suspicious traffic, the next step is to determine the specific attack responsible. Since manually inspecting individual flows or packets is prohibitively time-consuming and resource-intensive, a natural design is to route flagged traffic to an automated classifier (*e.g.*, a multi-class DNN). Such a classifier, trained jointly on multiple attack classes alongside a benign baseline, can rapidly categorise anomalous flows, distinguishing among diverse threats while also recognising normal traffic with minimal analyst effort. Remarkably, recent studies have reported classification performance exceeding 99% [6, 128, 221], leading to the suggestion that network-attack classification is, in effect, a solved problem.

However, this performance often does not hold up outside the lab. As discussed in §2.4, most evaluation protocols assume stationarity, treating training and testing traffic originating from the

same distribution. In practice, traffic is far from stationary: known attack types evolve, and entirely new types emerge. Our primary focus in this chapter is on the latter—the emergence of novel classes. When the classifier encounters a flow from an unseen class, it is still required to assign it to one of its training labels. In the best-case scenario, it misclassifies the traffic as another attack, leading to an inappropriate response. In the worst-case scenario, it labels the flow as benign, allowing the intrusion to go undetected. A practical, deployment-grade multi-class NIDS must therefore (i) detect the appearance of novel attack classes with minimal delay and high recall, and (ii) incorporate them into its model at modest manual labelling cost.

Challenges and Requirements. Meeting both objectives is challenging for two main reasons. First, deep networks frequently attach very high confidence to inputs that lie far outside their training manifold, so thresholds based on softmax scores miss many novelties [185]. As a result, raw classifier scores cannot be relied upon to detect novel samples. Second, even when all novelties are detected, the number of flagged samples can be enormous, making it infeasible to label every flow. For a system to be effective, it must reliably detect most novel samples while identifying only a small, representative subset for manual annotation. Crucially, this subset must include samples from every novel class, even the rarest, so that subsequent retraining equips the classifier to recognise the entire spectrum of new threats.

Our Method. To tackle both challenges, we propose `Rasd`, an auxiliary module that sits beside the classifier and combines high-recall detection of novel classes with data-efficient adaptation. During training, `Rasd` minimises a centroid-based loss that pulls each sample embedding toward its class centroid, while the centroids themselves are well separated, thereby creating low-density buffer regions around the known classes. These low-density regions are precisely where novel samples are expected to appear. At inference time, the module computes the distance of each incoming sample to all class centroids; if every distance exceeds its corresponding class-specific detection threshold, the sample is flagged as novel. The flagged stream is processed in temporal order, and `Rasd` selects a small subset that is both informative, drawn from dense portions of the detected samples, and diverse, drawn from sparse portions, for manual annotation. Once these samples are labelled, the classifier is incrementally retrained with the newly selected samples together with historical samples from the old classes, enabling it to incorporate novel classes without degrading performance on the original ones.

Evaluation. We evaluated `Rasd` on the IDS17 and CICIDS2018 datasets, two large-scale, time-stamped data sets designed to mimic the evolution of network attacks. We compared its detector with two baselines, including the SoTA `CADE`. `Rasd` detected 99.5% of novel samples on CICIDS2018, whereas the closest baseline detected only 62.21%. Training time on IDS17 was roughly 41 minutes for `Rasd`, compared with 1.2 hours for the closest baseline. We also compared `Rasd`'s sample-selection strategy with farthest-point sampling and least-confidence sampling. `Rasd` was the most effective: labelling only 1% of the novel samples on IDS17 improved the classifier's mF1 score by 26.18% over the strongest competing strategy.

Contributions. This chapter has three main contributions:

- We design a cost-effective, distance-based objective that extends center loss [280] to the novel class detection setting, facilitating high recall in identifying new classes.
- We propose an adaptation strategy that selects the most informative and diverse novel samples for manual labelling via Farthest-First Traversal (FFT) [218], with a user-controlled sampling rate that keeps annotation effort low.
- We integrate these components into the `Rasd` framework and demonstrate, on the IDS17 and CICIDS2018 datasets, that it surpasses SoTA baselines in detection performance, adaptation effectiveness, and training speed. We release `Rasd`'s code publicly[1] to enable reproducibility and further research.

4.2 Preliminaries

In this section, we outline two principles that underpin `Rasd`'s design. First, we review contrastive learning, with an emphasis on supervised objectives, and discuss their vulnerability to class imbalance as well as their high computational cost (§4.2.1). Second, we examine centroid-based objectives, focusing on center loss—the loss that `Rasd` extends to obtain memory-efficient training that is less sensitive to class imbalance (§4.2.2).

4.2.1 Contrastive Learning

Contrastive learning is a type of representation learning that focuses on learning robust representations by contrasting and comparing different examples [47, 243]. For a given anchor sample $\mathbf{x}_i$, which serves as the reference point in the comparison, the goal is to pull it closer to positive samples $\mathbf{x}_i^+$ that share semantic similarities with it, while pushing it away from negative samples $\mathbf{x}_i^-$. This approach can be utilised in both unsupervised and supervised contexts. In the unsupervised framework `SimCLR` [47], two augmentations of the same sample create a positive pair, allowing the model to learn without needing labels. In contrast, in supervised settings, labels guide the pairings, as in Dimensionality Reduction by Learning an Invariant Mapping (`DrLIM`) [105] and Lifted Structured Loss (`LSL`) [243]. Here, samples with the same label are considered positives, while those with different labels are considered negatives.

During training, an encoder g_θ, or a classifier along with a projection head h_ϕ[2], embeds each of the N samples in a mini-batch, and a contrastive loss such as `DrLIM`, or `LSL` is computed over those embeddings. `DrLIM` generates exactly one pair per anchor, meaning a mini-batch of size n results in only n pairs. On a heavily imbalanced NIDS dataset that might contain 10^6 benign samples but only 10^2 examples of a rare attack, most minority-class anchors may not be matched to a positive. Consequently, their representations tend to be shaped predominantly by the majority-class negatives, leading to fragmentation of the minority class in the latent space unless costly hard-positive mining and extensive over-sampling are employed. In contrast, `LSL` calculates all

[1] https://github.com/ICL-ml4csec/Rasd/

[2] A projection head is a small neural network (often a linear layer or a short MLP) attached on top of the encoder or classifier that maps intermediate representations into the embedding space on which the contrastive loss is computed; it is typically used only during contrastive training and is often discarded after training so downstream tasks use the encoder (or classifier) representations.

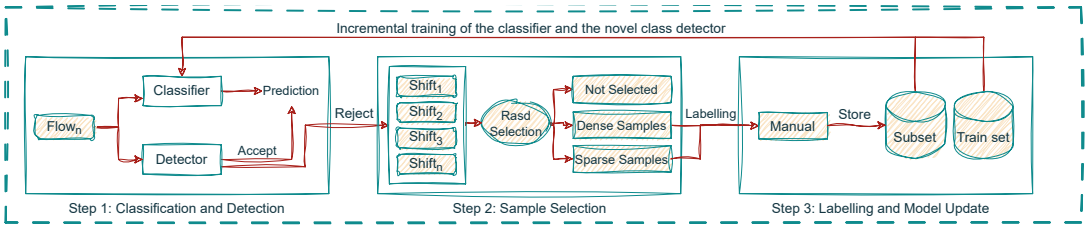


Figure 4.1: Overview of the Rasd framework.

$n(n-1)/2$ pairwise distances within the batch, enabling the few minority-class samples present to directly influence each other. However, this quadratic increase in pairs raises memory and computational demands. Additionally, the sheer volume of cross-class negative pairs still skews the gradient towards the majority class, potentially leaving rare classes inadequately separated without further re-weighting or class-balanced strategies.

4.2.2 Centroid-based Functions

The idea of summarising a class with a single centroid is one of the earliest concepts in Machine Learning (ML). In the unsupervised setting, K-Means clustering [166] repeats two steps: (i) assign every sample to its nearest centroid, and (ii) update each centroid to minimise the within-cluster sum of squared distances. Because every sample interacts with only one centroid, the algorithm is memory-light and its cost grows linearly with the data size.

A supervised analogue is the center loss of Wen *et al.* [280]. Given an encoder (or projection head) g_θ and a learnable centroid $\boldsymbol{c}_y$ for each class y, the loss

$$L_{\text{center}} = \frac{1}{2}\sum_{i=1}^{n} \|g_\theta(\mathbf{x}_i) - \boldsymbol{c}_{y_i}\|_2^2 \tag{4.1}$$

shrinks intra-class variance, while a concurrent cross-entropy term maintains inter-class separation. After each mini-batch, centroids corresponding to classes present in the batch are updated via a damped moving-average step:

$$\boldsymbol{c}_y \leftarrow \boldsymbol{c}_y - \alpha \frac{\sum_{i:y_i=y}\big(\boldsymbol{c}_y - g_\theta(\mathbf{x}_i)\big)}{1+N_y},$$

where N_y is the number of class-y samples in the batch and $\alpha \in (0,1]$ is a learning rate. Because a centroid is updated only when its class appears in the current mini-batch, minority-class centroids shift toward their embeddings far more slowly, whereas majority-class centroids are adjusted at every step and keep pace with their embeddings; simultaneously, the loss gradients are proportional to sample counts, so the model concentrates on minimising distances for abundant classes and exerts a weaker pull on the small ones, allowing their embeddings to remain comparatively dispersed. Despite this imbalance sensitivity, this loss remains memory- and computation-efficient, requiring just one centroid comparison per sample and scaling linearly with batch and dataset size.

4.3 The Rasd Framework

Rasd augments a multi-class NIDS classifier f_θ, trained on an initial labelled set $D_0 = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N_0}$, with three co-operating modules: (i) classification and shift detection, (ii) sample selection, and (iii) labelling and model update. Together these modules enable f_θ to recognise traffic from previously unseen classes and to adapt to them with minimal annotation effort. Figure 4.1 sketches the end-to-end workflow; the components are summarised below.

Classification and shift detection. An encoder g_θ, pre-trained on D_0 with the centroid-based Rasd loss, embeds every incoming sample $\mathbf{x}$ into a latent space. For each training class y we store a centroid $\mathbf{c}_y$ and a detection threshold t_y equal to the k-th percentile of the in-class training distances. At inference time we compute $d_y = \|g_\theta(\mathbf{x}) - \mathbf{c}_y\|_2$ for all y. If $d_y > t_y$ for every class, the sample is flagged as novel and placed in a streaming buffer U; otherwise the label produced by f_θ is accepted.

Sample selection. Buffered samples U arrive in temporal order and are grouped into mini-batches $MB_1, MB_2, \ldots$ of fixed size ρ. For every batch MB_n, and under the labelling budget δ, Rasd selects a small subset $S_n \subseteq MB_n$ that is both informative (dense regions) and diverse (sparse regions) for manual annotation.

Labelling and model update. The newly labelled subset S is merged with the historical dataset D_0. Both the classifier f_θ and the detector g_θ are then incrementally retrained on this enlarged dataset, completing the adaptation cycle and readying the system for the next round of classification and detection.

The remainder of this section describes the two main building blocks in detail: shift detection (§4.3.1) and shift adaptation (§4.3.2).

4.3.1 Shift Detection

Learning robust latent representations. Rasd's detector uses a centroid-based objective that combines the intuition of supervised contrastive learning with the efficiency of center loss. Traditional contrastive losses such as LSL compare every sample with all positives and negatives in the mini-batch, incurring $O(n^2)$ distance computations, where n is the batch size, and producing gradients dominated by majority classes. Center loss achieves linear complexity by summarising each class with a single centroid. Yet, centroids are only updated when their class appears in the batch, so under long-tailed data distributions, minority classes receive fewer updates and their embeddings remain less well aligned, while majority classes dominate the representation. Rasd preserves this linear complexity while reducing imbalance sensitivity: it fixes a synthetic centroid for every class before training and averages the per-class losses, ensuring that every class contributes equally regardless of its frequency. As Figure 4.2 shows, the encoder no longer contrasts cross-class pairs; instead, it minimises the distance between a sample and the fixed centroid of its own class.

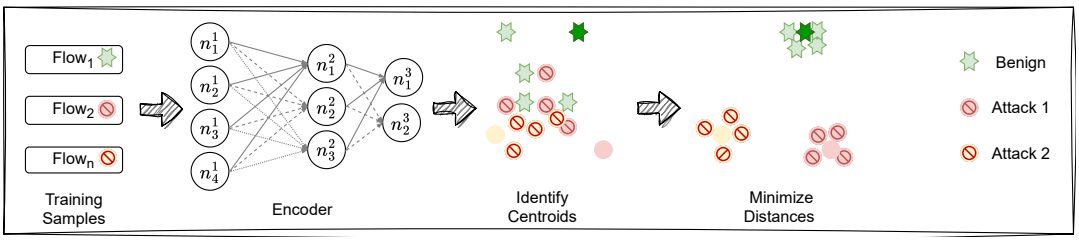


Figure 4.2: The high-level idea of `Rasd` loss function for shift detection.

Loss definition. An encoder g_θ maps a mini-batch of n network flows to latent vectors $Z = \{z_1, \ldots, z_n\}$ with corresponding labels $Y = \{y_1, \ldots, y_n\}$. Let $K = \{\ell_1, \ldots, \ell_k\} \subseteq Y$ be the set of distinct labels in the batch, and let $C = \{c_\ell \mid \ell \in K\}$ contain one fixed centroid per label.

For every class ℓ we measure the mean squared Euclidean distance between its samples and its centroid:

$$L_\ell(Z,Y,C) = \frac{\sum_{i=1}^{n} \mathbb{1}[y_i = \ell] \, \|z_i - c_\ell\|_2^2}{\sum_{i=1}^{n} \mathbb{1}[y_i = \ell]},$$

where $\mathbb{1}[\cdot]$ is the indicator function.

The `Rasd` loss is the average of these class-wise terms,

$$L_{\texttt{Rasd}}(Z,Y,C) = \frac{1}{k} \sum_{\ell \in K} L_\ell(Z,Y,C),$$

which tightens each class cluster while giving every class equal influence on the gradient.

Centroid search. Before we can train g_θ with the proposed loss, we must fix a set of synthetic centroids. We therefore run a Genetic Algorithm (GA) whose goal is to maximise the separation among centroids. The GA needs two ingredients: a fitness function, and an initial population of centroid sets.

Fitness. For a candidate set $C = \{c_1, \ldots, c_k\}$, the fitness is

$$\text{fit}(C) = -\min_{i \neq j} \|c_i - c_j\|_2,$$

i.e. the negative of the smallest pairwise distance.

Initial population. To obtain a stable starting point we first project the training data with the untrained encoder g_θ and run K-Means++ on the resulting embeddings. The resulting centroids initialise every chromosome in the population, while subsequent GA operations (mutation, crossover) explore alternative configurations. Using K-Means++ in lieu of random initialisation avoids the numerical instabilities that can arise when centroids are placed arbitrarily far from the data manifold.

Training pipeline. The procedure unfolds in three consecutive stages. First, the untrained encoder embeds the training data and K-Means++ yields an initial centroid set C_0. Second, the GA refines C_0 to maximise the minimum distance, producing a well-separated set C^*. Finally, keeping C^* fixed, we train the encoder parameters with the `Rasd` loss, so that each class cluster becomes more compact around its centroid.

Detecting shift samples. After training, the encoder serves as a novel class detector. The training data is embedded once to recompute the centroid of each class and to define a class-specific detection threshold t_y, set at the k-th percentile of in-class distances. During inference, an incoming sample $\mathbf{x}$ is embedded, and its distance to each centroid is computed as

$$d_y = \|g_\theta(\mathbf{x}) - c_y\|_2.$$

If $d_y > t_y$ for all classes, the sample is flagged as a novel-class sample.

4.3.2 Shift Adaptation

Once shift is detected, the incoming stream could, in principle, be fully labelled to update both the classifier and the detector. In practice, however, the traffic volume is far too large for exhaustive annotation. `Rasd` addresses this by selecting a small, chronologically ordered subset that is both informative and diverse, and incrementally updates the models on this subset together with the historical training data.

Selection strategy. We build on FFT [218], which iteratively selects the point farthest from those already chosen. Distances are measured in the latent space produced by g_θ using the Euclidean distance. Applied directly, however, FFT has two drawbacks: quadratic complexity in dataset size and a tendency to over-select outliers. `Rasd` mitigates both with two refinements.

First, buffered samples are partitioned into sequential mini-batches $MB_1, MB_2, \ldots$ of fixed size ρ. Within each mini-batch, every sample is assigned a similarity score, defined as its average cosine similarity to all other samples in the batch. Samples are then ranked by this score and split into two halves: the most similar (dense) 50% and the least similar (sparse) 50%. FFT is applied separately to each half, yielding candidates that are both informative (dense samples capture dominant behaviours) and diverse (sparse samples cover under-represented regions), while reducing the likelihood of over-selecting outliers. From each mini-batch, the number of selected samples is allocated according to the labelling budget δ (*i.e.*, $\delta/2$ to dense and $\delta/2$ to sparse regions), and their union forms the final subset S. Together, these refinements reduce the overall complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(m\rho^2/2)$, where $m = n/\rho$.

Incremental update. The selected subset S is manually labelled and merged with the historical training set D_0. Both the classifier f_θ and the detector g_θ are then incrementally updated on the expanded dataset, enabling the system to incorporate new classes while retaining knowledge of the old ones.

Table 4.1: Processed distribution of the IDS17 dataset.

Set	Days	Traffic Distribution (%)	# Flows
1	Mon–Wed	Benign (84.98), DoS Hulk (13.31), DoS GoldenEye (0.63), FTP Patator (0.33), DoS Slowloris (0.33), SSH Patator (0.25), DoS SlowHTTPTest (0.14), Heartbleed (0.0009)	1 190 531
2	Thu–Fri	Benign (71.96), Port Scan (17.48), DDoS (10.44), Bot (0.08), Brute Force (0.01), Infiltration (0.003), XSS (0.002), SQL Injection (0.001)	910 283

Table 4.2: Processed distribution of the CICIDS2018 dataset.

Set	Days	Traffic Distribution (%)	# Flows
1	14–16 Feb	Benign (83.09), DoS Hulk (10.02), FTP Brute-Force (0.0067), SSH Brute-Force (4.43), DoS SlowHTTPTest (0.0074), DoS GoldenEye (1.95), DoS Slowloris (0.47)	698 771
2	20–23 Feb	Benign (71.42), DDoS HOIC (8.68), DDoS LOIC HTTP (19.79), Bot (0.059), Infiltration (0.020), DDoS LOIC UDP (0.0079), Web Brute Force (0.0027)	960 137

4.4 Experimental Setup

We evaluate `Rasd` on the public IDS17 [77] and CICIDS2018 [230] NIDS datasets, comparing it with two detection and two adaptation baselines. The rest of this section describes the dataset preprocessing pipeline (§4.4.1), the baselines (§4.4.2), the hyperparameter search strategy (§4.4.3), and the computing platform (§4.4.4).

4.4.1 Datasets

IDS17 [77] and CICIDS2018 [230] were selected because they (i) were captured recently, (ii) span multiple consecutive days, and (iii) contain a diverse range of attack types, several of which appear only in the later segments of the dataset. By preserving the original temporal ordering when partitioning each dataset, previously unseen attack classes surface naturally as time progresses. This chronological split yields a natural "novel-attacks-emerge-over-time" scenario that simultaneously exercises the detection and adaptation capabilities of `Rasd`.

IDS17. We split the IDS17 into two sets, Set 1 with the first three days of data, and Set 2 with the last two days. Each set contains 7 kinds of attacks as well as background benign traffic. The kind of attacks present in Set 1 are different from the ones in Set 2. The resulting division is shown in Table 4.1. We then extract randomly 20% of the data from Set 1, and add it to Set 2, and we use these new sets respectively as Train and Test sets for our experiments. This last transformation introduced a limited and controlled amount of temporal bias, but makes it possible to test the behaviour of our framework in presence of both known attack classes (coming from the Set 1 samples) and new attacks (from Set 2).

CICIDS2018. We designated the first week of data for training and the second week for testing. Because the full dataset exceeds 60 million flows, we down-sampled it: from every file we ran-

domly selected 33% of the flows for each label. This sampling left the data highly imbalanced, with benign traffic making up more than 90% of all flows. To address this, we reduced the proportion of benign traffic in the testing set to 71.42%. Although this adjustment may influence our findings, particularly with respect to precision, we argue that our system is intended to operate downstream of an anomaly detector (*i.e.*, after `Mateen`), which would already filter out much of the benign traffic. The final class distribution is reported in Table 4.2. The remaining preprocessing step (*i.e.*, the random selection of 20% of Set 1) follows the same methodology previously described for the IDS17 dataset.

For both datasets, we removed features directly tied to labelling, namely ID, Dst Port, Flow ID, Src IP, Src Port, Dst IP, and Timestamp, in order to minimise shortcut learning[3].

4.4.2 Baselines

For detection, we employ two baselines. `CADE` [295], the current SoTA novel-class detector, combines an AE with the `DrLIM` contrastive term: the decoder minimises the reconstruction error, while the encoder simultaneously pulls together embeddings from the same class and pushes apart those from different classes. As a second baseline, `LSL` [243] shapes the latent space solely through the lifted-structured contrastive loss, omitting any reconstruction objective.

To evaluate `Rasd`'s adaptation module we replace its sample-selection step with two alternative strategies, both subject to the same labelling budget δ. Least Confidence (`LC`) [18] trains a multi-class classifier on the initial training set and queries labels for the samples on which the classifier is least confident. Farthest Sampling (`FS`) follows the strategy used in `CADE`: each detected-novel sample is embedded by g_θ, ranked by its Euclidean distance to the nearest class centroid $\mathbf{c}_y$, and the most distant samples are selected for labelling.

4.4.3 Architecture and Hyperparameters

Network architecture. All detectors share an encoder with three fully connected hidden layers whose widths are fixed to 75 %, 50 %, and 25 % of the input dimension d, followed by a bottleneck of 10 % d (sizes rounded to the nearest integer). For `IDS17` ($d = 77$) this yields layer sizes $58-39-19-7$. For `CICIDS2018` ($d = 76$) the corresponding sizes are $57-38-19-7$. In `CADE` the decoder is an exact mirror of the encoder, *e.g.* $19-39-58-77$ for IDS17. The adaptation classifier f_θ is a MLP with three hidden layers of 512, 256, and 128 neurons, respectively.

Training protocol. We initialise all weights using the Xavier–uniform scheme and optimise with Adam. For the detectors, we train for 250 epochs with a learning rate of 10^{-4} and a batch size of 1024. These choices follow recent findings in contrastive learning, which highlight the benefits of large batches and extended training schedules for representation quality [133, 48]. For the classifier f_θ, we use the same initialisation, but train for 140 epochs with Adam (learning rate 10^{-3}, batch size 128).

[3] Shortcut learning refers to a model exploiting easy, non-causal correlations in the input, such as identifiers or labelling artefacts, rather than learning meaningful behavioural patterns. For example, if a particular IP address or port is strongly associated with malicious traffic in a dataset, the model may learn to rely on that identifier instead of more informative features, which can artificially inflate performance.

hyperparameter optimisation. The raw training loss is not a reliable criterion for hyperparameter selection. For example, CADE introduces a weight parameter λ to balance the contrastive loss with the reconstruction loss, and a margin parameter m to define the minimum distance between dissimilar pairs. Increasing either λ or m inflates the numerical loss, even when the latent representations improve. Hence, lower loss values do not necessarily imply better cluster quality.

Our optimisation strategy instead follows the rationale of contrastive learning for novel-class detection: (i) samples with the same label should form compact clusters, while (ii) clusters with different labels should remain well separated [295, 48]. To capture these criteria, we use the Dunn Index (DI), which measures the ratio of inter-cluster separation to intra-cluster compactness. Hyperparameters are tuned on a small labelled validation split (5% of the training set) by maximising the DI:

$$\mathrm{DI} = \min_{i \neq j} \frac{d(C_i, C_j)}{\max_k s(C_k)}, \tag{4.2}$$

where $d(C_i, C_j)$ is the minimum Euclidean distance between clusters C_i and C_j, and $s(C_k)$ is the maximum intra-cluster distance within C_k. The hyperparameter setting that achieves the highest DI is selected.

For CADE we perform a grid search over $m \in \{1, 5, 10, 15, 20\}$ and $\lambda \in \{1, 0.1, 0.01, 0.001\}$ (20 combinations). The best configuration is $m = 20,\ \lambda = 1.0$ on IDS17 and $m = 1,\ \lambda = 0.1$ on CICIDS2018. For LSL we sweep $m \in \{1.0, 1.25, 1.5, 1.75, 2.0, 2.25, 2.5, 2.75, 3.0\}$. The optimum is $m = 1.5$ on IDS17 and $m = 2.25$ on CICIDS2018.

Rasd has no margin term, but its centroid search depends on the GA parameters. We fix the population size to 100 and try the number of generations in $\{50, 100, 150, 200, 250, 300, 350, 450, 500, 550\}$; 50 generations yields the best Dunn Index on both datasets.

4.4.4 Computing Platform

All experiments were run on a Linux machine with a 16-core AMD EPYC CPU, 70 GB RAM, and two NVIDIA A30 GPUs (24 GB each). We implemented CADE, LSL, and Rasd in PyTorch 2.0.1,[197] with CUDA 12.2 and Python 3.11.3, while GA was implemented using DEAP 1.4.

4.5 Evaluation Results

In this section we evaluate Rasd through four complementary studies: (i) detector performance (§4.5.1), (ii) latent-representation quality (§4.5.2), (iii) adaptation effectiveness (§4.5.3), and (iv) a sensitivity analysis (§4.5.4).

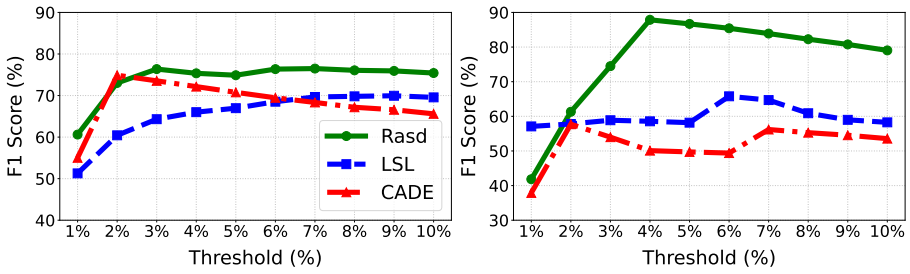
4.5.1 Rasd Detector Performance

Metrics. We treat novel samples as positives and training-class samples as negatives. To compare detectors under the same operating point, we set each method's detection threshold so that it

Table 4.3: Novel classes detection performance of Rasd and baselines

IDS17						
Framework	FPR	TPR	mTPR	F1-Score	Accuracy	Train Cost
CADE	**11.69**	68.79	61.22	65.60	83.96	1.47 h
LSL	12.83	77.29	70.89	69.56	84.97	1.20 h
Rasd	14.95	**92.23**	**93.75**	**75.42**	**86.64**	**40.62 min**

CICIDS2018						
Framework	FPR	TPR	mTPR	F1-Score	Accuracy	Train Cost
CADE	21.83	60.61	66.22	53.56	73.78	52.75 min
LSL	**17.06**	62.21	71.73	58.26	77.76	41.08 min
Rasd	17.36	**99.50**	**81.78**	**79.04**	**86.84**	**24.70 min**



(a) IDS17 (b) CICIDS2018

Figure 4.3: F1 performance of Rasd detector and baselines over different thresholds.

rejects exactly 10% of the Train set. We then report five metrics.

FPR is the proportion of non-novel samples mistakenly rejected. TPR (recall) is the fraction of novel samples correctly rejected. mTPR is the macro-averaged TPR across novel classes; it shows whether the detector works equally well on frequent and rare novel classes. For example, a detector that reaches 99% TPR with only 1% FPR is accurate overall, but a low mTPR would reveal that it still misses minority classes. F1-score combines precision and recall in one harmonic mean, and accuracy gives the overall fraction of correct decisions.

Results. Table 4.3 presents the detection results of the Rasd and the selected baselines. Among the detectors, Rasd stands out as a less computationally demanding method while achieving the best results on most of the detection metrics. When looking at the datasets, we can notice high differences, especially in the TPR metrics, with Rasd having the highest performance. Specifically, when looking at the recall at the IDS17 dataset, we can notice that Rasd outperformed the closest system by 14.94%. The difference is even higher when looking at the CICIDS2018 dataset, where the difference between Rasd and LSL is 37.29%. The other metrics are also favoring Rasd, however, with less differences when compared to the TPR. For example, in the IDS17 dataset, Rasd accuracy outperforms LSL by only 1.67%. However, when looking at the FPR, Rasd performs less than most of the baselines.

Table 4.4: Performance comparison of Rasd and the baselines in detecting novel class samples, with selecting the best model and a targeted false positive rate of 10%.

IDS17				
Framework	TPR	mTPR	F1-Score	Acc
CADE	87.97	63.69	78.70	89.41
LSL	77.17	76.93	72.37	86.90
Rasd	**92.87**	**91.18**	**81.28**	**90.49**

CICIDS2018				
Framework	TPR	mTPR	F1-Score	Acc
CADE	55.91	61.31	59.81	81.25
LSL	61.00	58.19	63.63	82.61
Rasd	**99.22**	**74.55**	**86.32**	**92.16**

Effect of the threshold value. One key factor that may influence performance is the detection threshold; therefore, we examine possible rejection rates ranging from 1% to 10%. Figure 4.3 presents the F1 score across all the considered rates. Generally, Rasd still achieves a higher score compared to the other methods at most threshold rates. However, on minor occasions, particularly at lower rates, Rasd may perform worse than the baselines. For example, when considering a 1% threshold, LSL outperforms Rasd in the CICIDS2018 dataset; however, Rasd surpasses LSL in the IDS17 dataset at the same threshold rate. Furthermore, even when evaluating the peak scores for each method, Rasd remains at the forefront. For instance, in the CICIDS2018 dataset, Rasd achieves 87.87% at a 4% threshold, while the nearest baseline, LSL, achieves 65.78% at a 6% threshold rate. Another insight is that the performance of all methods generally improves at higher threshold rates, particularly between 4% and 8%, due to an increase in the number of detected novel samples.

Calibrating the FPR. The performance outcomes of the first experiment can be significantly influenced by two main factors: the approach to hyperparameter optimization and the threshold rate at which false positives occur. As highlighted in Table 4.3, Rasd registers a higher FPR in comparison to other baselines, which may simultaneously leads to increase in the TPR. For every baseline model, we conduct a thorough evaluation of all possible hyperparameters and prioritise the one delivering the highest F1 score. Subsequently, we calibrate the FPR to a fixed target of 10%. While this particular setting might not be realistic in practical scenarios, our intention is to simulate an environment where both the thresholding and hyperparameter search techniques reliably pinpoint the most efficient model, whilst maintaining a predetermined false positive rate.

Table 4.4 reports the detection scores after each method is retuned to an FPR of 10%. On IDS17 almost all baseline values improve relative to Table 4.3, yet Rasd still tops every metric. The picture differs on CICIDS2018. Reducing CADE from its original 21.83% FPR to 10% lowers its TPR markedly, and LSL shows a similar drop in TPR despite modest gains in F1 and Acc. By contrast, Rasd loses only a few TPR points and thus retains the highest F1 and overall accuracy. These results confirm that Rasd maintains its advantage even when the false-positive rate is capped.

Table 4.5: Evaluating the quality of generated latent representations using clustering metrics.

Dataset	System	SS	NMI	ACC
IDS17	CADE	69.42	38.00	64.82
	LSL	57.62	59.69	79.85
	Rasd	**80.59**	**85.78**	**96.30**
CICIDS2018	CADE	71.17	14.37	34.34
	LSL	56.52	66.36	79.02
	Rasd	**84.58**	**92.32**	**98.46**

4.5.2 Latent Representation Quality

Although every baseline is evaluated in its best hyperparameter configuration, each one still trails Rasd. We conjecture that their contrastive objectives did not learn a sufficiently discriminative latent space. This idea is supported by recent work showing that contrastive losses are sensitive to class imbalance [48]. To probe this hypothesis we run K-Means++ on the test embeddings of the known classes (the 20% of Set 1 injected into Set 2) and evaluate the resulting clusters. The rationale is straightforward: if the representation is cleanly separated by class, an unsupervised algorithm like K-Means++ should recover the true labels with high fidelity.

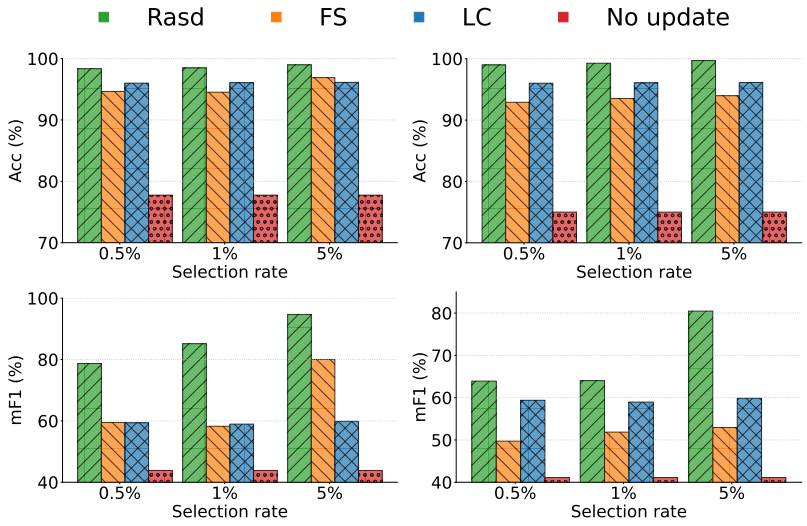
Metrics. We report three metrics. Normalised Mutual Information (NMI) and clustering Accuracy (ACC) compare the cluster assignments with ground-truth labels; Silhouette Score (SS) is label-free and gauges the distance between samples within a cluster relative to the nearest neighbouring cluster. A large SS coupled with a low accuracy indicates that the clusters are well separated but carry mixed labels, while a large NMI alongside a low accuracy suggests systematic confusion between specific classes.

Results. Table 4.5 shows that Rasd learns the cleanest latent space by a large margin. On CICIDS2018, it achieves 98.46% clustering accuracy and 92.32% NMI, far ahead of the baselines. In contrast, CADE attains reasonable separation (SS 69.42%) but very low alignment with labels (NMI 38%), indicating that embeddings from different classes mix together. Its performance on CICIDS2018 collapses further, consistent with its weak detection scores in Table 4.4. LSL is more stable, with accuracies above 79% on both datasets, yet its low SS suggests that clusters are not well spaced and neighboring classes remain difficult to distinguish. Overall, the baselines fail to learn robust, separable representations, whereas Rasd produces a latent space that is both compact and label-aligned.

4.5.3 Rasd Adaptation Strategy Performance

In this experiment, we evaluate how each sample-selection strategy (Rasd, LC, and FS) influences the performance of f_θ.

Metrics. The Test set is divided into an adaptation split (70%) and an evaluation split (30%). Within the adaptation portion, we first apply the Rasd detector to identify shifted samples. From



(a) IDS17. (b) CICIDS2018.

Figure 4.4: Adaptation performance across different selection rates δ.

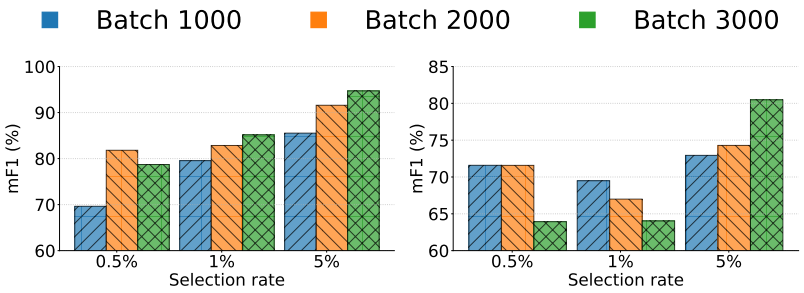
these detected shifts, we select a subset S under a labelling budget $\delta \in \{0.5\%, 1\%, 5\%\}$ (*i.e.*, δ percent of the detected shifted samples). The elements of S are manually labelled and used to adapt the model. For `Rasd`, the mini-batch size ρ is fixed to 3000. Performance is reported using two metrics: accuracy (Acc) and macro-F1 (mF1).

Results. Figure 4.4 reports the results for IDS17 (4.5a) and CICIDS2018 (4.5b). Several patterns stand out. First, every sampling strategy improves upon the frozen baseline that is trained only on the Train set, confirming the value of incremental learning. Second, all methods surpass 90% accuracy, yet their mF1 scores remain lower, a gap that reveals a bias toward majority-class over-selection. Third, the proposed `Rasd` sampler yields the best performance across both metrics and all δ values. Finally, raising the δ value further boosts `Rasd`'s mF1, showing that the method assimilates minority classes particularly well when given a higher labelling budget.

4.5.4 Sensitivity Study.

This experiment probes the robustness of the adaptation strategy to three factors. First, we vary the mini-batch size ρ, testing values of 1000, 2000, and 3000. Second, we inject controlled amounts of random label noise into S and assess its impact on the performance of f_θ. Third, we explore the use of an oracle to pseudo-label the remaining samples in the adaptation split.

Mini-batch sizes (ρ). Figure 4.5 compares three batch sizes ($\rho \in \{1000, 2000, 3000\}$) when samples are chosen with `FFT`. With a generous annotation budget ($\delta = 5\%$), the largest batch ($\rho = 3000$) performs best: the budget is sufficient for `FFT` to select representatives from several regions of the batch before being exhausted. When the budget tightens ($\delta \leq 1\%$), however, the trend reverses, particularly for CICIDS2018. Because `FFT` can then select only a handful of points per batch, a very large batch funnels those selections towards the global extremes, leaving much of the interior unexplored. Smaller batches, by contrast, force `FFT` to restart more frequently, spreading



(a) IDS17. (b) CICIDS2018.

Figure 4.5: Effect of ρ on the performance of the f_θ.

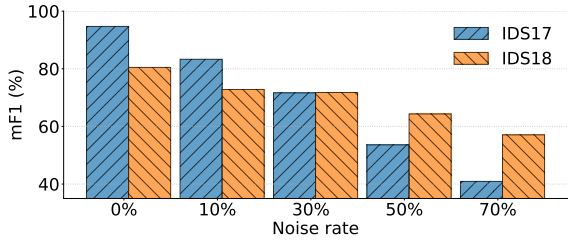


Figure 4.6: Effect of random label noise on f_θ mF1 Score with δ at 5%.

the limited queries across a broader portion of the buffer and thereby improving coverage and performance at low selection rates.

Label noise. Previous experiments assumed perfectly accurate annotations for the selected set—a condition seldom met in practice. To gauge the effect of imperfect labels, we randomly flipped the labels of 10%, 30%, 50%, and 70% of the samples in the selected subset. Figure 4.6 illustrates the impact. As expected, performance declines monotonically with increasing noise: in IDS17, overall accuracy drops from 94.70% in the noise-free setting to 83.32% at 10% noise, and to 40.93% when 70% of the labels are corrupted.

Incremental learning with oracle. In our adaptation experiment (§4.5.3), we incrementally trained the classifier f_θ on the historical training data (*Train*) together with the selected subset S from the adaptation split, leaving the remaining adaptation samples unused. Here, we ask whether S can guide labelling of the entire adaptation split. Following [18], we use a supervised model as an *oracle* for pseudo-labelling: we train five oracles—Random Forest (`RF`) [39], Decision Tree (`DTree`) [205], Support Vector Machine (`SVM`) [56], K-Nearest Neighbors (`KNN`) [57], and `NCN` [75]—on S (5% of the adaptation split) and use each to pseudo-label the remaining samples. We then assess both pseudo-labelling accuracy (agreement with ground truth labels) and the downstream effect on f_θ.

As shown in Table 4.6, `DTree` attains the highest overall accuracy, while `RF` is a close second but performs better on minority classes (higher mF1). Because preserving recall on rare attack types is critical, we focus on `RF` as the oracle. When the classifier f_θ is retrained on the union of the historical training data, S, and the `RF`-pseudo-labelled samples, it achieves 98.63% Acc and 93.95% mF1. Interestingly, training f_θ only on the historical training data and S yields slightly

Table 4.6: Label correctness vs. classifier performance.

	Label Correctness		Classifier Performance	
Model	**mF1**	**Acc**	**mF1**	**Acc**
RF	**81.24**	96.34	**93.95**	98.63
DTree	65.78	**96.59**	91.46	**98.73**
SVM	31.56	83.83	70.65	95.39
KNN	72.40	95.19	92.82	98.33
NCN	13.06	60.73	41.75	89.53

better performance (Acc: 98.98%, mF1: 94.70%). This small drop suggests that residual pseudo-labelling noise can marginally affect downstream performance.

4.6 Discussion and Future Work

Centroids Selection. We currently estimate class centroids with a genetic algorithm operating in the latent space of an initially random network. Because this optimisation is tightly coupled to the backbone architecture and its starting weights, its reliability can vary. A more principled way to obtain robust prototypes would therefore be valuable. One promising direction is to augment Rasd with a lightweight contrastive-learning pre-training stage: supplied with only a few labelled examples, a contrastive objective would align same-class embeddings and separate different classes, yielding improved class prototypes. Rasd could then refine these prototypes by minimising the distance from every sample to its class centroid. Developing and evaluating such a hybrid prototype–contrastive approach is left for future work.

Adversarial Machine Learning and Explainability. Our research intersects with three important challenges: robustness to poisoning attacks [259, 216], evasive attempts [216], and explaining system decisions [28]. While our focus was on identifying novel samples and adapting to them with the lowest possible annotation cost, further evaluation is necessary to understand the robustness of our system to evasive and poisonous samples. Equally crucial is the ability to justify decisions—explaining why a specific net flow is deemed novel sample and why querying that flow is expected to be more informative than querying alternative samples. A comprehensive investigation of adversarial robustness and explainability within Rasd is left to future work.

Attack Correlation and Order. To emulate deployment, we constructed a single chronological split per dataset, training on earlier days (Set 1) and evaluating novelty detection on later days (Set 2). This choice fixes the order and co-occurrence of attack families and therefore implicitly defines what counts as novel. In IDS17, for example, Set 1 contains mainly DoS and Patator attacks, whereas PortScan/DDoS and web/infiltration appear only in later days; CICIDS2018 similarly follows a week-by-week progression. Reordering the same days into a different but equally realistic sequence would change which attack behaviours are seen during training versus evaluation, and could therefore alter measured novelty-detection performance. We did not quantify

sensitivity to alternative orderings or to different attack co-occurrence patterns. A useful direction for future work is to conduct such a sensitivity analysis and determine how strongly performance depends on the dataset-induced attack schedule.

Learning under Noisy Labels. `Rasd` workflow periodically selects a small subset of samples for manual annotation. Then, it incrementally re-trains the classifier and the detector on the growing historical data and the newly labelled subset. This pipeline implicitly assumes error-free labels—an assumption rarely satisfied in practice. Even a modest fraction of mislabelled samples can cause the classifier to memorise noise rather than generalise to clean data. To counteract this risk, Chapter 5 introduces a dedicated noise-mitigation framework that detects and corrects label noise in training datasets, thereby preserving performance in the presence of label noise.

Oracle-based Labelling. In our published work [11], we used a `RF` oracle to pseudo-label the remainder of the adaptation split to avoid discarding those samples. That paper did not include an ablation of the oracle. In this thesis, we ablate it explicitly (Section 4.5.3) and find that, under our setting, oracle-based pseudo-labelling provides no measurable downstream benefit (Table 4.6). Accordingly, we de-emphasise the oracle and omit it from the `Rasd` framework (Section 4.3), treating it as an optional variant. More robust pseudo-labelling strategies (*e.g.*, semi-supervised learning) could still improve the classifier performance; we leave their exploration to future work.

4.7 Conclusion

In this chapter, we introduced `Rasd`, a framework that operates alongside the classifier to detect and adapt to novel classes. For detection, `Rasd` adjusts the center-loss objective so that it emulates the benefits of contrastive learning while remaining cheaper to train and less sensitive to class imbalance. For adaptation, `Rasd` applies a selection strategy that samples from both dense and sparse regions of the data distribution. The strategy builds on the `FFT` algorithm and keeps runtime low by grouping flagged samples into chronological mini-batches.

We evaluated `Rasd` on the widely used IDS17 and CICIDS2018 datasets. On IDS17, `Rasd` detected 92.23% of novel samples and achieved 86.64% overall accuracy, outperforming the strongest baseline by 14.94 and 1.67 points, respectively. On CICIDS2018 it detected 99.50% of novelties with 86.84% accuracy, exceeding the next-best approach by 37.29 and 9.08 points. Crucially, `Rasd` remained effective on minority novel classes, reaching a mTPR of 93.75% on IDS17 compared with 70.89% for the next-best approach. Owing to its lightweight objective, training time is 40.62 minutes on IDS17, compared with 1.20 hours for the fastest baseline. The adaptation component proved equally effective. With a selection budget of just 0.5% on IDS17, `Rasd` raised the mF1 score from 43.85% (no update) to 78.71% and lifted accuracy from 77.76% to 98.35%. Under the same budget, the strongest competing approach reached only 59.46% mF1 and 96.01% accuracy.

These results confirm that `Rasd` achieves SoTA novelty detection, efficient adaptation, and significant speed-ups compared with existing baselines. The framework nonetheless has several limitations that point to future research.

First, its reliability relies on the quality of the class centroids used in the customised center loss. At present, those centroids are produced by a genetic search in the latent space after the encoder has been initialised. A more robust option would pretrain the encoder with a contrastive objective and derive the centroids directly from the resulting, better-aligned embeddings. Second, `Rasd` has not been tested under adversarial conditions and is not explicitly designed to withstand targeted poisoning or evasion attacks. Investigating how adversarial techniques affect the framework, and designing safeguards, is an important future work. Third, the current implementation offers no human-readable explanations: it cannot yet clarify why a given flow is flagged as novel or why that flow is prioritised for annotation over others. Enhancing interpretability therefore remains a critical area of improvement. Finally, both `Rasd` and its underlying classifier remain vulnerable to label noise. The next chapter is devoted to strategies that lessen the impact of noisy labels on DNN classifiers.

Chapter 5

SLB: Label Noise Mitigation in Cyber Security

The previous two chapters presented our contributions toward reducing the impact of distribution shift in DNN-based NIDS. They also highlighted a related and equally critical challenge: label noise. In Section 2.5, we discussed this problem in detail and reviewed existing techniques for addressing label noise in both malware and NIDS datasets. That review revealed a key research gap—namely, the lack of a learning approach that can transfers across security datasets and scales beyond binary settings.

In this chapter, we fill this gap by introducing SLB, a framework that combats label noise across security DNN pipelines and is validated through two representative case studies: DNN-based NIDS and malware detection and classification. SLB provides security analysts with two tangible outputs: (i) a classifier that remains accurate in the presence of noisy labels, and (ii) a sanitised training set in which many erroneous labels have been repaired. This work has been published as:

Fahad Alotaibi, Euan Goodbrand, and Sergio Maffeis. "**Deep Learning from Imperfectly labelled Malware Data**." In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*, October 13–17, 2025, Taipei, Taiwan. ACM, New York, NY, USA, 15 pages.
DOI: 10.1145/3719027.3765197

5.1 Introduction

DNNs now underpin many frontline security applications. Trained on large, labelled datasets of network traffic, executable binaries, and incident timelines, they (i) flag and classify intrusions in NIDS [6, 128, 221], (ii) detect malware and assign each sample to its correct family [290, 90, 306], and (iii) reconstruct precise incident timelines and event sequences [13, 68, 16], consistently achieving SoTA results on benchmark tasks.

Yet assembling the large, accurately labelled datasets that deep networks demand is challenging. Ideally, every sample would be carefully validated by multiple security experts to guarantee high-

quality annotations, yet this is infeasible when dealing with hundreds of thousands or millions of security samples. Therefore, the community turns to automated labelling strategies, as outlined earlier in Section 2.5.

Malware research offers a clear illustration of automated labelling. VT, an online aggregator of many Anti-Virus (AV) engines, is a popular source of labels. Researchers typically combine its reports using threshold heuristics in order to scale dataset creation [46, 25, 44, 207, 199, 48]. These heuristics are error-prone because VT verdicts evolve: a file that is initially marked benign or categorised as greyware can later be flagged as malicious, or reassigned to a different threat family once new intelligence appears [125, 286, 180, 273]. The resulting dataset are therefore large but imperfect, with a significant fraction of samples carrying incorrect labels, a phenomenon known as label noise [125, 286, 273].

Imperfect datasets introduce two major complications for ML experiments. First, when learning-based models are trained on data with incorrect labels, they tend to overfit these errors. This overfitting reduces their performance on real-world data (*i.e.*, the data the model will encounter when deployed in practice) [286, 180]. Second, evaluating models on validation or holdout sets that include labelling errors yields misleading performance metrics. Consequently, researchers may prematurely embrace approaches that appear effective based on these flawed evaluations, even though such methods may merely be learning noise rather than meaningful patterns [273].

Our Method. In this chapter, we introduce `SLB`, a technique designed to mitigate label noise by progressively correcting and expanding an automatically identified clean subset. `SLB` operates in two stages. First, an ensemble of training epochs partitions the entire dataset into (1) a high-confidence clean subset, samples whose predictions are consistently aligned with their observed labels, and (2) a noisy subset containing ambiguous or inconsistently labelled samples. The noisy subset receives pseudo-labels to address its uncertain annotations. Second, `SLB` executes a continuous label revision process throughout training. Samples in the noisy set that repeatedly match either their observed label or their pseudo-label are promoted to the clean subset, occasionally flipping labels if necessary. Conversely, samples in the clean set that exhibit inconsistent predictions are reclassified as noisy. This iterative correction mechanism steadily denoises the dataset—expanding the clean set and reducing overall label noise. Furthermore, the classifier trained in parallel becomes increasingly resilient to noise and can be used immediately for multi-class classification or detection tasks.

Evaluation. We evaluated `SLB` on five security datasets that together cover network intrusion, Windows malware, and Android malware. The suite includes IDS17 for attack classification [77], VirusShare 2018 for Android-malware family classification [176], Malware PE for Windows PE malware classification [286], and APIGraph for Android-malware detection [48]. Across these datasets, we studied both real-world label noise derived from VT reports and synthetic noise that we injected at several controlled levels. Across high noise levels (70% synthetic), `SLB` improved the baseline model's macro F1 score by 13.55% on VirusShare 2018 and by 26.32% on Malware PE. Even under a real-world noise rate of 31.44% in VirusShare 2018, `SLB` offered a 10.12% gain in macro F1, while having minimal impact on performance in noise-free settings. Moreover, `SLB`

outperformed existing SoTA label-noise handling techniques for malware data, MORSE [286] and Differential Training (DT) [289], while requiring fewer computational resources.

Beyond improving classification robustness, SLB substantially revises dataset labels themselves. For example, it reduced 25% injected noise in APIGraph to below 1.5% and lowered 30% noise in Malware PE to under 6%. This label revision also boosts classical ML models: on Malware PE with 70% noise, RF [39] accuracy increased from 59.36% to 86.33%. Furthermore, we tested SLB with advanced deep architectures (*e.g.*, ResNet18 [114]), conducted an ablation of its core components, and performed a hyperparameter sensitivity analysis. Collectively, these experiments demonstrate that SLB generalises across diverse models, each of its components is crucial to overall effectiveness, and it remains robust even with suboptimal hyperparameter settings.

Contributions. In summary, the contributions of this chapter are as follows:

1. We propose an ensemble-based data cleaning technique that aggregates predictions from multiple epochs to identify correctly labelled samples. This method partitions the dataset into a clean and a noisy subset, and assigns high-quality pseudo-labels to uncertain samples.

2. We introduce a continuous revision strategy that dynamically updates sample labels during training. This strategy leverages both the observed labels and the assigned pseudo-labels, and employs a label flipping mechanism that promotes samples from the noisy set to the clean set.

3. We integrate these components into a single framework, SLB, which continuously revises labels, reduces noise in security datasets, and simultaneously trains a robust classifier. We release our code and datasets to promote reproducibility and further research in this area.[1]

4. We evaluate SLB extensively across various deep learning architectures (*e.g.*, ResNet18), diverse datasets (*e.g.*, NIDS, Android malware, and Windows malware), and traditional ML algorithms (*e.g.*, RF) under both real-world and synthetic noise at multiple noise levels. Our results demonstrate that SLB significantly enhances model robustness and generalisation while yielding a less noisy dataset. It surpasses existing binary and multi-class label noise handling methods for malware and proves more cost-efficient in practice.

5.2 The SLB Framework

In this Section, we introduce SLB, a framework designed to robustly train security classifiers under label noise while simultaneously producing a cleaner dataset. We begin by formally defining the problem setup (§5.2.1), then provide a high-level overview of the framework (§5.2.2). Finally, we describe the two main components of SLB: *Data Split* (§5.2.3) and *Continuous Revision* (§5.2.4).

[1] https://github.com/ICL-ml4csec/SLB/

5.2.1 Problem Definition

We consider a security dataset $D = \{(x_i, y_i)\}_{i=1}^{N}$, where each of the N samples is represented by a feature vector $x_i \in \mathbb{R}^d$. The feature extractor is task-agnostic: for malware, x_i might encode static artifacts (*e.g.*, imported functions, PE-header fields, strings) or dynamic behaviours (*e.g.*, API-call traces); for NIDS, it could capture flow statistics, packet headers, or packet payloads. Each sample belongs to one of C classes, denoted by y_i. Typical scenarios include multi-family malware classification (multiple malicious families plus a benign class), binary malware detection (benign vs. malicious), binary NIDS (benign traffic vs. attack), or multi-class NIDS (distinct attack classes plus a benign class).

In practice, large-scale labelling relies on automated tools, such as ensembles of AV engines, signature-based IDS, or test-based labelling, to generate an observed label $\tilde{y}_i$. Disagreements between tools, stale signatures, and heuristic errors introduce label noise, so that $\tilde{y}_i \neq y_i$ for a non-negligible fraction of the data.

When a DNN is trained on the noisy dataset $\tilde{D} = \{(x_i, \tilde{y}_i)\}_{i=1}^{N}$ using the standard cross-entropy loss

$$L_{\mathrm{CE}} = -\frac{1}{N}\sum_{i=1}^{N} \log p_i(\tilde{y}_i), \tag{5.1}$$

where $p_i(\tilde{y}_i)$ is the softmax probability of $\tilde{y}_i$, the model tends to over-fit the mislabelled samples, thereby degrading detection and classification performance in real-world deployments [286, 273, 309].

Our objective is twofold. First, we aim to learn a robust classifier

$$\hat{f}_\theta : \mathbb{R}^d \to \{1, \ldots, C\}$$

that can effectively mitigate the impact of label noise and generalise well on unseen data. Second, we seek to produce a revised dataset

$$\hat{D} = \{(x_i, \hat{y}_i)\}_{i=1}^{N}$$

whose labels $\hat{y}_i$ closely approximate the true labels y_i, formally:

$$\hat{y}_i \approx y_i.$$

By systematically identifying and handling mislabelled samples, we preserve the valuable diversity of the security dataset, strengthening the classifier's robustness and generalisation while also leaving behind a dataset with substantially fewer incorrect labels.

5.2.2 Overview of the `SLB` Framework

The `SLB` framework is built on two key ideas that help combat label noise in security datasets. The first idea leverages the tendency of DNNs to learn simpler, correctly labelled examples more quickly than mislabelled or complex ones [27, 106, 309, 23]. By tracking the consistency of each

sample predictions over multiple training epochs, we derive an initial partition of the dataset $\tilde{D}$ into a high-confidence clean set D^c and a noisy set D^n. For samples deemed noisy, we also generate a pseudo-label that may later replace the originally observed label.

Algorithm 2 Data Split

Require: Noisy dataset $\tilde{D} = \{(x_i, \tilde{y}_i)\}_{i=1}^N$, epochs e, class-balanced loss hyperparameter β

Initialize model f_θ^{DS}
for $t = 1$ to e **do** ▷ Model training
  Train f_θ^{DS} on $\tilde{D}$ using the loss:
  $L_{\text{CB}}^{\text{noisy}} = -\frac{1}{N}\sum_{i=1}^N \frac{1}{\text{EN}_{\tilde{y}_i}} \log\big(p_i(\tilde{y}_i)\big)$, where $\text{EN}_{\tilde{y}_i} = \frac{1-\beta^{n_{\tilde{y}_i}}}{1-\beta}$
  Save predicted label $\hat{y}_{i,t}$ for each sample x_i in $\mathbf{P}_i$
end for
$D^c \leftarrow \emptyset, \quad D^n \leftarrow \emptyset$
for $i = 1$ to N **do** ▷ Data partitioning
  Compute consistency ratio:
  $r_i = \frac{|\{\hat{y} \in \mathbf{P}_i : \hat{y} = \tilde{y}_i\}|}{e}$
  if $r_i = 1$ **then**
    Mark x_i as clean:
    $D^c \leftarrow D^c \cup \{(x_i, \tilde{y}_i)\}$
  else
    Derive pseudo-label:
    $y_i^{\text{maj}} \leftarrow \text{majority}(\mathbf{P}_i)$
    Mark x_i as noisy:
    $D^n \leftarrow D^n \cup \{(x_i, \tilde{y}_i, y_i^{\text{maj}})\}$
  end if
end for
return D^c and D^n

The second idea addresses the fact that relying on a single, static split can inadvertently discard genuinely correct but harder-to-learn samples, while still keeping mislabelled examples in the clean set. To resolve these issues, SLB incorporates a *Continuous Revision* stage that periodically re-checks each sample's predictions using an Exponential Moving Average (EMA). During revision, samples can move between D^c and D^n, and their assigned label can switch between the observed label and the pseudo-label, depending on the model's evolving confidence.

5.2.3 Data Split

The first component of SLB is the *Data Split*, where we divide the observed dataset

$$\tilde{D} = \{(x_i, \tilde{y}_i)\}_{i=1}^N$$

into a clean set D^c and a noisy set D^n. One widely used strategy for this split is the *small-loss trick* [286, 106]. In this approach, the model trains for a brief warm-up period, and the final-epoch training losses are used to rank samples. Those with the lowest loss (*e.g.*, the bottom 15% [286]) are designated as clean.

This simple strategy suffers from several limitations. Setting the threshold is difficult without access to a clean validation set or prior knowledge of the noise rate. In addition, it struggles to

handle class-specific noise effectively, particularly when certain classes are underrepresented or frequently mislabelled. Moreover, it is highly sensitive to the choice of warm-up epoch; if the model has not stabilized or begins to overfit during this period, it can lead to substantial noise in the resulting clean set.

To address these challenges, we propose a method that automatically adapts the size of the clean set based on the model's evolving predictions during the warm-up period. Rather than relying on a single epoch's loss values, we collect and aggregate each sample's hard-label predictions across warm-up epochs. If the model consistently predicts the same label as the observed label, we classify the sample as clean; otherwise, we mark it as noisy. In this way, we reduce the need for strict global thresholds or per-class adjustments, and therefore, this method is more robust to different noise levels and distributions. Algorithm 2 details the step-by-step process.

Epoch-wise Training and Prediction Consistency. To detect label noise, we initially train a single DNN model, f_θ^{DS}, for a predetermined warm-up period consisting of e epochs. At each epoch t during this period, we obtain and record the model's prediction $\hat{y}_{i,t}$ for every sample x_i. These predictions are aggregated into a prediction set:

$$\mathbf{P}_i \;=\; \{\hat{y}_{i,t} \mid t = 1,\ldots,e\}. \tag{5.2}$$

Our method builds on the intuition that predictions for clean samples generally stabilize quickly during early epochs, whereas noisy samples exhibit noticeable fluctuations. This is distinct from the existing ensemble-based approach [248], which relies on multiple models and use early training snapshots to improve inference-time prediction quality. Instead, we utilize the prediction consistency from a single model's multiple epochs explicitly for data cleansing purposes prior to final training.

To measure the consistency of predictions, we define a consistency ratio r_i as the fraction of epochs in which the model's predicted label matches the observed (potentially noisy) label $\tilde{y}_i$:

$$r_i = \frac{|\{\,\hat{y} \in \mathbf{P}_i : \hat{y} = \tilde{y}_i\,\}|}{e}. \tag{5.3}$$

Partitioning into Clean and Noisy Sets. After gathering predictions from the epoch-wise warm-up period, we partition the dataset into clean and noisy subsets based on the consistency of predictions. Using the consistency ratio r_i, computed as defined previously, we apply a strict threshold to determine set membership:

$$\begin{aligned} D^c &= \big\{(x_i,\tilde{y}_i) \;\big|\; r_i = 1\big\}, \\ D^n &= \big\{(x_i,\tilde{y}_i,y_i^{\text{maj}}) \;\big|\; r_i < 1\big\}. \end{aligned} \tag{5.4}$$

Samples with complete prediction consistency across all epochs ($r_i = 1$) are confidently labelled as clean and form the clean set D^c. These represent highly reliable examples that the model

consistently classifies correctly throughout the brief training phase.

Conversely, samples with any inconsistency ($r_i < 1$) are considered noisy D^n. For these samples, we further compute a pseudo-label, y_i^{maj}, representing the most frequently predicted label across the warm-up epochs:

$$y_i^{\text{maj}} = \text{majority}\big(\mathbf{P}_i\big), \tag{5.5}$$

where $\mathbf{P}_i$ is the aggregated prediction set from epoch-wise training.

The clean set thus collects examples the epochs consistently agrees upon, while the noisy set retains samples that exhibit any disagreement. Crucially, storing both the observed label and the majority-voted pseudo-label for noisy samples preserves opportunities for future correction. By structuring the data in this way, we ensure that the initial training can focus on high-confidence labels, laying a strong foundation for the *Continuous Revision* phase that will further interrogate D^n and resolve lingering ambiguities.

Class-Balanced Loss. Although the above procedure helps distinguish clean vs. noisy samples, class imbalance can severely hinder its effectiveness. In many real-world security datasets, the benign class alone may constitute up to 90% of all samples, leaving only about 10% for malicious samples (*e.g.*, in malware datasets [199]). If we further split those malicious samples into multiple classes, each class may represent only a small fraction of the dataset. Under such skewed conditions, empirical risk minimisation drives the model to prioritise the majority class, leading to the most significant reduction in training error. As a result, the model may show high consistency on the majority class and, in some cases, even overfit to its noisy labels. Meanwhile, truly correct samples in underrepresented classes may never achieve perfect consistency because the model lacks sufficient incentive to learn their characteristics thoroughly.

To address these challenges, we adopt the Class-Balanced (CB) loss [61] during model training. For each class y_b with n_{y_b} samples, we compute the Effective Number (EN):

$$\text{EN}_{y_b} = \frac{1-\beta^{n_{y_b}}}{1-\beta}, \tag{5.6}$$

where $\beta \in [0,1)$ is a hyperparameter that controls how aggressively we penalize classes based on their sizes. Each sample's loss term is weighted by $1/\text{EN}_{\tilde{y}_i}$, giving minority classes more influence in the training objective. Concretely, we modify the standard cross-entropy loss in Eq (5.1) to

$$L_{\text{CB}}^{\text{noisy}} \;=\; -\frac{1}{N}\sum_{i=1}^{N}\frac{1}{\text{EN}_{\tilde{y}_i}}\log\big(p_i(\tilde{y}_i)\big), \tag{5.7}$$

where $p_i(\tilde{y}_i)$ is the model's softmax probability for the observed label $\tilde{y}_i$. By re-weighting samples in this way, the model learns to pay more attention to underrepresented classes, which helps it better identify mislabelled examples within those classes. This becomes especially valuable when the majority class harbors significant noise, since otherwise the network might memorize those

incorrect labels and neglect the rare classes, ultimately preventing truly correct minority-class samples from being recognised as clean.

5.2.4 Continuous Revision

After partitioning the dataset into a clean subset D^c and a noisy subset D^n during the *Data Split* stage, `SLB` enters an iterative *Continuous Revision* phase, as described in Algorithm 3. In contrast to prior approaches that discard or ignore noisy samples entirely [186, 106], our method accounts for the possibility that some noisy samples may be correctly labelled or pseudo-labelled, and that some samples in the clean set may be mislabelled. To handle this uncertainty, we leverage both D^c and D^n to iteratively revise label quality and expand the set of reliably labelled samples.

EMA Initialization from Epoch-wise Predictions. Before training begins, we initialize an EMA of soft predictions for each sample x_i. Formally, we define:

$$\bar{p}_i^{(0)} = \frac{1}{e} \sum_{t=1}^{e} p_{i,t}, \tag{5.8}$$

where $p_{i,t}$ denotes the softmax output of the model f_θ^{DS} at epoch t during the *Data Split* stage. This aggregated prediction acts as an ensemble-based teacher signal, smoothing out noisy fluctuations from individual epochs and providing a stable initialization for subsequent revision.

Warm-up Training on D^c. We then perform a warm-up phase by training a new model $\hat{f}_\theta$ solely on the clean set D^c for m epochs. At the end of each epoch, the model produces soft predictions $p_i^{(r)}$ for each sample x_i from $\tilde{D}$, which are incorporated into the EMA:

$$\bar{p}_i^{(r)} = \alpha\, p_i^{(r)} + (1-\alpha)\, \bar{p}_i^{(r-1)}, \tag{5.9}$$

where $\alpha \in (0,1)$ is a smoothing parameter and $\bar{p}_i^{(0)}$ is initialized from the *Data Split* stage. After the final warm-up epoch, a hard label is assigned to each sample based on the EMA:

$$\hat{y}_i^{(m)} = \arg\max \bar{p}_i^{(m)}. \tag{5.10}$$

These revised labels serve as the starting point for the iterative revision process that follows.

Rebuilding the Datasets. Immediately after obtaining the updated hard labels $\hat{y}_i^{(m)}$, we reassemble the dataset by revising the assignments to D^c and D^n. Specifically, a sample $x_i \in D^c$ is demoted to D^n if its revised label $\hat{y}_i^{(m)}$ no longer matches its observed label $\tilde{y}_i$. Conversely, a sample $x_i \in D^n$ is promoted to D^c if $\hat{y}_i^{(m)}$ matches either its observed label $\tilde{y}_i$ or its pseudo-label y_i^{maj}.

This mechanism recognises that the initial separation could be imperfect, allowing incorrectly excluded samples to become part of the clean set once the model's predictions align. It also expels from D^c any samples that appear to have been mislabelled or were too complex to retain their observed label.

Algorithm 3 Continuous Revision

Require: Noisy dataset $\tilde{D}$, observed labels $\tilde{y}_i$, origin flags $o_i \in \{\texttt{c},\texttt{n}\}$, pseudo-labels y_i^{maj}, initial soft predictions $\bar{p}_i^{(0)}$, total epochs T, warm-up $m < T$, smoothing $\alpha \in (0,1)$

Initialize model $\hat{f}_\theta$
for $r = 1$ to m **do** ▷ Warm-up phase
  Train f_θ for one epoch on D_0^c
  for $i = 1$ to N **do**
    $p_i^{(r)} = \text{softmax}(\hat{f}_\theta(x_i)); \quad \bar{p}_i^{(r)} = \alpha p_i^{(r)} + (1-\alpha)\bar{p}_i^{(r-1)}$
  end for
end for
for $i = 1$ to N **do** ▷ Derive initial hard labels
  $\hat{y}_i^{(m)} = \arg\max \bar{p}_i^{(m)}$
end for
$D_{m+1}^c \leftarrow \emptyset, \quad D_{m+1}^n \leftarrow \emptyset$
for $i = 1$ to N **do** ▷ Rebuild sets after warm-up
  if $\hat{y}_i^{(m)} = \tilde{y}_i$ **then**
    $D_{m+1}^c \leftarrow D_{m+1}^c \cup \{(x_i, \tilde{y}_i)\}$
  else if $o_i = \texttt{n} \wedge \hat{y}_i^{(m)} = y_i^{\text{maj}}$ **then**
    $D_{m+1}^c \leftarrow D_{m+1}^c \cup \{(x_i, y_i^{\text{maj}})\}$
  else
    $D_{m+1}^n \leftarrow D_{m+1}^n \cup \{(x_i, \tilde{y}_i, \texttt{maj})\}$, where $\texttt{maj} \leftarrow \tilde{y}_i$ if $o_i = \texttt{c}$, else y_i^{maj}
  end if
end for
for $t = m+1$ to T **do** ▷ Continuous refinement
  Train $\hat{f}_\theta$ for one epoch on D_t^c
  for $i = 1$ to N **do**
    $p_i^{(t)} = \text{softmax}(f_\theta(x_i)); \quad \bar{p}_i^{(t)} = \alpha p_i^{(t)} + (1-\alpha)\bar{p}_i^{(t-1)}$
    $\hat{y}_i^{(t)} = \arg\max \bar{p}_i^{(t)}$
  end for
  $D_{t+1}^c \leftarrow \emptyset, \quad D_{t+1}^n \leftarrow \emptyset$
  for $i = 1$ to N **do** ▷ Update label sets
    if $\hat{y}_i^{(t)} = \tilde{y}_i$ **then**
      $D_{t+1}^c \leftarrow D_{t+1}^c \cup \{(x_i, \tilde{y}_i)\}$
    else if $o_i = \texttt{n} \wedge \hat{y}_i^{(t)} = y_i^{\text{maj}}$ **then**
      $D_{t+1}^c \leftarrow D_{t+1}^c \cup \{(x_i, y_i^{\text{maj}})\}$
    else
      $D_{t+1}^n \leftarrow D_{t+1}^n \cup \{(x_i, \tilde{y}_i, \texttt{maj})\}$, where $\texttt{maj} \leftarrow \tilde{y}_i$ if $o_i = \texttt{c}$, else y_i^{maj}
    end if
  end for
end for
return D_{T+1}^c, D_{T+1}^n, $p_i^{(T+1)}$, and final model $\hat{f}_\theta$

Iterative Revision over T Epochs. After the first reassembly, SLB continues in iterative fashion for T revision epochs. At the start of each epoch t, we train the model $\hat{f}_\theta$ on the current revised clean set D_t^c for one pass. Next, the newly obtained soft predictions $p_i^{(t)} = \text{softmax}(\hat{f}_\theta(x_i))$ for each sample x_i are used to update the EMA:

$$\bar{p}_i^{(t)} = \alpha\, p_i^{(t)} + (1-\alpha)\, \bar{p}_i^{(t-1)}, \tag{5.11}$$

A new hard label $\hat{y}_i^{(t)} = \arg\max \bar{p}_i^{(t)}$ is then derived. Finally, the sets D_t^c and D_t^n are rebuilt by comparing $\hat{y}_i^{(t)}$ with each sample's observed label $\tilde{y}_i$ and pseudo-label y_i^{maj}.

Repeating these steps over T epochs systematically adjusts which samples reside in the clean or noisy subsets. If the EMA consistently favors either $\tilde{y}_i$ or y_i^{maj} for a sample in D_n, that sample migrates into D_c. Conversely, if a sample in D_c conflicts with the model's revised predictions, it is moved to D_n. This dynamic approach ensures that the clean set is not fixed to its initial composition and can gradually incorporate valid (but previously misjudged) samples while filtering out entrenched mislabels. As a result, the classifier benefits from a growing reservoir of high-quality labels and avoids being constrained by early-stage partitioning errors.

Flipping Labels Between $\tilde{y}_i$ and y_i^{maj}. A critical facet of *Continuous Revision* is the ability to flip a sample's training label between the observed label $\tilde{y}_i$ and the pseudo-label y_i^{maj}. Specifically, a noisy sample might initially join D^c with its observed label, yet the EMA predictions could steadily favor y_i^{maj}. In that case, the label is flipped to y_i^{maj} in subsequent epochs to align with the model's increasingly confident predictions. Conversely, if the sample was integrated with y_i^{maj} but later stabilizes around $\tilde{y}_i$, we revert the label to $\tilde{y}_i$. This dynamic relabelling mechanism is vital for correcting errors that emerge during both the *Data Split* and early revision epochs.

SLB **Outputs.** At the end of this iterative training, EMA updates, and data reassembly, SLB yields three primary outcomes: (1) a robust classifier $\hat{f}_\theta$ that is both more robust to label noise and better generalises to unseen samples, ultimately providing a more reliable foundation for downstream malicious detection or multi-class classification tasks, (2) a refined subset D_{T+1}^c containing samples confidently identified as correctly labelled; and (3) a revised dataset $\hat{D}$, in which labels are updated based on the final EMA probabilities $p_i^{(T+1)}$, capturing the full dataset while allowing for minor residual label inaccuracies.

5.3 Experimental Setup

We evaluated SLB using five security-focused datasets that feature both real-world and synthesised label noise. The subsections that follow present the datasets (§5.3.1), describe the baseline methods (§5.3.2), define the evaluation metrics (§5.3.3), and detail the computing environment and model architecture (§5.3.4).

5.3.1 Datasets.

We used five security datasets: CICIDS 2017 (IDS17) [156, 230], Malware PE (PE) [286], API-Graph 2017–2018 (AG) [48, 311], VirusShare 2018 (VS18) [176], and Virus-MNIST (VM) [189]. In addition, we included a variant of PE and VS18 with real-world label noise, denoted as PE-Real and VS18-Real. Below, we briefly describe each dataset; additional details are provided in Appendix B.3.

CICIDS 2017 (IDS17). This multi-class dataset contains both benign traffic and flows from several network attacks. It was generated in a controlled lab setting with prior knowledge of the attacker's identity and actions, which enabled precise label assignments and minimized the risk of unrecognised label noise. The version used here, obtained from [156], contains about 1.91 million samples representing 15 classes, with roughly 78.28% deemed benign. Owing to its large size, we sampled 12,000 benign instances and 1,920 samples from eight primary attack classes. We allocated 80% of this subset for training (11,600 samples) and 20% for testing (2,320 samples).

Malware PE (PE). This dataset comprises 6,674 Windows executable samples, comprising 500 benign files and 6,174 malicious binaries from 11 families [286]. Each sample was labelled by at least three security experts, each having over five years of professional experience, making the dataset highly reliable and likely to be free from label noise. To create a test partition, the authors randomly selected 100 samples per class, leaving the remainder for training. They also introduced a variant called PE-Real by injecting label noise derived from VT reports. Specifically, if any AV engine assigned a label different from the given ground-truth, its label was changed accordingly. This resulted in approximately 13.88% of the labels being incorrect.

APIGraph 2017–2018 (AG). This Android dataset originates from [311], with the version we used provided by [48]. It comprises 69,241 samples collected from 2017 to 2018, comprising 62,993 benign and 6,248 malicious samples. The authors collected VT reports between 2022 and 2023[2] and applied a threshold-based heuristic for labelling: a sample is labelled as malicious if at least *sixteen* AV engines flag it, as benign if no detections are reported, and samples with intermediate results (*i.e.*, grayware) are discarded. We randomly split the remaining samples into an 80% training set (55,394 samples) and a 20% test set (13,847 samples).

VirusShare 2018 (VS18). This dataset originally comprised 28,543 Android samples [176], which we labelled by applying the threshold-based heuristic described in [48] to VT reports that we scanned and collected in 2025. Specifically, a sample was labelled as malicious if at least *sixteen* AV engines flagged it, and as benign if no engines reported detections. Samples with intermediate detection counts were considered ambiguous (*i.e.*, grayware) and excluded. Family labels were assigned using AVClass2 [226]. However, we identified some inconsistencies in the labelling. In particular, certain family names were reported by only a single AV engine, and some samples had conflicting or similarly weighted family assignments. For example, the sample with MD5 hash `9c1349f4569637323b8aa5ff8688afd1` was assigned both the `Triada` and `SMSReg`

[2]We thank Yizheng Chen for confirming the timeframe of VT report collection.

families, each by two engines. To address this, we applied an additional thresholding step: a family label was only accepted if it was supported by more than *five* AV engines. Otherwise, the sample was treated as grayware and excluded from the dataset. After this procedure, 1,417 benign samples and 8,529 malicious samples remained, distributed across 7 malware families. We randomly allocated 20% of the data for testing (1,989 samples) and 80% for training (7,955 samples). To simulate real-world label noise, we created VS18-Real by labelling the training subset exclusively with a single AV engine, following the methodology in [286, 46]. We selected the `Lionic` engine for its comprehensive family coverage, which ultimately produced an incorrect label rate of 31.44%.

Virus-MNIST (VM). This dataset is an image-based representation of Windows portable executables [192]. It comprises 51,880 images, including 2,516 benign samples and 49,364 malicious samples. We used the version extended by [189]. The authors subdivided the malicious class into nine distinct categories using K-Means clustering [166]. The dataset comes with a pre-defined split, allocating 3,458 samples to testing and 48,422 samples to training.

5.3.2 Baselines.

We evaluate `SLB` against three baselines: `Vanilla`, `DT` [289], and `MORSE` [286]. `Vanilla` serves as a basic baseline that trains directly on the noisy dataset $\tilde{D}$, without any mechanism for handling label noise. `DT` is a SoTA approach for noise mitigation in malware detection. It trains two deep neural networks over n rounds; at the end of each round, it identifies suspicious samples and flips their labels. For instance, if a sample is labelled as malicious and flagged as noisy, its label is flipped to benign. However, this binary label-flipping strategy is not applicable to multi-class settings. To address this limitation, we re-engineered `DT` to support multi-class datasets by replacing the binary flip rule with a pseudo-labelling strategy inspired by `SLB`. Specifically, we aggregate per-epoch predictions during each round and assign the majority-voted label to samples identified as noisy. We refer to this variant as `DT` modified (`DTm`). Notably, we found that this adjustment improves `DT`'s performance on the binary dataset (see §5.4.1). `MORSE` is the SoTA approach for noise mitigation in malware classification. At each epoch, it selects low-loss samples for supervised training and applies consistency regularization to the remaining samples using the FixMatch [240] framework. For all methods, `DT`, `MORSE`, and our proposed `SLB`, we performed a dedicated hyperparameter search. Full details are provided in Appendix B.2.

5.3.3 Evaluation Metrics.

We repeated each experiment ten times to reduce the effect of randomness and obtain reliable performance estimates. For each run, we computed two metrics. The first is accuracy (Acc), a standard measure that quantifies the proportion of correctly classified samples out of the total. Accuracy is widely used for evaluating both label correction and model classification performance, especially in the context of label noise [303, 106, 236, 212, 161, 210, 314]. However, as noted in prior work [286], label noise tends to disproportionately affect minority classes. In such cases, relying solely on accuracy can be misleading, as it may obscure poor performance on underrepresented classes. To address this limitation, we also report the macro F1-score (mF1). This metric

computes the F1-score for each class individually, and then averages them. As a result, mF1 gives equal importance to all classes, making it more robust to class imbalance and better suited for noisy settings in imbalanced datasets. Final results are reported as the average of each metric across all ten runs.

5.3.4 Computing Platform and Architecture.

All experiments were run on a Linux system (Ubuntu 22.04.5 LTS) with the following hardware: an AMD EPYC 7502P 32-core processor, 128 GB of RAM, and four NVIDIA A40 GPUs (each with 48 GB of memory). The software environment comprised Python 3.8.2, PyTorch 2.1.2, and CUDA 11.8. We used a MLP with three hidden layers containing 512, 256, and 128 neurons, respectively. Each layer was initialized using Xavier initialization. The MLP network was trained for 140 epochs (i.e., $T = 140$) with the Adam optimizer, a learning rate of 0.001, and a batch size of 128. All experiments and baselines were executed with this configuration unless otherwise stated.

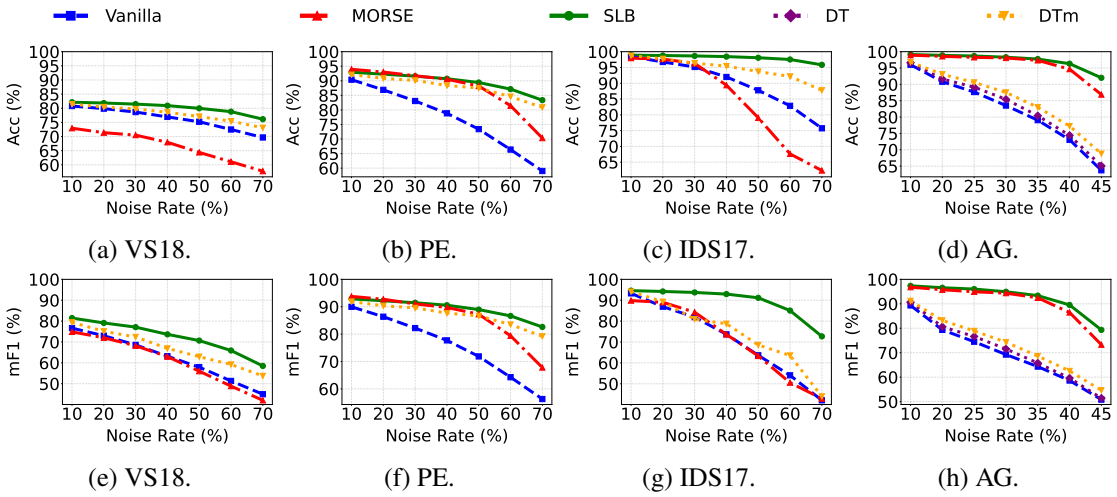
5.4 Evaluation Results

We designed our experiments to comprehensively evaluate the performance and robustness of `SLB` under diverse conditions. Our evaluation covers three key dimensions: Classification Performance (§5.4.1), Label Correction Performance (§5.4.2), and Sensitivity Analysis (§5.4.3). The following sections present and discuss the corresponding results.

5.4.1 Classification Performance

We evaluate the effectiveness of the final model $\hat{f}_\theta$ produced by the *Continuous Revision* phase across three settings: random label noise, real-world noise, and clean datasets. For synthetic noise, we inject random label noise into clean datasets (PE, VS18, AG, IDS17) at rates ranging from 10% to 70% to assess how performance deteriorates as noise increases. For real-world noise, we simulate label inconsistencies based on VT reports to evaluate the robustness of `SLB` and baseline methods under realistic conditions. Finally, we test `SLB` on noise-free datasets to verify that it does not degrade performance when the labels are already accurate.

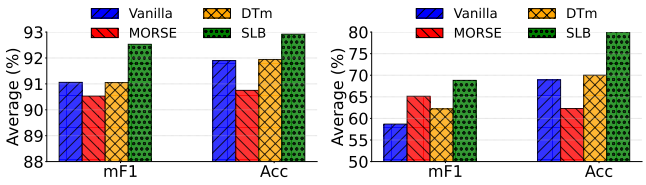
Performance on Random Noise. We generate random label noise by selecting a fixed percentage of samples from each class and flipping their labels to a different class. This method, which is widely used in both security [286, 289, 273] and non-security [106, 186] research, allows us to control the noise rate and examine its effect on model performance. In our experiments, we varied the noise rate from 10% to 70% across four datasets (VS18, PE, IDS17, and AG), as illustrated in Figure 5.1. The experimental results reveal three key findings. First, `SLB` consistently mitigates the impact of label noise across all tested noise rates. For instance, at a 10% noise rate, `SLB` improves the mF1 score over the `Vanilla` baseline by 1.3% on IDS17 and by 8.02% on AG. At higher noise levels, this advantage grows even more pronounced: `SLB` surpasses the `Vanilla` baseline by 30.62% on IDS17 (70% noise rate) and by 28.51% on AG (45% noise rate). Similar gains are observed in terms of accuracy, indicating that `SLB` improves classification performance for both major and minor classes.



(a) VS18. (b) PE. (c) IDS17. (d) AG.

(e) VS18. (f) PE. (g) IDS17. (h) AG.

Figure 5.1: Performance on synthetic noise datasets.



(a) PE-Real. (b) VS18-Real.

Figure 5.2: Performance on real noise datasets.

Second, existing SoTA noise-handling techniques, `MORSE` and `DT`, are more sensitive to specific datasets and noise rates. Although `MORSE` tends to perform well on AG and PE, it often underperforms relative to the `Vanilla` baseline on VS18 and IDS17. Meanwhile, `DT` and its re-engineered variant, `DTm`, offer a noticeable improvement over `Vanilla` in most cases, yet their effectiveness still varies by dataset. `DTm` exhibits large performance margins on VS18 and PE but less so on the other two datasets. Notably, `DTm` consistently outperforms `DT` on the binary AG dataset, confirming the positive impact of our proposed modifications. Third, `SLB` generally outperforms these SoTA methods. For example, under 70% noise on IDS17, `SLB` exceeds the best SoTA method (`DTm`) by 28.76% (mF1) and 8.07% (Acc). At lower noise levels, `SLB` generally outperforms SoTA baselines across most datasets, with one minor exception observed on the PE dataset. There, `MORSE` achieves slightly higher performance, with improvements of 0.9% in mF1 and 1.01% in Acc at a 10% noise rate, and gains of 0.56% in mF1 and 0.73% in Acc at a 20% noise rate.

Performance on Real Noise. So far, we have shown that `SLB` can effectively mitigate random label noise. However, real-world noise tends to be more complex and follows a different distribution. To examine whether our framework can handle such complexities, we evaluated `SLB` on two real-world noisy malware datasets: PE-Real and VS18-Real. In these datasets, label noise was introduced using threshold-based heuristics on VT reports, mirroring current automatic labelling practices. The results, shown in Figure 5.2, reveal three key points. First, `SLB` improves performance under real-world noise, providing measurable gains over the `Vanilla` baseline. For example, `SLB` increases the mF1 and Acc scores on PE-Real by 1.47% and 1.02%, respectively, and

Table 5.1: Performance on completely clean data (mean ± std)

Method	PE		VS18	
	Acc	**mF1**	**Acc**	**mF1**
Vanilla	94.12 ± 0.28	94.04 ± 0.27	82.01 ± 0.20	**82.59** ± 0.94
SLB	**94.31** ± 0.32	**94.33** ± 0.31	**82.21** ± 0.32	82.45 ± 0.60

Method	IDS17		AG	
	Acc	**mF1**	**Acc**	**mF1**
Vanilla	**99.54** ± 0.04	**98.08** ± 0.19	**99.36** ± 0.03	**98.06** ± 0.09
SLB	99.06 ± 0.06	95.53 ± 0.48	99.29 ± 0.03	97.85 ± 0.09

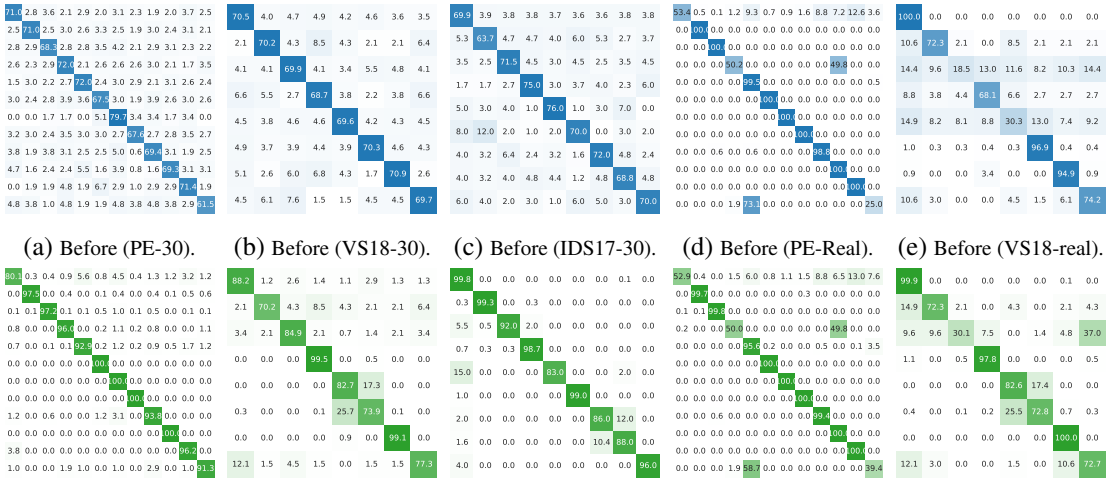
on VS18-Real by 10.12% (mF1) and 11.1% (Acc). Second, these improvements are particularly meaningful because SLB's performance approaches that of a model trained on a noise-free dataset (Table 5.1). For instance, on PE-Real, SLB achieves a 92.92% accuracy, compared to 91.90% for the noisy Vanilla baseline and 94.12% for a fully noise-free model. A similar pattern emerges on VS18-Real, where SLB attains an 80.03% accuracy, approaching the noise-free model's 82.01% while surpassing the Vanilla baseline's 68.93%. Third, SLB consistently surpasses the SoTA techniques: on VS18-Real, it outperforms the best baseline method in terms of mF1 (MORSE) by 3.68% and accuracy (DTm) by 10.01%.

Performance on Clean Datasets. It is often challenging to ascertain whether a security dataset is entirely free of label noise. Consequently, any noise handling technique should ideally have minimal adverse impact when applied to clean data. To evaluate this, we investigated the effect of SLB on completely clean datasets. Table 5.1 summarises the results of this experiment. Out of 8 measurements, in 3 cases SLB outperformed Vanilla by 0.23% on average, and in 4 cases it underperformed by the same amount (0.23%, on average). In the worst-case scenario, the mF1 on the IDS17 dataset decreased by 2.55%. Note that already at 10% noise SLB outperforms Vanilla on the same dataset by 1.3%, and at 70% it outperforms by 30.62%. This indicates that our approach is robust enough to be worth applying when the noise level is low or uncertain, as it does not significantly harm performance even on perfectly labelled data.

5.4.2 Label Correction Performance

We assess the quality of the revised dataset $\hat{D}$ produced by SLB through two experiments: first, by comparing the corrected labels $\hat{y}_i$ to the ground-truth clean labels y_i to quantify the accuracy of the correction process; and second, by training a variety of ML models on $\hat{D}$ to evaluate its downstream utility and generalisation beyond our own training framework.

Labels Accuracy. To evaluate the effectiveness of our label correction process, we compute the accuracy of the final labels $\hat{y}_i$ obtained from $p_i^{(T+1)}$ relative to the true labels y_i. As shown in Figures 5.3 and 5.4, SLB substantially reduces the noise rate across different datasets. For instance, in the VS18-Real dataset, the proportion of correctly labelled samples increases from 68.56% to 80.72%. Under random noise conditions, SLB also achieves notable improvements: at a 70%



(a) Before (PE-30). (b) Before (VS18-30). (c) Before (IDS17-30). (d) Before (PE-Real). (e) Before (VS18-real).

(f) After (PE-30). (g) After (VS18-30). (h) After (IDS17-30). (i) After (PE-Real). (j) After (VS18-Real).

Figure 5.3: Confusion matrices of the labels before and after revision. Each subfigure is annotated with the dataset name and noise rate (*e.g.* "VS18-30" denotes VirusShare 2018 with 30% label noise).

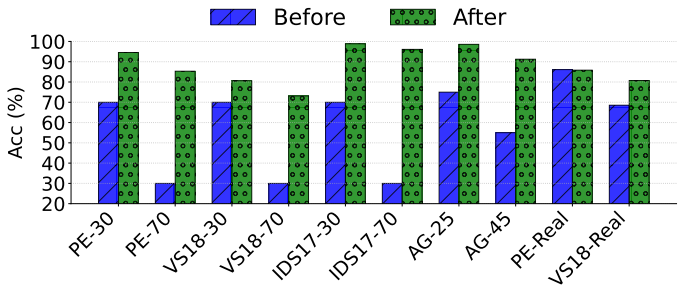


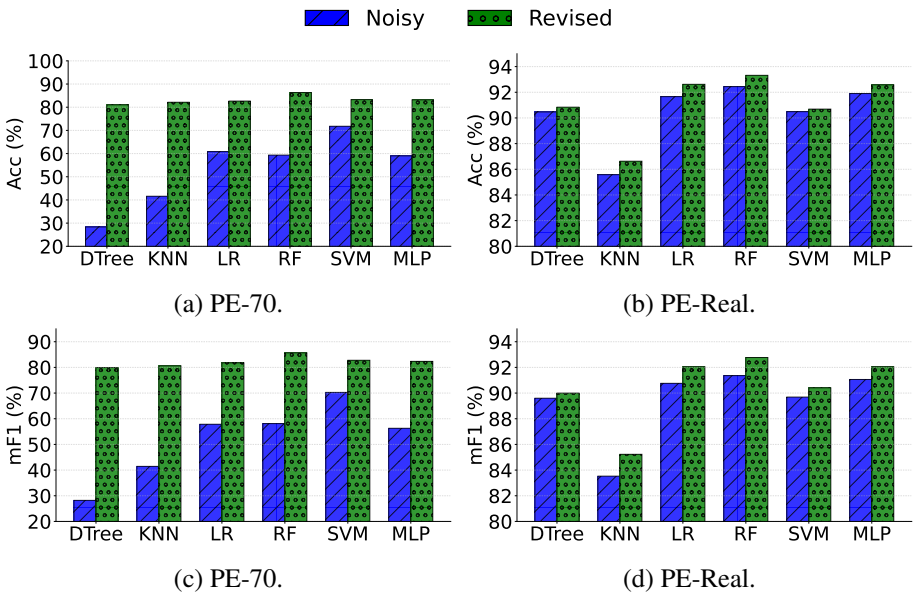
Figure 5.4: Label accuracy before and after correction.

noise rate, it lowers noise to under 15% in PE, 27% in VS18, and 4% in IDS17. On the binary AG dataset with 45% noise rate, SLB reduces it to below 9%.

Performance of ML Systems. We have demonstrated that SLB not only increases the resilience of DNN models to incorrect labels but also produces revised (corrected) labels with high accuracy. Although our initial experiments concentrated on deep models, classical ML algorithms can also be effective for detecting and classifying security samples in certain settings (*e.g.*, [34, 124, 7]). Accordingly, we used the SLB-revised labels to train five ML models: SVM, RF, DTree, Logistic Regression (LR), KNN, and a MLP. As illustrated in Figure 5.5, the revised labels significantly enhance the performance of these classifiers.

5.4.3 Sensitivity Analysis

We analyse SLB's robustness to architecture changes, component removal, and hyperparameter variations. Specifically, we test SLB on more complex deep learning architectures based on image representations of malware, replacing the original MLP; isolate key components such as continuous revision to measure their individual contributions under both real and synthetic noise; and evaluate the impact of key hyperparameters: the number of epochs e used during data split and the



(a) PE-70. (b) PE-Real.

(c) PE-70. (d) PE-Real.

Figure 5.5: Performance of machine learning classifiers trained on SLB revised labels.

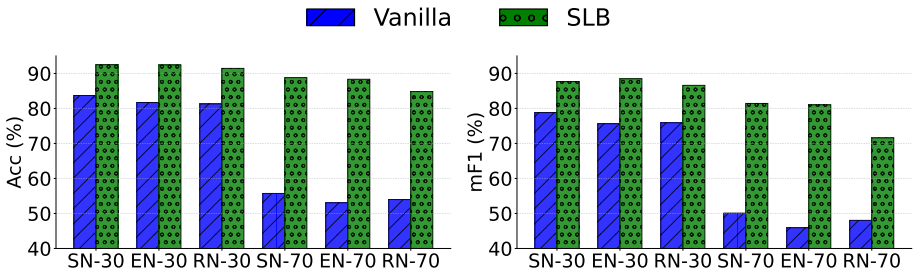
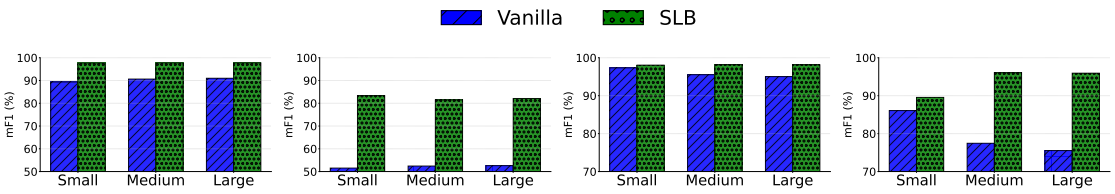


Figure 5.6: Performance across complex architectures using random noise. The x-axis tick labels indicate the architecture and the corresponding noise level (*e.g.* SN-70 represents a ShuffleNet model evaluated under 70% noise).

warm-up period m in continuous revision.

Performance on Complex Architectures. While SLB demonstrated robust performance on a simple deep learning model with three hidden layers, real-world applications often employ more complex architectures. To evaluate the impact of SLB on such models, we integrated it with three well-established image-based networks: ResNet18 (RN) [114], EfficientNet B0 (EN) [254], and ShuffleNet v2 (SN) [163]. We conducted experiments on the Virus-MNIST dataset [189], which provides image-based representations of malware. Due to the high computational cost (*e.g.*, ResNet18 comprises approximately 11.7 million parameters), we limited the experiments to five runs. As shown in Figure 5.6, SLB significantly enhances performance on the noisy dataset, indicating that the framework generalises well across a variety of deep learning architectures.

Large Datasets vs. Large Models. So far, we have shown that SLB consistently improves performance across both simple and complex models on relatively small datasets. An open question is whether these benefits extend to large-scale settings, where both dataset size and model capacity are substantially higher. To explore this, we evaluate SLB under random label noise on two large



(a) LargeAG (0.1). (b) LargeAG (0.45). (c) LargeIDS (0.1). (d) LargeIDS (0.7).

Figure 5.7: Performance of SLB across large datasets and model scales.

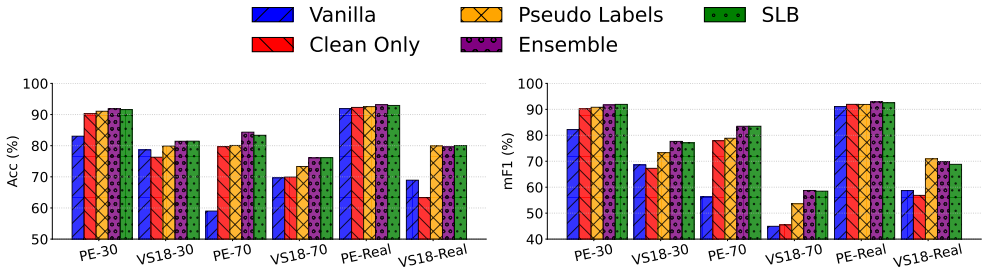
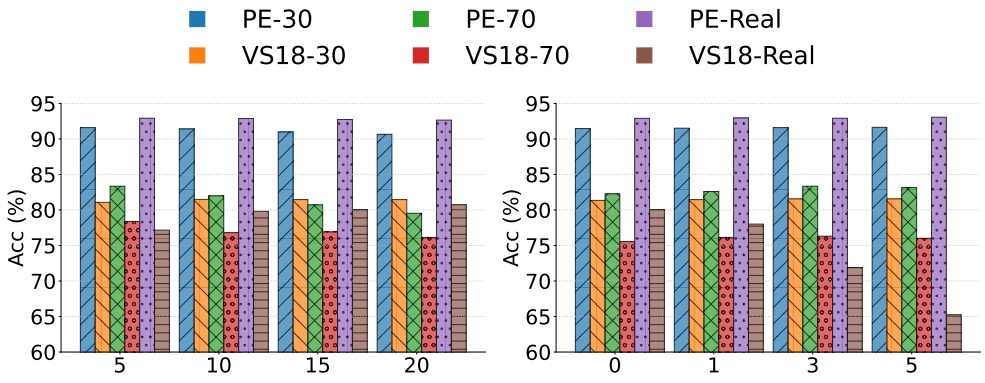


Figure 5.8: Ablation study.

benchmarks: LargeIDS (the full IDS17 dataset) and LargeAG (the complete APIGraph dataset from 2012–2018). For each dataset, we train three networks of increasing capacity: a small model ([512, 256, 128]), a medium model ([1024, 1024, 512]), and a large model ([1024, 2048, 1024]). To keep experiments computationally feasible, we fix the batch size to 1024 and report results at two representative noise rates: 10% and 70% for LargeIDS, and 10% and 45% for LargeAG. As shown in Figure 5.7, SLB consistently produces noise-resistant classifiers across different dataset sizes and model capacities, suggesting that its robustness extends to larger and more demanding scenarios.

Ablation Study. We conduct an ablation study with three alternative configurations to systematically assess our design choices. First, `Clean Only` omits the continuous revision step and trains a new model solely on the initially identified clean set. Second, `Pseudo Labels` also discards continuous revision but trains a new model on the entire dataset, relying on the pseudo labels generated by the majority vote. Third, we explore a more complex partitioning approach by training an ensemble of five models for e epochs each, generating a total of $5 \times e$ predictions per sample during the data split process. Figure 5.8 summarises the results of these three configurations. Overall, the findings suggest that preserving all SLB features yields the best results. While `Clean Only` shows decent results in certain cases, it proves inadequate for datasets like VS18-30 and VS18-Real. `Pseudo Labels` exceeds the `Vanilla` baseline yet typically falls short of the full SLB, aside from a slight mF1 advantage on VS18-Real. Finally, `Ensemble` delivers a small performance gain, up to 1.06% mF1 on VS18-Real, but increases computational costs. This makes `Ensemble` an optional extension of SLB where resource constraints are less critical.

Hyperparameters Sensitivity Figure 5.9 shows the sensitivity of SLB to its hyperparameters. As illustrated in Figure 5.9a, the hyperparameter e has a measurable but modest effect on per-



(a) Data Split Epochs (e). (b) Revision Warm-up (m).

Figure 5.9: Hyperparameter sensitivity.

formance. Notably, the PE-70 dataset exhibits the largest variation, with performance decreasing by 3.81 when e increases from 5 to 20. Based on these observations, we recommend setting e in the range of 5 to 15, which balances sufficient training iterations against the risk of overfitting to noise. Figure 5.9b analyses the impact of the hyperparameter m. For most datasets, the difference between the minimum and maximum performance values is modest (averaging around 0.384). However, VS18-Real is an exception; increasing m in this dataset significantly degrades performance, with scores dropping from 80.03 at $m = 0$ to 78.02 at $m = 1$, 71.88 at $m = 3$, and 65.23 at $m = 5$. This is because initial clean set contain approximately 20% noise. Consequently, we recommend keeping m low (between 0 and 3) to ensure prompt label corrections, enabling the SLB to quickly eliminate any noise present in the initial clean set.

5.5 Discussion

Application and Generalisation of SLB. Our experiments show that SLB performs well across varied settings, yet several obstacles remain before it can be adopted with confidence in operational environments. First, our real-label noise study focused mainly on VT reports, the most common labelling source for malware datasets, even though alternative methods (*e.g.*, [125, 220, 291, 77, 123]) may introduce different noise characteristics; future work should test SLB under these methods. Second, although we followed current practice by using two datasets for the real-noise evaluation [286, 273], broader empirical research is needed to build more comprehensive and representative benchmarks. Third, we evaluated SLB only on malware and NIDS datasets, while many other security tasks suffer from similar labelling problems. The DARPA host-based provenance datasets illustrate this gap; they provide only scenario-level summaries, forcing researchers to derive event-level labels with inconsistent heuristics that inject substantial and often hidden noise [123, 100, 211, 122, 51, 137]. Assessing SLB on such data and quantifying its resilience to this form of noise is therefore a promising direction for future study.

Fourth, although SLB is intended to be broadly applicable, it does not currently use domain- or protocol-specific knowledge that could improve correction reliability. For example, in flow-based NIDS data, attack-aware cues such as payload presence, transferred bytes, and traffic directionality

can help distinguish payload-driven attacks from scanning or denial of service activity. These cues can narrow the set of plausible label candidates before correction. Incorporating such lightweight signals as constraints or priors is a promising direction for reducing ambiguous relabelling and improving robustness. Finally, even though SLB has been validated on real datasets, its long-term performance and robustness in operational settings remain to be studied. As a result, further research is needed to confirm SLB's effectiveness over time in real-world environments.

Overhead of SLB. SLB introduces two additional components compared to the Vanilla baseline: a preliminary training phase for e epochs to perform the data split and a continuous label revision process during the main T-epoch training phase. These steps incur a modest overhead relative to the baseline. For example, on the VS18-30 dataset, the Vanilla model trains in approximately *44* seconds, while SLB takes about *50* seconds. In contrast, SoTA methods impose substantially higher costs, with MORSE taking roughly *133* seconds and DTm about *4284* seconds. Thus, we argue that the slight extra cost of SLB is justified by its significant performance gains.

Label-flipping Attacks. SLB is specifically designed to mitigate label noise resulting from legitimate user labelling methods. However, a closely related challenge is label flipping (or label poisoning) attacks, where an attacker deliberately injects label noise into the training set. Such attacks aim to degrade the classifier's overall performance or induce targeted misclassification [259]. We argue that most existing label noise mitigation techniques may be susceptible to these adversarial manipulations. Consequently, further research is needed to understand how poisoned samples can be crafted to remain undetected as clean, as well as to develop strategies to effectively counter these attacks. We consider this a critical avenue for future work.

5.6 Conclusion

In this chapter, we presented SLB, a framework that simultaneously trains a robust DNN classifier on noisy security data and corrects the dataset's labels. SLB operates in two stages. First, an epoch-based ensemble in the early learning phase flags likely correct labels and assigns pseudo labels to samples that appear mislabelled. Second, a continuous revision strategy progressively refines the noisy labels as training proceeds.

We conducted a comprehensive evaluation of SLB and found that it both strengthens the trained classifier and cleans the underlying data. With 70% random label noise, SLB boosts the mF1 score over a Vanilla baseline by 30.62% on IDS17, 13.55% on VS18, and 26.32% on PE. Under real-world noise, it achieves gains of 1.47% on PE-Real and 10.12% on VS18-Real. SLB also proves highly effective at label correction: on IDS17 with 30% injected noise, it reduces the noise to just 1.16%. Using these revised labels to train ML models further improves their performance—for example, an RF's mF1 increases by 27.56% on IDS17 with 70% noise.

Compared to SoTA methods, SLB delivers superior performance. At a 70% noise rate, it surpasses the strongest rival by 28.76% on IDS17, 4.78% on VS18, and 3.43% on PE, and under real-world noise, it outperforms these methods by 1.48% on PE-Real and 3.68% on VS18-Real. Moreover, it

is substantially more efficient, requiring only about 38% of MORSE's training time and just 1% of DTm's—roughly 2.6× and 86× faster, respectively.

We further evaluated SLB in several additional scenarios: (i) training on noise-free data, (ii) employing deeper network architectures, (iii) scaling to larger datasets and models, (iv) conducting component-level ablations, and (v) analysing hyperparameter sensitivity. When applied to datasets presumed clean, SLB does not impair performance. It also generalises across architectures—for instance, incorporating SLB into a ResNet-18 raises the mF1 score by 10.70% and 23.55% on an image-based malware dataset injected with 30% and 70% label noise, respectively. Each component of SLB contributes to its overall effectiveness, whilst a more computationally intensive ensemble variant yields an additional improvement of up to 1.06% in mF1. Finally, the framework displays reasonable sensitivity to its hyperparameters: across the tested range of m on most datasets, the average mF1 fluctuation remains low at 0.384.

These findings demonstrate that SLB can both train a robust classifier and reduce dataset noise, achieving SoTA performance with lower computational cost than previous methods. Nevertheless, SLB still has limitations that highlight promising directions for future research.

First, our study examined only VT–based real-label noise, even though other labelling practices can introduce different noise patterns. Second, we evaluated SLB solely on malware and NIDS datasets, whereas many additional security tasks also suffer from label noise. Future work should therefore test SLB across a wider range of security problems and construct new real-noise benchmarks that reflect diverse labelling methodologies. Third, our experiments assumed a static data distribution; assessing SLB's robustness under temporal shift remains an important open question. Fourth, we did not incorporate domain knowledge into SLB's design, which could improve the reliability of label corrections. Finally, we did not investigate SLB in the face of adversarial label-flipping attacks—an out-of-scope scenario here but still a relevant direction for future research.

Chapter 6

Conclusion

This thesis examined the resilience of DNN-based NIDS to two unavoidable data challenges: distribution shift and label noise. Distribution shift occurs whenever the traffic observed in deployment diverges from the data used for training—a constant in security, where attackers continuously devise new techniques and legitimate behaviour evolves over time. Label noise, by contrast, arises from the sheer scale of modern security datasets: with millions of flows or malware samples, exhaustive manual annotation is infeasible, so practitioners rely on automated labelling tools that inevitably introduce errors.

Because both problems originate in the training data, they can undermine even state-of-the-art models. We first address shifts in benign traffic with an adaptive zero-positive ensemble that reacts rapidly to diverse shift patterns. The ensemble integrates a lightweight shift detector, a coreset-style sampler, and a complexity-reduction module that prunes obsolete models and merges redundant ones, allowing it to track evolving benign behaviour with minimal labelling effort. In the multi-class setting, we propose a distance-based rejection function that flags traffic dissimilar to all known classes (*i.e.*, novel classes), paired with a sample-selection strategy that selects a small, informative, and diverse subset of samples for manual labelling and incremental model updates. To mitigate label noise, we introduce a self-labelling framework that first isolates a subset of highly reliable examples and then progressively corrects uncertain labels during training. By correcting the supervision signal on the fly, the framework preserves model performance while producing a substantially cleaner dataset.

In this concluding chapter, we first summarise the main contributions of the thesis (§6.1), then examine its limitations and outline research directions that may guide future work on DNN-based NIDS (§6.2), and finally offer our perspective on the role of DNNs in intrusion detection and their prospects moving forward (§6.3).

6.1 Summary of Thesis Achievements

In this thesis, we begin with a critical survey of recent techniques for mitigating distribution shift in DNN-based NIDS (Section 2.4) and label noise in malware and NIDS datasets (Section 2.5).

From this survey we derive three taxonomies: one that categorises shift types, another that groups shift-mitigation strategies, and a third that organises label-noise countermeasures. Our analysis shows that concept shift is one of the most serious challenges to DNN-based security models, appearing as benign-traffic shift in zero-positive learning, in-class evolution in both binary and multi-class settings, and the emergence of entirely new classes in multi-class tasks.

Next, we break a typical shift-mitigation workflow into five components: shift detection, sample selection, sample labelling, model adaptation, and robust learning. For each component we review the state of the art, noting strengths and limitations. For example, incremental learning is widely used for model adaptation because it preserves knowledge of earlier distributions; however, it is computationally intensive and may underfit when only a few samples represent a sharply different new distribution.

We then examine label noise in a broader security context that spans malware and NIDS datasets. Current learning approaches to counteract label noise fall into four modules: sample selection, label correction, robust learning, and semi-supervised learning. We examine each work in isolation, assessing its generalisability across binary and multi-class tasks and across different security domains. Each method offers distinct advantages and drawbacks. For instance, semi-supervised learning limits exposure to noisy labels by training on a small trusted subset, but the reduced supervised data can constrain the capacity of deep models to generalise. We close this discussion by outlining the gaps our work addresses and by suggesting additional open problems for future research.

Guided by these research gaps, we design three new systems. The first, presented in Chapter 3, tackles benign-traffic shift in zero-positive DNN-based NIDS that use AEs. We test two adaptation strategies, batch-based ensemble learning and fully incremental learning; each alone is unreliable, but together with a lightweight transfer-learning step they improve performance across every shift scenario. Three hurdles remain: detecting shift, limiting annotation cost, and containing ensemble growth. We solve them by (i) comparing traffic distributions to flag shift, (ii) selecting a coreset-style subset of flows that fits a user-specified labelling budget, and (iii) pruning weak models while merging redundant ones. Combined in the `Mateen` framework, these ideas handle diverse benign shifts and deliver state-of-the-art performance on all evaluated datasets.

The second system, presented in Chapter 4, is a plug-in module that sits beside any multi-class DNN-based NIDS to tackle novel classes. It introduces a customised distance-based function inspired by contrastive learning and center loss; this function flags traffic that lies far from all known class centroids while staying computationally light and robust to class imbalance. We pair the function with a sample selection strategy that selects a small, diverse, and informative subset of flows for manual annotation and incremental retraining. Combined as the `Rasd` framework, these components deliver high recall for novel classes detection and raise overall classifier performance even under tight labelling budgets.

The third system, presented in Chapter 5, helps security practitioners train noise-robust DNN classifiers on imperfect datasets. It builds on the observation that, in early epochs, networks learn genuine patterns from correctly labeled samples, and only later begin to memorize mislabeled

ones. To exploit this, an epoch-wise ensemble separates the data into a tentative clean set and a set of suspected noisy samples. A progressive correction stage then refines both sets, preserving the integrity of the clean set while updating dubious labels with ensemble-guided corrections when the evidence is strong. Together these steps form the `SLB` framework, which delivers classifiers that remain accurate under heavy label noise and returns a significantly cleaner dataset. To our knowledge, `SLB` is the current state-of-the-art defence against label noise in malware and NIDS datasets.

6.2 Limitations and Future Work

Shift Adaptation Cost. `Mateen` and `Rasd` accommodate distribution shifts through incremental learning, a strategy validated by earlier shift-aware ML-based security solutions [18, 48, 199]. The approach, however, brings two accumulating expenses. First, each adaptation round appends fresh flows to a replay buffer, so the training dataset expands without limit. Over months of deployment this growth can demand terabytes of storage and stretch retraining runs from hours to days. Second, every update, even after sample selection, still requires a security analyst to review the selected flows. Although our samplers restrict the batch to only a few instances, high-throughput networks or frequent shift can turn that modest requirement into a persistent drain on analyst time.

Future work could ease both pressures by pairing incremental learning with data-reduction techniques. Mature attack clusters could be distilled into signature rules and then removed from the replay buffer, while coreset selection could maintain a memory-bounded yet representative subset of benign traffic. Both ideas are well understood individually (*e.g.*, AI-assisted signature generation [131] and coresets that shrink training sets [182]), but integrating them into a shift-aware, DNN-driven NIDS remains an open and promising direction.

Labeling costs might be reduced further through pseudo-labeling. Although recent work has applied pseudo-labels to shift-aware DNN-based NIDS [18] and ML-based malware detectors [291], current techniques are still fragile: the noise they introduce can noticeably degrade classifier performance [129]. Designing more reliable pseudo-labeling strategies that enable learning without ground-truth labels, yet still enhance performance, therefore represents another valuable avenue for future research.

Synthetic Data. Most of our experiments rely on the IDS2017 and IDS2018 datasets. Both datasets were generated in a laboratory using automated attack scripts, so their traffic lacks the heterogeneity and stealth found in operational networks. Realistic traces with a substantial number of labelled security events are rarely released for open research [22]; consequently, many recent NIDS studies also depend on synthetic datasets, most often IDS2017 and IDS2018 [111, 144, 84, 67, 96, 150, 18]. Although this practice ensures that our results are comparable with the literature, it limits external validity. To lessen this risk we broadened the evaluation in the final technical chapter (§5) by incorporating four malware datasets, yet the absence of large, real-world network traces with ground-truth labels remains an open challenge for the community.

Looking forward, recent progress in LLMs may offer a path toward richer synthetic data. A practical design could link several LLM agents, each assigned a specific role, to operate a simulated enterprise network. For example, one group of agents might represent employees in a small real-estate company, another group could act as customers, and a separate set could play attackers. Early studies already show that LLM-driven agents are capable of sophisticated, automated penetration testing [62, 234, 112]. Extending this idea to full network simulations could yield diverse, realistic, and fully labelled traffic without exposing sensitive operational data.

Such simulated environments could move us toward an almost noise-free labelling regime. Because the simulator observes the complete sequence of agent actions, it can provide gold-standard ground truth, reducing the ambiguity inherent in manual and heuristic-based annotation. This could also ease the labelling bottleneck: LLMs could analyse shifted flows to propose high-confidence pseudo-labels, while triaging and cross-checking the uncertain cases surfaced by the `SLB` data split. Beyond labelling, the same setup would enable controlled, repeatable drift scenarios, improving the rigour and comparability of evaluations under non-stationarity.

Deployment challenges. Our systems were designed to meet the objectives set out in this thesis, but a production deployment would surface additional constraints that we treated as out of scope. The most immediate is throughput: an operational NIDS must process sustained high-rate links while also supporting periodic model updates. This creates pressure at both training and inference time. On the training side, shift-aware operation implies recurring retraining cycles that must complete within operational change windows, and that typically must be accompanied by the checks practitioners expect before promotion (*e.g.,* threshold re-tuning). These requirements interact directly with our architectures: `Mateen` incurs overhead from maintaining and updating an ensemble (including training multiple models, member evaluation and occasional merging), while `Rasd` relies on two coupled learners whose updates must remain synchronised. On the inference side, the cost can exceed a single forward pass per flow: `Rasd` combines classification with a novelty decision, introducing an additional computation path.

A second practical hurdle is resource scaling, spanning both storage and operational state. In realistic deployments, models are rarely ephemeral artifacts: ensembles, multiple model snapshots, and calibration state are often retained to support auditing, incident forensics, and safe rollback. This is precisely where `Mateen` can become expensive, since preserving past knowledge across shifts encourages retaining multiple members and historical checkpoints. `Rasd` similarly requires persistent reference statistics and decision thresholds for novelty scoring in addition to its two-model design. Beyond model artifacts, deployment also demands careful management of data retention (for monitoring and investigation) and of any replay or buffering used during updates; without explicit budgets, these components can grow into a non-negligible operational cost.

Finally, our evaluation largely abstracts away adversarial pressure, yet adversarial machine learning is central in a full security setting. An adaptive attacker may attempt evasion by shaping flows to resemble benign feature profiles, undermining reconstruction- and distance-based criteria, and may also attempt poisoning by manipulating the data stream that drives shift alarms, sample selection, and subsequent updates. The same attack surface extends to `SLB`: if an attacker can influence

the training dataset, they can inject carefully crafted instances that survive early-learning filters, bias label correction, and ultimately steer the deployed classifier.

Looking forward, these constraints motivate several concrete directions. Inference latency could be reduced by restructuring execution so that `Rasd`'s novelty component is invoked conditionally rather than by default (*e.g.,* when lightweight drift indicators trigger). Training-time latency could be addressed by engineering update cycles that are compatible with operational change windows, including lightweight but reliable validation. Scalability would benefit from explicit resource budgets over retained models, checkpoints, and replay data, coupled with stress-testing under realistic streaming load rather than offline replay alone. In parallel, a deployment-grade evaluation should include adaptive attackers and quantify both evasion and poisoning: the shift alarms and selection signals used by `Mateen` and `Rasd` should be tested against manipulation, and `SLB` should be evaluated under backdoor and data-poisoning regimes in which malicious samples are constructed to appear clean to early-learning heuristics.

Lack of Explanation. Our three frameworks successfully mitigate data issues such as distribution drift and label noise, yet their internal reasoning remains opaque. For instance, while `Rasd` and `Mateen` can detect distribution shifts, they do not explain which features changed, why the shift occurred, or why a particular flow is flagged as novel sample. Likewise, their sample-selection modules return only numerical scores, without revealing which attributes drive those scores or how they map onto the problem space. The same shortcoming appears in `SLB`: it labels some instances clean, corrects others, and progressively rejects noisy samples, yet it never links these actions to interpretable features that provides a clear rationale for its choices.

Looking ahead, a compelling research avenue is to devise explainability methods that connect a framework's internal feature space to the higher-level problem space in which practitioners reason. Concretely, `Rasd` and `Mateen` should augment every shift alert with the specific flow-level statistics whose distributions have moved (*e.g.*, a sudden drop in median payload size) and propose plausible network explanations (*e.g.*, a new QUIC-based service). Likewise, `SLB` should annotate every clean, corrected, or rejected label with the attributes that influenced the decision (*e.g.*, an impossible port pairing), enabling analysts to assess its plausibility. A promising technical direction is counterfactual explanation [270]: generating the smallest perturbation to a network-flow record that flips the verdict of the distance metric (*e.g.*, from shift to non-shift). The altered attributes directly highlight which features drive the model's judgement and translate seamlessly into actionable insights for practitioners.

Randomness in DNNs. We ran each shift-aware framework, `Rasd` and `Mateen`, only once, mirroring the common practice in NIDS research that assumes the stochasticity of deep networks has little practical impact [18, 295, 107, 48]. Yet recent computer-vision study shows that GPU type, architecture, weight initialisation, and random seed can materially change active-learning outcomes [121]. Motivated by these findings, we performed a pilot replication study for shift-aware NIDS and observed comparable sensitivity [247]. These results call into question the adequacy of single-run evaluations, even though we believe our main conclusions remain robust.

Unfortunately, a thorough repetition study is prohibitively expensive: our randomness experiment alone required 390 executions of `INSOMNIA`, each $\geq$ 1 hour on a single GPU; extending this protocol to all frameworks would demand thousands of GPU hours—well beyond our present budget.

This steep cost likely explains why most studies default to a single execution. To raise the field's empirical maturity, we propose a community-hosted evaluation platform, analogous to HuggingFace leaderboards for LLMs, that runs each submitted NIDS under diverse permutations of random seeds, hardware, and datasets. Centralising the compute in this way would make large-scale robustness testing feasible for all researchers and provide standardised reproducibility metrics.

6.3 Final Thoughts on DNN-based NIDS

We conclude this thesis with some final thoughts on using DNN for intrusion detection. Inspired by the breakthrough successes of DNNs and, more recently, LLMs in a variety of non-security domains [114, 254, 119, 71, 65, 41, 262, 2, 154], security researchers have increasingly imported these techniques into the intrusion detection context [78, 253, 12, 96, 89, 32]. While this transfer has driven notable progress, it has also introduced assumptions that misalign with the realities of network security and often lead to inflated performance claims. Two prominent examples are the treatment of data distributions as stationary during evaluation and the presumption of access to abundant, high-quality labelled datasets.

This thesis challenges these assumptions, arguing that they do not hold in practice and showing that their violation can significantly degrade model performance. In response, it contributes systems that enable DNN-based NIDS to adapt to distribution shift in multi-class and zero-positive settings and remain resilient against label noise. These contributions are model-centric, reflecting the main research direction at the time of writing (*e.g.*, [295, 194, 104, 312, 173, 188]). Looking forward, however, the next wave of advances will need to emphasise data-centric solutions (see [307] for a recent survey). Promising directions include constructing more realistic datasets that capture the diversity and stealth of real-world traffic, designing feature spaces that are inherently robust to both noise and shift, and developing expert-informed, data-driven strategies for denoising labels.

Together, these observations suggest that the path to practical and trustworthy DNN-based NIDS lies in refining and strengthening the data foundations on which these systems are built. By jointly advancing model-centric and data-centric approaches, the research community can move closer to intrusion detection systems that are both adaptive to real-world change and resilient to the imperfections of security data.

Bibliography

[1] Martín Abadi et al. "TensorFlow: A System for Large-Scale Machine Learning". In: *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016*. Ed. by Kimberly Keeton and Timothy Roscoe. USENIX Association, 2016, pp. 265–283. URL: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi.

[2] Josh Achiam et al. "Gpt-4 technical report". In: *arXiv preprint arXiv:2303.08774* (2023).

[3] Vincenzo Agate et al. "Enhancing IoT Network Security with Concept Drift-Aware Unsupervised Threat Detection". In: *IEEE Symposium on Computers and Communications, ISCC 2024, Paris, France, June 26-29, 2024*. IEEE, 2024, pp. 1–6. DOI: 10.1109/ISCC61673.2024.10733733. URL: https://doi.org/10.1109/ISCC61673.2024.10733733.

[4] Andrea Agiollo et al. "GNN4IFA: Interest Flooding Attack Detection With Graph Neural Networks". In: *8th IEEE European Symposium on Security and Privacy, EuroS&P 2023, Delft, Netherlands, July 3-7, 2023*. IEEE, 2023, pp. 615–630. DOI: 10.1109/EUROSP57164.2023.00043. URL: https://doi.org/10.1109/EuroSP57164.2023.00043.

[5] Cem Akkaya et al. "Amazon Mechanical Turk for Subjectivity Word Sense Disambiguation". In: *Proceedings of the 2010 Workshop on Creating Speech and Language Data with Amazon's Mechanical Turk, Los Angeles, USA, June 6, 2010*. Association for Computational Linguistics, 2010, pp. 195–203. URL: https://aclanthology.org/W10-0731/.

[6] Samed Al and Murat Dener. "STL-HDL: A new hybrid network intrusion detection system for imbalanced dataset on big data environment". In: *Computers & Security* 110 (2021), p. 102435.

[7] Almuthanna Alageel and Sergio Maffeis. "Hawk-Eye: holistic detection of APT command and control domains". In: *SAC '21: The 36th ACM/SIGAPP Symposium on Applied Computing, Virtual Event, Republic of Korea, March 22-26, 2021*. Ed. by Chih-Cheng Hung et al. ACM, 2021, pp. 1664–1673. DOI: 10.1145/3412841.3442040. URL: https://doi.org/10.1145/3412841.3442040.

[8] Kevin Allix et al. "Empirical assessment of machine learning-based malware detectors for Android - Measuring the gap between in-the-lab and in-the-wild validation scenarios". In: *Empir. Softw. Eng.* 21.1 (2016), pp. 183–211. DOI: 10.1007/S10664-014-9352-6. URL: https://doi.org/10.1007/s10664-014-9352-6.

[9] Fahad Alotaibi, Euan Goodbrand, and Sergio Maffeis. "Deep Learning from Imperfectly Labeled Malware Data". In: *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. CCS '25. Taipei, Taiwan: Association for Computing Machinery, 2025, pp. 3990–4004. ISBN: 9798400715259. DOI: 10.1145/3719027.3765197. URL: https://doi.org/10.1145/3719027.3765197.

[10] Fahad Alotaibi and Sergio Maffeis. "Mateen: Adaptive Ensemble Learning for Network Anomaly Detection". In: *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses*. RAID '24. Padua, Italy: Association for Computing Machinery, 2024, pp. 215–

234. ISBN: 9798400709593. DOI: 10.1145/3678890.3678901. URL: https://doi.org/10.1145/3678890.3678901.

[11] Fahad M. Alotaibi and Sergio Maffeis. "Rasd: Semantic Shift Detection and Adaptation for Network Intrusion Detection". In: *ICT Systems Security and Privacy Protection - 39th IFIP International Conference, SEC 2024, Edinburgh, UK, June 12-14, 2024, Proceedings*. Ed. by Nikolaos Pitropakis et al. Vol. 710. IFIP Advances in Information and Communication Technology. Springer, 2024, pp. 16–30. DOI: 10.1007/978-3-031-65175-5_2. URL: https://doi.org/10.1007/978-3-031-65175-5%5C_2.

[12] Ali Mohammed Alsaffar, Mostafa Nouri-Baygi, and Hamed M Zolbanin. "Shielding networks: enhancing intrusion detection with hybrid feature selection and stack ensemble learning". In: *Journal of Big Data* 11.1 (2024), p. 133.

[13] Abdulellah Alsaheel et al. "ATLAS: A Sequence-based Learning Approach for Attack Investigation". In: *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*. Ed. by Michael D. Bailey and Rachel Greenstadt. USENIX Association, 2021, pp. 3005–3022. URL: https://www.usenix.org/conference/usenixsecurity21/presentation/alsaheel.

[14] Suresh Kumar Amalapuram, Sumohana S. Channappayya, and Bheemarjuna Reddy Tamma. "Augmented Memory Replay-based Continual Learning Approaches for Network Intrusion Detection". In: *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. Ed. by Alice Oh et al. 2023. URL: http://papers.nips.cc/paper%5C_files/paper/2023/hash/3755a02b1035fbadd5f93a022170e46f-Abstract-Conference.html.

[15] Suresh Kumar Amalapuram, Bheemarjuna Reddy Tamma, and Sumohana S. Channappayya. "SPIDER: A Semi-Supervised Continual Learning-based Network Intrusion Detection System". In: *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications, Vancouver, BC, Canada, May 20-23, 2024*. IEEE, 2024, pp. 571–580. DOI: 10.1109/INFOCOM52122.2024.10621428. URL: https://doi.org/10.1109/INFOCOM52122.2024.10621428.

[16] Yonatan Amaru et al. "RAPID: Robust APT detection and investigation using context-aware deep learning". In: *arXiv preprint arXiv:2406.05362* (2024).

[17] Giuseppina Andresini et al. "A Network Intrusion Detection System for Concept Drifting Network Traffic Data". In: *Discovery Science - 24th International Conference, DS 2021, Halifax, NS, Canada, October 11-13, 2021, Proceedings*. Ed. by Carlos Soares and Luís Torgo. Vol. 12986. Lecture Notes in Computer Science. Springer, 2021, pp. 111–121. DOI: 10.1007/978-3-030-88942-5_9. URL: https://doi.org/10.1007/978-3-030-88942-5%5C_9.

[18] Giuseppina Andresini et al. "INSOMNIA: Towards Concept-Drift Robustness in Network Intrusion Detection". In: *AISec@CCS 2021: Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security, Virtual Event, Republic of Korea, 15 November 2021*. Ed. by Nicholas Carlini, Ambra Demontis, and Yizheng Chen. ACM, 2021, pp. 111–122. DOI: 10.1145/3474369.3486864. URL: https://doi.org/10.1145/3474369.3486864.

[19] Fabrizio Angiulli. "Fast Nearest Neighbor Condensation for Large Data Sets Classification". In: *IEEE Transactions on Knowledge and Data Engineering* (2007).

[20] Manos Antonakakis et al. "Understanding the Mirai Botnet". In: *26th USENIX Security Symposium, USENIX Security 2017, Vancouver, BC, Canada, August 16-18, 2017*. Ed. by Engin Kirda and Thomas Ristenpart. USENIX Association, 2017, pp. 1093–1110. URL: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis.

[21] Giovanni Apruzzese, Aurore Fass, and Fabio Pierazzi. "When Adversarial Perturbations meet Concept Drift: an Exploratory Analysis on ML-NIDS". In: *The 17th ACM Workshop on Artificial Intelligence Security (AISec)*. 2024.

[22] Giovanni Apruzzese, Pavel Laskov, and Johannes Schneider. "SoK: Pragmatic Assessment of Machine Learning for Network Intrusion Detection". In: *8th IEEE European Symposium on Security and Privacy, EuroS&P 2023, Delft, Netherlands, July 3-7, 2023*. IEEE, 2023, pp. 592–614. DOI: 10.1109/EUROSP57164.2023.00042. URL: https://doi.org/10.1109/EuroSP57164.2023.00042.

[23] Eric Arazo et al. "Unsupervised Label Noise Modeling and Loss Correction". In: *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2019, pp. 312–321. URL: http://proceedings.mlr.press/v97/arazo19a.html.

[24] Daniel Arp et al. "Dos and Don'ts of Machine Learning in Computer Security". In: *31st USENIX Security Symposium, USENIX Security 2022, Boston, MA, USA, August 10-12, 2022*. Ed. by Kevin R. B. Butler and Kurt Thomas. USENIX Association, 2022, pp. 3971–3988. URL: https://www.usenix.org/conference/usenixsecurity22/presentation/arp.

[25] Daniel Arp et al. "DREBIN: Effective and Explainable Detection of Android Malware in Your Pocket". In: *21st Annual Network and Distributed System Security Symposium, NDSS 2014, San Diego, California, USA, February 23-26, 2014*. The Internet Society, 2014. URL: https://www.ndss-symposium.org/ndss2014/drebin-effective-and-explainable-detection-android-malware-your-pocket.

[26] Daniel Arp et al. "Lessons Learned on Machine Learning for Computer Security". In: *IEEE Security & Privacy Magazine* (2023).

[27] Devansh Arpit et al. "A Closer Look at Memorization in Deep Networks". In: *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017*. Vol. 70. Proceedings of Machine Learning Research. PMLR, 2017, pp. 233–242. URL: http://proceedings.mlr.press/v70/arpit17a.html.

[28] Alejandro Barredo Arrieta et al. "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI". In: *Information Fusion* 58 (2020), pp. 82–115. DOI: 10.1016/j.inffus.2019.12.012. URL: https://doi.org/10.1016/j.inffus.2019.12.012.

[29] AustralianSuper. *Cyber Fraud Incident – Protecting Your Account*. Accessed 3 Jun 2025. AustralianSuper. Apr. 2025. URL: https://www.australiansuper.com/about-us/newsroom/2025/04/cyber-incident-and-service-issues-member-update-7-april-2025.

[30] Avast Threat Intelligence Team. *Additional information regarding the recent CCleaner APT security incident*. Accessed 3 Jun 2025. Avast Blog. Sept. 2017. URL: https://blog.avast.com/additional-information-regarding-the-recent-ccleaner-apt-security-incident.

[31] Avast Threat Intelligence Team. *New investigations into the CCleaner incident point to a possible third stage that had keylogger capacities*. Accessed 3 Jun 2025. Avast Blog. Mar. 2018. URL: https://blog.avast.com/new-investigations-in-ccleaner-incident-point-to-a-possible-third-stage-that-had-keylogger-capacities.

[32] Sukhvinder Singh Bamber et al. "A hybrid CNN-LSTM approach for intelligent cyber intrusion detection system". In: *Computers & Security* 148 (2025), p. 104146.

[33] BankInfoSecurity. *WannaCry Outbreak Hits Chipmaker, Could Cost $170 Million*. Aug. 7, 2018. URL: https://www.bankinfosecurity.com/chipmaker-tsmc-wannacry-attack-could-cost-us170-million-a-11285 (visited on 06/21/2025).

[34] Federico Barbero et al. "Transcending TRANSCEND: Revisiting Malware Classification in the Presence of Concept Drift". In: *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22-26, 2022*. IEEE, 2022, pp. 805–823. DOI: 10.1109/SP46214.2022.9833659. URL: https://doi.org/10.1109/SP46214.2022.9833659.

[35] BBC News. *Cyber attacks briefly knock out top sites*. Accessed 3 Jun 2025. BBC News. Oct. 2016. URL: https://www.bbc.co.uk/news/technology-37728015.

[36] Abhijit Bendale and Terrance E. Boult. "Towards Open Set Deep Networks". In: *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016, pp. 1563–1572. DOI: 10.1109/CVPR.2016.173. URL: https://doi.org/10.1109/CVPR.2016.173.

[37] Albert Bifet and Ricard Gavaldà. "Learning from Time-Changing Data with Adaptive Windowing". In: *Proceedings of the Seventh SIAM International Conference on Data Mining, April 26-28, 2007, Minneapolis, Minnesota, USA*. SIAM, 2007, pp. 443–448. DOI: 10.1137/1.9781611972771.42. URL: https://doi.org/10.1137/1.9781611972771.42.

[38] Albert Bifet, Geoffrey Holmes, and Bernhard Pfahringer. "Leveraging Bagging for Evolving Data Streams". In: *Machine Learning and Knowledge Discovery in Databases, European Conference, ECML PKDD 2010, Barcelona, Spain, September 20-24, 2010, Proceedings, Part I*. Ed. by José L. Balcázar et al. Vol. 6321. Lecture Notes in Computer Science. Springer, 2010, pp. 135–150. DOI: 10.1007/978-3-642-15880-3_15. URL: https://doi.org/10.1007/978-3-642-15880-3%5C_15.

[39] Leo Breiman. "Random forests". In: *Machine learning* 45 (2001), pp. 5–32.

[40] Robert A. Bridges et al. "A Survey of Intrusion Detection Systems Leveraging Host Data". In: *ACM Computing Surveys (CSUR)* 52.6 (2020), 128:1–128:35. DOI: 10.1145/3344382. URL: https://doi.org/10.1145/3344382.

[41] Tom B. Brown et al. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*. Ed. by Hugo Larochelle et al. 2020. URL: https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html.

[42] Nengfu Cai et al. "HCRP-OSD: Fine-Grained Open-Set Intrusion Detection Based on Hybrid Convolution and Adversarial Reciprocal Points Learning". In: *Concurrency and Computation: Practice and Experience* 37.4-5 (2025). DOI: 10.1002/CPE.70010. URL: https://doi.org/10.1002/cpe.70010.

[43] Saihua Cai et al. "CDDA-MD: An efficient malicious traffic detection method based on concept drift detection and adaptation technique". In: *Computers & Security* 148 (2025), p. 104121. DOI: 10.1016/J.COSE.2024.104121. URL: https://doi.org/10.1016/j.cose.2024.104121.

[44] Zhenquan Cai and Roland H. C. Yap. "Inferring the Detection Logic and Evaluating the Effectiveness of Android Anti-Virus Apps". In: *Proceedings of the Sixth ACM on Conference on Data and Application Security and Privacy, CODASPY 2016, New Orleans, LA, USA, March 9-11, 2016*. ACM, 2016, pp. 172–182. DOI: 10.1145/2857705.2857719. URL: https://doi.org/10.1145/2857705.2857719.

[45] Carlos Adrián Catania, Facundo Bromberg, and Carlos García Garino. "An autonomous labeling approach to support vector machines algorithms for network traffic anomaly detection". In: *Expert Systems with Applications* 39.2 (2012), pp. 1822–1829. DOI: 10.1016/J.ESWA.2011.08.068. URL: https://doi.org/10.1016/j.eswa.2011.08.068.

[46] Mahinthan Chandramohan, Hee Beng Kuan Tan, and Lwin Khin Shar. "Scalable malware clustering through coarse-grained behavior modeling". In: *20th ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE-20), SIGSOFT/FSE'12, Cary, NC, USA - November 11 - 16, 2012*. ACM, 2012, p. 27. DOI: 10.1145/2393596.2393627. URL: https://doi.org/10.1145/2393596.2393627.

[47] Ting Chen et al. "A Simple Framework for Contrastive Learning of Visual Representations". In: *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July*

2020, Virtual Event. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 1597–1607. URL: http://proceedings.mlr.press/v119/chen20j.html.

[48] Yizheng Chen, Zhoujie Ding, and David A. Wagner. "Continuous Learning for Android Malware Detection". In: *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023*. Ed. by Joseph A. Calandrino and Carmela Troncoso. USENIX Association, 2023, pp. 1127–1144. URL: https://www.usenix.org/conference/usenixsecurity23/presentation/chen-yizheng.

[49] Zekai Chen et al. "Learning Graph Structures With Transformer for Multivariate Time-Series Anomaly Detection in IoT". In: *IEEE Internet of Things Journal* (2022).

[50] Zhi Chen et al. "Is It Overkill? Analyzing Feature-Space Concept Drift in Malware Detectors". In: *2023 IEEE Security and Privacy Workshops (SPW), San Francisco, CA, USA, May 25, 2023*. IEEE, 2023, pp. 21–28. DOI: 10.1109/SPW59333.2023.00007. URL: https://doi.org/10.1109/SPW59333.2023.00007.

[51] Zijun Cheng et al. "Kairos: Practical Intrusion Detection and Investigation using Whole-system Provenance". In: *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*. IEEE, 2024, pp. 3533–3551. DOI: 10.1109/SP54263.2024.00005. URL: https://doi.org/10.1109/SP54263.2024.00005.

[52] Euijin Choo et al. "A Large Scale Study and Classification of VirusTotal Reports on Phishing and Malware URLs". In: *Proceedings of the ACM on Measurement and Analysis of Computing Systems (POMACS)* 7.3 (2023), 59:1–59:26. DOI: 10.1145/3626790. URL: https://doi.org/10.1145/3626790.

[53] Cisco Systems. *Cisco Consumer Security Alert: Millions of Users Threatened by Breach of Consumer Application, Piriform's CCleaner*. Accessed 3 Jun 2025. Cisco Systems Investor Relations. Sept. 2017. URL: https://investor.cisco.com/news/news-details/2017/Cisco-Consumer-Security-Alert-Millions-of-Users-Threatened-by-Breach-of-Consumer-Application-Piriforms-CCleaner/default.aspx.

[54] Cody Coleman et al. "Selection via Proxy: Efficient Data Selection for Deep Learning". In: *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. URL: https://openreview.net/forum?id=HJg2b0VYDr.

[55] Andrea Corsini and Shanchieh Jay Yang. "Are Existing Out-Of-Distribution Techniques Suitable for Network Intrusion Detection?" In: *IEEE Conference on Communications and Network Security, CNS 2023, Orlando, FL, USA, October 2-5, 2023*. IEEE, 2023, pp. 1–9. DOI: 10.1109/CNS59707.2023.10288685. URL: https://doi.org/10.1109/CNS59707.2023.10288685.

[56] Corinna Cortes and Vladimir Vapnik. "Support-vector networks". In: *Machine learning* 20.3 (1995), pp. 273–297.

[57] Thomas Cover and Peter Hart. "Nearest neighbor pattern classification". In: *IEEE transactions on information theory* 13.1 (1967), pp. 21–27.

[58] Steve Dias Da Cruz et al. "Autoencoder Attractors for Uncertainty Estimation". In: *26th International Conference on Pattern Recognition, ICPR 2022, Montreal, QC, Canada, August 21-25, 2022*. IEEE, 2022, pp. 2553–2560. DOI: 10.1109/ICPR56361.2022.9956240. URL: https://doi.org/10.1109/ICPR56361.2022.9956240.

[59] Ekin Dogus Cubuk et al. "RandAugment: Practical Automated Data Augmentation with a Reduced Search Space". In: *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*. Ed. by Hugo Larochelle et al. 2020. URL: https://proceedings.neurips.cc/paper/2020/hash/d85b63ef0ccb114d0a3bb7b7d808028f-Abstract.html.

[60] Jian Cui et al. "Tweezers: A Framework for Security Event Detection via Event Attribution-centric Tweet Embedding". In: *32nd Annual Network and Distributed System Security Symposium, NDSS*

2025, San Diego, California, USA, February 24-28, 2025. The Internet Society, 2025. URL: https://www.ndss-symposium.org/ndss-paper/tweezers-a-framework-for-security-event-detection-via-event-attribution-centric-tweet-embedding/.

[61] Yin Cui et al. "Class-Balanced Loss Based on Effective Number of Samples". In: *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019*. Computer Vision Foundation / IEEE, 2019, pp. 9268–9277. DOI: 10.1109/CVPR.2019.00949.

[62] Gelei Deng et al. "PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing". In: *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*. Ed. by Davide Balzarotti and Wenyuan Xu. USENIX Association, 2024. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/deng.

[63] Jia Deng et al. "ImageNet: A large-scale hierarchical image database". In: *2009 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2009), 20-25 June 2009, Miami, Florida, USA*. IEEE Computer Society, 2009, pp. 248–255. DOI: 10.1109/CVPR.2009.5206848. URL: https://doi.org/10.1109/CVPR.2009.5206848.

[64] Google Developers. *Classification: ROC and AUC*. Google Machine Learning Crash Course. 2024. URL: https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc (visited on 06/22/2025).

[65] Jacob Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*. Ed. by Jill Burstein, Christy Doran, and Thamar Solorio. Association for Computational Linguistics, 2019, pp. 4171–4186. DOI: 10.18653/V1/N19-1423. URL: https://doi.org/10.18653/v1/n19-1423.

[66] Terrance DeVries and Graham W Taylor. "Improved regularization of convolutional neural networks with cutout". In: *arXiv preprint arXiv:1708.04552* (2017).

[67] Tiago Fontes Dias et al. "Unravelling Network-Based Intrusion Detection: A Neutrosophic Rule Mining and Optimization Framework". In: *Computer Security. ESORICS 2023 International Workshops - CPS4CIP, ADIoT, SecAssure, WASP, TAURIN, PriST-AI, and SECAI, The Hague, The Netherlands, September 25-29, 2023, Revised Selected Papers, Part II*. Ed. by Sokratis K. Katsikas et al. Vol. 14399. Lecture Notes in Computer Science. Springer, 2023, pp. 59–75. DOI: 10.1007/978-3-031-54129-2_4. URL: https://doi.org/10.1007/978-3-031-54129-2%5C_4.

[68] Hailun Ding et al. "AIRTAG: Towards Automated Attack Investigation by Unsupervised Learning with Log Texts". In: *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023*. Ed. by Joseph A. Calandrino and Carmela Troncoso. USENIX Association, 2023, pp. 373–390. URL: https://www.usenix.org/conference/usenixsecurity23/presentation/ding-hailun-airtag.

[69] Simone Disabato and Manuel Roveri. "Learning convolutional neural networks in presence of concept drift". In: *2019 International Joint Conference on Neural Networks (IJCNN)*. IEEE. 2019, pp. 1–8.

[70] Gregory Ditzler and Robi Polikar. "Incremental Learning of Concept Drift from Streaming Imbalanced Data". In: *IEEE Transactions on Knowledge and Data Engineering* (2013).

[71] Alexey Dosovitskiy et al. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". In: *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL: https://openreview.net/forum?id=YicbFdNTTy.

[72] Gerard Draper-Gil et al. “Characterization of Encrypted and VPN Traffic using Time-related Features”. In: *Proceedings of the 2nd International Conference on Information Systems Security and Privacy, ICISSP 2016, Rome, Italy, February 19-21, 2016*. SciTePress, 2016, pp. 407–414. DOI: 10.5220/0005740704070414. URL: https://doi.org/10.5220/0005740704070414.

[73] Lei Du et al. “A Few-Shot Class-Incremental Learning Method for Network Intrusion Detection”. In: *IEEE Transactions on Network and Service Management* 21.2 (2024), pp. 2389–2401. DOI: 10.1109/TNSM.2023.3332284. URL: https://doi.org/10.1109/TNSM.2023.3332284.

[74] Min Du et al. “Lifelong Anomaly Detection Through Unlearning”. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*. Ed. by Lorenzo Cavallaro et al. ACM, 2019, pp. 1283–1297. DOI: 10.1145/3319535.3363226. URL: https://doi.org/10.1145/3319535.3363226.

[75] Richard O Duda and Peter E Hart. “Pattern classification and scene analysis”. In: *A Wiley-interscience publication* (1973).

[76] Ryan Elwell and Robi Polikar. “Incremental Learning of Concept Drift in Nonstationary Environments”. In: *IEEE Transactions on Neural Networks* (2011).

[77] Gints Engelen, Vera Rimmer, and Wouter Joosen. “Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study”. In: *IEEE Security and Privacy Workshops, SP Workshops 2021, San Francisco, CA, USA, May 27, 2021*. IEEE, 2021, pp. 7–12. DOI: 10.1109/SPW53761.2021.00009. URL: https://doi.org/10.1109/SPW53761.2021.00009.

[78] Osama Faker and Erdogan Dogdu. “Intrusion Detection Using Big Data and Deep Learning Techniques”. In: *Proceedings of the 2019 ACM Southeast Conference, ACM SE ’19, Kennesaw, GA, USA, April 18-20, 2019*. Ed. by Dan Lo, Donghyun Kim, and Eric Gamess. ACM, 2019, pp. 86–93. DOI: 10.1145/3299815.3314439. URL: https://doi.org/10.1145/3299815.3314439.

[79] Yasir Ali Farrukh et al. “AIS-NIDS: An intelligent and self-sustaining network intrusion detection system”. In: *Computers & Security* 144 (2024), p. 103982. DOI: 10.1016/J.COSE.2024.103982. URL: https://doi.org/10.1016/j.cose.2024.103982.

[80] Federal Trade Commission. *Equifax Data Breach Settlement*. Nov. 2024. URL: https://www.ftc.gov/enforcement/refunds/equifax-data-breach-settlement (visited on 06/21/2025).

[81] Helmut Finner and Veronika Gontscharuk. “Two-sample Kolmogorov–Smirnov-type tests revisited: old and new tests in terms of local levels”. In: *The Annals of Statistics* (2018).

[82] Fahimeh Fooladgar et al. “Manifold DivideMix: A Semi-Supervised Contrastive Learning Framework for Severe Label Noise”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024 - Workshops, Seattle, WA, USA, June 17-18, 2024*. IEEE, 2024, pp. 4012–4021. DOI: 10.1109/CVPRW63382.2024.00405. URL: https://doi.org/10.1109/CVPRW63382.2024.00405.

[83] Scott Freitas, Rahul Duggal, and Duen Horng Chau. “MalNet: A Large-Scale Image Database of Malicious Software”. In: *Proceedings of the 31st ACM International Conference on Information & Knowledge Management, Atlanta, GA, USA, October 17-21, 2022*. Ed. by Mohammad Al Hasan and Li Xiong. ACM, 2022, pp. 3948–3952. DOI: 10.1145/3511808.3557533. URL: https://doi.org/10.1145/3511808.3557533.

[84] Chuanpu Fu et al. “Point Cloud Analysis for ML-Based Malicious Traffic Detection: Reducing Majorities of False Positive Alarms”. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26-30, 2023*. Ed. by Weizhi Meng et al. ACM, 2023, pp. 1005–1019. DOI: 10.1145/3576915.3616631. URL: https://doi.org/10.1145/3576915.3616631.

[85] Chuanpu Fu et al. “Realtime Robust Malicious Traffic Detection via Frequency Domain Analysis”. In: *CCS ’21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*. Ed. by Yongdae Kim et al. ACM, 2021,

pp. 3431–3446. DOI: 10.1145/3460120.3484585. URL: https://doi.org/10.1145/3460120.3484585.

[86] Sean Fuhrman, Onat Güngör, and Tajana Rosing. "CND-IDS: Continual Novelty Detection for Intrusion Detection Systems". In: *CoRR* abs/2502.14094 (2025). DOI: 10.48550/ARXIV.2502.14094. arXiv: 2502.14094. URL: https://doi.org/10.48550/arXiv.2502.14094.

[87] João Gama, Raquel Sebastião, and Pedro Pereira Rodrigues. "On evaluating stream learning algorithms". In: *Machine learning* (2013).

[88] João Gama et al. "A survey on concept drift adaptation". In: *ACM Computing Surveys* (2014).

[89] Sunanda Gamage and Jagath Samarabandu. "Deep learning methods in network intrusion detection: A survey and an objective comparison". In: *Journal of Network and Computer Applications* 169 (2020), p. 102767.

[90] Han Gao, Shaoyin Cheng, and Weiming Zhang. "GDroid: Android malware detection and classification with graph convolutional network". In: *Computers & Security* 106 (2021), p. 102264. DOI: 10.1016/J.COSE.2021.102264. URL: https://doi.org/10.1016/j.cose.2021.102264.

[91] Cristiano Mesquita Garcia et al. "Concept Drift Adaptation in Text Stream Mining Settings: A Systematic Review". In: *ACM Trans. Intell. Syst. Technol.* 16.2 (2025), 27:1–27:67. DOI: 10.1145/3704922. URL: https://doi.org/10.1145/3704922.

[92] Saurabh Garg et al. "Complementary Benefits of Contrastive Learning and Self-Training Under Distribution Shift". In: *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. Ed. by Alice Oh et al. 2023. URL: http://papers.nips.cc/paper%5C_files/paper/2023/hash/26f96550613971371c5d07f37f0e06c0-Abstract-Conference.html.

[93] Jean-Gabriel Gaudreault and Paula Branco. "Detection of Novel Cyberattacks Using Multi-Binary Classifiers". In: *2024 34th International Conference on Collaborative Advances in Software and COmputiNg (CASCON)*. IEEE. 2024, pp. 1–9.

[94] Mathieu Germain et al. "MADE: Masked Autoencoder for Distribution Estimation". In: *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*. Ed. by Francis R. Bach and David M. Blei. Vol. 37. JMLR Workshop and Conference Proceedings. JMLR.org, 2015, pp. 881–889. URL: http://proceedings.mlr.press/v37/germain15.html.

[95] Jalal Ghadermazi et al. "GTAE-IDS: Graph Transformer-Based Autoencoder Framework for Real-Time Network Intrusion Detection". In: *IEEE Transactions on Information Forensics and Security.* 20 (2025), pp. 4026–4041. DOI: 10.1109/TIFS.2025.3557741. URL: https://doi.org/10.1109/TIFS.2025.3557741.

[96] Dimitrios Giagkos et al. "ZeekFlow: Deep Learning-Based Network Intrusion Detection a Multimodal Approach". In: *Computer Security. ESORICS 2023 International Workshops - CPS4CIP, ADIoT, SecAssure, WASP, TAURIN, PriST-AI, and SECAI, The Hague, The Netherlands, September 25-29, 2023, Revised Selected Papers, Part II*. Ed. by Sokratis K. Katsikas et al. Vol. 14399. Lecture Notes in Computer Science. Springer, 2023, pp. 409–425. DOI: 10.1007/978-3-031-54129-2_24. URL: https://doi.org/10.1007/978-3-031-54129-2%5C_24.

[97] Jacob Goldstein-Greenwood. *ROC Curves and AUC for Models Used for Binary Classification*. UVA Library StatLab article. 2022. URL: https://library.virginia.edu/data/articles/roc-curves-and-auc-for-models-used-for-binary-classification (visited on 06/22/2025).

[98] Dong Gong et al. "Memorizing Normality to Detect Anomaly: Memory-Augmented Deep Autoencoder for Unsupervised Anomaly Detection". In: *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019, Seoul, Korea (South), October 27 - November 2, 2019*. IEEE,

2019, pp. 1705–1714. DOI: 10.1109/ICCV.2019.00179. URL: https://doi.org/10.1109/ICCV.2019.00179.

[99] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016, pp. 168–227. URL: http://www.deeplearningbook.org.

[100] Akul Goyal, Gang Wang, and Adam Bates. "R-CAID: Embedding Root Cause Analysis within Provenance-based Intrusion Detection". In: *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*. IEEE, 2024, pp. 3515–3532. DOI: 10.1109/SP54263.2024.00253. URL: https://doi.org/10.1109/SP54263.2024.00253.

[101] Timothy Grance et al. *Guide to Information Technology Security Services*. NIST Special Publication 800-35. Gaithersburg, MD: National Institute of Standards and Technology (NIST), Oct. 2003. DOI: 10.6028/NIST.SP.800-35. URL: https://doi.org/10.6028/NIST.SP.800-35.

[102] Jorge L. Guerra, Carlos Adrián Catania, and Eduardo E. Veas. "Datasets are not enough: Challenges in labeling network traffic". In: *Computers & Security* 120 (2022), p. 102810. DOI: 10.1016/J.COSE.2022.102810. URL: https://doi.org/10.1016/j.cose.2022.102810.

[103] Xin Fan Guo et al. "Knowml: Improving generalization of ML-NIDS with attack knowledge graphs". In: *arXiv preprint arXiv:2506.19802* (2025).

[104] Ragini Gupta et al. "Generative Active Adaptation for Drifting and Imbalanced Network Intrusion Detection". In: *CoRR* abs/2503.03022 (2025). DOI: 10.48550/ARXIV.2503.03022. arXiv: 2503.03022. URL: https://doi.org/10.48550/arXiv.2503.03022.

[105] Raia Hadsell, Sumit Chopra, and Yann LeCun. "Dimensionality Reduction by Learning an Invariant Mapping". In: *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2006), 17-22 June 2006, New York, NY, USA*. IEEE Computer Society, 2006, pp. 1735–1742. DOI: 10.1109/CVPR.2006.100. URL: https://doi.org/10.1109/CVPR.2006.100.

[106] Bo Han et al. "Co-teaching: Robust training of deep neural networks with extremely noisy labels". In: *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*. Ed. by Samy Bengio et al. 2018, pp. 8536–8546. URL: https://proceedings.neurips.cc/paper/2018/hash/a19744e268754fb0148b017647355b7b-Abstract.html.

[107] Dongqi Han et al. "Anomaly Detection in the Open World: Normality Shift Detection, Explanation, and Adaptation". In: *30th Annual Network and Distributed System Security Symposium, NDSS 2023, San Diego, California, USA, February 27 - March 3, 2023*. The Internet Society, 2023. URL: https://www.ndss-symposium.org/ndss-paper/anomaly-detection-in-the-open-world-normality-shift-detection-explanation-and-adaptation/.

[108] Dongqi Han et al. "DeepAID: Interpreting and Improving Deep Learning-based Anomaly Detection in Security Applications". In: *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*. Ed. by Yongdae Kim et al. ACM, 2021, pp. 3197–3217. DOI: 10.1145/3460120.3484589. URL: https://doi.org/10.1145/3460120.3484589.

[109] Dongqi Han et al. "Evaluating and Improving Adversarial Robustness of Machine Learning-Based Network Intrusion Detectors". In: *IEEE Journal on Selected Areas in Communications* (2021).

[110] Xueying Han et al. "ECNet: Robust Malicious Network Traffic Detection With Multi-View Feature and Confidence Mechanism". In: *IEEE Transactions on Information Forensics and Security* 19 (2024), pp. 6871–6885. DOI: 10.1109/TIFS.2024.3426304. URL: https://doi.org/10.1109/TIFS.2024.3426304.

[111] Zijun Hang et al. "Flow-MAE: Leveraging Masked AutoEncoder for Accurate, Efficient and Robust Malicious Traffic Classification". In: *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses, RAID 2023, Hong Kong, China, October 16-18,*

2023. ACM, 2023, pp. 297–314. DOI: 10.1145/3607199.3607206. URL: https://doi.org/10.1145/3607199.3607206.

[112] Andreas Happe and Jürgen Cito. "Can LLMs Hack Enterprise Networks? Autonomous Assumed Breach Penetration-Testing Active Directory Networks". In: *arXiv preprint arXiv:2502.04227* (2025).

[113] Mohammad J. Hashemi, Greg Cusack, and Eric Keller. "Towards Evaluation of NIDSs in Adversarial Setting". In: *Proceedings of the 3rd ACM CoNEXT Workshop on Big DAta, Machine Learning and Artificial Intelligence for Data Communication Networks, Big-DAMA@CoNEXT 2019, Orlando, FL, USA, December 9, 2019*. ACM, 2019, pp. 14–21. DOI: 10.1145/3359992.3366642. URL: https://doi.org/10.1145/3359992.3366642.

[114] Kaiming He et al. "Deep Residual Learning for Image Recognition". In: *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90. URL: https://doi.org/10.1109/CVPR.2016.90.

[115] Zeyu He et al. "If in a Crowdsourced Data Annotation Pipeline, a GPT-4". In: *Proceedings of the CHI Conference on Human Factors in Computing Systems, CHI 2024, Honolulu, HI, USA, May 11-16, 2024*. ACM, 2024, 1040:1–1040:25. DOI: 10.1145/3613904.3642834. URL: https://doi.org/10.1145/3613904.3642834.

[116] Geoffrey Hinton et al. "Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups". In: *IEEE Signal processing magazine* 29.6 (2012), pp. 82–97.

[117] Seong-Cheol Hong et al. "Traffic growth analysis over three years in enterprise networks". In: *2009 15th Asia-Pacific Conference on Communications*. IEEE. 2009, pp. 896–899.

[118] Soumyadeep Hore et al. "A sequential deep learning framework for a robust and resilient network intrusion detection system". In: *Computers & Security* 144 (2024), p. 103928. DOI: 10.1016/J.COSE.2024.103928. URL: https://doi.org/10.1016/j.cose.2024.103928.

[119] Andrew G. Howard et al. "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications". In: *CoRR* abs/1704.04861 (2017). arXiv: 1704.04861. URL: http://arxiv.org/abs/1704.04861.

[120] Rahma Jablaoui and Noureddine Liouane. "An effective deep CNN-LSTM based intrusion detection system for network security". In: *International Conference on Control, Automation and Diagnosis, ICCAD 2024, Paris, France, May 15-17, 2024*. IEEE, 2024, pp. 1–6. DOI: 10.1109/ICCAD60883.2024.10553826. URL: https://doi.org/10.1109/ICCAD60883.2024.10553826.

[121] Yilin Ji et al. "Randomness is the Root of All Evil: More Reliable Evaluation of Deep Active Learning". In: *IEEE/CVF Winter Conference on Applications of Computer Vision, WACV 2023, Waikoloa, HI, USA, January 2-7, 2023*. IEEE, 2023, pp. 3932–3941. DOI: 10.1109/WACV56688.2023.00393. URL: https://doi.org/10.1109/WACV56688.2023.00393.

[122] Zian Jia et al. "MAGIC: Detecting Advanced Persistent Threats via Masked Graph Representation Learning". In: *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*. Ed. by Davide Balzarotti and Wenyuan Xu. USENIX Association, 2024. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/jia-zian.

[123] Baoxiang Jiang et al. "ORTHRUS: Achieving High Quality of Attribution in Provenance-based Intrusion Detection Systems". In: *Security Symposium (USENIX Sec'25). USENIX*. 2025.

[124] Roberto Jordaney et al. "Transcend: Detecting Concept Drift in Malware Classification Models". In: *26th USENIX Security Symposium, USENIX Security 2017, Vancouver, BC, Canada, August 16-18, 2017*. Ed. by Engin Kirda and Thomas Ristenpart. USENIX Association, 2017, pp. 625–642. URL: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/jordaney.

[125] Robert J Joyce et al. “Motif: A malware reference dataset with ground truth family labels”. In: *Computers & Security* 124 (2023), p. 102921.

[126] Md. Alamgir Kabir et al. “Assessing the Significant Impact of Concept Drift in Software Defect Prediction”. In: *43rd IEEE Annual Computer Software and Applications Conference, COMPSAC 2019, Milwaukee, WI, USA, July 15-19, 2019, Volume 1*. Ed. by Vladimir Getov et al. IEEE, 2019, pp. 53–58. DOI: 10.1109/COMPSAC.2019.00017. URL: https://doi.org/10.1109/COMPSAC.2019.00017.

[127] Hesham Kamal and Maggie Mashaly. “Advanced Hybrid Transformer-CNN Deep Learning Model for Effective Intrusion Detection Systems with Class Imbalance Mitigation Using Resampling Techniques”. In: *Future Internet* 16.12 (2024), p. 481.

[128] Hesham Kamal and Maggie Mashaly. “Enhanced Hybrid Deep Learning Models-Based Anomaly Detection Method for Two-Stage Binary and Multi-Class Classification of Attacks in Intrusion Detection Systems”. In: *Algorithms* 18.2 (2025), p. 69.

[129] Zeliang Kan et al. “Investigating Labelless Drift Adaptation for Malware Detection”. In: *AISec@CCS 2021: Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security, Virtual Event, Republic of Korea, 15 November 2021*. Ed. by Nicholas Carlini, Ambra Demontis, and Yizheng Chen. ACM, 2021, pp. 123–134. DOI: 10.1145/3474369.3486873. URL: https://doi.org/10.1145/3474369.3486873.

[130] ElMouatez Billah Karbab and Mourad Debbabi. “PetaDroid: Adaptive Android Malware Detection Using Deep Learning”. In: *Detection of Intrusions and Malware, and Vulnerability Assessment - 18th International Conference, DIMVA 2021, Virtual Event, July 14-16, 2021, Proceedings*. Vol. 12756. Lecture Notes in Computer Science. Springer, 2021, pp. 319–340. DOI: 10.1007/978-3-030-80825-9_16. URL: https://doi.org/10.1007/978-3-030-80825-9%5C_16.

[131] Sanmeet Kaur and Maninder Singh. “Hybrid intrusion detection and signature generation using deep recurrent neural networks”. In: *Neural Computing and Applications* 32.12 (2020), pp. 7859–7877.

[132] Nitish Shirish Keskar et al. “On large-batch training for deep learning: Generalization gap and sharp minima”. In: *5th International Conference on Learning Representations, ICLR 2017*. 2017.

[133] Prannay Khosla et al. “Supervised Contrastive Learning”. In: *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*. Ed. by Hugo Larochelle et al. 2020. URL: https://proceedings.neurips.cc/paper/2020/hash/d89a66c7c80a29b1bdbab0f2a1a94af8-Abstract.html.

[134] Ansam Khraisat et al. “Survey of intrusion detection systems: techniques, datasets and challenges”. In: *Cybersecurity* 2.1 (2019), pp. 1–22.

[135] KrishnaTeja Killamsetty et al. “GLISTER: Generalization based Data Subset Selection for Efficient and Robust Learning”. In: *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*. AAAI Press, 2021, pp. 8110–8118. DOI: 10.1609/AAAI.V35I9.16988. URL: https://doi.org/10.1609/aaai.v35i9.16988.

[136] KrishnaTeja Killamsetty et al. “GRAD-MATCH: Gradient Matching based Data Subset Selection for Efficient Deep Model Training”. In: *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*. Ed. by Marina Meila and Tong Zhang. Vol. 139. Proceedings of Machine Learning Research. PMLR, 2021, pp. 5464–5474. URL: http://proceedings.mlr.press/v139/killamsetty21a.html.

[137] Isaiah J. King et al. “EdgeTorrent: Real-time Temporal Graph Representations for Intrusion Detection”. In: *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions*

and Defenses, RAID 2023, Hong Kong, China, October 16-18, 2023. ACM, 2023, pp. 77–91. DOI: 10.1145/3607199.3607201. URL: https://doi.org/10.1145/3607199.3607201.

[138] Diederik P. Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*. Ed. by Yoshua Bengio and Yann LeCun. 2015. URL: http://arxiv.org/abs/1412.6980.

[139] Marius Kloft and Pavel Laskov. "Online Anomaly Detection under Adversarial Impact". In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, AISTATS 2010, Chia Laguna Resort, Sardinia, Italy, May 13-15, 2010*. Ed. by Yee Whye Teh and D. Mike Titterington. Vol. 9. JMLR Proceedings. JMLR.org, 2010, pp. 405–412. URL: http://proceedings.mlr.press/v9/kloft10a.html.

[140] David Korczynski and Heng Yin. "Capturing Malware Propagations with Code Injections and Code-Reuse Attacks". In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*. ACM, 2017, pp. 1691–1708. DOI: 10.1145/3133956.3134099. URL: https://doi.org/10.1145/3133956.3134099.

[141] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "Imagenet classification with deep convolutional neural networks". In: *Advances in neural information processing systems* 25 (2012).

[142] Aditya Kuppa and Nhien-An Le-Khac. "Learn to adapt: Robust drift detection in security domain". In: *Computers and Electrical Engineering* (2022).

[143] Aditya Kuppa et al. "Black Box Attacks on Deep Anomaly Detectors". In: *Proc. of ARES*. 2019.

[144] Maxime Lanvin et al. "Towards Understanding Alerts raised by Unsupervised Network Intrusion Detection Systems". In: *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses, RAID 2023, Hong Kong, China, October 16-18, 2023*. ACM, 2023, pp. 135–150. DOI: 10.1145/3607199.3607247. URL: https://doi.org/10.1145/3607199.3607247.

[145] Arash Habibi Lashkari et al. "Characterization of Tor Traffic using Time based Features". In: *Proceedings of the 3rd International Conference on Information Systems Security and Privacy, ICISSP 2017, Porto, Portugal, February 19-21, 2017*. Ed. by Paolo Mori, Steven Furnell, and Olivier Camp. SciTePress, 2017, pp. 253–262. DOI: 10.5220/0006105602530262. URL: https://doi.org/10.5220/0006105602530262.

[146] Legal Aid Agency and Ministry of Justice. *Legal Aid Agency data breach*. Accessed 3 Jun 2025. GOV.UK. May 2025. URL: https://www.gov.uk/government/news/legal-aid-agency-data-breach.

[147] Charles Leigh. *Hackers Steal $500,000 from Australian Super Funds*. Accessed 3 Jun 2025. University of Hawai'i–West O'ahu Cybersecurity Program. Apr. 2025. URL: https://westoahu.hawaii.edu/cyber/global-weekly-exec-summary/hackers-steal-500000-from-australian-super-funds/.

[148] Christophe Leys et al. "Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median". In: *Journal of experimental social psychology* 49.4 (2013), pp. 764–766.

[149] Adrian Shuai Li et al. "Revisiting Concept Drift in Windows Malware Detection: Adaptation to Real Drifted Malware with Minimal Samples". In: *32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24-28, 2025*. The Internet Society, 2025. URL: https://www.ndss-symposium.org/ndss-paper/revisiting-concept-drift-in-windows-malware-detection-adaptation-to-real-drifted-malware-with-minimal-samples/.

[150] Ruoyu Li et al. “Interpreting Unsupervised Anomaly Detection in Security via Rule Extraction”. In: *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. Ed. by Alice Oh et al. 2023. URL: http://papers.nips.cc/paper%5C_files/paper/2023/hash/c43b987f23fd5ea840df2b2be426315c-Abstract-Conference.html.

[151] Xiang Li et al. “On the Convergence of FedAvg on Non-IID Data”. In: *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. URL: https://openreview.net/forum?id=HJxNAnVtDS.

[152] XuKui Li et al. “Building Auto-Encoder Intrusion Detection System based on random forest feature selection”. In: *Computers & Security* (2020).

[153] Yuhua Li and Liam P. Maguire. “Selecting Critical Patterns Based on Local Geometrical and Statistical Information”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2011).

[154] Aixin Liu et al. “Deepseek-v3 technical report”. In: *arXiv preprint arXiv:2412.19437* (2024).

[155] Anjin Liu et al. “Accumulating regional density dissimilarity for concept drift detection in data streams”. In: *Pattern Recognition* (2018).

[156] Lisa Liu et al. “Error Prevalence in NIDS datasets: A Case Study on CIC-IDS-2017 and CSE-CIC-IDS-2018”. In: *10th IEEE Conference on Communications and Network Security, CNS 2022, Austin, TX, USA, October 3-5, 2022*. IEEE, 2022, pp. 254–262. DOI: 10.1109/CNS56114.2022.9947235. URL: https://doi.org/10.1109/CNS56114.2022.9947235.

[157] Sheng Liu et al. “Early-Learning Regularization Prevents Memorization of Noisy Labels”. In: *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*. 2020. URL: https://proceedings.neurips.cc/paper/2020/hash/ea89621bee7c88b2c5be6681c8ef4906-Abstract.html.

[158] Yunhui Liu et al. “OIDMD: A Novel Open-Set Intrusion Detection Method Based on Mahalanobis Distance”. In: *8th International Conference on Data Science in Cyberspace, DSC 2023, Hefei, China, August 18-20, 2023*. IEEE, 2023, pp. 253–260. DOI: 10.1109/DSC59305.2023.00044. URL: https://doi.org/10.1109/DSC59305.2023.00044.

[159] Jie Lu et al. “Learning under Concept Drift: A Review”. In: *IEEE Transactions on Knowledge and Data Engineering* 31.12 (2019), pp. 2346–2363. DOI: 10.1109/TKDE.2018.2876857. URL: https://doi.org/10.1109/TKDE.2018.2876857.

[160] Yang Lu, Yiu-Ming Cheung, and Yuan Yan Tang. “Adaptive Chunk-Based Dynamic Weighted Majority for Imbalanced Data Streams With Concept Drift”. In: *IEEE Transactions on Neural Networks and Learning Systems* (2020).

[161] Yangdi Lu and Wenbo He. “SELC: Self-Ensemble Label Correction Improves Learning with Noisy Labels”. In: *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI 2022, Vienna, Austria, 23-29 July 2022*. ijcai.org, 2022, pp. 3278–3284. DOI: 10.24963/IJCAI.2022/455. URL: https://doi.org/10.24963/ijcai.2022/455.

[162] Xiang Luo et al. “Identifying malicious traffic under concept drift based on intraclass consistency enhanced variational autoencoder”. In: *Science China Information Sciences* 67.8 (2024). DOI: 10.1007/S11432-023-4010-4. URL: https://doi.org/10.1007/s11432-023-4010-4.

[163] Ningning Ma et al. “ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design”. In: *Computer Vision - ECCV 2018 - 15th European Conference, Munich, Germany, September 8-14, 2018, Proceedings, Part XIV*. Vol. 11218. Lecture Notes in Computer Science. Springer, 2018, pp. 122–138. DOI: 10.1007/978-3-030-01264-9_8. URL: https://doi.org/10.1007/978-3-030-01264-9%5C_8.

[164] Shang Ma et al. “Careful About What App Promotion Ads Recommend! Detecting and Explaining Malware Promotion via App Promotion Graph”. In: *32nd Annual Network and Distributed Sys-*

tem Security Symposium, NDSS 2025, San Diego, California, USA, February 24-28, 2025. The Internet Society, 2025. URL: https://www.ndss-symposium.org/ndss-paper/careful-about-what-app-promotion-ads-recommend-detecting-and-explaining-malware-promotion-via-app-promotion-graph/.

[165] Xingjun Ma et al. "Normalized Loss Functions for Deep Learning with Noisy Labels". In: *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 6543–6553. URL: http://proceedings.mlr.press/v119/ma20c.html.

[166] James MacQueen. "Some methods for classification and analysis of multivariate observations". In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics*. Vol. 5. University of California press. 1967, pp. 281–298.

[167] AA Makarov and GI Simonova. "Some properties of two-sample kolmogorov–smirnov test in the case of contamination of one of the samples". In: *Journal of Mathematical Sciences* (2016).

[168] Mandiant, Google Cloud. *M-Trends 2024: Our View from the Frontlines*. Tech. rep. Accessed: 2025-09-02. Mandiant, a Google Cloud Company, 2024. URL: https://cloud.google.com/blog/topics/threat-intelligence/m-trends-2024.

[169] David M Mason and John H Schuenemeyer. "A modified Kolmogorov-Smirnov test sensitive to tail alternatives". In: *The Annals of Statistics* (1983).

[170] Dominic Masters and Carlo Luschi. "Revisiting small batch training for deep neural networks". In: *arXiv preprint arXiv:1804.07612* (2018).

[171] Warren S McCulloch and Walter Pitts. "A logical calculus of the ideas immanent in nervous activity". In: *The bulletin of mathematical biophysics* 5.4 (1943), pp. 115–133.

[172] Brendan McMahan et al. "Communication-Efficient Learning of Deep Networks from Decentralized Data". In: *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, AISTATS 2017, 20-22 April 2017, Fort Lauderdale, FL, USA*. Ed. by Aarti Singh and Xiaojin (Jerry) Zhu. Vol. 54. Proceedings of Machine Learning Research. PMLR, 2017, pp. 1273–1282. URL: http://proceedings.mlr.press/v54/mcmahan17a.html.

[173] Qianwei Meng et al. "Beyond known threats: A novel strategy for isolating and detecting unknown malicious traffic". In: *Journal of Information Security and Applications* 89 (2025), p. 103920. DOI: 10.1016/J.JISA.2024.103920. URL: https://doi.org/10.1016/j.jisa.2024.103920.

[174] Aleksandar Milenkoski et al. "Evaluating computer intrusion detection systems: A survey of common practices". In: *ACM Computing Surveys (CSUR)* 48.1 (2015), pp. 1–41.

[175] Brad Miller et al. "Reviewer Integration and Performance Measurement for Malware Detection". In: *Detection of Intrusions and Malware, and Vulnerability Assessment - 13th International Conference, DIMVA 2016, San Sebastián, Spain, July 7-8, 2016, Proceedings*. Vol. 9721. Lecture Notes in Computer Science. Springer, 2016, pp. 122–141. DOI: 10.1007/978-3-319-40667-1_7. URL: https://doi.org/10.1007/978-3-319-40667-1%5C_7.

[176] Tomás Concepción Miranda et al. "Debiasing Android Malware Datasets: How Can I Trust Your Results If Your Dataset Is Biased?" In: *IEEE Transactions on Information Forensics and Security (TIFS)* 17 (2022), pp. 2182–2197. DOI: 10.1109/TIFS.2022.3180184. URL: https://doi.org/10.1109/TIFS.2022.3180184.

[177] Yisroel Mirsky et al. "Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection". In: *25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18-21, 2018*. The Internet Society, 2018. URL: http://wp.internetsociety.org/ndss/wp-content/uploads/sites/25/2018/02/ndss2018%5C_03A-3%5C_Mirsky%5C_paper.pdf.

[178] Baharan Mirzasoleiman, Jeff A. Bilmes, and Jure Leskovec. "Coresets for Data-efficient Training of Machine Learning Models". In: *Proceedings of the 37th International Conference on Machine*

Learning, ICML 2020, 13-18 July 2020, Virtual Event. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 6950–6960. URL: http://proceedings.mlr.press/v119/mirzasoleiman20a.html.

[179] Abedelaziz Mohaisen and Omar Alrawi. "Unveiling Zeus: automated classification of malware samples". In: *22nd International World Wide Web Conference, WWW '13, Rio de Janeiro, Brazil, May 13-17, 2013, Companion Volume*. International World Wide Web Conferences Steering Committee / ACM, 2013, pp. 829–832. DOI: 10.1145/2487788.2488056. URL: https://doi.org/10.1145/2487788.2488056.

[180] Aziz Mohaisen and Omar Alrawi. "AV-Meter: An Evaluation of Antivirus Scans and Labels". In: *Detection of Intrusions and Malware, and Vulnerability Assessment - 11th International Conference, DIMVA 2014, Egham, UK, July 10-11, 2014. Proceedings*. Vol. 8550. Lecture Notes in Computer Science. Springer, 2014, pp. 112–131. DOI: 10.1007/978-3-319-08509-8_7. URL: https://doi.org/10.1007/978-3-319-08509-8%5C_7.

[181] Jose G. Moreno-Torres et al. "A unifying view on dataset shift in classification". In: *Pattern Recognition* (2012).

[182] Brian B Moser et al. "A Coreset Selection of Coreset Selection Literature: Introduction and Recent Advances". In: *arXiv preprint arXiv:2505.17799* (2025).

[183] Vinod Nair and Geoffrey E. Hinton. "Rectified Linear Units Improve Restricted Boltzmann Machines". In: *Proceedings of the 27th International Conference on Machine Learning (ICML-10), June 21-24, 2010, Haifa, Israel*. Ed. by Johannes Fürnkranz and Thorsten Joachims. Omnipress, 2010, pp. 807–814. URL: https://icml.cc/Conferences/2010/papers/432.pdf.

[184] Behnam Neyshabur, Hanie Sedghi, and Chiyuan Zhang. "What is being transferred in transfer learning?" In: *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*. Ed. by Hugo Larochelle et al. 2020. URL: https://proceedings.neurips.cc/paper/2020/hash/0607f4c705595b911a4f3e7a127b44e0-Abstract.html.

[185] Anh Mai Nguyen, Jason Yosinski, and Jeff Clune. "Deep neural networks are easily fooled: High confidence predictions for unrecognizable images". In: *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015*. IEEE Computer Society, 2015, pp. 427–436. DOI: 10.1109/CVPR.2015.7298640. URL: https://doi.org/10.1109/CVPR.2015.7298640.

[186] Duc Tam Nguyen et al. "SELF: Learning to Filter Noisy Labels with Self-Ensembling". In: *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. URL: https://openreview.net/forum?id=HkgsPhNYPS.

[187] Quoc Phong Nguyen et al. "GEE: A Gradient-based Explainable Variational Autoencoder for Network Anomaly Detection". In: *7th IEEE Conference on Communications and Network Security, CNS 2019, Washington, DC, USA, June 10-12, 2019*. IEEE, 2019, pp. 91–99. DOI: 10.1109/CNS.2019.8802833. URL: https://doi.org/10.1109/CNS.2019.8802833.

[188] Xuan-Ha Nguyen and Kim-Hung Le. "nNFST: A single-model approach for multiclass novelty detection in network intrusion detection systems". In: *Journal of Network and Computer Applications* 236 (2025), p. 104128. DOI: 10.1016/J.JNCA.2025.104128. URL: https://doi.org/10.1016/j.jnca.2025.104128.

[189] David A. Noever and Samantha E. Miller Noever. "Virus-MNIST: A Benchmark Malware Dataset". In: *CoRR* abs/2103.00602 (2021). arXiv: 2103.00602. URL: https://arxiv.org/abs/2103.00602.

[190] Curtis G. Northcutt, Anish Athalye, and Jonas Mueller. "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks". In: *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December*

2021, virtual. 2021. URL: https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/f2217062e9a397a1dca429e7d70bc6ca-Abstract-round1.html.

[191] Curtis G. Northcutt, Lu Jiang, and Isaac L. Chuang. "Confident Learning: Estimating Uncertainty in Dataset Labels". In: *Journal of Artificial Intelligence Research (JAIR)* 70 (2021), pp. 1373–1411. DOI: 10.1613/JAIR.1.12125. URL: https://doi.org/10.1613/jair.1.12125.

[192] Angelo Oliveira. *Malware Analysis Datasets: Raw PE as Image*. 2019. DOI: 10.21227/8brp-j220. URL: https://dx.doi.org/10.21227/8brp-j220.

[193] Palo Alto Networks. *What's New in PAN-OS Nebula 10.2*. URL: https://www.paloaltonetworks.com/network-security/whats-new-in-nebula (visited on 06/21/2025).

[194] Mengying Pan et al. "Anomaly Detection Under Normality-Shifted IoT Scenario: Filter, Detection, and Adaption". In: *Wireless Artificial Intelligent Computing Systems and Applications - 18th International Conference, WASA 2024, Qindao, China, June 21-23, 2024, Proceedings, Part II*. Ed. by Zhipeng Cai et al. Vol. 14998. Lecture Notes in Computer Science. Springer, 2024, pp. 426–438. DOI: 10.1007/978-3-031-71467-2_34. URL: https://doi.org/10.1007/978-3-031-71467-2%5C_34.

[195] Cheong Hee Park and Youngsoon Kang. "An active learning method for data streams with concept drift". In: *2016 IEEE International Conference on Big Data (Big Data)*. IEEE. 2016, pp. 746–752.

[196] Daniel S. Park et al. "SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition". In: *20th Annual Conference of the International Speech Communication Association, Interspeech 2019, Graz, Austria, September 15-19, 2019*. Ed. by Gernot Kubin and Zdravko Kacic. ISCA, 2019, pp. 2613–2617. DOI: 10.21437/INTERSPEECH.2019-2680. URL: https://doi.org/10.21437/Interspeech.2019-2680.

[197] Adam Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*. Ed. by Hanna M. Wallach et al. 2019, pp. 8024–8035. URL: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[198] Marek Pawlicki, Rafal Kozik, and Michal Choras. "Towards Deployment Shift Inhibition Through Transfer Learning in Network Intrusion Detection". In: *ARES 2022: The 17th International Conference on Availability, Reliability and Security, Vienna,Austria, August 23 - 26, 2022*. ACM, 2022, 58:1–58:6. DOI: 10.1145/3538969.3544428. URL: https://doi.org/10.1145/3538969.3544428.

[199] Feargus Pendlebury et al. "TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time". In: *28th USENIX Security Symposium, USENIX Security 2019, Santa Clara, CA, USA, August 14-16, 2019*. Ed. by Nadia Heninger and Patrick Traynor. USENIX Association, 2019, pp. 729–746. URL: https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury.

[200] Peng Peng et al. "Opening the Blackbox of VirusTotal: Analyzing Online Phishing Scan Engines". In: *Proceedings of the Internet Measurement Conference, IMC 2019, Amsterdam, The Netherlands, October 21-23, 2019*. ACM, 2019, pp. 478–485. DOI: 10.1145/3355369.3355585. URL: https://doi.org/10.1145/3355369.3355585.

[201] Guolou Ping. "OpenCADP: Open-Set Intrusion Detection with a Cluster Anomaly Detection Plugin". In: *IEEE International Conference on Systems, Man, and Cybernetics, SMC 2023, Honolulu, Oahu, HI, USA, October 1-4, 2023*. IEEE, 2023, pp. 1951–1956. DOI: 10.1109/SMC53992.2023.10393991. URL: https://doi.org/10.1109/SMC53992.2023.10393991.

[202] Yuqi Qing et al. "Low-Quality Training Data Only? A Robust Framework for Detecting Encrypted Malicious Network Traffic". In: *31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26 - March 1, 2024*. The Internet Society,

2024. URL: https://www.ndss-symposium.org/ndss-paper/low-quality-training-data-only-a-robust-framework-for-detecting-encrypted-malicious-network-traffic/.

[203] Zhiyin Qiu et al. "VAEMax: Open-set intrusion detection based on OpenMax and variational autoencoder". In: *2024 5th Information Communication Technologies Conference (ICTC)*. IEEE. 2024, pp. 98–105.

[204] Miguel Quebrado, Edoardo Serra, and Alfredo Cuzzocrea. "Android Malware Identification and Polymorphic Evolution Via Graph Representation Learning". In: *2021 IEEE International Conference on Big Data (Big Data), Orlando, FL, USA, December 15-18, 2021*. IEEE, 2021, pp. 5441–5449. DOI: 10.1109/BIGDATA52589.2021.9671437. URL: https://doi.org/10.1109/BigData52589.2021.9671437.

[205] J. Ross Quinlan. "Induction of decision trees". In: *Machine learning* 1.1 (1986), pp. 81–106.

[206] Adityanarayanan Radhakrishnan, Mikhail Belkin, and Caroline Uhler. "Overparameterized neural networks implement associative memory". In: *Proceedings of the National Academy of Sciences* 117.44 (2020), pp. 27162–27170.

[207] Moheeb Abu Rajab et al. "CAMP: Content-Agnostic Malware Protection". In: *20th Annual Network and Distributed System Security Symposium, NDSS 2013, San Diego, California, USA, February 24-27, 2013*. The Internet Society, 2013. URL: https://www.ndss-symposium.org/ndss2013/camp-content-agnostic-malware-protection.

[208] Sebastian Raschka. "Model evaluation, model selection, and algorithm selection in machine learning". In: *arXiv preprint arXiv:1811.12808* (2018).

[209] Cyrus Rashtchian et al. "Collecting Image Annotations Using Amazon's Mechanical Turk". In: *Proceedings of the 2010 Workshop on Creating Speech and Language Data with Amazon's Mechanical Turk, Los Angeles, USA, June 6, 2010*. Association for Computational Linguistics, 2010, pp. 139–147. URL: https://aclanthology.org/W10-0721/.

[210] Scott E. Reed et al. "Training Deep Neural Networks on Noisy Labels with Bootstrapping". In: *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Workshop Track Proceedings*. 2015. URL: http://arxiv.org/abs/1412.6596.

[211] Mati Ur Rehman, Hadi Ahmadi, and Wajih Ul Hassan. "Flash: A Comprehensive Approach to Intrusion Detection via Provenance Graph Representation Learning". In: *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*. IEEE, 2024, pp. 3552–3570. DOI: 10.1109/SP54263.2024.00139. URL: https://doi.org/10.1109/SP54263.2024.00139.

[212] Mengye Ren et al. "Learning to Reweight Examples for Robust Deep Learning". In: *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*. Ed. by Jennifer G. Dy and Andreas Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018, pp. 4331–4340. URL: http://proceedings.mlr.press/v80/ren18a.html.

[213] Pengzhen Ren et al. "A Survey of Deep Active Learning". In: *ACM Computing Surveys* (2022).

[214] Reuters. *UnitedHealth to take up to $1.6 billion hit this year from Change hack*. Apr. 16, 2024. URL: https://www.reuters.com/business/healthcare-pharmaceuticals/unitedhealth-warns-115-135share-hit-this-year-hack-2024-04-16/ (visited on 06/21/2025).

[215] Phillip Rieger et al. "ARGUS: Context-Based Detection of Stealthy IoT Infiltration Attacks". In: *32nd USENIX Security Symposium (USENIX Security 23)*. 2023.

[216] Ishai Rosenberg et al. "Adversarial Machine Learning Attacks and Defense Methods in the Cyber Security Domain". In: *ACM Computing Surveys (CSUR)* 54.5 (2022), 108:1–108:36. DOI: 10.1145/3453158. URL: https://doi.org/10.1145/3453158.

[217] Frank Rosenblatt. “The perceptron: a probabilistic model for information storage and organization in the brain.” In: *Psychological review* 65.6 (1958), p. 386.

[218] Daniel J. Rosenkrantz, Richard Edwin Stearns, and Philip M. Lewis II. “An Analysis of Several Heuristics for the Traveling Salesman Problem”. In: *SIAM Journal on Computing* 6.3 (1977), pp. 563–581. DOI: 10.1137/0206041. URL: https://doi.org/10.1137/0206041.

[219] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors”. In: *nature* 323.6088 (1986), pp. 533–536.

[220] Aleieldin Salem. “Towards Accurate Labeling of Android Apps for Reliable Malware Detection”. In: *CODASPY ’21: Eleventh ACM Conference on Data and Application Security and Privacy, Virtual Event, USA, April 26-28, 2021*. ACM, 2021, pp. 269–280. DOI: 10.1145/3422337.3447849. URL: https://doi.org/10.1145/3422337.3447849.

[221] Raisa Imani Sani and M Agni Catur Bhakti. “Intrusion Detection System Using Convolution Neural Network with Feature Selection for Multiclass Classification”. In: *2024 Ninth International Conference on Informatics and Computing (ICIC)*. IEEE. 2024, pp. 1–6.

[222] SC Media (CyberRisk Alliance). *Colorado firm claims ransomware attack behind closure*. Sept. 14, 2018. URL: https://www.scworld.com/news/colorado-firm-claims-ransomware-attack-behind-closure (visited on 06/21/2025).

[223] Karen Scarfone and Peter Mell. *Guide to Intrusion Detection and Prevention Systems (IDPS). Draft Revision 1*. NIST Special Publication 800-94 Rev. 1 (Draft). Draft for public comment; accessed 3 Jun 2025. Gaithersburg, MD: National Institute of Standards and Technology (NIST), 2012, pp. 5–10. URL: https://csrc.nist.gov/publications/detail/sp/800-94/rev-1/draft.

[224] Peter Schneider and Konstantin Böttinger. “High-performance unsupervised anomaly detection for cyber-physical system networks”. In: *Proceedings of the 2018 workshop on cyber-physical systems security and privacy*. 2018, pp. 1–12.

[225] Bernhard Schölkopf et al. “Estimating the Support of a High-Dimensional Distribution”. In: *Neural Comput.* 13.7 (2001), pp. 1443–1471. DOI: 10.1162/089976601750264965. URL: https://doi.org/10.1162/089976601750264965.

[226] Silvia Sebastián and Juan Caballero. “AVclass2: Massive Malware Tag Extraction from AV Labels”. In: *ACSAC ’20: Annual Computer Security Applications Conference, Virtual Event / Austin, TX, USA, 7-11 December, 2020*. ACM, 2020, pp. 42–53. DOI: 10.1145/3427228.3427261. URL: https://doi.org/10.1145/3427228.3427261.

[227] Rico Sennrich, Barry Haddow, and Alexandra Birch. “Improving Neural Machine Translation Models with Monolingual Data”. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, August 7-12, 2016, Berlin, Germany, Volume 1: Long Papers*. The Association for Computer Linguistics, 2016. DOI: 10.18653/V1/P16-1009. URL: https://doi.org/10.18653/v1/p16-1009.

[228] Burr Settles. *Active learning literature survey*. Accessed 3 Jun 2025. University of Wisconsin-Madison Department of Computer Sciences. 2009. URL: http://digital.library.wisc.edu/1793/60660.

[229] Ali Shafahi et al. “Poison Frogs! Targeted Clean-Label Poisoning Attacks on Neural Networks”. In: *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*. Ed. by Samy Bengio et al. 2018, pp. 6106–6116. URL: https://proceedings.neurips.cc/paper/2018/hash/22722a343513ed45f14905eb07621686-Abstract.html.

[230] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization”. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy, ICISSP 2018, Funchal, Madeira - Portugal, January 22-24, 2018*. Ed. by Paolo Mori, Steven Furnell, and Olivier Camp. SciTePress,

2018, pp. 108–116. DOI: 10.5220/0006639801080116. URL: https://doi.org/10.5220/0006639801080116.

[231] Iman Sharafaldin et al. “Towards a reliable intrusion detection benchmark dataset”. In: *Software Networking* 2018.1 (2018), pp. 177–200.

[232] Yam Sharon et al. “TANTRA: Timing-Based Adversarial Network Traffic Reshaping Attack”. In: *IEEE Trans. Inf. Forensics Secur.* 17 (2022), pp. 3225–3237. DOI: 10.1109/TIFS.2022.3201377. URL: https://doi.org/10.1109/TIFS.2022.3201377.

[233] Ahmed Shebl et al. “DCNN: a novel binary and multi-class network intrusion detection model via deep convolutional neural network”. In: *EURASIP Journal on Information Security* 2024.1 (2024), p. 36.

[234] Xiangmin Shen et al. “Pentestagent: Incorporating llm agents to automated penetration testing”. In: *arXiv preprint arXiv:2411.05185* (2024).

[235] Yuge Shi et al. “How robust are pre-trained models to distribution shift?” In: *ICML 2022: Workshop on Spurious Correlations, Invariance and Stability*. 2022.

[236] Jun Shu et al. “Meta-Weight-Net: Learning an Explicit Mapping For Sample Weighting”. In: *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*. Ed. by Hanna M. Wallach et al. 2019, pp. 1917–1928. URL: https://proceedings.neurips.cc/paper/2019/hash/e58cc5ca94270acaceed13bc82dfedf7-Abstract.html.

[237] Methaq A Shyaa et al. “Evolving cybersecurity frontiers: A comprehensive survey on concept drift and feature dynamics aware machine and deep learning in intrusion detection systems”. In: *Engineering Applications of Artificial Intelligence* 137 (2024), p. 109143.

[238] Methaq A. Shyaa et al. “Reinforcement Learning-Based Voting for Feature Drift-Aware Intrusion Detection: An Incremental Learning Framework”. In: *IEEE Access* 13 (2025), pp. 37872–37903. DOI: 10.1109/ACCESS.2025.3544221. URL: https://doi.org/10.1109/ACCESS.2025.3544221.

[239] João A. Simioni et al. “An Early Exit Deep Neural Network for Fast Inference Intrusion Detection”. In: *Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing, SAC 2025, Catania International Airport, Catania, Italy, 31 March 2025 - 4 April 2025*. Ed. by Jiman Hong et al. ACM, 2025, pp. 730–737. DOI: 10.1145/3672608.3707974. URL: https://doi.org/10.1145/3672608.3707974.

[240] Kihyuk Sohn et al. “FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence”. In: *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*. 2020. URL: https://proceedings.neurips.cc/paper/2020/hash/06964dce9addb1c5cb5d6e3d9838f733-Abstract.html.

[241] Mahdi Soltani et al. “An adaptable deep learning-based intrusion detection system to zero-day attacks”. In: *Journal of Information Security and Applications* 76 (2023), p. 103516. DOI: 10.1016/J.JISA.2023.103516. URL: https://doi.org/10.1016/j.jisa.2023.103516.

[242] Robin Sommer and Vern Paxson. “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection”. In: *31st IEEE Symposium on Security and Privacy, SP 2010, 16-19 May 2010, Berleley/Oakland, California, USA*. IEEE Computer Society, 2010, pp. 305–316. DOI: 10.1109/SP.2010.25. URL: https://doi.org/10.1109/SP.2010.25.

[243] Hyun Oh Song et al. “Deep Metric Learning via Lifted Structured Feature Embedding”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016, pp. 4004–4012. DOI: 10.1109/CVPR.2016.434. URL: https://doi.org/10.1109/CVPR.2016.434.

[244] Jungsuk Song et al. “Statistical analysis of honeypot data and building of Kyoto 2006+ dataset for NIDS evaluation”. In: *Proceedings of the First Workshop on Building Analysis Datasets and Gathering Experience Returns for Security, BADGERS@EuroSys 2011, Salzburg, Austria, April 10, 2011*. Ed. by Engin Kirda and Thorsten Holz. ACM, 2011, pp. 29–36. DOI: 10.1145/1978672.1978676. URL: https://doi.org/10.1145/1978672.1978676.

[245] Anna Sperotto et al. “A Labeled Data Set for Flow-Based Intrusion Detection”. In: *IP Operations and Management, 9th IEEE International Workshop, IPOM 2009, Venice, Italy, October 29-30, 2009. Proceedings*. Ed. by Giorgio Nunzi, Caterina M. Scoglio, and Xing Li. Vol. 5843. Lecture Notes in Computer Science. Springer, 2009, pp. 39–50. DOI: 10.1007/978-3-642-04968-2_4. URL: https://doi.org/10.1007/978-3-642-04968-2%5C_4.

[246] Nitish Srivastava et al. “Dropout: a simple way to prevent neural networks from overfitting”. In: *Journal of Machine Learning Research* 15.1 (2014), pp. 1929–1958. DOI: 10.5555/2627435.2670313. URL: https://dl.acm.org/doi/10.5555/2627435.2670313.

[247] Lucy Steele, Fahad Alotaibi, and Sergio Maffeis. “Poster: Randomness Unmasked: Towards Reproducible and Fair Evaluation of Shift-Aware Deep Learning NIDS”. In: *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*. Taipei, Taiwan: ACM, Oct. 2025, pp. 1–3. ISBN: 979-8-4007-1525-9/2025/10. DOI: 10.1145/3719027.3760743.

[248] Uri Stern, Daniel Shwartz, and Daphna Weinshall. “United We Stand: Using Epoch-Wise Agreement of Ensembles to Combat Overfit”. In: *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada*. AAAI Press, 2024, pp. 15075–15082. DOI: 10.1609/AAAI.V38I13.29429. URL: https://doi.org/10.1609/aaai.v38i13.29429.

[249] Zixian Su et al. “Navigating distribution shifts in medical image analysis: A survey”. In: *arXiv preprint arXiv:2411.05824* (2024).

[250] Chen Sun et al. “Revisiting Unreasonable Effectiveness of Data in Deep Learning Era”. In: *IEEE International Conference on Computer Vision, ICCV 2017, Venice, Italy, October 22-29, 2017*. IEEE Computer Society, 2017, pp. 843–852. DOI: 10.1109/ICCV.2017.97. URL: https://doi.org/10.1109/ICCV.2017.97.

[251] Tiezhu Sun et al. “Temporal-incremental learning for Android malware detection”. In: *ACM Transactions on Software Engineering and Methodology* (2024).

[252] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. “Sequence to sequence learning with neural networks”. In: *Advances in neural information processing systems* 27 (2014).

[253] Md Alamin Talukder et al. “Machine learning-based network intrusion detection for big and imbalanced data using oversampling, stacking feature embedding and feature extraction”. In: *Journal of big data* 11.1 (2024), p. 33.

[254] Mingxing Tan and Quoc V. Le. “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks”. In: *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2019, pp. 6105–6114. URL: http://proceedings.mlr.press/v97/tan19a.html.

[255] Ruming Tang et al. “Zerowall: Detecting zero-day web attacks through encoder-decoder recurrent neural networks”. In: *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE. 2020, pp. 2479–2488.

[256] Antti Tarvainen and Harri Valpola. “Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results”. In: *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, De-*

cember 4-9, 2017, Long Beach, CA, USA. 2017, pp. 1195–1204. URL: https://proceedings.neurips.cc/paper/2017/hash/68053af2923e00204c3ca7c6a3150cf7-Abstract.html.

[257] The Guardian. *Travelex falls into administration, with loss of 1,300 jobs*. Aug. 6, 2020. URL: https://www.theguardian.com/business/2020/aug/06/travelex-falls-into-administration-shedding-1300-jobs (visited on 06/21/2025).

[258] Kurt Thomas et al. "Investigating Commercial Pay-Per-Install and the Distribution of Unwanted Software". In: *25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016*. USENIX Association, 2016, pp. 721–739. URL: https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/thomas.

[259] Zhiyi Tian et al. "A Comprehensive Survey on Poisoning Attacks and Countermeasures in Machine Learning". In: *ACM Computing Surveys (CSUR)* 55.8 (2023), 166:1–166:35. DOI: 10.1145/3551636. URL: https://doi.org/10.1145/3551636.

[260] Robert Tibshirani et al. "Diagnosis of multiple cancer types by shrunken centroids of gene expression". In: *Proceedings of the National Academy of Sciences* 99.10 (2002), pp. 6567–6572.

[261] Lionel N Tidjon, Marc Frappier, and Amel Mammar. "Intrusion detection systems: A cross-domain overview". In: *IEEE Communications Surveys & Tutorials* 21.4 (2019), pp. 3639–3681.

[262] Hugo Touvron et al. "Llama 2: Open foundation and fine-tuned chat models". In: *arXiv preprint arXiv:2307.09288* (2023).

[263] TSMC. *TSMC Details Impact of Computer Virus Incident*. Aug. 5, 2018. URL: https://pr.tsmc.com/english/news/1969 (visited on 06/21/2025).

[264] Daniele Ucci, Leonardo Aniello, and Roberto Baldoni. "Survey of machine learning techniques for malware analysis". In: *Computers & Security* 81 (2019), pp. 123–147. DOI: 10.1016/J.COSE.2018.11.001. URL: https://doi.org/10.1016/j.cose.2018.11.001.

[265] Md Ashraf Uddin et al. "A dual-tier adaptive one-class classification IDS for emerging cyberthreats". In: *Computer Communications* 229 (2025), p. 108006.

[266] UK Parliament. *Legal Aid Agency: Cyber-security Incident (Commons debate, Vol. 767)*. Accessed 3 Jun 2025. Hansard. May 2025. URL: https://hansard.parliament.uk/Commons/2025-05-19/debates/C6B0C827-D166-4DF8-90AD-ECBDA4951D37/LegalAidAgencyCyberSecurityIncident.

[267] Unit 42, Palo Alto Networks. *Today's Attack Trends — Unit 42 Incident Response Report*. Tech. rep. Accessed: 2025-09-02. Palo Alto Networks, Unit 42, 2024. URL: https://www.paloaltonetworks.com/blog/2024/02/unit-42-incident-response-report.

[268] UnitedHealth Group. *Change Healthcare Cyberattack Support*. Accessed 3 Jun 2025. 2024. URL: https://www.unitedhealthgroup.com/ns/health-data-breach.html.

[269] Pablo Velarde-Alvarado et al. "A Novel Framework for Generating Personalized Network Datasets for NIDS Based on Traffic Aggregation". In: *Sensors* 22.5 (2022), p. 1847. DOI: 10.3390/S22051847. URL: https://doi.org/10.3390/s22051847.

[270] Sahil Verma et al. "Counterfactual explanations and algorithmic recourses for machine learning: A review". In: *ACM Computing Surveys* 56.12 (2024), pp. 1–42.

[271] Haoyu Wang et al. "RmvDroid: towards a reliable Android malware dataset with app metadata". In: *Proceedings of the 16th International Conference on Mining Software Repositories, MSR 2019, 26-27 May 2019, Montreal, Canada*. IEEE / ACM, 2019, pp. 404–408. DOI: 10.1109/MSR.2019.00067. URL: https://doi.org/10.1109/MSR.2019.00067.

[272] Jingjing Wang et al. "Re-measuring the Label Dynamics of Online Anti-Malware Engines from Millions of Samples". In: *Proceedings of the 2023 ACM on Internet Measurement Conference, IMC 2023, Montreal, QC, Canada, October 24-26, 2023*. ACM, 2023, pp. 253–267. DOI: 10.1145/3618257.3624800. URL: https://doi.org/10.1145/3618257.3624800.

[273] Liu Wang et al. “MalWhiteout: Reducing Label Errors in Android Malware Detection”. In: *37th IEEE/ACM International Conference on Automated Software Engineering, ASE 2022, Rochester, MI, USA, October 10-14, 2022*. ACM, 2022, 69:1–69:13. DOI: 10.1145/3551349.3560418. URL: https://doi.org/10.1145/3551349.3560418.

[274] Minxiao Wang et al. “On the robustness of ML-based network intrusion detection systems: An adversarial and distribution shift perspective”. In: *Computers* 12.10 (2023), p. 209.

[275] Weixing Wang et al. “Feature Distribution Shift Mitigation with Contrastive Pretraining for Intrusion Detection”. In: *2024 IEEE International Conference on Machine Learning for Communication and Networking (ICMLCN)*. IEEE. 2024, pp. 177–182.

[276] Xian Wang. “ENIDrift: A Fast and Adaptive Ensemble System for Network Intrusion Detection under Real-world Drift”. In: *Annual Computer Security Applications Conference, ACSAC 2022, Austin, TX, USA, December 5-9, 2022*. ACM, 2022, pp. 785–798. DOI: 10.1145/3564625.3567992. URL: https://doi.org/10.1145/3564625.3567992.

[277] Hongxin Wei et al. “Mitigating Memorization of Noisy Labels by Clipping the Model Prediction”. In: *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*. Ed. by Andreas Krause et al. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 36868–36886. URL: https://proceedings.mlr.press/v202/wei23e.html.

[278] Jiaheng Wei et al. “Learning with Noisy Labels Revisited: A Study Using Real-World Human Annotations”. In: *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL: https://openreview.net/forum?id=TBWA6PLJZQm.

[279] Qi Wei et al. “Self-Filtering: A Noise-Aware Sample Selection for Label Noise with Confidence Penalization”. In: *Computer Vision - ECCV 2022 - 17th European Conference, Tel Aviv, Israel, October 23-27, 2022, Proceedings, Part XXX*. Vol. 13690. Lecture Notes in Computer Science. Springer, 2022, pp. 516–532. DOI: 10.1007/978-3-031-20056-4_30. URL: https://doi.org/10.1007/978-3-031-20056-4%5C_30.

[280] Yandong Wen et al. “A Discriminative Feature Learning Approach for Deep Face Recognition”. In: *Computer Vision - ECCV 2016 - 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part VII*. Ed. by Bastian Leibe et al. Vol. 9911. Lecture Notes in Computer Science. Springer, 2016, pp. 499–515. DOI: 10.1007/978-3-319-46478-7_31. URL: https://doi.org/10.1007/978-3-319-46478-7%5C_31.

[281] Zack Whittaker. *UnitedHealth confirms 190 million Americans affected by Change Healthcare data breach*. Accessed 3 Jun 2025. TechCrunch. Jan. 2025. URL: https://techcrunch.com/2025/01/24/unitedhealth-confirms-190-million-americans-affected-by-change-healthcare-data-breach/.

[282] Jennifer Williams and Charlie Dagli. “Twitter Language Identification Of Similar Languages And Dialects Without Ground Truth”. In: *Proceedings of the Fourth Workshop on NLP for Similar Languages, Varieties and Dialects, VarDial 2017, Valencia, Spain, April 3, 2017*. Association for Computational Linguistics, 2017, pp. 73–83. DOI: 10.18653/v1/W17-1209. URL: https://doi.org/10.18653/v1/w17-1209.

[283] Ka Wong, Praveen Paritosh, and Kurt Bollacker. “Are ground truth labels reproducible? An empirical study”. In: *Proceedings of ML Evaluation Standards Workshop at ICLR*. 2022, pp. 25–29.

[284] Miuyin Yong Wong et al. “An Inside Look into the Practice of Malware Analysis”. In: *CCS ’21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*. Ed. by Yongdae Kim et al. ACM, 2021, pp. 3053–3069. DOI: 10.1145/3460120.3484759. URL: https://doi.org/10.1145/3460120.3484759.

[285] Dongrui Wu. "Pool-Based Sequential Active Learning for Regression". In: *IEEE Trans. Neural Networks Learn. Syst.* 30.5 (2019), pp. 1348–1359. DOI: 10.1109/TNNLS.2018.2868649. URL: https://doi.org/10.1109/TNNLS.2018.2868649.

[286] Xian Wu et al. "From Grim Reality to Practical Solution: Malware Classification in Real-World Noise". In: *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*. IEEE, 2023, pp. 2602–2619. DOI: 10.1109/SP46215.2023.10179453. URL: https://doi.org/10.1109/SP46215.2023.10179453.

[287] Yueming Wu et al. "MalScan: Fast Market-Wide Mobile Malware Scanning by Social-Network Centrality Analysis". In: *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019, San Diego, CA, USA, November 11-15, 2019*. IEEE, 2019, pp. 139–150. DOI: 10.1109/ASE.2019.00023. URL: https://doi.org/10.1109/ASE.2019.00023.

[288] Shaoke Xi et al. "MineShark: Cryptomining Traffic Detection at Scale". In: *32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24-28, 2025*. The Internet Society, 2025. URL: https://www.ndss-symposium.org/ndss-paper/mineshark-cryptomining-traffic-detection-at-scale/.

[289] Jiayun Xu, Yingjiu Li, and Robert H. Deng. "Differential Training: A Generic Framework to Reduce Label Noises for Android Malware Detection". In: *28th Annual Network and Distributed System Security Symposium, NDSS 2021, virtually, February 21-25, 2021*. The Internet Society, 2021. URL: https://www.ndss-symposium.org/ndss-paper/differential-training-a-generic-framework-to-reduce-label-noises-for-android-malware-detection/.

[290] Ke Xu et al. "DeepRefiner: Multi-layer Android Malware Detection System Applying Deep Neural Networks". In: *2018 IEEE European Symposium on Security and Privacy, EuroS&P 2018, London, United Kingdom, April 24-26, 2018*. IEEE, 2018, pp. 473–487. DOI: 10.1109/EUROSP.2018.00040. URL: https://doi.org/10.1109/EuroSP.2018.00040.

[291] Ke Xu et al. "DroidEvolver: Self-Evolving Android Malware Detection System". In: *IEEE European Symposium on Security and Privacy, EuroS&P 2019, Stockholm, Sweden, June 17-19, 2019*. IEEE, 2019, pp. 47–62. DOI: 10.1109/EuroSP.2019.00014. URL: https://doi.org/10.1109/EuroSP.2019.00014.

[292] Lijuan Xu et al. "ADTCD: An Adaptive Anomaly Detection Approach Toward Concept Drift in IoT". In: *IEEE Internet of Things Journal* (2023).

[293] Jian Yang et al. "Conditional Variational Auto-Encoder and Extreme Value Theory Aided Two-Stage Learning Approach for Intelligent Fine-Grained Known/Unknown Intrusion Detection". In: *IEEE Transactions on Information Forensics and Security* 16 (2021), pp. 3538–3553. DOI: 10.1109/TIFS.2021.3083422. URL: https://doi.org/10.1109/TIFS.2021.3083422.

[294] Jingkang Yang et al. "Generalized Out-of-Distribution Detection: A Survey". In: *International Journal of Computer Vision* 132.12 (2024), pp. 5635–5662. DOI: 10.1007/S11263-024-02117-4. URL: https://doi.org/10.1007/s11263-024-02117-4.

[295] Limin Yang et al. "CADE: Detecting and Explaining Concept Drift Samples for Security Applications". In: *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*. Ed. by Michael Bailey and Rachel Greenstadt. USENIX Association, 2021, pp. 2327–2344. URL: https://www.usenix.org/conference/usenixsecurity21/presentation/yang-limin.

[296] Limin Yang et al. "Jigsaw Puzzle: Selective Backdoor Attack to Subvert Malware Classifiers". In: *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*. IEEE, 2023, pp. 719–736. DOI: 10.1109/SP46215.2023.10179347. URL: https://doi.org/10.1109/SP46215.2023.10179347.

[297] Shuo Yang et al. "ReCDA: Concept Drift Adaptation with Representation Enhancement for Network Intrusion Detection". In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2024, Barcelona, Spain, August 25-29, 2024*. Ed. by Ricardo

Baeza-Yates and Francesco Bonchi. ACM, 2024, pp. 3818–3828. DOI: 10.1145/3637528.3672007. URL: https://doi.org/10.1145/3637528.3672007.

[298] Yu Yang, Hao Kang, and Baharan Mirzasoleiman. "Towards Sustainable Learning: Coresets for Data-efficient Deep Learning". In: *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*. Ed. by Andreas Krause et al. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 39314–39330. URL: https://proceedings.mlr.press/v202/yang23g.html.

[299] Nong Ye et al. "Multivariate Statistical Analysis of Audit Trails for Host-Based Intrusion Detection". In: *IEEE Transactions on Computers* 51.7 (2002), pp. 810–820. DOI: 10.1109/TC.2002.1017701. URL: https://doi.org/10.1109/TC.2002.1017701.

[300] Zinuo Yin et al. "CAEAID: An incremental contrast learning-based intrusion detection framework for IoT networks". In: *Computer Networks* 262 (2025), p. 111161. DOI: 10.1016/J.COMNET.2025.111161. URL: https://doi.org/10.1016/j.comnet.2025.111161.

[301] Susik Yoon et al. "Adaptive Model Pooling for Online Deep Anomaly Detection from a Complex Evolving Data Stream". In: *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14 - 18, 2022*. Ed. by Aidong Zhang and Huzefa Rangwala. ACM, 2022, pp. 2347–2357. DOI: 10.1145/3534678.3539348. URL: https://doi.org/10.1145/3534678.3539348.

[302] Shujian Yu and Zubin Abraham. "Concept Drift Detection with Hierarchical Hypothesis Testing". In: *Proceedings of the 2017 SIAM International Conference on Data Mining, Houston, Texas, USA, April 27-29, 2017*. Ed. by Nitesh V. Chawla and Wei Wang. SIAM, 2017, pp. 768–776. DOI: 10.1137/1.9781611974973.86. URL: https://doi.org/10.1137/1.9781611974973.86.

[303] Xingrui Yu et al. "How does Disagreement Help Generalization against Label Corruption?" In: *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*. Ed. by Kamalika Chaudhuri and Ruslan Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2019, pp. 7164–7173. URL: http://proceedings.mlr.press/v97/yu19b.html.

[304] Qingjun Yuan et al. "BoAu: Malicious traffic detection with noise labels based on boundary augmentation". In: *Computers & Security* 131 (2023), p. 103300. DOI: 10.1016/J.COSE.2023.103300. URL: https://doi.org/10.1016/j.cose.2023.103300.

[305] Qingjun Yuan et al. "MCRe: A Unified Framework for Handling Malicious Traffic With Noise Labels Based on Multidimensional Constraint Representation". In: *IEEE Transactions on Information Forensics and Security* 19 (2024), pp. 133–147. DOI: 10.1109/TIFS.2023.3318962. URL: https://doi.org/10.1109/TIFS.2023.3318962.

[306] Zhenlong Yuan et al. "Droid-Sec: deep learning in android malware detection". In: *ACM SIGCOMM 2014 Conference, SIGCOMM'14, Chicago, IL, USA, August 17-22, 2014*. ACM, 2014, pp. 371–372. DOI: 10.1145/2619239.2631434. URL: https://doi.org/10.1145/2619239.2631434.

[307] Daochen Zha et al. "Data-centric Artificial Intelligence: A Survey". In: *ACM Computing Surveys (CSUR)* 57.5 (2025), 129:1–129:42. DOI: 10.1145/3711118. URL: https://doi.org/10.1145/3711118.

[308] Chaoyun Zhang, Xavier Costa-Pérez, and Paul Patras. "Tiki-Taka: Attacking and Defending Deep Learning-based Intrusion Detection Systems". In: *CCSW'20, Proceedings of the 2020 ACM SIGSAC Conference on Cloud Computing Security Workshop, Virtual Event, USA, November 9, 2020*. Ed. by Yinqian Zhang and Radu Sion. ACM, 2020, pp. 27–39. DOI: 10.1145/3411495.3421359. URL: https://doi.org/10.1145/3411495.3421359.

[309] Chiyuan Zhang et al. “Understanding deep learning requires rethinking generalization”. In: *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. 2017.

[310] Xiaobo Zhang et al. “CoSaR: Combating Label Noise Using Collaborative Sample Selection and Adversarial Regularization”. In: *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, CIKM 2023, Birmingham, United Kingdom, October 21-25, 2023*. ACM, 2023, pp. 3184–3194. DOI: 10.1145/3583780.3614826. URL: https://doi.org/10.1145/3583780.3614826.

[311] Xiaohan Zhang et al. “Enhancing State-of-the-art Classifiers with API Semantics to Detect Evolved Android Malware”. In: *CCS ’20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*. Ed. by Jay Ligatti et al. ACM, 2020, pp. 757–770. DOI: 10.1145/3372297.3417291. URL: https://doi.org/10.1145/3372297.3417291.

[312] Xinchen Zhang et al. “Continual Learning with Strategic Selection and Forgetting for Network Intrusion Detection”. In: *CoRR* abs/2412.16264 (2024). DOI: 10.48550/ARXIV.2412.16264. arXiv: 2412.16264. URL: https://doi.org/10.48550/arXiv.2412.16264.

[313] Ziming Zhao et al. “Trident: A Universal Framework for Fine-Grained and Class-Incremental Unknown Traffic Detection”. In: *Proceedings of the ACM on Web Conference 2024, WWW 2024, Singapore, May 13-17, 2024*. Ed. by Tat-Seng Chua et al. ACM, 2024, pp. 1608–1619. DOI: 10.1145/3589334.3645407. URL: https://doi.org/10.1145/3589334.3645407.

[314] Songzhu Zheng et al. “Error-Bounded Correction of Noisy Labels”. In: *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 11447–11457. URL: http://proceedings.mlr.press/v119/zheng20c.html.

[315] Ying Zhong et al. “RFG-HELAD: A Robust Fine-Grained Network Traffic Anomaly Detection Model Based on Heterogeneous Ensemble Learning”. In: *IEEE Transactions on Information Forensics and Security* 19 (2024), pp. 5895–5910. DOI: 10.1109/TIFS.2024.3402439. URL: https://doi.org/10.1109/TIFS.2024.3402439.

[316] Shuofei Zhu et al. “Measuring and Modeling the Label Dynamics of Online Anti-Malware Engines”. In: *29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020*. USENIX Association, 2020, pp. 2361–2378. URL: https://www.usenix.org/conference/usenixsecurity20/presentation/zhu.

[317] Binghui Zou et al. “FACILE: A capsule network with fewer capsules and richer hierarchical information for malware image classification”. In: *Computers & Security* 137 (2024), p. 103606. DOI: 10.1016/J.COSE.2023.103606. URL: https://doi.org/10.1016/j.cose.2023.103606.

Appendix A

Mateen

A.1 Dataset Processing

In the following, we provide additional details about the datasets utilised and the methods employed in their processing.

IDS17. We used an improved version of the IDS17 dataset, initially developed by the Canadian Institute for Cybersecurity [77]. The original dataset [230], which included packet captures and network flows extracted with the CICFlowMeter tool [145, 72], had inaccuracies in feature extraction and labeling. To overcome these issues, Engelen *et al.* [77] released a revised version, which we adopted to enhance the accuracy and reliability of our findings.

For the purposes of our experiments, the dataset was divided into two segments: the training segment and the testing segment. The training segment, covering the first two days of data, contained 693,702 network flows. This portion was primarily benign traffic (99%), but it also included instances of two network attacks: FTP-Patator and SSH-Patator. The testing segment covered the last three days of data, comprising 1,406,274 network flows, with 64.55% being benign traffic alongside a variety of network attacks (12 different types). The testing segment was divided into batches of 50,000 network flows, resulting in a total of 29 batches of network flows. Crucially, to ensure integrity and objectivity in our analysis, we omitted certain features from the dataset that were used as a basis for original labeling. These are Flow ID, Src IP, Src Port, Dst IP, and Timestamp. This step was taken to avoid any bias, such as the potential for models to use these features as shortcuts in classification and anomaly detection.

IDS18. IDS18, an extended version of the IDS17 dataset, incorporates data from over 450 machines across five departments, including 50 machines designated for attack simulations, collected over a three-week period. Similar to the original IDS17, this larger dataset initially suffered from flaws in the labeling and feature extraction process. However, these issues were addressed in a revised version, as detailed in [156], which we used for our experiments. Our experimental design was tailored to accommodate the substantial size of the dataset. We partitioned the data from the initial two weeks into training and testing segments. The training segment, including data from February 14th to 16th, consisted of over 10 million flows before processing. The testing segment

covered data from February 20th to 23rd. Due to the large size of both datasets, we implemented a strategic sampling approach, randomly selecting 30% of flows from each label in every file. This selection was carried out in a time-based sequential order to avoid temporal bias [199, 22].

Post-processing, the training set was reduced to 2,548,780 flows, among which 1.47% were classified as attacks across six distinct classes. Likewise, the testing set was reduced to 7,501,307 flows, with 12.70% identified as attacks. These attacks span 10 classes, all of which differ from those in the training set. We segmented the test set into batches, each containing 50,000 flows, resulting in a total of 151 batches. To ensure unbiased analysis, we removed features that were used for labeling, specifically ID, Flow ID, Src IP, Src Port, Dst IP, and Timestamp. Furthermore, we adjusted the testing batches to consist of a fixed benign-to-malicious ratio to eliminate the impact of ratio shifts.

Kitsune. The Kitsune dataset [177], tailored for IoT networks, comprises 115 statistical features derived from network packets. It is organized into separate files by attack type, each containing a mix of benign and attack-specific records. In total, the combined files yield over 21 million records, presenting a significant computational challenge. To manage this, we adopted the same approach as with the IDS18 dataset, randomly sampling 30% of records from each label within each file. We selected the ARP Man-in-the-Middle file for training, which, after post-processing, contained 751,280 records, with 45% classified as malicious traffic. For testing, we chose other files, resulting in 5,324,759 records, of which 20.86% were identified as attacks. Segmenting the test set into batches of 50,000 each yielded a total of 107 batches.

Additionally, we created two new testing sets: one with a fixed malicious-to-benign ratio (mKitsune), and another designed to simulate a recurring concept (rKitsune). For mKitsune, we adjusted the already generated testing set to maintain this ratio, distributing attacks evenly across batches in line with the methodology employed in our processing of IDS18. For rKitsune, we generated a new test set by sampling 50% from both the original ARP Man-in-the-Middle and Active Wiretap datasets. We then interspersed two batches of Active Wiretap after every four batches of ARP Man-in-the-Middle. This arrangement produced a recurring concept test set with 1,739,344 records, which, when divided, yielded 35 batches.

A.2 Hyperparameter Search & Analysis

In the following sections, we provide detailed information on the baselines (Section A.2.1), the architecture used for each model (Section A.2.2), our approach to conducting a hyperparameter search for each baseline (Section A.2.3), and conclude with an analysis of hyperparameter sensitivity for `Mateen` (Section A.2.4).

A.2.1 Baselines Details

In this subsection, we provide detailed descriptions of each baseline utilised. Emphasis is placed on their high-level functions, specifically how they detect, select, and adapt to shift samples.

`OWAD`. `OWAD` is an adaptive framework designed specifically for detecting, selecting, and adapting to

benign shifts in one-class AD security applications. It employs a distribution-based shift detection method, comparing the data distribution of new upcoming batches against a control set derived from the training data. Upon detecting a shift, OWAD uses a custom optimization function to identify the top δ most significant samples for labeling and adaptation. This function is designed to align the distribution of the control set with that of the incoming batch, achieved by substituting portions of the control set's samples with samples from the incoming batch. After labeling the top δ most critical samples, OWAD updates the underlying AE through a customized version of the UNLEARN framework [74]. In this framework, UNLEARN precisely regulates updates to the neural network weights, striking a balance between preventing catastrophic forgetting of the old distribution and adapting to the new one.

INSOMNIA. INSOMNIA is an adaptive, incremental learning framework specifically designed to improve the performance of multi-class NIDS amidst changing data distributions. Initially tailored for softmax classifiers, its flexibility allows for extension to any binary classification model that can generate probability scores. INSOMNIA offers two variants: the first requires human intervention, wherein the top δ most uncertain samples in each batch are manually labeled, added to the training set, and followed by retraining of the classifier. The second variant streamlines the process by using a Nearest Centroid Neighbor classifier [260] to estimate labels for new batches, thereby enabling automatic retraining. It is important to highlight that in both variants, shifts are not detected; instead, updates occur with each incoming batch of data. Comparative evaluations conducted by the authors of INSOMNIA underscore the benefits of manual labeling in significantly improving classifier accuracy. Accordingly, we have chosen the manually labeled version of INSOMNIA as the standard for our experiments.

CADE. CADE is a framework designed for detecting, selecting, and explaining shifts within multi-class security applications. It specifically targets concept shifts by identifying new classes, choosing relevant samples for adaptation, and providing explanations for why these samples are marked as shifts. At the heart of the CADE framework is supervised contrastive learning, particularly through the use of DrLIM [105]. This approach trains DrLIM in conjunction with a AE to optimize the latent space by minimizing distances between samples of the same class while maximizing the distances between different classes. Once trained, CADE employs the Median Absolute Deviation (MAD) [148] method to detect shifted samples. This involves measuring the distance of each sample from the centroids of each training class and applying a class-specific MAD criterion to determine whether a sample significantly diverges from existing classes. A sample is flagged as a shift if it deviates significantly from all the training classes. The samples exhibiting the most significant distance from the classes' centroids are selected, in accordance with the δ selection rate, for labeling and subsequent adaptation. It should be noted that CADE itself does not introduce a novel adaptation function; rather, it suggests employing the standard incremental learning approach to demonstrate the significance of its detection and selection mechanisms.

A.2.2 Architecture and Training

In both OWAD and Mateen, we employed an identical AE architecture. Specifically, the encoder followed a progressive reduction in dimensions: from the original dimensions to 75%, then to 50%,

followed by 25%, and finally 10%. Conversely, the decoder mirrored this structure in reverse. Each layer was coupled with a ReLU activation function, except for the encoder input and the decoder output layers. It is important to note that this architectural configuration is consistent with the one originally proposed by OWAD, which was inherited from Kitsune [177]. We then trained the AE utilising the MSE loss function. The AE was trained with a batch size of 1024 for 100 epochs, employing the Adam optimizer with a learning rate of 0.00001. Regarding CADE, we adopted the training hyperparameters specified by the authors, with the exception being the architectural design, which remains consistent with that of OWAD and Mateen. These hyperparameters were retained due to their proven effectiveness in a contrastive learning setting, where, for instance, training with longer batches (*e.g.* 250) is advantageous [48, 47, 133].

Regarding the softmax classifier (INSOMNIA), we followed the hyperparameter search methodology outlined by the authors [18]. This search aimed to determine the best combinations of batch size, learning rate, dropout rate, and the number of neurons per hidden layer, with a limitation of 3 layers. The classifier is trained for 150 epochs using the Adam optimizer, with early stopping conducted after 10 epochs on the validation set. When it came to updating the models, we configured the number of epochs to be 10% of the original training epochs.

A.2.3 Hyperparameter Space and Configuration

The baseline frameworks, OWAD and CADE, require hyperparameter fine-tuning to optimize performance. Consequently, we undertook the following hyperparameter search:

OWAD. We explored the hyperparameter space of the OWAD selection method, with a primary focus on weight hyperparameters. These weights, namely 'acc_wgt,' 'ohd_wgt,' and 'det_wgt,' correspond to ensuring accurate representation of the shifted distribution, reducing the need for manual labeling in test samples, and emphasizing deterministic selection. We conducted empirical evaluations across various combinations, including the original values (5, 10, and 1, respectively), represented as (acc_wgt, ohd_wgt, det_wgt), as well as other configurations such as (3, 3, 3), (2, 3, 5), (3, 2, 5), (5, 2, 3), (5, 3, 2), (3, 5, 2), and (2, 5, 3). The most effective configurations, for a δ of 1%, were identified, such as (3, 2, 5) for IDS17, (3, 5, 2) for IDS18, (5, 10, 1) for Kitsune, (3, 5, 2) for mKitsune, and (2, 5, 3) for rKitsune. We also conducted the same empirical searches for all the considered δ values and experimental scenarios.

CADE. CADE's primary principle is that well-separated label clusters improve the accuracy of detecting shift samples. To achieve this, CADE combines MSE loss and contrastive loss [105], thereby creating distinct representations in latent space. This approach introduces two crucial hyperparameters: m and λ. The parameter m controls the separation distance between representations of different classes, while λ adjusts the contribution of contrastive loss to the overall objective. To optimize CADE, we conducted a hyperparameter search with a focus on the separation of class clusters and the compactness of individual clusters. For this purpose, we employed the Dunn Index, a metric measuring the smallest inter-cluster distance against the largest intra-cluster distance. Our search explored various combinations of m values (1, 5, 10, 15, 20) and λ values (1, 0.1, 0.01, 0.001) to identify the optimal settings that maximize the Dunn Index. The most effective hyperparameters were found to be $m = 1.0$ and $\lambda = 1.0$ for IDS 17, and $m = 10.0$ and $\lambda = 0.01$ for

mKitsune.

A.2.4 Hyperparameter Sensitivity

We conducted an empirical hyperparameter search for `Mateen`, starting by setting the ensemble size to 3 and fixing λ_0 at 0.1. Recognizing that representativeness more accurately mirrors the distribution than uniqueness, we uniformly set λ_1 to 1.0. This was followed by a grid search covering σ% of values 30%, 50%, and 90%, and ρ of values 500, 1000, and 1500. utilising the optimal values obtained, we further explored the best setting for λ_0 (considering values 0.5 and 1.0), and then determined the most effective ensemble size (options being 5 and 7).

In the following sections, we present the results of our hyperparameter search across the datasets at a δ of 1%, and include a comparison of sensitivity between `Mateen` and `OWAD`.

ρ	σ%	AUC-ROC
500	30%, 50%, 90%	96.42, 96.39, 95.87
1000	**30%**, 50%, 90%	**96.79**, 95.85, 95.65
1500	30%, 50%, 90%	96.23, 96.63, 95.66

Table A.1: The impact of σ% and ρ values on IDS17.

IDS17. Table A.1 presents the impact of different σ% and ρ combinations. The most effective combination was identified as a σ% of 30% with a ρ of value 1000. Further analysis was conducted on λ_0 with values of 0.5 and 1.0, yielding results of 96.43 and 94.26, respectively. However, the initially set value of 0.1 for λ_0 proved to be superior. Additionally, we explored varying ensemble sizes, specifically 5 and 7, and found that an ensemble size of 3 yielded the best results. In detail, the original ensemble size of 3 achieved an AUC-ROC of 96.79, while sizes of 5 and 7 resulted in AUC-ROCs of 96.69 and 96.49, respectively. The final optimal parameters were determined as: λ_0 at 0.1, a σ% value of 30%, an ensemble size of 3, and a ρ of value 1000.

IDS18. Table A.2 illustrates the effects of various combinations of σ% and ρ. The optimal combination was found to be 30% for σ and 500 for ρ. Subsequent evaluations of λ_0 at 0.5 and 1.0 produced scores of 97.44 and 97.17, respectively. Therefore, the initial λ_0 setting of 0.1 was deemed most effective. Investigations into different ensemble sizes, notably 5 and 7, revealed that a smaller ensemble of 3 was most efficacious, with an original ensemble size of 3 reaching an AUC-ROC score of 98.17, compared to the 97.42 and 97.76 achieved by ensemble sizes of 5 and 7, respectively. Ultimately, the best-performing parameters were identified as λ_0 set at 0.1, σ at 30%, an ensemble size of 3, and ρ valued at 500.

Kitsune. Table A.3 shows the F1 scores from the hyperparameter optimization performed on

ρ	σ%	AUC-ROC
500	**30%**, 50%, 90%	**98.17**, 97.41, 97.49
1000	30%, 50%, 90%	97.83, 97.64, 97.22
1500	30%, 50%, 90%	97.32, 98.11, 96.92,

Table A.2: The impact of σ% and ρ values on IDS18.

ρ	σ%	F1-Score
500	30%, 50%, 90%	96.72, 96.72, 96.61
1000	30%, 50%, 90%	96.79, 96.80, 96.56
1500	30%, **50%**, 90%	97.04, **97.10**, 96.90

Table A.3: The impact of σ% and ρ values on Kitsune.

the Kitsune dataset. In this analysis, the AUC-ROC score was not considered a primary metric due to the limited number of batches containing a mix of labels. Nonetheless, for reference, the average and standard deviation of the AUC-ROC scores are provided in Table A.6. The F1 results indicate only minor variations, with the lowest score recorded at 96.56 and the highest at 97.10, achieved with a ρ of value 1500 and a σ% value of 50%. Further assessment was made on the influence of λ_0 and the ensemble size. λ_0 values of 0.5 and 1.0 yielded scores of 96.55 and 97.05 respectively, while ensemble sizes of 5 and 7 led to scores of 96.43 and 96.57 respectively. The initial combination, featuring λ_0 at 0.1 and an ensemble size of 3, continued to show the strongest performance. Consequently, the optimal configuration for this dataset was determined to be a ρ value of 1500, a σ% value of 50%, λ_0 set at 0.1, and an ensemble size of 3.

ρ	σ%	AUC-ROC
500	30%, 50%, 90%	94.17, 93.90, 93.79
1000	30%, 50%, 90%	93.90, 93.85, 94.39
1500	**30%**, 50%, 90%	**94.42**, 93.78, 94.00

Table A.4: The impact of σ% and ρ values on mKitsune.

mKitsune. Table A.4 showcases the AUC-ROC outcomes from the hyperparameter optimization conducted on the mKitsune dataset. The highest AUC-ROC value observed was 94.42, achieved with a ρ of value 1500 and a σ% value of 30%. Subsequent analysis focused on various combinations of λ_0 and ensemble size. A λ_0 value of 0.5 yielded a superior AUC-ROC score of 94.76, surpassing both the initial setting of 94.42 and the score obtained with λ_0 at 1.0 (94.02). In terms of ensemble size, the initial value of 3 proved to be the most effective, outperforming sizes of 5 and 7, which resulted in scores of 94.54 and 94.66, respectively. Thus, the final optimal settings for this dataset were determined to be a ρ of value 1500, a σ% value of 30%, λ_0 at 0.5, and an ensemble size of 3.

rKitsune. Table A.5 presents the F1 scores obtained from the rKitsune dataset, which encounters a similar challenge to the Kitsune dataset: a limited number of batches is suitable for AUC-ROC computation, primarily due to the prevalence of batches dominated by a single label. Despite this, the average and standard deviation of AUC-ROC for this dataset are available in Table A.6 for reference.

ρ	σ%	F1
500	30%, 50%, 90%	93.95, 92.87, 94.11
1000	30%, 50%, **90%**	93.40, 93.81, **94.13**
1500	30%, 50%, 90%	93.32, 92.90, 93.20

Table A.5: The impact of σ% and ρ values on rKitsune.

Regarding the hyperparameter combinations tested, the pairing of a ρ of value 1000 with a $\sigma\%$ value of 90% emerged as the most effective, achieving an F1 score of 94.13. Subsequent evaluation of λ_0 values revealed that both 0.5 and 1.0 offered slight performance improvements. Specifically, a λ_0 value of 0.5 resulted in an F1 score of 94.38, while a value of 1.0 achieved a score of 94.20. Further analysis of the ensemble size indicated that a size of 7, with an F1 score of 94.52, outperformed the initial value of 3 (F1 score of 94.38). In contrast, an ensemble size of 5 yielded an F1 score of 93.75. Therefore, the final optimal configuration for this dataset is identified as a ρ of value 1000, a $\sigma\%$ value of 90%, λ_0 set at 0.5, and an ensemble size of 7.

Dataset	OWAD		Mateen	
	AUC-ROC	F1-Score	AUC-ROC	F1-Score
IDS17	93.75±0.24	83.69±0.54	96.19±0.64	91.93±0.42
IDS18	90.80±0.97	91.97±0.26	97.61±0.39	97.41±0.04
Kitsune	66.09±3.01	50.29-±25.33	70.39±2.22	96.80±0.22
mKitsune	77.37±2.42	71.21±20.35	94.22±0.34	95.83±0.38
rKitsune	87.10±0.46	89.52±1.14	92.55±0.99	93.80±0.52

Table A.6: Comparative evaluation of hyperparameter sensitivity: Mateen vs. OWAD.

Sensitivity Comparison. Table A.6 presents the comparative results of the AUC-ROC and F1-Score from the hyperparameter sensitivity analysis of Mateen versus OWAD. The results reveal that Mateen consistently outperforms OWAD across all datasets, as indicated by the higher average scores and lower standard deviations. Notably, Mateen demonstrates minimal variability in its performance, with the exception of the AUC-ROC on the Kitsune dataset, which can be attributed to the limited number of batches containing a mix of labels. We can also observe that OWAD's performance diminishes on datasets with concept shifts, contrasting its relatively stable performance on datasets with covariate shifts. Overall, Mateen exhibits robustness to variations in its hyperparameters, often maintaining strong performance even with suboptimal settings.

A.3 Guidelines for Hyperparameter Selection

Mateen includes multiple hyperparameters that can be adjusted for different scenarios. Below, we provide guidance on how to set these values.

Performance threshold (t): This describes the acceptable performance level. Specifically, if the ensemble's performance on a selected subset exceeds a threshold t, no update is needed. This threshold can be set low (*e.g.* 90%), resulting in fewer updates, or high (*e.g.* 99%), leading to more frequent updates. The choice of this value balances desired performance against the computational cost of updates. If computational cost is not a concern, setting a higher threshold is recommended.

Maximum Ensemble Size: This establishes a maximum size for the ensemble. Once this limit is reached, the complexity module activates to merge and remove temporary models. Setting a lower limit, such as 3, means the module activates more often, optimizing the ensemble for recent shifts and potentially enhancing performance. However, this requires additional processing. Conversely, a higher limit, such as 10, results in less frequent activations but requires more storage to maintain more models.

Selection Rate (δ): This parameter sets the percentage of samples from a shifted batch that are selected for manual labeling. A higher setting (*e.g.* 10%) selects more samples, which can improve ensemble performance but increases the risk of mislabeling, latency, and labor costs. A lower setting (*e.g.* 0.5%) results in fewer labeled samples, which might lower performance but also reduces labor costs, likelihood of errors, and latency. Our findings indicate that the performance difference between high and low settings is slight, so selecting the lower value is recommended.

Mini-Batch Size (ρ): This represents the size of the mini-batches created from each shifted batch of data. Specifically, we divide each shifted batch into mini-batches of size ρ and then select a few samples from each mini-batch using distance-based measures. A smaller mini-batch size, such as 500, lowers computational costs, whereas a larger size, such as 1500, increases them. The impact on performance varies with the dataset. For instance, the smallest batch size achieves good performance at lower costs in the IDS17 and mKitsune datasets, but larger sizes can slightly improve performance if resources permit.

Retention Rate (σ%): This specifies the percentage of samples kept from a mini-batch to create a condensed version. We form this condensed mini-batch by retaining σ% of the samples and discarding the remaining $100\% - \sigma\%$, which are similar and thus redundant. The choice of σ is based on the level of redundancy observed in the network. For the datasets examined in this paper, setting σ at 30% generally results in good performance. However, it is important to note that this setting results in the removal of a substantial portion of the samples, specifically 70%.

Uniqueness vs. Informativeness (λ_0 & λ_1): These parameters assign weights to Uniqueness ($\hat{U}$) and Informativeness ($\hat{I}$) during the sample selection process for manual labeling. $\hat{I}$ indicates how many samples are similar to a particular sample, while $\hat{U}$ measures how distinct a sample is from the rest. Setting $\hat{U}$ to a high value and $\hat{I}$ to a low value prioritizes the selection of outliers. Conversely, setting $\hat{I}$ high and $\hat{U}$ low focuses the selection on more common, similar samples. Typically, $\hat{I}$ is set high (*e.g.* 1) to ensure the model learns from the majority of data, and $\hat{U}$ is set low (*e.g.* 0.1 or 0.5) to select only a few outliers.

Appendix B

SLB

B.1 Implementation

We used the open-source implementation of MORSE provided by its authors (https://github.com/nuwuxian/morse/). For DT, we adopted the implementation made available by [273] (https://github.com/MalTools/MalWhiteout/tree/master/dt). To ensure a fair comparison, all frameworks, including SLB and the Vanilla baseline, were trained with the same model architecture, hyperparameters (*e.g.*, batch size), and initial weights. For each experiment, we fixed the initialisation seeds (using seeds from 1 to 10) and initialised the model weights with Xavier initialisation. This setup ensures that the results of each method are not affected by randomness.

B.2 Hyperparameter Space

We performed a grid search to determine the optimal hyperparameters for each method. Our search criterion was the highest mF1 score obtained using seed 1; once the best configuration was identified, we ran experiments with seeds 1 through 10. For random noise experiments, we selected the hyperparameters that performed best at a 10% noise rate and then applied this configuration consistently across all noise levels. Although ideally we would perform a separate grid search for each seed and noise rate, this approach is computationally prohibitive; hence, we adopt this strategy as a practical compromise between cost and tuning rigor.

For SLB, we focused on three key parameters: the number of epochs for the data split e (see Equation 5.2), the number of epochs before label flipping in the continuous revision phase m (see Equation 5.10), and the CB loss weighting parameter β (see Equation 5.6). The EMA parameter α was fixed at 0.95 (see Equation 5.9) due to its minimal impact on performance. Our search spaces were defined as $e \in \{2,5,10,15,20\}$, $m \in \{0,1,3,5\}$, and $\beta \in \{0.0,0.9999\}$, with 0.0 indicating no weighting.

For MORSE, which also utilises CB loss, we further tuned the parameter that determines when reweighting begins by testing values $\{20,30,40\}$. The remaining hyperparameters for MORSE were set analogously to those for SLB: the number of warm-up epochs before the data split used the

Table B.1: Statistics of the PE Malware dataset

Class	Training Samples		Noise Rate (%)	Testing Samples
	Clean	Noisy		
Benign	749	400	0.00	100
VirLock	796	800	0.50	100
WannaCry	818	820	0.24	100
Upatre	656	340	3.23	100
Cerber	801	944	15.57	100
Urelas	467	472	1.05	100
WinActivator	59	66	10.60	100
Pykspa	632	644	1.86	100
Ramnit	160	224	29.46	100
Gamarue	127	508	75.00	100
InstallMonster	105	199	47.23	100
Locky	59	57	54.38	100

same range as e, and the β parameter shared the same search space, while the ε parameter was tuned over $\{0.7, 0.9\}$ in line with the authors' recommendations. For `DT`, we tuned two key hyper-parameters: the number of rounds $n \in \{3, 5, 7, 10\}$ and the number of training epochs per round $e \in \{2, 5, 10, 15, 20\}$.

B.3 Datasets

PE Malware. We used a multi-class Windows PE malware dataset from [286]. The dataset comprises 1,024 features derived from four main sources: byte entropy histograms, PE imports, string histograms, and PE metadata. Each sample has two labels: a ground-truth label and a noisy label. Table B.1 outlines the dataset's details. The *noise rate* indicates the fraction of samples incorrectly assigned to a label; for example, among the 944 samples labeled as Cerber in the noisy dataset, 15.57% are actually not Cerber.

Table B.2: Statistics of the IDS17 and Virus-MNIST datasets

(a) IDS17

Class	Train	Test
Benign	10000	2000
DDoS	300	60
DoS Hulk	300	60
Portscan	250	50
Infiltration	250	50
DoS GoldenEye	200	40
DoS Slowloris	100	20
FTP-Patator	100	20
SSH-Patator	100	20

(b) Virus-MNIST

Class Examples	Train	Test
Benign[a]	2341	175
Adware	7188	496
Trojan 1	2832	205
Trojan 2	2228	176
Installer	738	58
Backdoor 1	6206	456
Crypto	14374	1003
Backdoor 2	7003	491
Downloader	2398	173
Heuristic	3114	225

[a] This class is completely benign

IDS2017. We used the enhanced multi-class CIC-IDS2017 dataset from [156], which contains packet captures and network flows generated by CICFlowMeter [145, 72]. Our experiments focused on the network flows. Labels were derived from attacker/victim IPs, ports, and timestamps,

Table B.3: Statistics of the VS18 dataset

Class	Training Samples		Noise Rate (%)	Testing Samples
	Clean	Noisy		
Benign	1132	1680	32.61	283
Shedun	3146	1000	4.60	786
SMSReg	3119	3452	12.42	780
Hiddad	182	438	71.68	46
Gappusin	146	300	91.00	37
Wapron	117	382	70.94	29
Youmi	66	379	87.07	16
Dowgin	47	324	89.50	12

which we removed to prevent shortcut learning. From the full dataset, we sampled 12,000 benign and 1,920 attack instances spanning eight primary attack classes. Table B.2a summarizes the distribution.

VirusShare 2018. We used the multi-class Android malware dataset produced by [176]. This dataset includes 215 features describing each app's characteristics, such as file size, the presence of embedded URLs (`has_url`), and requested permissions (*e.g.* `READ_SMS`). We labeled the dataset according to the following VT criteria: if at least *sixteen* AV engines detect an app as malicious and more than *five* engines agree on the same family name, we assign the sample to that malware family; if the above conditions are not met but at least *one* engine flags the app, we consider it *grayware* and remove it; and if no engines identify the app as malicious, we label it as benign. We also generated a noisy labeled set by relying solely on the `Lionic` AV engine's output. Table B.3 provides the class distribution of this dataset.

APIGraph 2017–2018. We utilised the two most recent years (2017–2018) of an Android application dataset from [48], originally introduced in [311]. This dataset was chosen for three reasons. First, its reports were collected at multiple time points (prior to 2020 in [311] and between 2022 and 2023 in [48]), which helps mitigate label noise. Second, a stringent labeling threshold was applied: a sample is considered malicious only if flagged by at least *sixteen* AV engines, ensuring high label confidence. Third, the dataset reflects real-world distributions, with only about 9% of the samples being malicious; accordingly, we formulate the task as a binary classification problem. In addition, the dataset comprises 1,159 features (excluding the label), which were extracted using DREBIN [25].

Virus-MNIST. We used this dataset to evaluate SLB on more advanced architectures (*e.g.* ResNet18), which are commonly employed in image-based malware analysis [317, 204, 251, 83]. Virus-MNIST transforms PE malware files into 32x32 grayscale images by sampling the first 1024 bytes of each PE [189]. Unlike a typical classification setting, the dataset includes a purely benign class, while the malicious samples are grouped into nine clusters using K-Means [166] to emulate an MNIST-like distribution. As a result, the focus shifts from classifying distinct malware families to identifying structurally similar clusters. The distribution of this dataset is shown in Table B.2b.